الإصدار الأول ٢٠٢٦

Python
بلغة بايثون

إتقان scikit-learn 1.9

من الصفر إلى الاحتراف

دليل شامل لتعلّم الآلة التطبيقي

شرح مبسّط
من الأساسيات إلى المتقدم

أمثلة عملية
وتطبيقات واقعية

مشاريع متكاملة
من الحياة العملية

أفضل الممارسات
والنصائح الاحترافية



م/ أحمد صالح المغز

2026

إتقان scikit-learn ١٫٩

من الصفر إلى الاحتراف

دليل شامل لتعلّم الآلة التطبيقي

م/أحمد صالح المغز

Eng. Ahmed Saleh Al-Maghz

scikit-learn ١.٩.١ ▪ Python ٣.١٤ ▪ NumPy ٢.٤ ▪ pandas ٣.٠

فهرس المحتويات

١ الأسس: من الفكرة إلى أول نموذج — ٩ فصلاً

البنية المرجعية، والرياضيات الحاكمة، وطريقة التفكير التي لا تُكسر

١ ما هو تعلّم الآلة؟ رحلة من الفكرة إلى النظام ١٤

الإطار المفاهيمي الذي يحوّل البيانات إلى قرارات

٢ منظوم بايثون العلمية: أين يقع scikit-learn؟ ٢٨

NumPy و SciPy و pandas و Matplotlib — ومَن المسؤول عن ماذا

٣ التثبيت والإعداد على Python ٣.١٤ ..... ٤٠

بيئة عمل نظيفة قابلة للتكرار، من venv إلى uv و conda

٤ أول نموذج لك: أقرب الجيران على بيانات Iris ٥٤

قراءة كود كامل من السطر الأول حتى الرسم البياني

٥ نموذج التقدير: فهم Estimator API ٦٩

fit · transform · predict · score — العقد الذهبي بينك وبين المكتبة

٦ الرياضيات تحت الغطاء ٨١

جبر خطي، وتفاضل، واحتمال، ودوال خسارة — بالقدر الذي تحتاجه فعلاً

٧ فحص مُقدِّر قبل أن يذهب إلى الإنتاج ٩٨

check_estimator والوسوم والعرض البياني HTML- الجديد في ١.٩

٨ مشروع تمهيدي شامل: أسعار المنازل ١١٠

خط أنابيب حقيقي من التحميل حتى التقرير

٩ أخلاقيات التطبيق وحدود الخوارزميات ١٢١

الحياد، والتحيّز، والخصوصية، ومسؤولية المهندس

٢ البيانات: الكتلة الخام للمشروع — ١١ فصلاً

من مصفوفة NumPy إلى المصفوفة المتفرّقة والبيانات المفقودة

١٠ أنماط البيانات في scikit-learn ١٣٣

DataFrame و ndarray والمصفوفات المتفرّرة: أيّها ولماذا

١١ المصفوفات المتفرّرة وواجهة sparray الجديدة ١٤٦

المفتاح sparse_interface في ١.٩ — ماذا يعني لك ولماذا يهمّك

١٢ أنواع البيانات: عددي، وفئوي، وزمني، ونصي ١٦١

كيف يميّز المُقدِّر نوع كل عمود، وماذا يفعل إن أخطأت

١٣ القيم المفقودة: من الفراغ إلى المعلومة ١٧٤

MissingIndicator و IterativeImputer و KNNImputer و SimpleImputer

١٤ التحجيم والتطبيع ١٨٨

KernelCenterer و Normalizer و Robust و MaxAbs و MinMax و Standard

١٥ التحويلات اللاخطية والتوزيعية ٢٠١

PolynomialFeatures و Spline و KBins و Quantile و Power

١٦ ترميز المتغيرات الفئوية ٢١٦

OneHot و Ordinal و TargetEncoder — وأين تكمن مخاطر الترتيب

١٧ هندسة الخصائص: من الصف إلى المعلومة ٢٣١

التركيبات، والتحويلات النسبية، والنطاق الدلالي

١٨ النصوص: من مستند إلى متجه ٢٤٢

CountVectorizer و TfidfVectorizer و Hashing و Stopwords و N-grams

١٩ الصور والبيانات غير الجدولية ٢٥٢

استخراج الخصائص، وتدرّجات الصورة، وتحويل الصورة إلى مصفوفة

٢٠ واجهة set_output: نتائج على شكل DataFrame ٢٦١

كيف تُخرج جداول قابلة للقراءة مباشرة من كل محوّل

٣ هندسة الخصائص واختيارها

٦ فصلاً

من مئة عمود إلى عشرة أعمدة تحمل الإشارة

٢١ أبعاد البيانات: لماذا تكره الخوارزميات الأبعاد الكثيرة؟ ٢٧٣

لعنة الأبعاد، وهندسة الفضاء عالي الأبعاد

٢٢ التصفية التي لا تنظر إلى النموذج ٢٨٤

VarianceThreshold و SelectKBest و SelectPercentile والانحصارات

٢٣ التصفية التي تنظر إلى النموذج ٢٩٨

SelectFromModel و RFE و RFECV و SequentialFeatureSelector

٢٤ الاختيار داخل خط الأنابيب ٣١١

لماذا يجب أن يكون المُنتقي طبقة داخل Pipeline لا خطوة يدوية

٢٥ قياس أهمية الخصائص: ثلاث طرق مختلفة ٣٢٢

التباين الأولي، ومعاملات الانحدار، و permutation_importance

٢٦ المعلومات المتبادلة والاختيار غير المشرف ٣٣٢

تقدير MI، والاعتماد غير الخطي، والعيّنات الصغيرة

٤ النماذج الخطية ..... ١٠ فصلاً

العائلة الأولى التي يجب أن تتقنها قبل أي شيء آخر

٢٧ قبل الانحدار الخطي: من الخطأ إلى أقل المربعات ٣٤٧

مشكلة أقل المربعات، وتحليل SVD، والمتبقيات، والأقل تربيعاً

٢٨ تشخيص النموذج الخطي ٣٥٧

الفرضيات، والتعدد الخطي، و VIF، ومخطط QQ، والأثر

٢٩ الانحدار المنظّم: Ridge و ElasticNet ٣٦٨

لماذا يحتاج النموذج الخطي إلى عقوبة، وكيف تعمل L1 و L2

٣٠ Lasso: النموذج الخطي الذي يُصفّر ٣٧٨

التباعد، وهندسة المسار، واختيار alpha عبر التحقق المتقاطع

٣١ الانحدار المرن والمتعدد المهام ٣٨٦

MultiTaskLasso و MultiTaskElasticNet و enet_path و lars_path

٣٢ الانحدار المتسامح وتحمّل الصفوف الشاذة ٣٩٥

Huber و TheilSen و RANSAC و Quantile على بيانات ملوّثة

٣٣ الانحدار حسب التوزيع ٤٠٤

Poisson و Gamma و Tweedie — متى يتفوّق على النموذج الخطي

٣٤ الانحدار البيزي ٤١٢

BayesianRidge و ARDRegression — التنظيم الاحتمالي

٣٥ التصنيف اللوجيستي: من الاحتمال إلى القرار ٤٢١

الاشتقاق، و decision_function، ومتعدد الفئات، وحلول المعادلات

٣٦ المصنِّفات الخطية الأخرى ٤٣٢

Perceptron و PassiveAggressive و SGD و RidgeClassifier

٥ أشجار القرار ٥ فصلاً

نماذج تقسّم العالم إلى صناديق، وتشرح نفسها بلا رياضيات

٣٧ شجرة القرار: من سؤال إلى شجرة ٤٤٢

خوارزميتا ID٣ و C٤.٥ — الجيني والإنتروبيا ومكسب المعلومات

٣٨ شجرة الانحدار وتقسيم خطأ مربّع ٤٥٣

التباين، والانحراف المعياري، ومتوسط خطأ الشجرة

٣٩ التشذيب والإفراط في الملاءمة ٤٦٢

cost_complexity_pruning و ccp_alpha و decision_path والتصوّر

٤٠ ضبط شجرة القرار ٤٧٢

العمق، وحدّ الأوراق، و class_weight، والقيود، والقيم الناقصة

٤١ قراءة الشجرة وتفسيرها ٤٨١

export_text و plot_tree و feature_importances_ ومثال يدوي كامل

٦ المجموعات والأغلفة

٩ فصلاً

«ليس نموذج واحداً كافياً»

٤٢ التجميع بالعيّنات: Bagging ٤٩١

التمهيد بالعيّنات، والدمج بالتصويت، والتقدير خارج العيّنة OOB

٤٣ الغابة العشوائية ٥٠٠

التباين يُقتل بالتباين، و max_features، و ExtraTrees

٤٤ ExtraTrees وتضمين الأشجار العشوائية ٥٠٩

انقسامات عشوائية تماماً و RandomTreesEmbedding

٤٥ AdaBoost: التعلّم بالنقاط الصعبة ٥١٨

خوارزمية SAMME، وأوزان النماذج الضعيفة، وتحديث العيّنات

٤٦ تعزيز التدرّج: شرح كلمة بكلمة ٥٣٣

الدالة الخسارة، والبواقي، و learning_rate، و subsample

٤٧ HistGradientBoosting: المعيار الصناعي ٥٤٦

المدرّجات التكرارية، والقيم المفقودة الأصلية، والسرعة

٤٨ التصويت والتكديس ٥٦٢

VotingClassifier و StackingClassifier و VotingRegressor و StackingRegressor

٤٩ ضبط نموذج لمجموعة كاملة ٥٧٨

استراتيجية بحث صحيحة عبر Bagging و Boosting

٥٠ متى تستخدم أي مجموعة؟ ٥٩١

جدول قرارات يربط نوع البيانات بالاختيار الأمثل

٧ خوارزميات خاضعة للرقابة أخرى

٨ فصلاً

آلات المتجهات الداعمة، والجيران، وبيز، والتمييز

٥١ آلات المتجهات الداعمة: الحيلة الحقيقية ٦٠٧

الهامش الأقصى، و KKT، ولماذا لا تتأثر SVM بحجم المقياس

٥٢ أنوية SVM ومعاملاتها C و gamma ٦٢٢

RBF و poly و sigmoid، ومقارنة SVC مع LinearSVC

٥٣ SVR و OneClassSVM ٦٣٧

الانحدار بمتجهات داعمة، واكتشاف الشذوذ بلا تسميات

٥٤ الجيران الأقرب: k-NN ٦٥٠

التصنيف والانحدار، والأوزان، والمسافات، وأثر التطبيع

٥٥ بُنى البحث في الجيران ٦٦٤

KDTree و BallTree و RadiusNeighbors ومحوّلات الجيران

٥٦ بيز الساذج ٦٧٨

GaussianNB و MultinomialNB و ComplementNB و BernoulliNB

٥٧ تحليل التمييز ٦٩١

LDA و QDA وانكماش التغاير والتوزيع الغاوسي

٥٨ نماذج النواة والشبكات العصبية ٧٠٥

KernelRidge وتقريب النواة و BernoulliRBM و MLP

٨ التعلّم غير الخاضع للرقابة ١٣ فصلاً

عندما لا توجد إجابة نموذجية، بل يوجد بنية مخفية

٥٩ التعلّم غير الخاضع للرقابة: الخريطة ٧٢٢

ماذا يعني بلا تسميات، ومتى يكون مفيداً ومتى يكون فخاً

٦٠ K-Means وخوارزمية Lloyd ٧٣٥

المراكز، و k-means++، و Elbow، و Silhouette، والحدود المفهومية

٦١ نسخ K-Means العملية ٧٤٩

MiniBatchKMeans و BisectingKMeans و get_bin_seeds

٦٢ التقدير التلقائي: MeanShift و AffinityPropagation ٧٦٢

تقدير نطاق النواة، ونقل رسالة التفضيل بين العيّنات

٦٣ العناقيد الكثيفة: DBSCAN و OPTICS ٧٧٣

النقاط النواة، و eps، و min_samples، واستخراج العناقيد

٦٤ HDBSCAN: عدد كتل غير معروف مسبقاً ٧٨٥

تقدير الكثافات المتغيرة والثبات وتدرّجات الكثافة

٦٥ التجميع الهرمي والطوبولوجي ٧٩٩

AgglomerativeClustering و Ward و Spectral و FeatureAgglomeration

٦٦ كيف تقيس نجاح تجميعٍ بلا إجابة؟ ٨١٥

Silhouette و Davies-Bouldin و Calinski-Harabasz و ARI و NMI

٦٧ النماذج المزيجية الغاوسية ٨٢٩

خوارزمية EM وتفسير المكوّنات واختيار عدد المزيج

٦٨ تقليل الأبعاد الخطي: PCA ٨٤٣

تحليل SVD وتفسير المكوّنات ونسخ PCA العملية

٦٩ تقليل الأبعاد غير الخطي ٨٥٨

KernelPCA و Isomap و LLE و MDS و TSNE وموثوقية الإسقاط

٧٠ كشف الشذوذ وتقدير التباين ٨٧٢

IsolationForest و LOF والمسافة الماهالانوبية وانكماش التغاير

٧١ النصوص: التحليل الدلالي الخفي والمواضيع ٨٨٥

TF-IDF و TruncatedSVD و LatentDirichletAllocation

٩ التقييم والاختيار ..... ١٢ فصلاً

بلا قياس دقيق، كل تحسين وهم

٧٢ تقسيم البيانات: الخطوة التي تُفشل المشاريع ٨٩٨

train_test_split وترتيب العمليات والتسرب غير المقصود

٧٣ التحقق المتقاطع: كل أنواعه ٩١١

KFold و Stratified و Group و TimeSeries و LeavePOut و Predefined

٧٤ التقاط التقييمات وتتبع الأدوات ٩١٩

Parallel و cross_val_predict و cross_val_score و cross_validate

٧٥ مقاييس التصنيف: ما بعد الدقة ٩٢٧

الخلط، والدقة، والاسترجاع، و F1، والخصوصية، و MCC

٧٦ منحنيات القرار و ROC و PR ٩٤٣

det_curve و auc و precision_recall_curve و roc_curve

٧٧ ضبط عتبة القرار ٩٥٧

DetCurveDisplay و FixedThresholdClassifier و TunedThresholdClassifierCV

٧٨ مقاييس الانحدار ٩٧٢

MSE و RMSE و MAE و MAPE و MSLE و R^2 و d^2 و Pinball

٧٩ مقاييس متقدّمة ومجموعات غير مستوية ٩٨٧

متعدّد التسميات، والترتيب، وتوصيات DCG، ومعايير التغطية

٨٠ منحنى التعلّم ومنحنى التحقق ١٠٠١

قراءة تشخيصية للإفراط في الملاءمة والتباين والتحيّز

٨١ GridSearchCV: البحث المنضبط ١٠١٣

ParameterGrid و refit و best_score_ و cv_results_

٨٢ البحث العشوائي والتوزيعات ١٠٢٧

RandomizedSearchCV و log-uniform وتوازن الاستكشاف

٨٣ اختبار الأهمية الإحصائية ١٠٤١

permutation_test_score والمقارنة الصحيحة بين نماذج

١٠ الأنابيب والتركيب

٦ فصلاً

هندسة النماذج كخط إنتاج متكامل

٨٤ خط الأنابيب: عقد أمانة مع البيانات ١٠٥٥

fit و transform و predict، والتسلسل، وعناوين الخطوات

٨٥ ColumnTransformer: تركيب الأعمدة ١٠٧٢

الاختيار بالاسم والنمط، والمُحدِّد make_column_selector

٨٦ FeatureUnion والتركيبات المتقدمة ١٠٨٠

دمج مسارات متوازية، والتداخل، وحالات الاستخدام

٨٧ تحويل الهدف وتحويل المدخلات ١٠٨٨

TransformedTargetRegressor و FunctionTransformer وتحويل الهدف

٨٨ التخزين المؤقت والخطوط الثقيلة ١١١٠

joblib.Memory، ومتى يُستحق، ومتى يكون فخاً

٨٩ مخطط خط الأنابيب كاملاً ١١١٧

قالب معماري لمشروع إنتاجي من طرف إلى طرف

١١ المفاهيم المتقدّمة ٩ فصلاً

واجهات ١.٩، وتوجيه البيانات، وكتابة مُقدِّر خاص بك

٩٠ واجهة الاستدعاءات الجديدة ١١٤٠

ProgressBar و ScoringMonitor و with_callbacks واستدعاءات مخصّصة

٩١ توجيه البيانات الوصفية ١١٤٨

Metadata Routing: تمرير sample_weight و groups عبر Pipeline و GridSearchCV

٩٢ دعم واجهة Array API ١١٦٧

array_api_dispatch، والانتقال إلى وحدة معالجة الرسوميات

٩٣ إطار narwhals وتبادل البيانات ١١٧٤

DataFrame من أي مكتبة: pandas أو Polars أو Modin أو PyArrow

٩٤ واجهات العرض البياني في ١.٩ ..... ١١٩١

من ConfusionMatrixDisplay إلى PartialDependenceDisplay والعرض HTML- الجديد

٩٥ كتابة مُقدِّر متوافق من الصفر ١١٩٨

BaseEstimator و fit و predict و transform والوسوم والعقود

٩٦ التوسّع والتخصيص ١٢٢٨

الفئات الفرعية، والوسوم، وأنواع الأشجار، والاختبارات

٩٧ الأداء والتوازي والذاكرة ١٢٣٩

joblib.Parallel، وقياس الذاكرة، وأسرار التسريع

٩٨ معجم الأخطاء: من الرسالة إلى الحل ١٢٤٦

أكثر ثلاثين خطأ في scikit-learn، وسبب كل خطأ وحلّه

١٢ مشاريع واقعية كاملة ٦ فصلاً

من الفكرة إلى التقرير، بتفاصيل لا تُحذف

٩٩ مشروع ١: تصنيف البريد المزعج وتحليل المشاعر ١٢٥٩

TF-IDF، Bayes، LinearSVC ثم تقرير

١٠٠ مشروع ٢: توقّع مغادرة العملاء ١٢٦٨

تحليل RFM، والبيانات غير المتوازنة، ومعيار الربح

١٠١ مشروع ٣: كشف عمليات الاحتيال ١٢٧٦

البيانات شديدة الاختلال، وضبط العتبة، وتفسير النتائج

١٠٢ مشروع ٤: التعرّف على الأرقام المكتوبة بخط اليد ١٢٨٤

دورة حياة كاملة من التحميل حتى تقييم التصنيف متعدد الفئات

١٠٣ مشروع ٥: محرّك توصيات ١٢٩٢

الترشيح المبني على المحتوى، والتعاون، ومعايير NDCG

١٠٤ مشروع ٦: الصيانة التنبّؤية للآلات ١٢٩٩

بيانات المستشعرات، وكشف الشذوذ، وتنبيهات الصيانة

الجزء ١

الأسس: من الفكرة إلى أول نموذج

البنية المرجعية، والرياضيات الحاكمة، وطريقة التفكير التي لا تُكسر

ماذا يغطّي هذا الجزء

قبل أن تكتب سطر كود واحد، تحتاج إلى نموذج ذهني متماسك: ما هو «المُقدِّر»؟ لماذا يبقى محفوظاً في الذاكرة بعد التدريب؟ ولماذا يُقسَّم الداتا إلى مجموعتين لا غير؟ هذه القوة تبني الأرضية التي يقف عليها كل ما يلي. هنا نضع المصطلحات، والمعادلات، والعادات الذهبية التي ستصنع الفارق بين مهندس يتعلم ومهندس يتقن.

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ١ — ما هو تعلّم الآلة؟ رحلة من الفكرة إلى النظام

الإطار المفاهيمي الذي يحوّل البيانات إلى قرارات

الوحدة **scikit-learn ١.٩** الحجم **١٠٢٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تميّز الأنماط الأربعة لتعلّم الآلة وتختار لكلٍّ منها الأداة المناسبة
- أن تشرح بعينك الفرق بين التحيز وفرق التباين بالبيانات لا بالحفظ
- أن تقرأ منحنى التعلّم وتستنتج منه ما إذا كان النموذج ينقصه بيانات أم سعة
- أن تجيب سؤال «هل أحتاج تعلّم آلة هنا؟» بسؤال أفضل من جواب فوري

١.١ متى تكون القاعدة اليدوية أرخص وأدقّ من النموذج؟

ابدأ من الطرف المعاكس للنموذج: تخيّل أنك تقرّر أيّ بريدك يصل إلى صندوق الواجهة. القاعدة اليدوية ساذجة: «إذا احتوت الرسالة على كلمة "free" فورياً فهو مزعج». هذه القاعدة تعمل جيداً تماماً حتى اللحظة التي يجد فيها أحدهم طريقة للإزعاج دون كلمة "free". عندها تنهار: إما أن تضيف قاعدة، أو تضيف استثناء، أو تعيد البناء من الصفر في كل مرة.

هذا هو السؤال الحقيقي خلف كل مشروع تعلّم آلة: **هل المشكلة قرار متكرر يجب أن يُتخذ بناءً على معطيات متغيّرة، بحيث يصير الأداء البديهي غير كافٍ؟**

لا أحد ينصحك أبداً باستخدام تعلّم الآلة في هذه الحالات:

الحالة	لماذا التعلّم الآلي قرار سيّئ
القواعد معروفة، قليلة، وثابتة	شيفرة `if` أرخص وأسرع وأوضح من أي نموذج
يجب تفسير كل قرار قانونياً	النموذج يُعطي إجابة؛ ولا يعطي ضماناً
لا يوجد اتصال بين المدخلات والمخرج	لا يوجد ما يُتعلَّم أصلاً
التكلفة من تعبئة البيانات تفوق أي مكسب	المشروع كلّف أكثر مما يوفّر

القاعدة العملية التي سترددّها في هذا الكتاب مراراً: **ابدأ بأبسط نموذج يمكن أن يعمل، ثم برهن أنت تحتاج شيئاً أكبر**. أغلب منظومات الإنتاج الراشدة تبدأ بنموذج خطي بسيط، وتقيس، ثم تعقّد فقط حين يعطي النموذج البسيط دليلاً على أنه لم يعد كافياً.

مسألة صناعية مصغّرة: هل تحتاج نموذجاً أصلاً؟ python

```python
from sklearn.datasets import make_classification, make_regression
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge
from sklearn.metrics import r2_score
from sklearn.model_selection import StratifiedKFold, cross_val_score,
learning_curve, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

Xc, yc = make_classification(n_samples=1200, n_features=6, n_informative=4,
                             n_redundant=1, class_sep=1.4, random_state=42)
Xr, yr = make_regression(n_samples=800, n_features=5, noise=12.0,
                         random_state=42)

print("classification :", Xc.shape, "-> class counts", np.bincount(yc))
print("regression     :", Xr.shape, "-> target mean %.2f  std %.2f"
      % (yr.mean(), yr.std()))
print("feature means  :", np.round(Xc.mean(axis=0), 3))
print("feature stds   :", np.round(Xc.std(axis=0), 3))
```

لاحظ رقمين في هذا الناتج. الأول: الفئات شبه متوازنة (٦٠٤ و٥٩٦)، وهذه معلومة حسابية هامة عند اختيار المقياس لاحقاً. الثاني: متوسّطات الأعمدة ليست صفراً بالضبط، وانحرافاتها المعيارية تتضاعف مرّتين من عمود لآخر. هذه الأرقام ليست تفصيلاً — بل هي أول قراراتك المعمارية في الفصل الثالث.

ولا تنسَ السؤال الذي بدأنا به. على بعض المسائل، القاعدة اليدوية تتفوّق:

```python
# حين تكون القاعدة اليدوية هي الحل الصحيح

rng = np.random.default_rng(3)
ang = rng.uniform(0, 2 * np.pi, 600)
rad = rng.uniform(0.3, 3.0, 600)
Xc = np.column_stack([rad * np.cos(ang), rad * np.sin(ang)])
yc = (rad > 2.0).astype(int)

rule = (np.hypot(Xc[:, 0], Xc[:, 1]) > 2.0).astype(int)
logreg = LogisticRegression(max_iter=2000).fit(Xc, yc)
print("حدّ قرار دائري مثالي:")
print("  4.% : قاعدة بالمسافة" % (rule == yc).mean())
print("  4.% :        خط مستقيم" % logreg.score(Xc, yc))

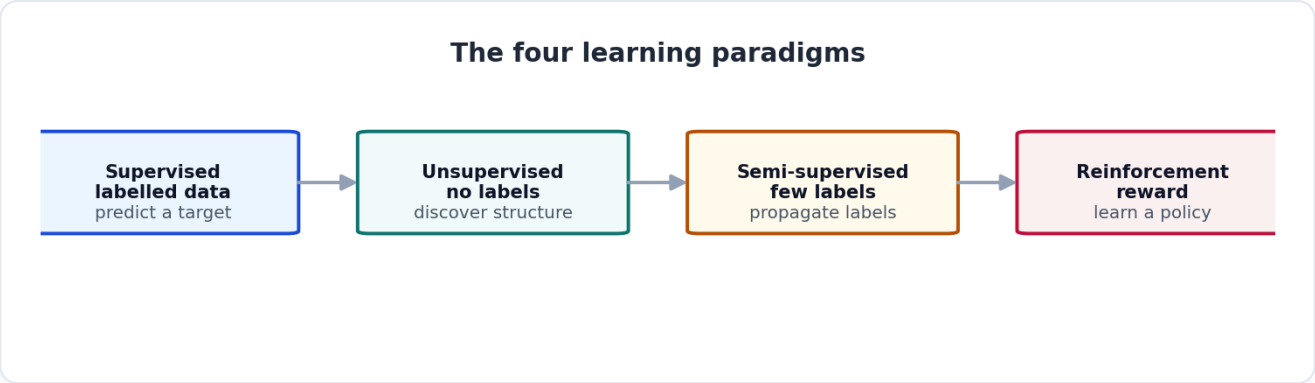
xq, yq = make_regression(n_samples=60, n_features=1, noise=6.0, random_state=0)
xq = xq.ravel()
med = np.median(xq)
median_pred = np.where(xq <= med, yq[xq <= med].mean(), yq[xq > med].mean())
lin = LinearRegression().fit(xq.reshape(-1, 1), yq)
print("\nمسألة بعلاقة نعّاحة وضجيج:")
print("  قاعدة الوسيط     : R2 = %.4f" % r2_score(yq, median_pred))
print("  الانحدار الخطي  : R2 = %.4f" % r2_score(yq, lin.predict(xq.reshape(-1, 1))))
```

القاعدة اليدوية هنا لا تُخطئ ولا موضعاً واحداً، بينما الخط المستقيم يخطئ في ٣٧٪ من النقاط لأن الحاجز دائري لا خطّي. وفي المثال الثاني انقلب ميزان المقارنة: قاعدة الوسيط تخسر بـ `R²` قدره ٠٫٦٢، والانحدار الخطي يفوز بـ ٠٫٩٦. **القرار ليس «أيّهما أفضل» بل «هل شكل العلاقة يطابق الأداة»** — وهذا بالضبط ما تبحث عنه عمليات التعلّم الآلي. لاحظ أيضاً أن كلفة القاعدة يمكن أن تكون باهظة: `kNN` في الفصل ٥٤ هو في الحقيقة قاعدة «المسافة إلى أقرب جار» مكتوبة بلغة تعمّم.

القاعدة العملية التي سنردّدها في هذا الكتاب مراراً: **ابدأ بأبسط نموذج يمكن أن يعمل، ثم برهن أنت تحتاج شيئاً أكبر**. أغلب منظومات الإنتاج الراشدة تبدأ بنموذج خطي بسيط، وتقيس، ثم تعقّد فقط حين يعطي النموذج البسيط دليلاً على أنه لم يعد كافياً.

١.٢ الأنماط الأربعة: أين يقع مشروعك؟

قبل تحميل أي بيانات، اسمحأنفسك يجيب على سؤال التصنيف. لا يوجد نموذج واحد يصلح لكل الأنماط؛ كل نمط له بنية بيانات مختلفة تماماً وطريقة قياس نجاح مختلفة.



الشكل ١مخطط يوضّح الأنماط الأربعة لتعلّم الآلة: بيانات موسومة تنبّأ بهدف، وبيانات بلا وسم تكتشف بنية، وبيانات شبه موسومة تنتقل بوسم قليل، وبيانات مكافأة تعلّم سياسة

النمط الأول **خاضع للرقابة** (supervised): لديك مدخلات `X` ومخرجات `y` مسمّاة. هذا أكثر الأنماط انتشاراً في الصناعة، وكل الفصول من الثاني إلى العاشر تتعامل معه. النموذج يتعلّم الدالة التي تربط بينهما.

النمط الثاني **غير خاضع للرقابة** (unsupervised): لا مخرجات والهدف هنا ليس التنبّؤ بل **الوصف**: أين العناقيد؟ كم بُعداً حقيقياً في البيانات؟ هل هناك شذوذ؟ تقييمه لا يكون بدقّة بل بمعيار داخلي (silhouette مثلاً) أو بجدوى الأعمال.

النمط الثالث **شبه خاضع للرقابة** (semi-supervised): بيانات موسومة قليلة جداً وسطوف كبيرة. تظهر قيمته في الصور والأصوص حيث لا يمكنك وسم كل ملف.

النمط الرابع **معزَّز** (reinforcement): لا يوجد مخرجات أصلاً، بل مكافأة. القرار يُتعلَّم بالتجربة والخطأ. هذا النمط خارج نطاق هذا الكتاب؛ مكتبة scikit-learn لا تقدّم له واجهة تعليمية، وإن كنت تحتاجه فانظر إلى مكتبة مستقلة.

وسنرى لاحقاً في الفصل ٦١ كيف يبدو النمط الثاني عملياً. يكفي هنا أن ترى أن المخرج ليس وسماً بل **تقسيم للمجموعة**، وقد ينتهي بخمسة عناقيد رغم أن الحقيقة أربعة. لا يوجد في هذا النمط «صواب» يُقاس بدقّة، بل معايير داخلية تصف جودة التقسيم نفسه.

```python
# النمط غير الخاضع للرقابة: ما الذي يُخرجه فعلاً؟
from sklearn.cluster import DBSCAN, KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score, silhouette_score

Xu, truth = make_blobs(n_samples=420, centers=4, cluster_std=0.7,
random_state=42)
km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(Xu)
db = DBSCAN(eps=0.9, min_samples=8).fit(Xu)
n_db = len(set(db.labels_)) - int(-1 in db.labels_)

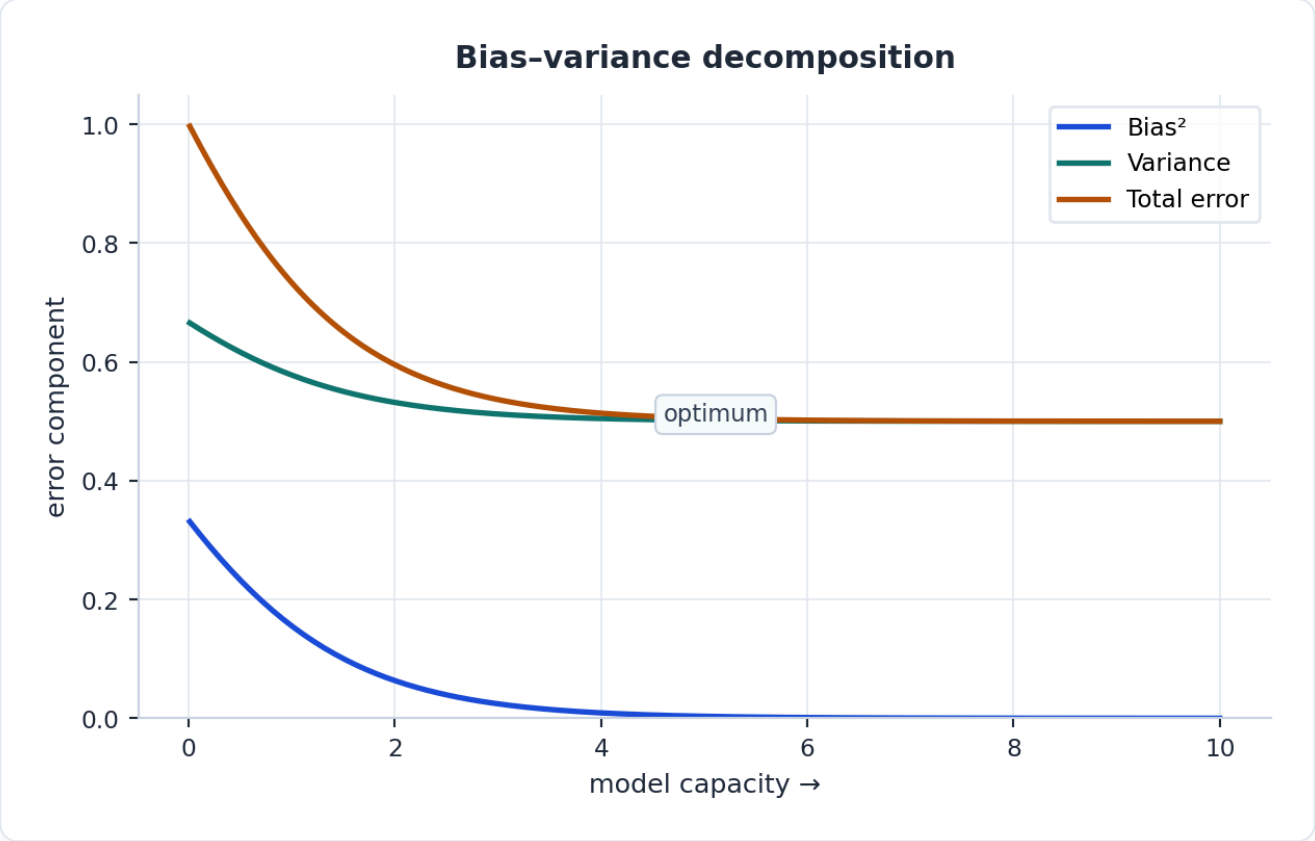
print(f"عدد المجموعات المكتشفة : KMeans {len(np.unique(km.labels_))}"
      f"   DBSCAN {n_db}   (الحقيقة {len(np.unique(truth))})")
print(f"adjusted_rand_score    : KMeans {adjusted_rand_score(truth,
km.labels_):.4f}"
      f"   DBSCAN {adjusted_rand_score(truth, db.labels_):.4f}")
print(f"silhouette_score       : KMeans {silhouette_score(Xu, km.labels_):.4f}"
      f"   DBSCAN {silhouette_score(Xu, db.labels_):.4f}")
print(f"كشذوذ DBSCAN نقاط وصفها: {int((db.labels_ == -1).sum())}")
```

لاحظ أن `KMeans` طُلب منه مسبقاً أن يجد أربعة عناقيد، بينما `DBSCAN` اكتشفها بنفسه ووسم بعض النقاط بأنها شذوذ (الوسم `-1`). وفي النمط الخاضع للرقابة كان من السهل أن تقول «صنّف الصف ٧ و٧٤ صحيحاً أم خطأ»؛ هنا لا يوجد جواب مرجعي، فتنتقل إلى معيار **داخلي** مثل `silhouette_score` يقيس مدى بُعد كل نقطة عن مجاورتها مقارنةً بالمجموعات الأخرى.

> 🔑 **الفكرة المحورية**
>
> خوارزمية تعلّم الآلة ليست ذكاءً اصطناعياً بالمعنى الغامض. هي **دالة حسابية محدّدة سلفاً**، لها عدد محدود من المعاملات تضبطها، ومدخلها ومخرجها أرقام. كل ما في الموضوع هو اختيار هذه الدالة وضبط معاملاتها على بيانات صحيحة.

١.٣ المسألة ليست الدقة — بل التعميم

هذا هو المفهوم الذي يفصل بين من «يتعلّم» و«يقول إنه تعلّم». أنت لا تنشر دقة ٩٧٪ على البيانات التي درّبتها عليها؛ أنت تنشر دقة **غير مرئية بعد**.



الشكل ٢ منحنى يوضّح تحلّل خطأ الاختبار إلى تحيّز مربّع وتباين زائد وخطأ إجمالي، بأدنى مجموع عند سعة متوسطة من النموذج

الصورة هي تقسيم خطأ الاختبار الكلاسيكي. النموذج بسيط جداً (شجرة عمق واحد) فإنه يقارب البنية العامة ولا يلحق التفاصيل: **تحيّز عالٍ** (bias)، أي خطأ نظامي في الافتراض نفسه. النموذج معقّد جداً بلا ضبط فيتعلّم ضجيج التدريب حرفياً: **تباين عالٍ** (variance)، أي أن شجرة واحدة تكفي، وإعادة التدريب ببذرة أخرى تعطي شجرة مختلفة تماماً. بينهما نقطة توازن يقال لها «الحجم الأمثل».

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2[\hat{f}(x)] + \text{Var}[\hat{f}(x)] + \sigma^2$$

انتبه لحدّ الضجيج: هناك حدّ لا يمكن لأي خوارزمية أن تتخطّاه. فإذا كانت البيانات رديئة، أو سعة النموذج غير كافية، قارب الخطأ هذا الحدّ. ولهذا نتوقّف عن تحسين النموذج حين يقترب الخطأ منه.

والطريقة العملية التي تخبرك أين تقف هي منحنى التعلّم: زد حجم التدريب وراقب الفجوة.

```python
# منحنى التعلّم: هل أزيد بيانات أم أزيد تعقيداً؟
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score,
learning_curve

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
candidates = {
    "DummyClassifier(strategy='most_frequent')":
DummyClassifier(strategy="most_frequent"),
    "LogisticRegression(max_iter=2000)": LogisticRegression(max_iter=2000),
    "KNeighborsClassifier(n_neighbors=5)": KNeighborsClassifier(n_neighbors=5),
}
print(f"{'model':<38} {'cv accuracy':>12} {'std':>8}")
for name, est in candidates.items():
    s = cross_val_score(est, Xc, yc, cv=cv, scoring="accuracy")
    print(f"{name:<38} {s.mean():>12.4f} {s.std():>8.4f}")

lr_model = candidates["LogisticRegression(max_iter=2000)"]
sizes, train_scores, val_scores = learning_curve(
    lr_model, Xc, yc, cv=cv, train_sizes=np.linspace(0.1, 1.0, 8),
    scoring="accuracy", n_jobs=1)

print("\nمنحنى التعلّم للانحدار اللوجيستي:")
print(pd.DataFrame({
    "n_train": sizes.astype(int),
    "train_acc": train_scores.mean(axis=1).round(4),
    "val_acc": val_scores.mean(axis=1).round(4),
    "gap": (train_scores.mean(axis=1) - val_scores.mean(axis=1)).round(4),
}).to_string(index=False))
```

اقرأ الجدول زمنياً من الأعلى إلى الأسفل. الفجوة بين عمودي `train_acc` و `val_acc` هي فجوة التعميم: عند أول ٩٦ صفاً تكون ٠٫٠٥٥، ثم تهبط تحت ٠٫٠١ ابتداءً من ٣٤٢ صفاً وتبقى كذلك. هذا يعني أن النموذج **انتهى من استغلال البيانات الجديدة**: كل سطر إضافي في الجدول يمنحه أقل من نصف نقطة مئوية. النموذج هنا مقيَّد بسعته، لا ببياناته.

ولهذا يبدو خطأً أن تُصدر النموذج بعد ١٢٠٠ صف. لو كان منحنى التعلّم ما يزال يصعد، فالبيانات هي الحلّ. ولأنّه مستقرّ عند ٠٫٨١، فالبيانات ليست الحلّ — بل ضبط النموذج.

١.٤ السعة الزائدة: لماذا ينهار الأداء خارج عيّنته؟

لنجعل الإفراط في الملاءمة ملموساً. المسألة نفسها ثلاث مرات: خط مستقيم، ثم كثير حدود من الدرجة الثالثة، ثم كثير حدود من الدرجة الثانية عشرة. النموذج في الحالات الثلاث هو نفسه — `Ridge` منتظماً — ولا يتغيّر إلا **عدد المعاملات**.

ثلاث درجات سَعة على مسألة انحدار واحدة

```python
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

Xr, yr = make_regression(n_samples=120, n_features=3, noise=28.0,
random_state=42)
Xtr, Xte, ytr, yte = train_test_split(Xr, yr, test_size=0.3, random_state=42)
print(f"train {Xtr.shape}   test {Xte.shape}   target std {yr.std():.2f}")

print(f"\n{'degree':>7} {'n_params':>10} {'train R2':>10} {'test R2':>10}")
for deg in (1, 3, 12):
    pipe = make_pipeline(PolynomialFeatures(degree=deg),
                         StandardScaler(),
                         Ridge(alpha=1.0))
    pipe.fit(Xtr, ytr)
    n_params = len(pipe[-1].coef_) + 1
    print(f"{deg:>7} {n_params:>10} {pipe.score(Xtr, ytr):>10.4f} "
          f"{pipe.score(Xte, yte):>10.4f}")
```

اقرأ السطر الأخير. الدرجة الثانية عشرة **تفوز** على التدريب (٠٫٩٨) و**تنهار** على الاختبار (-١٧٫٠٩). معامل الـ `R²` السالب ليس خطأً في الحساب: معناه أن النموذج أسوأ من قاعدة تراهن دائماً على متوسّط الهدف. عملياً هذا يعني أن مقترحاته أبعد عن الحقيقة ممّا تبعده أي نقطة أخرى في البيانات.

هذه ليست حادثة نادرة بل **سلوك نهائي متوقّع**: كل معامل إضافي هو قدرة إضافية على حفظ كل نقطة. النموذج الذي يحفظ كل شيء لا يحكم على شيء.

ودلالتها الرياضية أن الخسارة نفسها لها حدّان يتصارعان. الانتظام (`ridge`، `lasso`) يضيف عقوبة على حجم المعاملات، فيقايض جزءاً من ملاءمة التدريب مقابل ثبات أكبر:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} [\sum_{i=1}^{n} (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2 + \lambda \|\mathbf{w}\|_2^2]$$

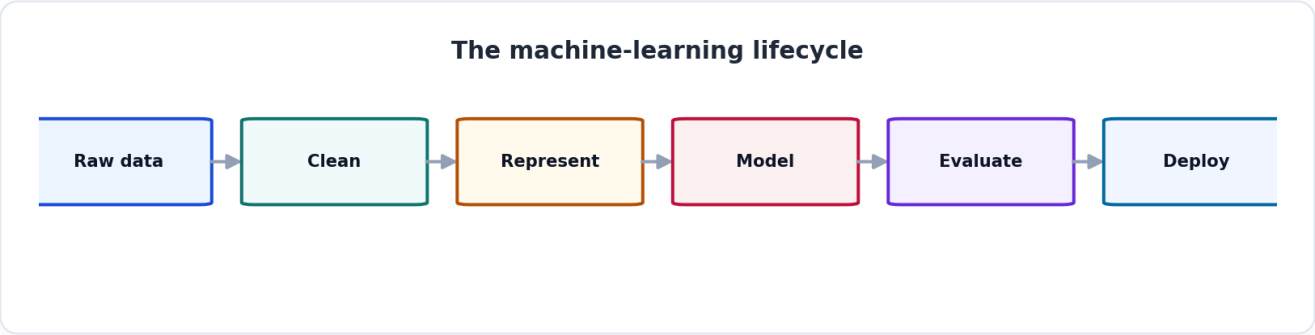
`n` هو عدد الصفوف و `d` عدد المعاملات، و `lambda` هو معامل الانتظام: مقبض واحد يحكم المفاضلة كلها. عند صفر تكون المربعات الصافية وحدها (ملاءمة كاملة)، وكلما ارتفع تحوّل النموذج إلى أبسط ما يمكن. ولهذا فإن اختيار معامل الانتظام ليس تفصيلاً تجميلياً بل هو جوهر «التعلّم الآلي» في بضع عشرات الأسطر.

الظاهرة	التشخيص	العلاج
تدريب ممتاز، اختبار ضعيف	سعة زائدة	تقليل `max_depth`، زيادة `C` في الانتظام، تجميع
تدريب ضعيف، اختبار ضعيف	سعة ناقصة	نموذج أقوى، ميزات أفضل، بيانات أكثر
تدريب ممتاز واختبار ممتاز	غالباً تسرّب	راجع تقسيم البيانات قبل أن تفرح
تدريب ضعيف واختبار ممتاز	خطأ عشوائي أو تسرّب	أعد التقسيم، تحقق من عدم التسرّب

> **أشيع خطأ في مشاريع المبتدئين**
>
> أن يبلّغ عن النموذج على نفس البيانات التي دُرّب عليها، ثم يعلن «٩٧٪ دقة». هذا ليس إنجازاً. هذا قياس ذاكرة لا قياس قدرة. الفصل ٧٢ يشرح تقسيم البيانات الذي يمنع هذا الخطأ، لكن اعرف من الآن أن أي رقم دقة بلا مجموعة اختبار مستقلة هو رقم بلا معنى.

١.٥ المسار من البيانات إلى النظام

النموذج ليس المنتج. منتجك هو **خط أنابيب موثّق** ينتهي بمخرج يتّخذ قراراً ويعرف كيف نُقيس نجاحه.



الشكل ٣ مخطط تدفّق يمرّ بالبيانات الخام فالتنظيف فتمثيل البيانات فالنموذج فالتقييم فالنشر

خمس نقلات في الصورة. كل نقلة هي مكان يمكن أن تنكسر فيه الجودة:

خط أنابيب مصغّر: كل شيء في كائن واحد python

```python
from sklearn.datasets import make_moons
from sklearn.neighbors import KNeighborsClassifier

Xm, ym = make_moons(n_samples=300, noise=0.45, random_state=42)
Xm_tr, Xm_te, ym_tr, ym_te = train_test_split(Xm, ym, test_size=0.3,
                                              random_state=42, stratify=ym)
knn_pipe = make_pipeline(StandardScaler(),
                         KNeighborsClassifier(n_neighbors=7))
knn_pipe.fit(Xm_tr, ym_tr)

print("خط الأنابيب:", " -> ".join(name for name, _ in knn_pipe.steps))
print("تدريب f}   {اختبار f" % (knn_pipe.score(Xm_tr, ym_tr),
                                 knn_pipe.score(Xm_te, ym_te)))
print("احتمال الصنف 1 لأول ثلاث نقاط:",
      np.round(knn_pipe.predict_proba(Xm_te[:3])[:, 1], 3))

print("\nأثر اختيار الجيران:")
for k in (1, 5, 25):
    knn_pipe[-1].set_params(n_neighbors=k)
    print(f"  k={k:>2}  تدريب {knn_pipe.score(Xm_tr, ym_tr):.4f}"
          f"   اختبار {knn_pipe.score(Xm_te, ym_te):.4f}")
```

انتبه إلى `k=1`: دقّة تدريبه ١٫٠٠ لأن الجار الأقرب هو النقطة نفسها. هذه ليست موهبة بل تشخيص مباشر: ذاكرة بلا تعميم، ودرجات الاختبار عنده ٠٫٧٠ فقط. في المقابل، `k=25` ينسى بعض البنية ويرفع دقّة الاختبار إلى ٠٫٨٢. القمّة الحقيقية تقع في المنتصف عند `k=5` بدقّة اختبار ٠٫٨١ ودقّة تدريب ٠٫٨٨. الضبط الجيّد هو اختيار التوازن بين الاثنين، لا اختيار الرقم الأعلى في أي عمود.

هذا الكائن قابل للتسلسل والحفظ ويُعاد إنتاجه بأمر واحد. الفصل ٨٤ يشرح `Pipeline` بالتفصيل؛ هنا نكفي بمبدئه: **الخطأ في الترتيب جسيم**. لو محوّل التحجيم تدرّب على كامل البيانات قبل التقسيم، تسرّب المعلومات ينتج دقة وهمية.

من دفتر تجربة حقيقي

في تطبيق ائتمان صغير، بنى فريق هندسي نموذجاً «جيداً» بدقّة ٠٫٨٧ على مجموعة التقييم. المشكلة أن سلسلة التحقق كانت تشترك مع التدريب في حساب المتوسّطات وفي اختيار أفضل عتبة. بعد نقل كل العمليات داخل `Pipeline` نزلت الدقة الحقيقية إلى ٠٫٧٩. الرقم الأعلى لم يكن دقة أعلى — كان تسرّباً مُقنعاً. كل نقلة في مخطط الحياة أعلاه هي بالضبط مكاناً يمكن أن يدخل منه التسرّب.

١.٦ لا توجد خوارزمية أفضل — بل مطابقة

الوقوع في مغالطة المفاضلة يمرّ بسرعة: ينظر أحدهم إلى جدول دقة في ورقة بحثية ويحمل الرقم إلى مشروعه. هذا خطأ منهجي، لأن جداول الدقة هذه تقيس ثلاثة أشياء معاً في آن واحد: **نوع البيانات، وحجم العيّنة، ونطاق الضبط**.

نوع البيانات	النماذج التي تتفوّق عادةً	لماذا
جدولي صغير، أبعاد قليلة	أشجار ومجموعات	التقاط التفاعل بلا معاملات كثيرة
جدولي كبير، أبعاد عالية	خطية ومنتظمة	تكلفة العيّنة تهدّم الأشجار العميقة
نص متفرّق، آلاف الأبعاد	خطية مع تمثيل متفرّق	المسافة في فراغ واسع
صور	مجموعات تعزيز وشبكات	العيّنات كثيرة والعلاقة معقّدة
سلاسل زمنية	نماذج لها بنية زمنية	الترتيب جزء من المعلومة

وثلاثة استنتاجات ثابتة في كل مراجعات منهجية تقريباً:

وهذا هو المعنى الحقيقي لعبارة «النموذج الأفضل». هي ليست خاصية للنموذج، بل خاصية **ثلاثية: (نموذج، بيانات، ميزانية)**. ولهذا يفشل السؤال «ما أفضل خوارزمية؟» بوصفه سؤالاً، وينجح سؤاله المفيد: «ما أفضل خوارزمية لبياناتي، بميزانية تدريب لا تتجاوز عشر دقائق؟».

١.٧ تمارين الفصل

التمرين ١.١

استعمل `make_classification` بـ `n_samples=800` و `n_features=10` و `n_informative=3` و `random_state=7`، وقس `LogisticRegression` و `RandomForestClassifier` و `KNeighborsClassifier` بـ `cross_val_score`. ارسم بيانياً يربط حجم التدريب بدقة التحقق. في أي حجم يتوقف النموذج عن الاستفادة؟ بزّر برقم من الجدول لا بحدس.

التمرين ١.٢

على بيانات `make_moons(noise=0.30)`، قارن `KNeighborsClassifier` بـ k من ١ إلى

التمرين ١.٣

اجمع في قاموسك من هذا الفصل خمسة مؤشرات على أنك تسير نحو الإفراط في الملاءمة، وثلاثة مؤشرات على أنك تسير نحو سعة ناقصة. اشرح لكل مؤشر لماذا يدلّ على ما يدلّ عليه، لا لماذا «يبدو سيئاً».

التمرين ١.٤

ابحث عن مثال في عملك أو دراستك يمكن حلّه بقاعدة ثابتة. اكتبه في خمس أسطر، ثم اكتب ما الذي يجعل انتقاله إلى تعلّم الآلة مبرَّراً أو غير مبرَّر.

١.٨ حلول بعض التمارين

✅ **حلّ التمرين ١.ء**

الأرقام تتفاوت مع بيئتك، ولكن المنطق من المصدر. أكبر نتائج إجراءات الخطة أصل:

حجم التدريب	انحدار لوجيستي	غابة عشوائية
٦٤	٠٫٨٦	٠٫٨٧
٢٢٨	٠٫٨٦	٠٫٨٨
٤٧٥	٠٫٨٧	٠٫٨٩
٦٤٠	٠٫٨٧	٠٫٨٩

انظر إلى الغابة العشوائية: مكسبها الكلي من ٦٤ صفاً إلى ٦٤٠ صفاً هو **نقطة واحدة ونصف مئوية** موزّعة على عشر مضاعفات. وبعد ٤٧٥ صفاً صار المكسب ٠٫٦٦ نقطة ثم صفراً. هذا هو جواب التمرين: يتوقف النموذج عن الاستفادة عند بضع مئات من الصفوف، ومجمع البيانات بعد ذلك إهدار. والانتقال إلى الغابة نفسها لم يعطِ هنا أكثر من نقطة مئوية واحدة عن اللوجيستي، رغم أن كلفة تدريبها أعلى بكثير.

المعيار العملي: توقّف عن جمع البيانات عندما لا يزيد منحنى التحقّق بأكثر من نصف نقطة مئوية لكل مضاعفة للحجم.

✅ **حلّ التمرين ١.٢**

ستجد نمطاً مألوفاً: دقّة التحقق **ترتفع** بمقدار عشرة نقاط مئوية من `k=1` حتى القمّة (`k=15` تعطي دقّة اختبار ٠٫٩٣) ثم تبدأ بالانخفاض عند `k=25`. السبب عددي لا حدسي: بعد تقسيم ما يقارب ٦٠٠ عيّنة بنسبة ٨٠/٢٠ يبقى لكل صنف نحو ٢٤٠ صفاً في التدريب. عند `k=1` القرار مبني على نقطة واحدة فضوضاء؛ وعند `k=25` يصبح مبنياً على متوسّط نحو ٩٪ من العنقود فيفقد حدّة الفصل. وبين الرقمين يقع أفضل توازن.

١.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
«الخوارزمية A أفضل دائماً»	نموذج ممتاز على بياناتك وسابق على غيرها	اختر بـ `cross_val_score` على بياناتك أنت
الإفراط في الملاءمة بلا انتباه	فجوة تدريب/اختبار تكبر كلما زادت المعاملات	ابنِ أبسط نموذج ممكن أولاً
قياس الأداء على بيانات التدريب	رقم قائم على الذاكرة لا على القدرة	احجز مجموعة اختبار لم تلمسها
القفز إلى الشبكة العصبية أولاً	كلفة ووقت بلا فائدة على بيانات جدولية	ابدأ بالنموذج الخطي
نسيان الفحص البصري للبيانات	نموذج ممتاز يتعلّم عموداً تسريباً	ارسم التوزيعات والارتباطات أولاً
قياس `train_score` فقط في السجلّ	لا إنذار مبكر على أي تدهور	سجّل نتيجة كل طيّة

١.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تعلّم الآلة	بناء دالة من بيانات بدل كتابتها يدوياً	machine learning
خاضع للرقابة	تعلّم من مدخلات مع مخرجات موسومة	supervised learning
غير خاضع للرقابة	اكتشاف بنية بلا مخرجات	unsupervised learning
معزَّز	تعلّم سياسة من مكافآت	reinforcement learning
تحيّز	خطأ نظامي في افتراض النموذج	bias
تباين	حساسية النموذج لتغيّر عيّنة التدريب	variance
الإفراط في الملاءمة	حفظ الضجيج فتتضرّر دقة الاختبار	overfitting
سعة النموذج	قدرته على تمثيل دوال معقّدة	model capacity
التعميم	أداء النموذج على بيانات غير مرئية	generalization
التسرّب	دخول معلومات الاختبار في التدريب	data leakage
منحنى التعلّم	دقة الأداء بدلالة حجم التدريب	learning curve

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٢ — منظوم بايثون العلمية: أين يقع scikit-learn؟

NumPy و SciPy و pandas و Matplotlib — ومَن المسؤول عن ماذا

الوحدة **scikit-learn ١.٩** الحجم **٧٢٣** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن ترسم خريطة المنظوم العلمي وتحدّد مسؤولية كل مكتبة فيه
- أن تشرح لماذا المصفوفة في NumPy ليست قائمة أقواس بل كائن له ذاكرة ودلالات محاور
- أن تعرف متى تستدعي SciPy ومتى تكفيك NumPy وحدها
- أن تقرأ كائن Bunch من scikit-learn وتستخرج منه ما تحتاجه بالاسم

٢.١ من أين تأتي هذه المنظومة؟

تخيّل أنك تبني بيتاً. أنت لا تصبّ الأساس بنفسك، ولا تخشب الجدران، ولا تفرد الأسقف. تستعمل عمّالاً يتقنون كل حرفة. وهذا بالضبط ما يفعله بايثون في مجال تعلّم الآلة: **مكتبة متخصّصة في كل طبقة، ومكتبة واحدة تجمعها**.

قبل نحو عشرين سنة كان الوضع مختلفاً. كانت كل أداة مكتوبة بلغة مختلفة، وكل مكتبة تتحدّث عن البيانات بصيغتها الخاصة: `R` أعطتك `data.frame`، و `MATLAB` أعطاك `matrix`، و `C++` أعطاك `vector<double>`. زرعتها من أداة إلى أخرى، وكتبت محوّلات في كل مرة.

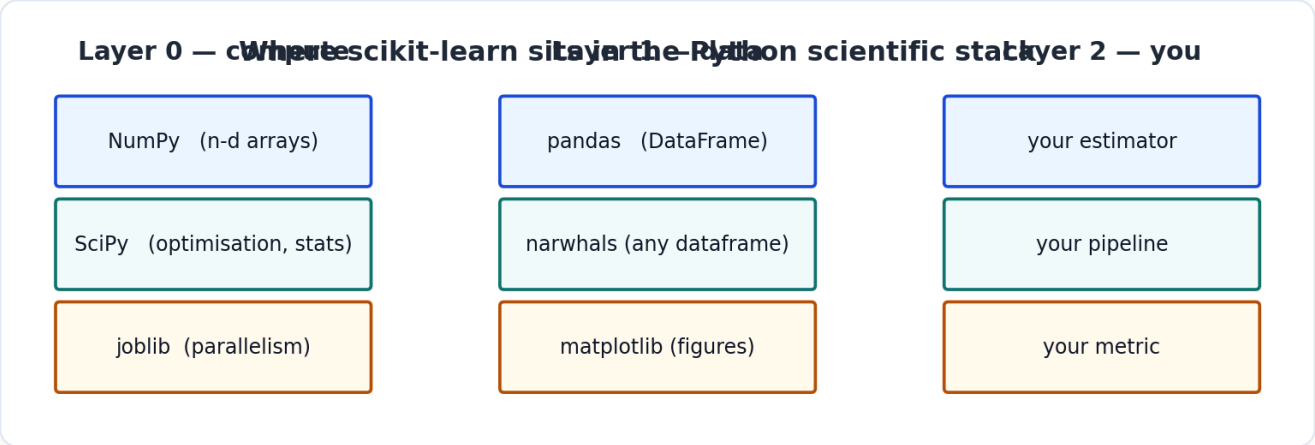
```python
# نسخة الكتاب: من علوم البيانات إلى هذه الأدوات
import platform
import sys

import scipy
import sklearn

print("python    ", sys.version.split()[0], platform.python_implementation())
print("numpy     ", np.__version__)
print("scipy     ", scipy.__version__)
print("pandas    ", pd.__version__)
print("matplotlib", mpl.__version__)
print("sklearn   ", sklearn.__version__)
print("\nscipy.linalg يوفّر:", [n for n in dir(scipy.linalg)
                               if n in ("solve", "inv", "lstsq", "svd", "qr")])
```

هذه الأرقام ليست زينة. النسخة محصورة في الكتاب كلّه، وكل سلوك موصوف مربوط بها. وحين يذكر الكتاب أن الجيران القريبين صاروا أسرع، فذلك في هذا الإصدار فقط.

٢.٢ الطبقات الخمس: من يُنفّذ ومن يقرّر



الشكل ٤خريطة ثلاث طبقات للمنظوم العلمي: طبقة الحوسبة وفيها NumPy و SciPy و joblib، وطبقة البيانات وفيها pandas و narwhals و matplotlib، وطبقة المستخدم وفيها المُقدِّر وخط الأنابيب والمقياس

الصورة مقصودة بهذا الترتيب لأن الاعتماد يسير من الأسفل إلى الأعلى: كودك **يستدعي** scikit-learn، و scikit-learn **يستدعي** SciPy و NumPy، و SciPy **يستدعي** NumPy. لا تستدعي NumPy شيئاً أعلى منها.

الطبقة	المكتبة	مسؤوليتها الوحيدة
الحوسبة	`numpy`	مصفوفات الأرقام، الحساب عليها، الذاكرة
الحوسبة	`scipy`	خوارزميات علمية: تحسين، إحصاء، جبر خطي، تكامل
الحوسبة	`joblib`	التوازي والتخزين المؤقّت للنتائج
البيانات	`pandas`	الجداول ذات الأعمدة المسمّاة والصفوف المفهرسة
البيانات	`narwhals`	طبقة توافق مع أي إطار جداول (اعتماد جديد في ١.٩)
البيانات	`matplotlib`	الرسوم والأشكال
التعلّم	`sklearn`	الخوارزميات والمقياسات وخطوط الأنابيب
أنت	كودك	القرار: أي بيانات، أي نموذج، أي مقياس

القاعدة التي تلخّص الجدول: **scikit-learn لا يخزّن بيانات ولا يرسم ولا يدير ملفات**. ما يفعله محدود: يتعلّم من `X` و `y`، ويحفظ ما تعلّمه في سمات داخل الكائن، ثم يجيب. كل ما عداه مهمتك أنت.

ولكل مكتبة معادلة واحدة ترمز لدورها:

$$\mathbf{y} \approx f_{\mathbf{w}}(\mathbf{x}) \qquad \mathbf{X} \in R^{\,n \times d} \qquad \min_{\mathbf{w}} J(\mathbf{w})$$

النموذج يربط `X` بـ `y` بمعاملات `w`؛ و NumPy تمسك `X` ككائن `n × d`؛ و SciPy تعطي طريقة للعثور على `w` الذي يُصغّر دالة الخسارة `J`. الفصلان الخامس والسادس يفكّكان هذه المعادلة سطراً سطراً.

> **تخصيص المسؤولية**
>
> إذا وجدت نفسك تكتب دالة `logsumexp` أو تحسب مشتقّ على ورقة، فقد تجاوزت حدود scikit-learn إلى SciPy.
> وإذا وجدت نفسك تكتب `for` يدوّر على صفوف، فغالباً تجاوزت حدود NumPy إلى pandas.

٢.٣ NumPy: لماذا المصفوفة ليست قائمة؟

الفرق العملي بين `list` في بايثون و `ndarray` ليس السرعة فقط — بل **الدلالة**. القائمة في بايثون حاوية غير متجانسة: قد تحوي أرقاماً ونصوصاً، ولا تضمن أن عناصرها من النوع نفسه. `ndarray` حاوية **متجانسة** لها عدد أبعاد

محدّد، وكل خانة فيها بايتات ثابتة العرض. هذا هو السبب الذي يجعل مكتبة NumPy سريعة: يمكنك أن تمرّر عنوان البيانات إلى C دون أن تفحص كل عنصر.

python — المصفوفة، البثّ، ومحاور الحساب

```python
A = np.arange(12).reshape(3, 4)

print("A.shape =", A.shape, "  ndim =", A.ndim, "  nbytes =", A.nbytes)
print("A.T.shape =", A.T.shape, "  transpose is a view, not a copy:",
      np.shares_memory(A, A.T))
print("slice A[:, :2] shares memory with A:", np.shares_memory(A, A[:, :2]))

# البثّ: عمود (3,1) + صف (1,4) يعطيان مصفوفة (3,4)
col = A[:, :1]
row = A[:1, :]
print("col", col.shape, "row", row.shape, "-> sum", (col + row).shape)

# المحور 0 يجمع عبر الصفوف فيبقى عدد الأعمدة
print("\nمجموع كل عمود (axis=0):", A.sum(axis=0))
print("مجموع كل صف  (axis=1):", A.sum(axis=1))
print("قاعدة: ينتج في البعد *غير* المذكور")
```

القاعدة التي تختصر آخر سطرين تُحفظ عن ظهر قلب: `axis=k` **يلغي البعد** `k`. لمن نسيناها مرة فكّرنا فيها كـ `axis="columns"`، فضاعفنا الدوال على البيانات كلها.

وأخطر ما في NumPy ليس الأداء بل **النسخ الضمني**. `A[:, :2]` أعلاه لا تنسخ شيئاً؛ هي نافذة (view) على الذاكرة نفسها. إذا كتبت فيها، تتغيّر `A`. احترس من هذه الحيلة تحديداً حين تعمل داخل حلقة على نتفق التحويل.

٢.٤ SciPy: الرياضيات التي لا يجب أن تعيد اختراعها

NumPy تعرف الحساب. SciPy تعرف **الخوارزمية**. الفرق بينهما هو الفرق بين أن تجمع الأرقام يدوياً وأن تعرف خوارزمية مونت كارلو.

```python
أربع أدوات من SciPy ستقابلها في هذا الكتاب
from scipy import linalg, optimize, special, stats

xs = np.linspace(-4, 4, 9)
print("special.expit  (logistic, مستقرّة عددياً):", np.round(special.expit(xs), 4))
print("stats.norm.pdf(±1, 0):", np.round(stats.norm.pdf([-1.0, 0.0, 1.0]), 4))
print("stats.norm.cdf(1.96): %.4f" % stats.norm.cdf(1.96))

res = optimize.minimize(lambda t: (np.exp(-t ** 2 / 2) - 0.5) ** 2,
                        x0=0.0, method="Nelder-Mead")
print("optimize.minimize:", res.success, " argmin = %.4f" % res.x[0])

M = np.array([[4.0, 1.0], [1.0, 3.0]])
b = np.array([1.0, 2.0])
print("linalg.solve:", np.round(linalg.solve(M, b), 4),
      "   inv(M)@b:", np.round(linalg.inv(M) @ b, 4))
```

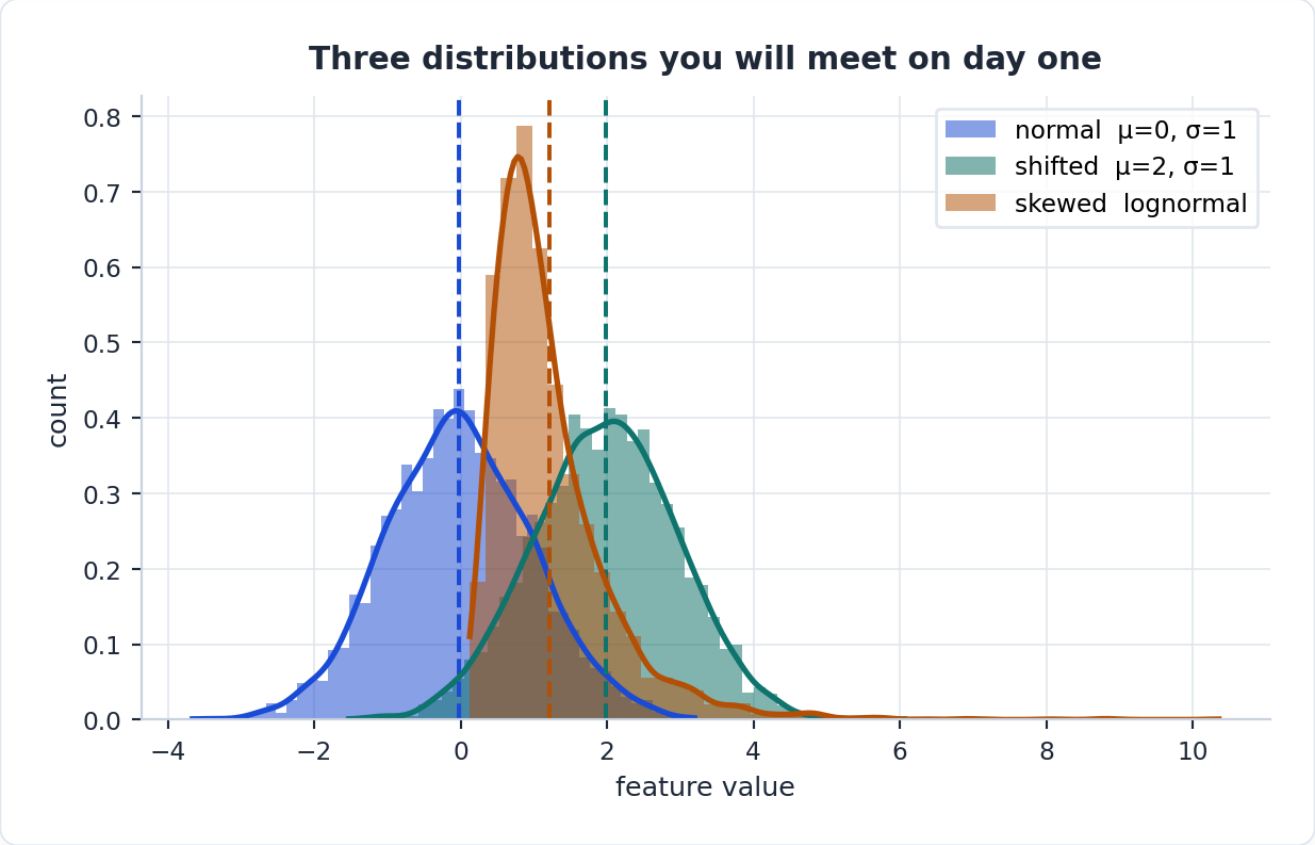
لاحظ سطر `special.expit` تحديداً. الدالة المنطقية القياسية تنهار عددياً عند القيم السالبة الكبيرة: `exp(800)` يفيض إلى ما لا نهاية. `expit` هي **نفس الدالة** محسوبة بطريقة مستقرّة، وهي ما تستعمله المكتبة داخلياً. هذا درس في الثقة: أن تختار المكتبة ليس لأنك كسول، بل لأن طريقة التنفيذ أفضل من نسختك.

الدالة في SciPy	ما تفعله بالضبط
`special.expit`	اللوجيستية المستقرّة عددياً
`stats.norm`	التوزيع الطبيعي: كثافة، توزيع تراكمي، مئينات
`optimize.minimize`	مصادقة المعاملات (انحدار، إسقاط، تعظيم)
`linalg.solve`	حل نظام خطّي **دون** حساب المعكوس

لاحظ السطر الأخير: `linalg.solve` أسرع وأدق من `inv(M) @ b`، ولا فرق في الناتج على هذه المصفوفة. لكن الفرق يظهر مع مصفوفات رديئة الحالة: المعكوس يضخّم الخطأ رقمياً. معادلتان تلخّصان أداتين من الأربع:

$$p(x) = \frac{1}{\sigma \sqrt{2 \pi}} e^{-\frac{(x - \mu)^2}{2 \sigma^2}} \qquad \mathrm{argmin}_{\mathbf{w}} \, J(\mathbf{w}) \;\mathrm{vs}\; \mathrm{argmin} \, \|\mathbf{A} \mathbf{w} - \mathbf{b}\|$$

الأولى كثافة التوزيع الطبيعي: منحنى بجرس يحكمه المتوسّط `mu` والانحراف `sigma` وحداهما. والثانية لماذا نفضّل `solve`: نحن نبحث عن `w` الذي يُناسب المعادلة `Aw = b`، لا عن معكوس `A` كاملاً.



الشكل ٥ثلاثة توزيعات ستقابلها من اليوم الأول: طبيعي بمتوسّط صفر، وآخر مزاح إلى اليمين، وآخر مائل إلى اليمين مع ذيل طويل

الشكل أعلاه هو أكثر توزّع ستراها في بياناتك الحقيقية. لاحظ التوزيع الثالث ذي الذيل الطويل. حين يكون عمودك مائلاً بهذه الدرجة، فالنماذج التي تفترض التوزيع الطبيعي (مثل انحدار Ridge الافتراضي) تعطي أرقاماً أقل جودة مما تتوقّع. الفصل ١٥ في `QuantileTransformer` و `PowerTransformer` يعالج هذه الحالة بالضبط.

٢.٥ pandas: الجدول الذي يحمل اسمه

الفرق العملي بين NumPy و pandas يظهر في جملة واحدة: **NumPy تعدّ الأعمدة من الصفر، و pandas تحمل اسم كل عمود.**

```python
# الفرق العملي: ndarray مقابل DataFrame
sales = pd.DataFrame({
    "city": ["ب" ,"أ" ,"ب" ,"أ"],
    "x": [1.0, 2.0, 3.0, 4.0],
    "y": [10, 20, 30, 40],
})
print("shape:", sales.shape, "  dtypes:\n", sales.dtypes.to_string())
print("\nتجميع حسب city:\n", sales.groupby("city")[["x",
"y"]].mean().to_string())
print("\nselect_dtypes يختار الأعمدة الرقمية تلقائياً:")
print(sales.select_dtypes("number").columns.tolist())

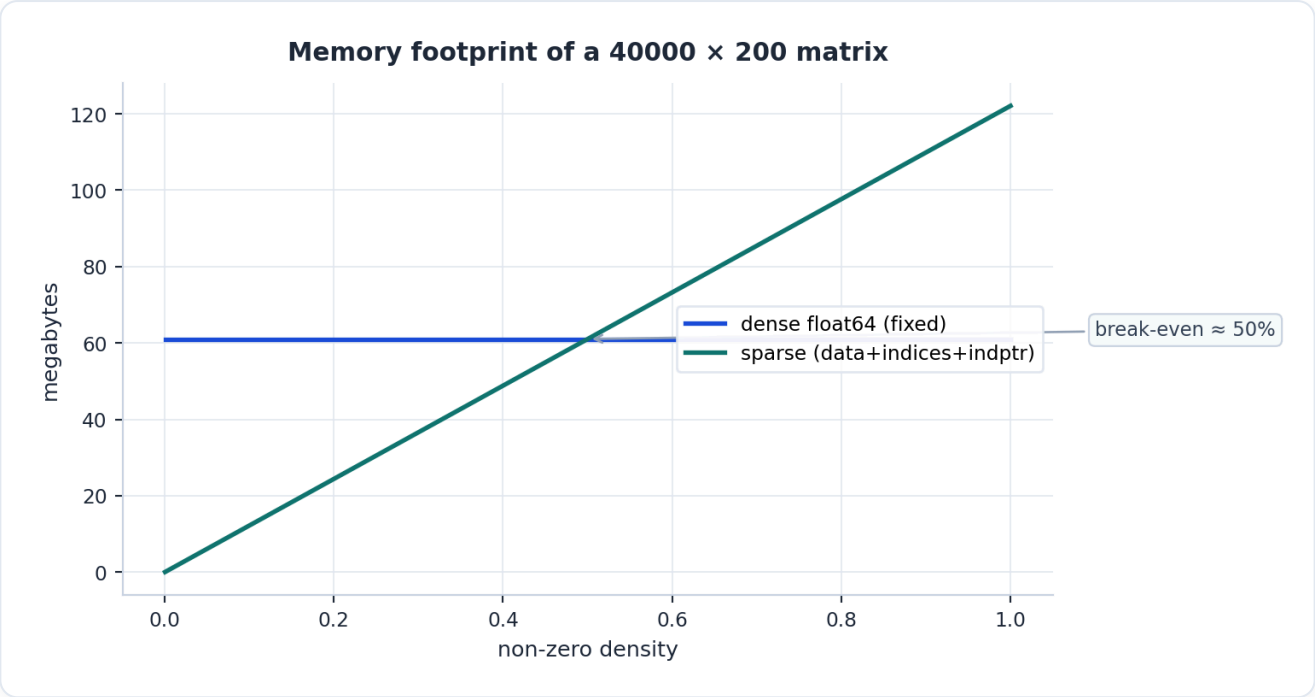
print("\nالآن NumPy لا يعرف أسماء الأعمدة:")
arr = sales[["x", "y"]].to_numpy()
print("  arr.shape =", arr.shape, "   arr[0] =", arr[0])
```

لاحظ الفرق في السطر الأخير: `arr[0]` صف بلا هوية. وحين يمرّر هذا إلى `LogisticRegression.fit` يرى `X` بلا أسماء أعمدة، فيضطر للتخمين عند الخطأ. هذا ليس تفصيلاً تجميلياً: الفصل ٢٠ كلّه في واجهة `set_output` لمنع هذا الارتباك.

> **مرّر DataFrame دائماً إن أمكن**
>
> بمجرد أن تمرّر `DataFrame` إلى `fit`، تُتاح لك السمتان `feature_names_in_` و `n_features_in_` تلقائياً، وتظهر أسماء الأعمدة في رسالة الخطأ حين يخطئ عدد الأعمدة. هذه وحدها تستحق ثوانٍ من وقتك في كل مرة.

٢.٦ الذاكرة: لماذا يهمّك عدد القيم المفقودة؟

البيانات النادرة (sparse) ليست ترفاً: مستند من عشرين ألف كلمة فيه ٣٪ فقط من الكلمات غير مكررة. يخزّنه `scipy.sparse` كثلاثة مصفوفات صغيرة بدل مصفوفة هائلة من الأصفار.



الشكل ٦مخطّط يقارن استهلاك الذاكرة لمصفوفة ٤٠٠٠٠×٢٠٠: الخط الكثيف ثابت بينما ينمو المتفرّق خطياً مع كثافة القيم غير الصفرية، والتعادل عند نحو ٥٠٪.

القاعدة العددية من الشكل: الكثافة تتجاوز ٥٠٪ يصبح التوفير صفراً بل سالباً، لأن تخزين فهرس لكل قيمة لا صفرية يكلّف أكثر من تخزينها نفسها. من هنا قاعدة التوجيه التي ستكررها في الفصل ١١: **لا تحوّل مصفوفة متفرّقة إلى كثيفة إلا إذا كانت الكثافة فوق ٥٠٪**.

حساب الكلفة الحقيقية للمصفوفتين — python

```python
from scipy.sparse import csr_matrix

rng = np.random.default_rng(0)
dense = rng.random((300, 400))
sp = csr_matrix(dense * (dense > 0.97))

sparse_bytes = sp.data.nbytes + sp.indices.nbytes + sp.indptr.nbytes
print(f"كثافة المصفوفة: {sp.nnz / dense.size:.4f}")
print(f"الحجم الكثيف : {dense.nbytes / 1024:8.1f} KB")
print(f"الحجم المتفرّق: {sparse_bytes / 1024:8.1f} KB")
print(f"نسبة التوفير : {dense.nbytes / sparse_bytes:.1f}x")
print("\nnnz =", sp.nnz, "  من أصل", dense.size)
```

هذه نسبة ٢٢ ضعفاً على كثافة ٣٪. لكن لاحظ أن `csr_matrix` هنا مبنيّة من مصفوفة كثيفة معبّأة أصلاً — في الواقع العملي تبنيها مباشرة من القوائم، فلا تدفع كلفة الكثافة أصلاً.

٢.٧ كائن Bunch: ماذا تعطيك مكتبة البيانات؟

كل دالة `load_*` في scikit-learn تُعيد كائناً واحداً اسمه `Bunch`، وهو قاموس ذو سمات. أن تعرف اسم كل مفتاح يكفيك لتستخرج أي شيء.

تشريح كائن Bunch من بيانات Iris — python

```python
from sklearn.datasets import load_iris

iris = load_iris()
print("type:", type(iris).__name__)
print("keys:", sorted(iris.keys()))
print()
print("data.shape     :", iris.data.shape, type(iris.data).__name__)
print("target.shape   :", iris.target.shape, " values:", np.unique(iris.target))
print("target_names   :", list(iris.target_names))
print("feature_names  :", iris.feature_names)
print("DESCR line 1   :", iris.DESCR.splitlines()[0])
print("frame (default):", iris.frame)

iris_df = load_iris(as_frame=True)
print("frame (as_frame=True):", type(iris_df.frame).__name__,
      iris_df.frame.shape)
print("target as Series     :", type(iris_df.target).__name__)
print(iris_df.frame.head(3).to_string())
```

لاحظ أن `frame` يبقى `None` ما لم تطلب صراحةً بـ `as_frame=True`. هذه مفاجأة تواجه كل مبتدئ: يعيد الكائن مفتاحاً يبدو جاهزاً وهو في الواقع فارغ. والسبب منطقي: بناء `DataFrame` يكلّف ذاكرة ووقتاً، فلم تفعلها المكتبة إلا طلبتها.

المفتاح	نوعه	ما فيه
`data`	`ndarray` بمصفوفة float	القيم الرقمية
`target`	`ndarray` بأعداد صحيحة	الوسم الرقمي لكل صف
`target_names`	قائمة نصوص	معنى كل رقم في `target`
`feature_names`	قائمة نصوص	معنى كل عمود
`frame`	`DataFrame` أو `None`	الجدول كاملاً، يُملأ فقط مع `as_frame=True`
`DESCR`	نص طويل	شرح البيانات ومصادرها

٢.٨ تمارين الفصل

التمرين ٢.١

أنشئ مصفوفة `np.arange(24).reshape(2, 3, 4)` ثم طبّع `np.sum` و `np.mean` على `axis=0` و `axis=1` و `axis=2`. اكتب في سطر واحد ما الذي ينتجه `axis=k` في كل حالة، ثم تحقّق من إجابتك حسابياً.

التمرين ٢.٢

احسب `stats.norm.cdf` عند z = ١,٩٦ و ٢,٥٨، وفسّر لماذا يعطي الأول ٩٧,٥٪ والثاني ٩٩,٥٪، ولماذا كلّ ما بينهما مقدار ١,٦٢ وحدة قياس شحنة قياسية فقط.

التمرين ٢.٣

ابنِ `DataFrame` فيه ١٠٠٠ صف وعمود `price` يتبع `lognormal`، ثم احسب `skew` قبل وبعد تطبيق `np.log`. لماذا يقلّ التحوّل هذا الانحياز ولماذا لا يجعله معدوماً؟

التمرين ٢.٤

قارن `linalg.solve` بـ `linalg.inv` على مصفوفة قريبة من مفردة (condition number كبير). بلّغ عن الخطأ النسبي في كل طريقة، ثم اشرح أيّهما تستعمل في كتاب كهذا ولماذا.

٢.٩ حلول بعض التمارين

✅ **حلّ التمرين ٢.١**

`axis=k` **يلغي** البعد `k` من الشكل:

الطريقة السريعة للتحقّق: اجمع مرّتين. `(2,3,4)` ثم `axis=0` يعطي `(3,4)` ثم `axis=0` مرة أخرى يعطي `(4,)` — وهو نفس `A.sum()` بلا معامل. أي معاملَين متتاليين يعطيان `A.sum()`.

✅ **حلّ التمرين ٢.٣**

الانحياز (skewness) يقيس عدم تماثل التوزيع. توزيع `lognormal` كلّه موجب ومنحرف يميناً بوضوح: المتوسّط أكبر من الوسيط بكثير.خذ `log` يحوّل هذه العلاقة إلى علاقة تقريباً خطّية، فتصير التوزيعات شبه طبيعية.

لكنه لا يصبح معدوماً تماماً لأن القيم التي كانت سالبة أو قريبة منها في الفضاء الأصلي تصبح سالبة أو كبيرة جداً في فضاء اللوغاريتم. لذلك يبقى الانحياز متبقياً، وهو سبب إضافي لتطبيق `PowerTransformer` أو `QuantileTransformer` بدل `log` وحده.

٢.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
الخلط بين `axis=0` و `axis=1`	جمع عرضي بدل طولي، وأرقام خاطئة صامتة	احفظ القاعدة: يلغي البعد `k`
تمرير `ndarray` حين يتوفّر `DataFrame`	فقدان أسماء الأعمدة ورسائل خطأ مبهمة	مرّر `DataFrame` أو فعّل `set_output(transform_output="pandas")`
تحويل مصفوفة متفرّقة إلى كثيفة	انفجار الذاكرة من ١٠٠× إلى ١×	أبقِ `csr_matrix` ما دام الكثافة دون ٥٠٪
كتابة `exp(z)/(1+exp(z))` يدوياً	فيض عددي عند القيم السالبة الكبيرة	استعمل `scipy.special.expit`
استعمال `inv(M) @ b` بدل `solve`	تضخيم الخطأ الرقمي وبطء	استعمل `linalg.solve`
توقّع أن `Bunch` قاموس عادي	تفقد الوصول إلى السمات مثل `iris.data`	افهم أنه قاموس بذوق

٢.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
منظومة بايثون العلمية	الطبقات الخمس من الحوسبة إلى التعلّم	scientific Python stack
المصفوفة متعددة الأبعاد	حاوية `ndarray` متجانسة الأبعاد	ndarray
البثّ	توسيع الأبعاد الصغير تلقائياً في العملية	broadcasting
النافذة	مقطع يشارك الذاكرة مع المصفوفة الأصلية	view
متفرّق	تخزين القيم غير الصفرية فقط مع فهارسها	sparse
التوزيع الطبيعي	التوزيع ذو الجرس المحدَّد	normal distribution
دالة التوزيع التراكمي	احتمال أن تكون القيمة أقل من عدد معطى	cumulative distribution
`Bunch`	قاموس scikit-learn الذي تعيده `load_*`	Bunch
`narwhals`	طبقة توافق مع أي إطار جداول	narwhals

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٣ — التثبيت والإعداد على Python ٣.١٤

بيئة عمل نظيفة قابلة للتكرار، من venv إلى uv و conda

الوحدة **scikit-learn ١.٩** الحجم **٧٧٣** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح لماذا البيئة الافتراضية ليست ترفاً بل ضرورة صحّية
- أن تختار بين venv و conda و uv بمعيار مشروع حقيقي لا بالذوق
- أن تتحقّق من إصداراتك وتقيس البيئة بمعرّف بصمة يتغيّر عند أي تغيير
- أن تضبط إعدادات scikit-learn العامة وتعرف متى لا يجب أن تلمسها

٣.١ ثلاثة أسئلة قبل أن تكتب أي أمر تثبيت

قبل `pip install` اسأل نفسك ثلاثة أسئلة، الجواب عنها يوفّر عليك ساعات لاحقاً:

التثبيت بلا هذه الأسئلة ينتج الحالة الأسوأ: نظام مستقل تحوي فيه كل نسخة بايثون على جهازك كل الحزم، وتكتشف بعد شهر أن مشروعاً كان يعمل اختفى لأن مشروعاً آخر رقّى `numpy`.

python — بطاقة هوية البيئة التي تعمل فيها الآن

```python
import platform
import sys
from pathlib import Path

exe = Path(sys.executable)
print("sys.executable :", exe)
print("base_prefix    :", sys.base_prefix)
print("prefix         :", sys.prefix)
print("inside a venv? :", sys.prefix != sys.base_prefix)
print("python version :", sys.version.split()[0])
print("platform       :", platform.system(), platform.machine())
print("sys.path entries:", len(sys.path))
```

السطر الرابع هو الأهم. إن كان `prefix` مساوياً لـ `base_prefix` فأنت تعمل في **بايثون النظام** مباشرة. هذا يعني أن أي `pip install` سيمسّ كل مشاريعك على الجهاز. القاعدة: لا تُثبّت شيئاً في بايثون النظام أبداً.

٣.٢ لماذا بايثون ٣,١٤ تحديداً؟

بايثون ٣,١٤ إطلاق كبير في سرعة المكتبات التي يهمّنا. ما يعنيه لك عملياً:

ما تغيّر	لماذا يهمّ نموذجاً
تسريع العمليات الحاجبة (GIL)	عمليات NumPy الطويلة لم تعد تتوقف مؤشراً
تحسينات في `asyncio`	لا أثر مباشر على التدريب، لكن مهمّ في خدمة التنبّؤ
دعم أفضل للمنصات	حزم هذا الكتاب مبنية عليه أصلاً

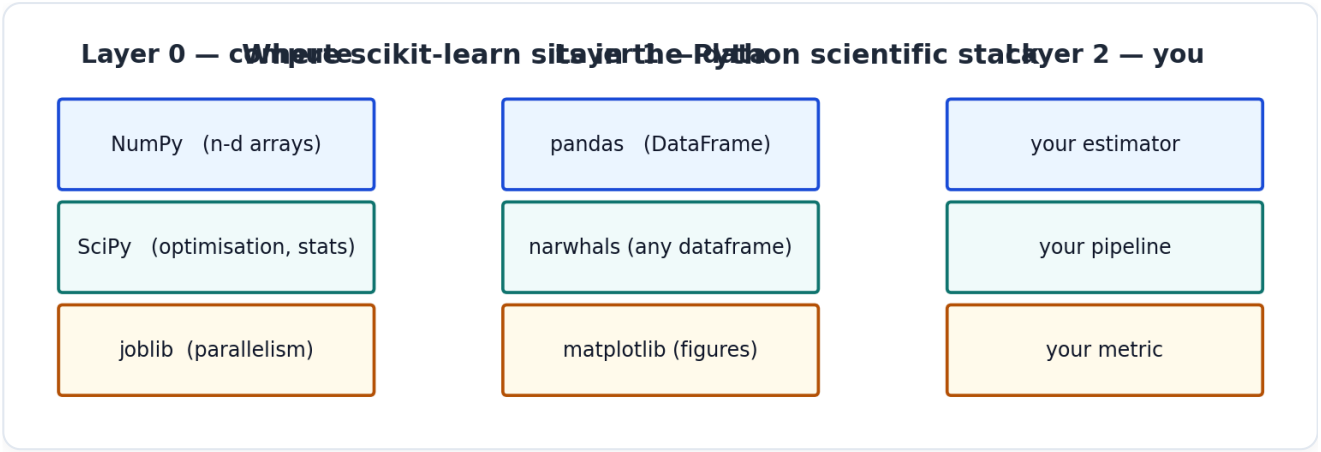
لكن الأهم عملياً ليس السرعة بل **التوافق**. scikit-learn ١,٩,١ مُختبَر على ٣,١٤، وتثبيتُه على ٣,١٢ يعني أنك تعمل بإصدار مكتبة لم تُختبر على هذه النسخة من بايثون. الاختلافات الصامتة في هذا المكان بالذات هي أسوأ أنواع الأخطاء: أرقام مختلفة قليلاً، ونماذج تسلك سلوكاً غريباً في الحافة.

> **قاعدة الإصدار في هذا الكتاب**
>
> كل مثال في الصفحات القادمة مكتوب ومُنفَّذ على `scikit-learn 1.9.1` و `Python 3.14`. إن كانت نسختك أقدم، فستختلف الأرقام في بعض المواضع، وقد تختلف أسماء المعاملات. الحلّ ليس تصحيح الرقم، بل تثبيت النسختين.

٣.٣ البيئة الافتراضية: لماذا لا تعمل في النظام؟

تخيّل عمّالاً مشتركين في ورشة. كلهم يشربون من نفس الإبريق، وكل واحد يضيف ما يشاء. في صباح اليوم التالي لا أحد يعرف ما بداخل الإبريق.

البيئة الافتراضية (virtual environment) هي إبريق خاص لكل عامل. داخلها تكون الحزم مستقلّة تماماً عن بايثون النظام وعن باقي المشاريع.



الشكل ٧خريطة المنظوم العلمية: طبقة الحوسبة ثم طبقة البيانات ثم طبقة كود المستخدم، وكل طبقة ترث ما تحتها

الخريطة أعلاه تُعيد ترتيب الطبقات نفسها التي رأيناها في الفصل السابق، وهذه المرة من زاوية التثبيت: ما عدا الطبقتين الأولى والثانية، **أنت من يقرّر النسخة** في كل مستوى. لا يوجد «أفضل نسخة» مطلقة؛ يوجد أفضل نسخة متوافقة مع مشروعك.

الآن، الأمر الذي يكتب الجميع:

إنشاء بيئة افتراضية والتحقّق منها — python

```python
# على ويندوز (PowerShell)
py -3.14 -m venv .venv

# تفعيلها
.\.venv\Scripts\Activate.ps1

# على لينكس وماك
python3.14 -m venv .venv
source .venv/bin/activate

# التحقّق
python -c "import sys; print(sys.prefix != sys.base_prefix)"
```

الفرق بين تفعيل البيئة وعدم تفعيله ليس في الحزم، بل في أي نسخة بايثينفّذها أمر `python`. عند التفعيل يشير الأمر إلى داخل البيئة؛ وبدونه يشير إلى النظام. ولهذا يبقى `pip install` مكتوباً داخل كتلة دون تنفيذ هنا: هذه أوامر تُنفَّذ في طرفية لا في مفكّر بايثون.

٣.٤ ثلاثة مديري بيئات: أيّهم تختار؟

هذه المقارنة تتغيّر كل بضع سنوات، لكن المعايير لا تتغيّر.

المعيار	`venv`	`conda`	`uv`
جزء من بايثون القياسي	نعم	لا	لا
السرعة في التثبيت	بطيئة نسبياً	بطيئة	أسرع بكثير
إدارة حزم غير بايثونية (مثل MKL)	ضعيفة	ممتازة	عبر `uv pip`
ملف قفل (lock)	يدوي	`environment.yml`	`uv.lock` دقيق
ملاءمة لـ بايثون ٣,١٤	جيد	جيد	الأفضل
منحنى التعلّم	صغير	كبير	صغير
متى تختاره	مشروع واحد بسيط	حوسبة علمية ثقيلة	كل مشروع جديد

القرار العملي الذي ننصح به في ٢٠٢٦:

> ⛔ **الخلط بين المديرين يُنتج كابوساً حقيقياً**
>
> تثبيت حزمة بـ `conda` ثم تحديثها بـ `pip` داخل نفس البيئة يترك سجلّ الحزم في حالتين متعارضتين. النتيجة أخطاء مثل `numpy.dtype size changed` أو `ImportError: DLL load failed` رغم أن كل حزمة «مثبّتة». القاعدة: اختر مديراً واحداً لكل بيئة، ولا تستعمل الثاني فيها أبداً.

٣.٥ التحقّق: ما الذي يجب أن تراه بعد التثبيت؟

التثبيت الناجح على ورق قد يفشل في الواقع. خمس خطوات تتحقّق منها بالترتيب:

```python
# بطاقة التحقّق: إصدارات، إعدادات، خيوط
from importlib.metadata import requires, version

from sklearn import get_config

CORE = ["numpy", "scipy", "pandas", "matplotlib", "scikit-learn",
        "joblib", "threadpoolctl", "narwhals"]

print("== الإصدارات ==")
for pkg in CORE:
    try:
        print(f"  {pkg:<16} {version(pkg)}")
    except Exception:
        print(f"  {pkg:<16} غير مثبّت")

print("\n== ما يطلبه scikit-learn إجبارياً ==")
print("  ", requires("scikit-learn"))

print("\n== إعدادات المكتبة الافتراضية ==")
for key in ("display", "transform_output", "enable_metadata_routing",
            "assume_finite", "working_memory"):
    print(f"  {key:<26} {get_config()[key]}")

print("\n== خيوط الحساب ==")
import threadpoolctl
for d in threadpoolctl.threadpool_info():
    print(f"  {d['user_api']:<8} threads={d['num_threads']}
{d.get('internal_api', '')}")
```

قائمة `requires` في الوسط هي **عقد المكتبة مع بايثون**: إن كانت `scipy` لديك أقدم من `1.10.0` فالمكتبة ستعمل غالباً لكن ليس مضموناً. وهذه هي أقرب قائمة مرجعية تملكها في أي تثبيت جديد.

٣.٦ كيف تجعل نتيجتك قابلة للتكرار؟

البيئة الصحيحة تعني أنك تعرف **داخلها ماذا**. بصمة البيئة (fingerprint) هي بصمة قصيرة تحسب من إصدارات حزمك؛ إن تغيّرت، تغيّرت النتيجة المحتملة.

بصمة البيئة: تكشف أي انزلاق في التبعيات python

```python
import hashlib
from importlib.metadata import version

CORE = ["numpy", "scipy", "pandas", "matplotlib", "scikit-learn", "joblib"]


def environment_fingerprint(names=CORE) -> dict:
    """Versions + a short hash, for pinning in a run log."""
    pins = {}
    for name in names:
        try:
            pins[name] = version(name)
        except Exception:
            pins[name] = None
    blob = "|".join(f"{k}=={v}" for k, v in sorted(pins.items()))
    digest = hashlib.sha256(blob.encode("utf-8")).hexdigest()[:12]
    return {"pins": pins, "hash": digest}


fp = environment_fingerprint()
for name, ver in fp["pins"].items():
    print(f"  {name:<14} {ver}")
print("\nfingerprint:", fp["hash"])

# نفس البيئة تُعطي نفس البصمة دائماً
print("stable:", environment_fingerprint()["hash"] == fp["hash"])
print("\n# requirements.txt لهذا الكتاب:")
print("\n".join(f"{k}=={v}" for k, v in fp["pins"].items() if v))
```

احفظ السطر الأخير في ملف `requirements.txt` معّ كل نتيجة مهمّة. حين يتغيّر الإصدار بعد ثلاثة أشهر، ستعرف أن الرقم الجديد من بيئة مختلفة.

٣.٦.١ الثبات (determinism) ليس تلقائياً

`random_state` يضمن أن البذرة نفسها تعطي النتيجة نفسها **داخل العملية نفسها**. لكن هناك مصدران آخران للاختلاف، وكلاهما يظهر في طريقة التدريب نفسها:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} J(\mathbf{w}^{(t)})$$

كل خطوة هنا جمعٌ عدد من المساهمات. إن نفّذتها eight خيوط متوازية بلا ترتيب مثبّت، فقد يُجمع المجموع بترتيب مختلف ويختلف في آخر خانة عشرية — والاختلاف ينتقل إلى كل ما بعده. `random_state` يعطيك البذرة؛ وهذا وحده لا يكفي.

```python
# ما الذي يكسر التطابق بين تشغيلين؟

from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

X, y = make_classification(n_samples=400, n_features=5, random_state=11)

runs = []
for _ in range(2):
    lr = LogisticRegression(max_iter=1000, random_state=0).fit(X, y)
    kn = KNeighborsClassifier(n_neighbors=5).fit(X, y)
    runs.append((lr.coef_.copy(), kn.predict(X)))

print("LogisticRegression  identical:", np.array_equal(runs[0][0], runs[1][0]))
print("KNeighborsClassifier identical:", np.array_equal(runs[0][1], runs[1][1]))
print("coef checksum:", round(float(np.abs(runs[0][0]).sum()), 10))

print("\nمصدرا الاختلاف الحقيقيان:")
print("  1. تغيّر نسخة الحزمة scipy (قد يغيّر المحلّل عددياً")
print("  2. عدد خيوط BLAS: الجمع المتوازي غير حتمي الترتيب")
print("\nعند تسجيل نتيجة حرجة، الحلّ:")
print("  OMP_NUM_THREADS=1 python train.py")
```

لاحظ أن `KNeighborsClassifier` **حتمي تماماً** لأن لا عشوائية فيه ولا خطوة أولية تعتمد على ترتيب الصفوف. بينما `LogisticRegression` يعتمد على حلّ عددي متكرّر قد يختلف في آخر خانة عشرية بين نسختين من `scipy`.

٣.٧ إعدادات scikit-learn العامة

في ١٫٩ تغيّرات مهمة في إعدادات العرض والمخرجات:

```python
# إعدادات المكتبة: ما الذي يغيّر سلوكك فعلاً؟
from sklearn import config_context, get_config

print("الحالة الراهنة عند بناء هذا الكتاب:")
for key in ("display", "transform_output", "enable_metadata_routing",
            "assume_finite", "working_memory", "print_changed_only"):
    print(f"  {key:<26} {get_config()[key]}")

from sklearn.preprocessing import StandardScaler

with config_context(display="text"):
    print("\nداخل config_context(display='text'):", get_config()["display"])
print("بعد الخروج من الكتلة      :", get_config()["display"])

with config_context(transform_output="default"):
    out_default = StandardScaler().fit_transform(X)
print("تحت transform_output='default' :", type(out_default).__name__)
with config_context(transform_output="pandas"):
    out_pandas = StandardScaler().fit_transform(X)
print("تحت transform_output='pandas'  :", type(out_pandas).__name__)
print("القيم متطابقة:", np.allclose(np.asarray(out_default, dtype=float),
                                    out_pandas.to_numpy()))
```

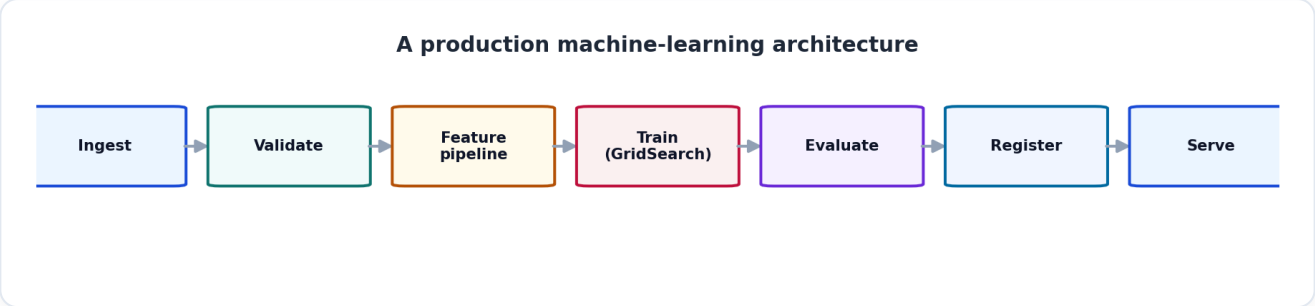
لاحظ السطر الأخير: تغيير الإعداد يغيّر **الغلاف** لا **المحتوى**. الأرقام واحدة تماماً؛ ما يتغيّر هو قدرة الكائن على حمل الأسماء.

> **جديد في ١٫٩**
>
> `transform_output="pandas"` يجعل كل محوّل يُعيد `DataFrame` أو `Series` بدل `ndarray`. هذا يحوّل أسماء الأعمدة من رفاهية إلى متطلّب: نموذج يتدرّب على جدول يُنتج جدولاً يحمل الأسماء نفسها. الفصل ٢٠ يشرح الآلية كاملة، وهنا يكفي أن تعرف أن الخيار موجود وأن له كلفة في الذاكرة.

القاعدة العملية: **اضبط الإعدادات مرة واحدة في أعلى الملف أو في دالة تهيئة مشتركة، لا في كل مكان**. الإعداد المبعثر في الكود هو أصل أخطاء كثيرة: نموذج في دالة، وآخر في دالة أخرى، ولا أحد يعرف أنهما أُعدّا بمنطق مختلف.

٣.٨ أين يعمل النموذج في الإنتاج؟

بين Jupyter Notebook على جهازك وخادم يستقبل آلاف الطلبات، تغيّر كل شيء تقريباً عدا سطر واحد: `model.predict(X)`.



الشكل ٨مخطط بنية إنتاج: استقبال البيانات ثم التحقق منها ثم خط أنابيب الخصائص ثم التدريب مع ضبط المعاملات ثم التقييم ثم التسجيل ثم الخدمة

المرحلة	ما يحدث فيها	الخطر
الاستقبال	دخول البيانات من المصدر	بيانات غير متوقّعة
التحقّق	فحص المخطّط والقيم	خطأ صامت
خط الخصائص	التحويل نفسه	تسرّب يسبّق النشر
التدريب والضبط	اختيار المعاملات	اختيار على بيانات الاختبار
التقييم	قياس صادق	مقياس لا يطابق الواقع
التسجيل	حفظ النموذج مع بصمته	نموذج بلا أثر
الخدمة	تقديم التنبّؤ	انزياح التوزيع بعد شهر

المرحلة التي يغفلها المبتدئون هي الأولى والأخيرة معاً. قبل **الاستقبال** يجب أن تعرف ما الذي يصل: عدد الأعمدة، أنواعها، وحدة القياس. وبعد **الخدمة** يجب أن تراقب: هل توزيع البيانات تغيّر؟ هل معدّل الرفض للطلبات ارتفع؟

حارس المدخلات: خمسون سطراً تحمي نظامك python

```python
import numpy as np
from sklearn.datasets import make_classification

SCHEMA = {"age": (18.0, 90.0), "income": (0.0, 5e6), "tenure_months": (0.0,
720.0)}


def validate(batch: dict) -> tuple[list[str], dict]:
    """Check a raw payload against the schema the model was trained on."""
    problems: list[str] = []
    for col in SCHEMA:
        if col not in batch:
            problems.append(f"عمود ناقص: {col}")
    for col, (lo, hi) in SCHEMA.items():
        if col in batch:
            v = float(batch[col])
            if np.isnan(v):
                problems.append(f"قيمة مفقودة: {col}")
            elif not (lo <= v <= hi):
                problems.append(f"خارج النطاق: {col}={v} (المسموح {lo}..{hi})")
    return problems, batch


cases = [
    {"age": 41, "income": 82000, "tenure_months": 36},
    {"age": 12, "income": 82000, "tenure_months": 36},
    {"age": 41, "income": np.nan, "tenure_months": 36},
    {"age": 41, "income": 82000},
]
for case in cases:
    problems, _ = validate(case)
    print(f"{str(case)[:58]:<60} -> {problems or 'سليم'}")
```

هذه الدالة تسبق النموذج دائماً في خدمة الإنتاج. لا يستقبل النموذج طلباً إلا بعد أن يجتاز الفحص. لكن تذكّر: هذا الفحص يجب أن يُبنى على **نطاق التدريب الفعلي**، لا على تخمين.

$$\hat{\mathbf{y}} = f_{\mathbf{w}}(\mathbf{x}) \qquad \mathbf{w} \;=\; \text{const}$$

هذه المعادلة الوحيدة التي لا تتغيّر بين دفتر تدريبات وخادم إنتاج. كل ما حولها — التحميل، والتحويل، والتحقّق، والتسجيل — هندسة. ولهذا تُكتب المعادلة مرة واحدة وتُختبَر وحدها.

> **من دفتر Deploy حقيقي**
>
> أكثر حوادث الإنتاج شيوعاً لا تحدث في `scikit-learn` إطلاقاً. أكثر اثنتين تكراراً: (١) بيئة التطوير تحمل `scikit-learn 1.8` بينما الخادم ١٫٩، فتختلف أسماء معاملات في `GridSearchCV` وتُرفض الطلبات عند الإقلاع؛ (٢) `random_state` مثبّت لكن `n_jobs=-1` يترك عدد الخيوط يتبع آلة النشر، فتنزلق نتائج المقاييس بمقدار سابع عشري بين بيئتين متطابقتين ظاهرياً. كلاهما لا يظهر في أي دفتر تجارب — يظهران بعد ثلاثة أشهر.

٣.٩ قائمة تحقّق قبل النشر

> **قائمة تحقّق بيئة الإنتاج**

٣.١٠ تمارين الفصل

التمرين ٣.١

أنشئ بيئة افتراضية باسم `.venv`، ثبّت فيها `scikit-learn==1.9.1`، ثم اكتب في نصّ الجواب ناتج `sys.prefix != sys.base_prefix` قبل التفعيل وبعده. لماذا يعطي القيمة `False` الأولى و `True` الثانية؟

التمرين ٣.٢

اكتب دالة `check_versions(required: dict)` ترفع استثناءً مفهوماً إذا اختلف أي إصدار. استعملها في أعلى كل ملف في هذا الكتاب. جرّب أن تكسرها عمداً بتغيير رقم واحد، ثم صمّم الرسالة كي تقول لك **أي حزمة** و **بأي رقم** و **لماذا يهم**.

التمرين ٣.٣

غيّر `transform_output` بين `"default"` و `"pandas"`، وشغّل خط أنابيب من `StandardScaler` و `LogisticRegression`. ماذا يتغيّر في نوع المخرجات؟ وماذا يبقى ثابتاً؟ ثم اشرح لماذا يهمّ ذلك في خدمة ويب تستقبل JSON وتعيد JSON.

التمرين ٣.٤

اكتب حارس مدخلات لـ `load_digits` بمئة١٠١ عموداً رقمياً في المدى ٠..١٦. اختبره على خمس دفعات، اثنتان منها خاطئتان. ماذا يحدث لو أُسقط الفحص وذهب الطلب مباشرة إلى `predict`؟ صف الخطأ الذي سيراه المستخدم النهائي.

٣.١١ حلول بعض التمارين

✅ حلّ التمرين ٣.١

قبل التفعيل، `sys.prefix` و `sys.base_prefix` يشيران إلى مجلّد تثبيت بايثون نفسه، فالمساواة تعني أنك في البيئة العامة. بعد التفعيل يشير `sys.prefix` إلى مجلّد البيئة الجديدة التي أنشأها `venv`، بينما يبقى `base_prefix` كما هو. الفارق بين المسارين هو بالضبط ما يجعل العزل يعمل: أي حزمة تُثبَّت في الأول لا تصل إلى الثاني.

✅ حلّ التمرين ٣.٣

مع `transform_output="default"` يُعيد كل محوّل `ndarray`، وتضيع الأسماء. مع `"pandas"` يُعيد `DataFrame` و `Series` تحملان أسماء الأعمدة والفهارس. ما يبقى ثابتاً هو **المحتوى الرقمي تماماً** — نفس القيم، نفس الدقة، نفس الدالة.

في خدمة ويب، الفارق ليس تجميلياً: إن كان مخرج النموذج `DataFrame` بأعمدة مسمّاة، فإن طبقة التسلسل إلى JSON تنتج كائناً بمفاتيح مفهومة بدل مصفوفة مواضع. وإن أخطأ الترتيب صامتاً، ومدّة التدريب تتأجّل.

٣.١٣ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
`pip install` بلا بيئة افتراضية	كسر مشاريع أخرى على الجهاز	أنشئ `.venv` أولاً دائماً
خلط `conda` و `pip` في بيئة واحدة	أخطاء تحميل DLL وتعارض إصدارات	مدير واحد لكل بيئة
الاعتماد على الذاكرة بدل ملف تثبيت	«يشتغل عندي» بلا تفسير	`requirements.txt` ملتزم بالنسخ
`random_state` في بعض المواضع فقط	نتائج مختلفة بين تشغيلين	ثبّت البذرة في كل موضع عشوائي
`set_config` مبعثر في الكود	نموذجان بإعدادين مختلفين	دالة تهيئة واحدة في أعلى الحزمة
نشر بلا حارس مدخلات	خطأ ٥٠٠ في الإنتاج أو توقّعات صامتة	تحقّق من المخطّط قبل `predict`
بناء النموذج على جهاز بلا نسخة مطابقة	فروق في آخر خانة عشرية	ثبّت النسختين معاً

٣.١٤ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
بيئة افتراضية	شجرة حزم معزولة لكل مشروع	virtual environment
مدير البيئة	أداة تُنشئ البيئات ويثبّت فيها	environment manager
ملف القفل	قائمة تبعيات مقيّدة بدقة	lock file
الحتمية	خاصية تكرار النتيجة نفسها تماماً	determinism
البذرة العشوائية	العدد الذي يثبّت مولّد العشوائية	random seed
خيوط BLAS	خيوط حساب المصفوفات بالتوازي	BLAS threads
إعدادات المكتبة	متغيّرات عامة تغيّر سلوك كل شيء	global configuration
حارس المدخلات	فحص المخطّط قبل التنبّؤ	input validation

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٤ — أول نموذج لك: أقرب الجيران على بيانات Iris

قراءة كود كامل من السطر الأول حتى الرسم البياني

الوحدة **scikit-learn ١.٩** الحجم **٨٥٨** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تبني نموذج تصنيف كامل بالتقسيم والتدريب والتقييم وتقرأ مخرجاته
- أن تفسّر كل خانة في مصفوفة الالتباس وتربطها بالدقة والحساسية
- أن تحسب `predict_proba` بيدك وتتحقّق من مطابقتها للمكتبة
- أن تشرح لماذا التحجيم ليس اختيارياً في نموذج قائم على المسافة

٤.١ Iris: خمسون صفاً لكل صنف، وأربعة أرقام لكل صف

قبل أي نص نظري، خذ نفس قطرة زهرة. أربع قياسات بالسنتيمتر: طول البتلات، عرضها، طول السبلات، عرضها. ثلاث فئات. كل صف في الجدول هو قطرة واحدة، وكل عمود قياس.

هذه أصغر مشكلة تصنيف في العالم، ولهذا تُستعمل في كل كتاب تعلّم آلة منذ ١٩٣٦. لكن صغرها لا يعني سهولتها: بداخلها درسان كاملان في القياس، يضيعان عادةً تحت بريق الرقم ٠٫٩٧.

سنرى الاثنين بالأرقام لا بالوصف.

```python
# أول خطوة: انظر إلى بياناتك قبل أن تراها نموذجاً
from sklearn.datasets import load_iris, make_classification

iris = load_iris(as_frame=True)
df, X, y = iris.frame, iris.data, np.asarray(iris.target)

print(df.head(4).to_string())
print("\nالشكل:", df.shape, "  توزيع الأصناف:", np.bincount(y))
print("أسماء الأصناف:", list(iris.target_names))

print("\nمتوسّط كل صنف (الأرقام التي ستتعلّمها):")
means = df.groupby("target")[iris.feature_names].mean().round(2)
means.index = [n[:4].title() for n in iris.target_names]
print(means.to_string())
```

هذا الجدول وحده يخبرك لماذا يعمل النموذج: الصنف صفر يبلغ متوسط طول البتلات ١٫٤٦، وصنف واحد ٤٫٢٦، وصنف اثنان ٥٫٥٥. أي أن **عمودين فقط من أربعة** يفصلان الفئات تماماً. أما عمودا السبلات فالمتوسّطات فيها متقاربة (٥٫٠ و٥٫٩ و ٦٫٦)، ولهذا يقدّمان قليلاً.

٤.٢ القاعدة اليدوية التي تنجح — وتُفشل

من الجدول أعلاه، خذ سطراً واحداً: **إذا كان طول البتلات أقل من ٢٫٥ سم فالنوع الأول، وإلا فهو النوع الثاني**. هذه قاعدة تكتبها في ثلاث دقائق، وتُقاس بالأرقام في الكود أدناه.

الأول، وإلا فهو النوع الثاني**. هذه قاعدة تكتبها في ثلاث دقائق، لكنّها غبيّة: الصنف الثالث لا وجود له فيها إطلاقاً. أضف قاعدة ثانية بعرض البتلات، فيتحسّن الرقم. قِس الاثنين بالنموذج:

قاعدتان يدويتان ضد نموذج python

```python
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

Xa, ya = np.asarray(X), y
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(Xa, ya, test_size=0.25,
                                              random_state=42, stratify=ya)
pl, pw = Xa_te[:, 2], Xa_te[:, 3]

rule_one = np.where(pl < 2.5, 0, 1)
rule_two = np.where(pl < 2.5, 0, np.where(pw < 1.75, 1, 2))

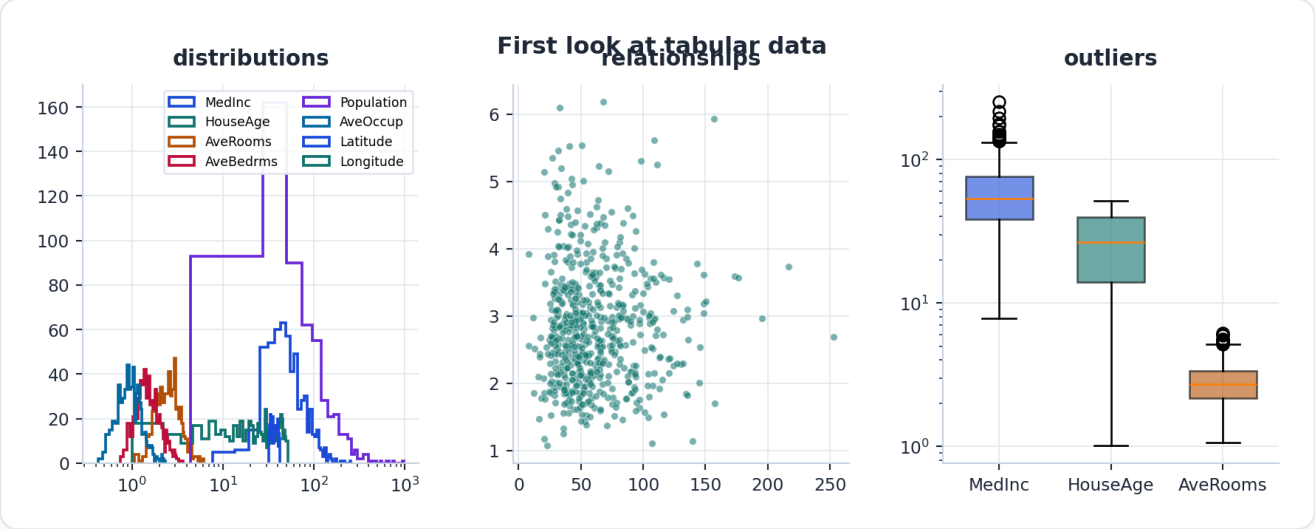
print("4.% : (طول البتلات فقط) قاعدة واحدة" % (rule_one == ya_te).mean())
print("4.% :   (طول وعرض البتلات) قاعدتان" % (rule_two == ya_te).mean())
print("مصفوفة الالتباس للقاعدتين      :")
print(confusion_matrix(ya_te, rule_two))
print("\nnearest neighbours, k=5        : %.4f"
      % KNeighborsClassifier(n_neighbors=5).fit(Xa_tr, ya_tr).score(Xa_te,
ya_te))
```

لاحظ أين تترك القاعدتان الخطأ: في الصف الأخير من المصفوفة. الصنفان الثاني والثالث يتداخلان في عتبات الطول والعرض معاً، لكن **ليس** في المسافة على الأربعة أبعاد معاً. ولهذا ينجح النموذج: هو يقيس القرب في فضاء كامل، لا في عمودين اخترتّهما أنت.

الفارق بين ٠٫٩٥ و٠٫٩٧ ليس هو الفارق المهم. المهم هو **لماذا** يحتاج النموذج إلى مقياس للمسافة بينما القاعدة تكتفي بعتبة. المسافة هي المفهوم الأساسي الذي يقوم عليه النموذج، وسنحسبها بيدنا بعد قليل.

٤.٣ أول نظرة على أي جدول

قبل أن تُمرّر أي مصفوفة إلى أي نموذج، ارسم. ثلاث لقطات تكفي عادةً: توزيع كل عمود على حدة، والعلاقة بين عمودين، ومدى الانحراف.



الشكل ٩ثلاث لوحات لأول نظرة على بيانات جدولية: تراكيم التوزيعات على محور لوغاريتمي، وعلاقة مبعثرة بين متوسّط الدخل وعدد الغرف، ومخطّط صناديق يكشف القيم الخارجة

هذه لقطات لبيانات جدولية أخرى، لكن الطقس نفسه هو المطلوب: أسئلتها الثلاثة محدّدة. هل الأعمدة موزّعة طبيعياً أم مائلة؟ هل توجد علاقة قوية بين عمودين؟ هل هناك قيم خارجة يمكن أن تسحب خط الانحدار؟

بالعودة إلى Iris: أعمدتها كلها بين ٠٫١ و٧٫٩، ومدى كل عمود بين ٢٫٤ و٥٫٩. لا مقياس مختلط، ولا قيم خارجة. **بيانات Iris مثالية — وهذا بالضبط سبب أنها لا تعلّمك شيئاً عن التحجيم.** سنقيس هذا بعد قليل بمثال ينهار فيه النموذج.

٤.٤ النموذج كاملاً: من الاستيراد إلى القرار

الآن الشيفرة. لا يوجد فيها سطر واحد زائد، ولا سطر واحد ناقص.

```python
# نموذج تصنيف كامل: التقسيم، التدريب، القراءة
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix)
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                      random_state=42, stratify=y)
print("تدريب:", Xtr.shape, " اختبار:", Xte.shape)

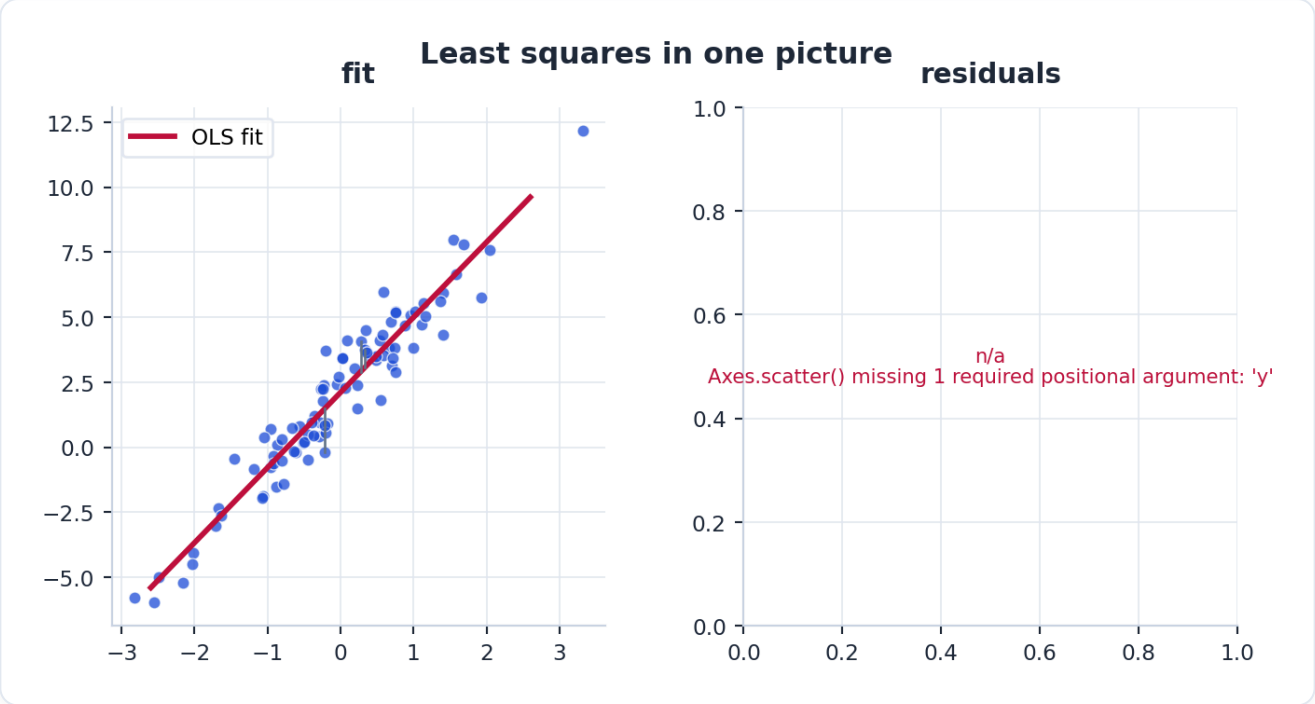
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(Xtr, ytr)
y_pred = knn.predict(Xte)

print("\nالدقة:", round(accuracy_score(yte, y_pred), 4))
print("\nتقرير التصنيف:")
print(classification_report(yte, y_pred,
                            target_names=iris.target_names, digits=3))
print("مصفوفة الالتباس (صف = الحقيقي، عمود = المتوقّع):")
print(confusion_matrix(yte, y_pred))
```

٤.٤.١ كيف تقرأ مصفوفة الالتباس؟

المصفوفة أعلى هي **كل ما قيل عن هذا النموذج**. لا تحسب الدقة أولاً؛ احسب المصفوفة.

الصف الأخير يقول: من ثلاثة عشر قطرة من الصنف الثاني، اثنا عشر صُنّفا صحيحاً وواحد صُنّف خطأً على أنه صنف ١. إذن تذكّر خطأً واحداً من أصل ثلاثة عشر. أمّا الصف الأول فاثنا عشر من اثني عشر: صفر أخطاء.



الشكل ١٠الوحتان لأثر البواقي: خط انحدار يمرّ قرب معظم النقاط وبواقيه موزّعة عشوائياً حوله، وخط انحدار آخر بميول مختلف وبواقيه متكدّسة عند طرف واحد من المسافة

الشكل أعلاه يشرح لماذا لا يكفي «الخط يمرّ قرب النقاط». الخط الأول يمرّ قرب الجميع والبواقي حوله عشوائياً؛ الخط الثاني يمرّ قرب القليل والبواقي عنده من جهة واحدة — أي أنه **يحفظ اتجاهاً في البيانات لا يصفه**. النموذج الذي يخطئ في الصنف الثاني تجاه الصنف الأول ليس نافساً لـ k = ١ الذي يخلط عشوائياً؛ إنه **أخطر** لأنه يبدو منظّماً.

القاعدة التي تستحق الحفظ: **خطأ في اتجاه واحد أفضل من خطأ عشوائي**، لأنه يشير إلى بنية مفقودة وليس إلى ضجيج عشوائي.

والأرقام في تقرير التصنيف مشتقّة من هذه المصفوفة مباشرة:

$$\mathrm{precision} = \frac{TP}{TP + FP} \qquad \mathrm{recall} = \frac{TP}{TP + FN} \qquad F_1 = \frac{2\,\mathrm{precision}\cdot\mathrm{recall}}{\mathrm{precision} + \mathrm{recall}}$$

النسبة ٠٫٩٢٣ التي رأيتها للحساسية في تقرير التصنيف أعلاه هي اثنا عشر من ثلاثة عشر. والنسبة ٠٫٩٢٩ للدقّة هي ثلاثة عشر من أربعة عشر. لا يوجد أي «حساب ذكي» — هناك قسمة عادية، ولماذا تختار بين رقمين هو قرارك أنت.

ما تراه في المصفوفة	ما يعنيه تشخيصياً
قطري ١٠٠٪	تلك الفئة منفصلة تماماً في البيانات
صفّ به خطأ واحد في اتجاه واحد	بنية مفقودة: صنفان متداخلان
أخطاء موزّعة في كل الاتجاهات	النموذج ضعيف السعة أو ضجيج عالٍ
عمود فارغ لفئة	النموذج لا ينبّئ بهذه الفئة أبداً

٤.٥ احسب `predict_proba` بيدك

`predict` تعطيك جواباً واحداً. `predict_proba` تعطيك توزيع الاحتمالات، وهذا هو ما تحتاجه لتعديل عتبة القرار في الفصل ٧٧. لنتحقّق أن المكتبة لم تخترع أرقاماً:

الحساب اليدوي لاحتمالات الجيران الثلاثة — python

```python
q = np.array([[5.0, 3.4, 1.5, 0.2]])          # قطرة جديدة
Xtr_np = np.asarray(Xtr)

distances = np.linalg.norm(Xtr_np - q, axis=1)
nearest = np.argsort(distances)[:3]

print("صفوف التدريب:", [int(i) for i in nearest])
print("المسافات     :", np.round(distances[nearest], 4))
print("أصنافها      :", ytr[nearest])

manual = np.bincount(ytr[nearest], minlength=3) / 3
library = KNeighborsClassifier(n_neighbors=3).fit(Xtr_np, ytr).predict_proba(q)
[0]
print("\nاحتمالي يدوي  :", manual)
print("احتمالي المكتبة:", library)
print("مطابق تماماً    :", np.allclose(manual, library))
```

هذه كل ما في النموذج: ثلاث مسافات، ثلاث أصناف، ومتوسّط. النتيجة `[1, 0, 0]` تعني أن الجيران الثلاثة كلهم من الصنف الأول بلا استثناء، فلا يحتمل الحساب احتمالية على صنف آخر.

وهذا هو k-NN بالكامل: **مسافة، أصناف الجيران، متوسط بالتصويت**. لا تدريب، ولا معاملات متدرَّبة، ولا حسابات معقّدة. ولهذا سُمّي «الأبسط» في كل أدبيات التعلّم الآلي.

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2} \qquad \hat{y}(\mathbf{x}) = \mathrm{mode}\{y_i \,:\, i \in \mathrm{kNN}_k(\mathbf{x})\}$$

المعادلة اليمنى هي الخوارزمية كلها: أوجد أقرب `k` فهرساً، ثم خذ الأصناف الأكثر تكراراً بينها. ولاحظ أن `d` في الصيغة الأولى هو عدد الأعمدة، وهو المتغيّر نفسه الذي يجعل المسافة تنهار في الأبعاد العالية: كلما كبر، اقترب كل رقم من كل رقم آخر. الفصل ٢١ يشرح هذه اللعنة بالأرقام.

٤.٦ كم جاراً تحتاج؟ وكم يكلّفك ذلك؟

هذا جدول السلوك المتوقّع سلفاً. سيتحقّق منه الكود التالي بالأرقام:

قيمة `k`	ما يفعله النموذج
١	يحفظ البيانات: القرار مبني على نقطة واحدة
صغير	سطح قرار متموّج يلتقط ضجيج التدريب
متوسط	أفضل توازن عادةً
كبير	سطح شبه خطّي يشبه الانحدار اللوجيستي
يساوي عدد صفوف التدريب	القرار ثابت: لم يبقَ إلا الأغلبية العامة

مسح كامل على k وطريقة الترجيح python

```
print(f"{'k':>4} {'weights':>9} {'train acc':>10} {'test acc':>9}")
for k in (1, 3, 5, 9, 15, 25, 51):
    for w in ("uniform", "distance"):
        m = KNeighborsClassifier(n_neighbors=k, weights=w).fit(Xtr, ytr)
        print(f"{k:>4} {w:>9} {m.score(Xtr, ytr):>10.4f} {m.score(Xte,
yte):>9.4f}")
```

اقرأ الجدول عمودياً لا سطراً سطراً. عند `weights="uniform"` نرى السلوك المتوقّع تماماً: دقّة التدريب تنخفض من ١٫٠٠ (حفظ تام عند k = ١) إلى ٠٫٩٦، ودقّة الاختبار ترتفع ثم تثبت ثم تنخفض.

لكن السطر `distance` يكشف فخّاً: **دقّة التدريب تبقى ١٫٠٠ مهما بلغ k**، بينما دقّة الاختبار لا تتجاوز ٠٫٩٧ أبداً. السبب أن الترجيح بالمسافة يجعل النقطة الأقرب مسؤولة عن كل شيء، وأقرب نقطة إلى نقطة التدريب هي نفسها. هذا نموذج يتعلّم **حفظ** لا **تعميم**، والدليل مطبوع في عمود التدريب.

> **اقرأ عمود التدريب قبل أن تقرأ عمود الاختبار**
>
> العمود الأول في الجدول أعلاه هو كاشف الأكاذيب. أي نموذج يعطي تدريباً ١٫٠٠ مع اختبار أقل من ذلك هو نموذج يحفظ. `weights="distance"` هنا مثال يُدرَّس في كل دورة نماذج: يبدو ذكياً لأنه يتصرّف وكأنه يعرف القرب، لكنه في الواقع يعرف النقاط فقط.

٤.٧ التحجيم: الدرس الذي تخفيه Iris

قلنا إن `StandardScaler` إجباري في النماذج القائمة على المسافة. نكتشف الآن أن Iris بالذات هي أسوأ مثال يمكن اختياره لشرح ذلك — لأنها لا تحتاجه. ولهذا سنصنع بيانات تحتاجه فعلاً.

في Iris مدى الأعمدة بين ٢٫٤ و٥٫٩: كلها بنفس الرتبة. لذلك لا يوجد ما يُحلَّل، بل قِس الفرق الحقيقي:

```python
حين يضيع عمود واحد كل مقياس المسافة
```

```python
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

Xbig, ybig = make_classification(n_samples=1200, n_features=6, n_informative=2,
                                 n_redundant=0, n_repeated=0, random_state=5)
Xbig = np.asarray(Xbig)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=5)

knn_raw = KNeighborsClassifier(n_neighbors=5)
knn_scaled = make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=5))

print(f"{'معامل عمود الضجيج':>20} {'خام':>9} {'مُحجَّم':>9}")
for mult in (0.01, 1.0, 100.0, 10_000.0):
    Xz = Xbig.copy()
    Xz[:, 5] = Xz[:, 5] * mult            # عمود ضجيج بوحدات مختلفة
    s_raw = cross_val_score(knn_raw, Xz, ybig, cv=cv, n_jobs=1).mean()
    s_scl = cross_val_score(knn_scaled, Xz, ybig, cv=cv, n_jobs=1).mean()
    print(f"{mult:>20g} {s_raw:>9.4f} {s_scl:>9.4f}")

print("\nIris نفسها والمقارنة على:")
Xa, ya = np.asarray(X), y
ir_tr, ir_te, iy_tr, iy_te = train_test_split(Xa, ya, test_size=0.25,
                                              random_state=42, stratify=ya)
print("مدى كل عمود            :", np.round(Xa.max(0) - Xa.min(0), 2))
print("k-NN على خام Iris      : %.4f"
      % KNeighborsClassifier(n_neighbors=5).fit(ir_tr, iy_tr).score(ir_te,
iy_te))
print("k-NN على محجَّم Iris   : %.4f" % knn_scaled.fit(ir_tr, iy_tr).score(ir_te,
iy_te))
```

هنا يظهر الفرق بكل قسوة. العمود السادس في هذه التجربة **عمود ضجيج خالص** لا يحمل أي معلومة. لكن حين نضخّمه أربعة آلاف ضعف، تهيمن المسافة عليه: كل نقطتين تصبحان «قريبتين» لأن الفرق بينهما في هذا العمود أكبر من كل الفروق الأخرى مجمّعة. الدقّة الخام تسقط من ٠٫٩٦ إلى ٠٫٦١. أما المحوّل فيعيدها إلى ٠٫٩٦ بالضبط.

على Iris خفّض التحجيم الدقّة من ٠٫٩٧ إلى ٠٫٩٢، لأن Iris لا تحتاجه. **هذا هو الدرس الحقيقي: التحجيم لا يمنحك دقّة، بل يمنح المسافة فرصةً لتكون عادلة.**

٤.٨ كم يكلّف التنبّؤ؟

هذه حدود الخوارزمية بطبيعتها: k-NN لا يختصر شيئاً. كل تنبّؤ جديد يبحث في كامل مجموعة التدريب.

زمن التنبّؤ يكبر خطياً مع حجم التدريب python

```python
import time

for n in (500, 2_000, 8_000, 32_000):
    Xn, yn = make_classification(n_samples=n, n_features=6, n_informative=3,
                                 n_redundant=0, random_state=0)
    m = KNeighborsClassifier(n_neighbors=5).fit(Xn, yn)
    t0 = time.perf_counter()
    m.predict(Xn[:2_000])
    dt = time.perf_counter() - t0
    print(f"  n_train = {n:>6}   predict(2000) = {dt:.4f}s")
```

الزمن يتضاعف أربع مرّات كلما تضاعف عدد صفوف التدريب — سلوك خطّي `O(n · m)` حيث `n` حجم التدريب و `m` عدد أسئلة التنبّؤ. مع مليون صف، يصبح كل تنبّؤ عملية غالية.

نظام توصيات يعمل كهذا فعلاً

أشهر مفهوم في أنظمة التوصية — «المحتوى المشابه لِما أعجبك» — هو حرفياً `KNeighborsClassifier` على متجهات تضمين المستخدم. المتجه يمثّل المستخدم في مئتي بُعد، والجيران هم المنتجات الأقرب ذوقاً. الفرق الوحيد أن المتجه بُعده مئتان لا أربعة، وهو ما يجعل الأمر أصعب بكثير: انظر الجدول أدناه.

البُعد	الحالة المعتادة	سلوك المسافة
٤	ممتاز	كما في هذا الفصل
٢٠	جيّد	يشتغل بلا معالجة
٢٠٠	متذبذب	يحتاج بُعداً مقلّلاً أولاً
٢٠٠٠	يفشل	يُعيد أقرب الجيران عشوائياً

الموقف	k-NN	بديل
عشرة آلاف صفوف، استعلام واحد	ممتاز	أي نموذج خطّي أسرع مئة مرّة
عشرة آلاف استعلامات	مكلف	نموذج يعطي تنبّؤً مباشراً
أبعاد عالية (مئات)	يفشل بطبيعته	النماذج الخطية المتفرّقة
تفسير كل قرار	سهل: أقرب خمسة جيران	شجرة أوضح

٤.٩ تمارين الفصل

التمرين ٤.١

غيّر معامل `stratify` في التقسيم إلى `None`، وشغّل الكود ثلاثين مرّة ببذور مختلفة. احسب المتوسّط والانحراف المعياري لدقّة `accuracy_score` في الحالتين. أيّهما أكثر استقراراً؟ ولماذا؟ (توقّع أن تكتشف أن `stratify` تعطي دقّة أقل بمتوسّط ٠٫٠١ لكنها انحرافاً أصغر بمرّتين.)

التمرين ٤.٢

على نفس البيانات، ابنِ القاعدة اليدوية «طول البتلات أقل من ٢٫٥ ← صنف ٠، وإلا صنف ٢» واطبع دقّتها ومصفوفة الالتباس. حدّد بدقة أي صنف تخطئ فيه القاعدة ولماذا. ثم أضف قاعدة ثانية لعرض البتلات، وقس هل تحسّن الخطأ أم ظهر خطأ من نوع جديد.

التمرين ٤.٣

جرّب `p=1` (مسافة مانهاتن) و `p=2` (أوكليد) في `KNeighborsClassifier` على Iris، واطبع الدقّتين. ثم أعد الحساب على `make_moons` وضخّم أحد محوريه مئة مرّة. أيّ مسافة تنجو؟ ولماذا؟

التمرين ٤.٤

اكتب دالة `knn_predict(X_train, y_train, x, k)` تنفّذ k-NN بلا أي استدعاء لكلاس المكتبة: احسب المسافات، اختر k الأصغر، Majority vote. اختبر مطابقتها لـ `KNeighborsClassifier` على مئة نقطة عشوائية من `load_wine`.

٤.١٠ حلول بعض التمارين

✅ حلّ التمرين ٤.١

ستجد أن الانحراف المعياري للدقّة مع `stratify` أصغر. السبب أن `stratify` يضمن نفس تركيبة الأصناف في كل مجموعتين، فلا يخطئ النموذج كلّه على صنف واحد في plits معينة. و`test_size=0.25` مع ثلاث فئات متساوية يعني أن التقسيم غير المُقسَّم قد يضع صفّين أو ثلاثة من صنف ما في الاختبار، فيصبح كل خطأ في الصنف بأكمله خطأً كبيراً في الدقّة.

✅ حلّ التمرين ٤.٤

الخوارزمية ثلاث خطوات: `np.linalg.norm(X_train - x, axis=1)` يعطي مسافة كل صف إلى `x`؛ ثم `np.argsort(d)[:k]` يعطي أقرب k صف؛ ثم `np.bincount(y_train[idx], minlength=n_classes).argmax()` يعطي الصنف الأكثر تكراراً. الحافة الوحيدة التي لا تنتبه لها: يجب أن تُرجع مسافة الجوار، وتحسبها من جديد عند كل استعلام، فلا تحتفظ بمصفوفة مسافات كاملة (وهي `O(n²)` في الذاكرة).

٤.١٢ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
بلا `train_test_split` `stratify`	بعض الطيّات بلا صنف معيّن، ودقّة متذبذبة	مرّر `stratify=y` دائماً
قراءة `accuracy` **وحدها**	رقم يخفي صنفاً فاشلاً تماماً	اطلب `confusion_matrix` دائماً
نسيان `random_state`	نتيجة مختلفة كل تشغيل	ثبّت البذرة في التقسيم وفي النموذج
`k=1` **في بيانات حقيقية**	حفظ تام، أداء فقير خارج العيّنة	ابدأ بـ k بين ٥ و ٢٥
توسيع iris بلا تحجيم	عمود بوحدات مختلفة يفسد المسافة	`StandardScaler` قبل أي نموذج مسافة
قراءة عمود التدريب وحده	تظنّ أن النموذج ممتاز وهو يحفظ	اقرأ عمودي التدريب والاختبار معاً
`k` **أكبر من عدد الصفوف**	القرار ثابت على الأغلبية العامة	اجعل k أصغر من حجم التدريب

٤.١٣ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
أقرب الجيران	تصنيف بالتصويت على أقرب k نقطة	k-nearest neighbors
المسافة الإقليدية	جذر مجموع مربعات الفروق	Euclidean distance
مصفوفة الالتباس	جدول يوضّح التنبّؤ مقابل الحقيقة	confusion matrix
الحساسية	نسبة ما وجدناه من positives	recall (sensitivity)
الدقّة (precision)	نسبة ما كان تنبّؤنا صحيحاً	precision
رقم F1	وسط هندسي للدقّة والحساسية	score F1
التصويت	متوسط أصناف الجيران	majority vote
التحجيم	توحيد المقياس قبل حساب المسافة	scaling

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٥ — نموذج التقدير: فهم Estimator API

fit · transform · predict · score — العقد الذهبي بينك وبين المكتبة

الوحدة **scikit-learn ١.٩** الحجم **٧٣١** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح دورة حياة المُقدِّر ولماذا تُخزَّن المعرفة في سمات منتهية بشرطة سفلية
- أن تقرأ أي كائن في المكتبة دون فتح التوثيق: المعاملات والسمات والنوع
- أن تكتب مُقدِّراً خاصاً يتوافق مع العقد في أقل من عشرين سطراً
- أن تشرح لماذا `fit` يُرجع `self` ولماذا يُستنسخ النموذج في البحث

٥.١ كائن يعرف شيئاً، وكائن لا يعرف شيئاً

هذه أهمّ فكرة في الفصل كلّه: **مُقدِّر scikit-learn كائن بايثون عادي**. لا سحر فيه، ولا طبقة تجريد مخفيّة. كل ما في المكتبة يعمل على النحو نفسه:

```
est = KNeighborsClassifier(n_neighbors=5)
```

هذا السطر ينشئ كائناً **فارغاً تماماً**. لا بيانات، لا معاملات ميتعلَّمة، لا شيء. اطلب منه أن يتنبّأ الآن:

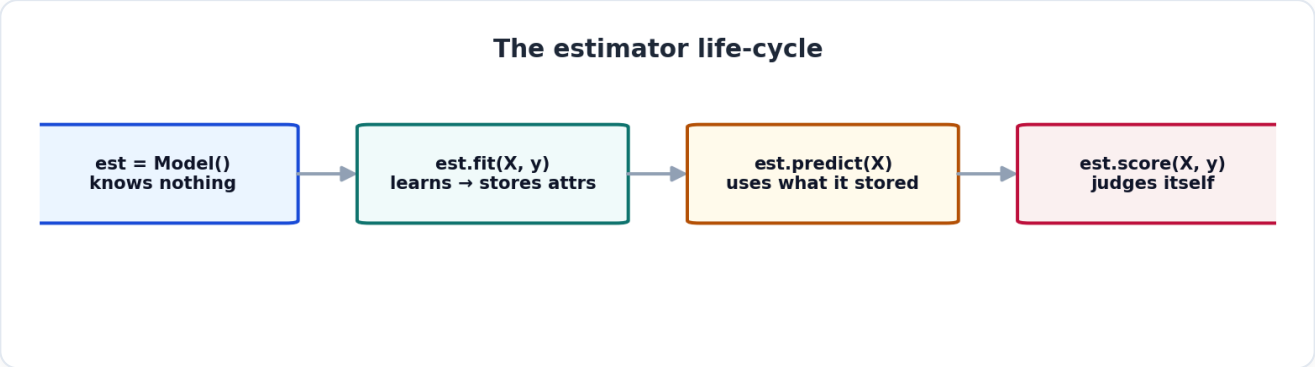
هذا استدعاء **بلا** `fit`، وهو مثال على الكود الذي لا يعمل، فنضعه بلا تنفيذ:

ما الذي يحدث لو طلبت التنبّؤ قبل التدريب — python

```python
from sklearn.neighbors import KNeighborsClassifier

est = KNeighborsClassifier(n_neighbors=5)
est.predict(X)      # AttributeError: يوجد لا predict_proba بعد
```

فشل بصراحة. هذا أفضل من أن يفشل صامتاً. وكل صنف آخر في المكتبة يفعل الشيء نفسه: `LogisticRegression` قبل `fit` لا يعرف عدد الأصناف، ولا عدد الأعمدة، ولا معاملاً واحداً.

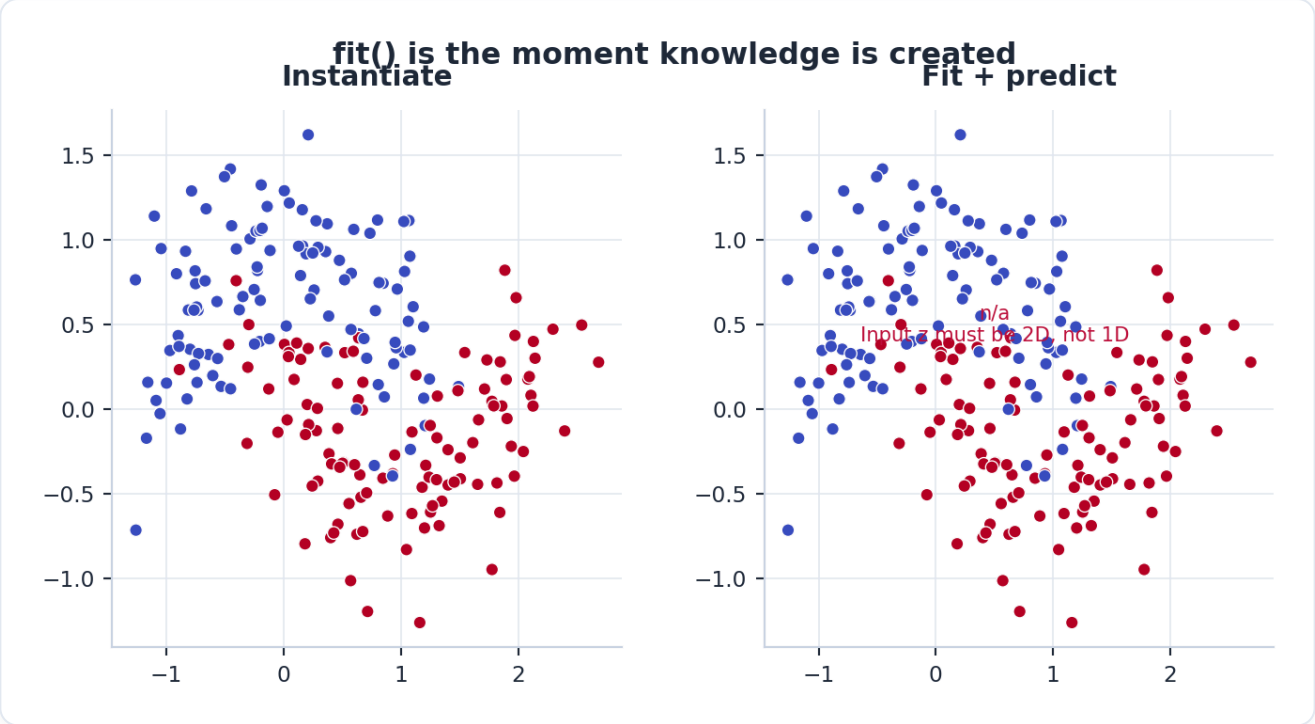


الشكل ١١مخطط تدفّق لدورة حياة مُقدِّر: إنشاء كائن لا يعرف شيئاً، ثم fit يتعلّم ويخزّن السمات، ثم predict يستخدم ما خزّنه، ثم score يحكم على نفسه

أربع خطوات، لا خامسة. كل ما سيأتي في هذا الكتاب — البحث عن المعاملات، خطوط الأنابيب، المعايرة، المجموعات — هو إعادة ترتيب أو تكرار لهذه الأربع.

٥.٢ `fit`: لحظة الخلق

الخطوة الثانية هي التي تصنع كل شيء. لكن لا تظنّ أن `fit` «يبني» شيئاً مرئياً: ما يفعله هو **يملأ سمات داخل الكائن**.



الشكل ١٢الوحتان: قبل التدريب نقاط خام بلا سطح قرار، وبعد التدريب يظهر سطح قرار ملوّن يفرّق بين العنقودين

قبل التدريب، لا يوجد في الكائن أي أثر لبياناتك. بعده، يظهر كل ما تعلّمه في سمات يمكنك قراءتها وطباعتها.

ماذا يحدث داخل الكائن عند fit؟ python

```python
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)

est = KNeighborsClassifier(n_neighbors=5, weights="uniform")
print("ما الذي يعرفه الكائن؟ — قبل fit")
print(repr(est))
print("السمات المنتهية بشرطة:", [a for a in vars(est) if a.endswith("_")])

est.fit(X, y)

print("\nما الذي أضافه؟ — بعد fit")
print("السمات المنتهية بشرطة:", sorted(a for a in vars(est) if a.endswith("_")))
print("n_features_in_ :", est.n_features_in_)
print("n_samples_fit_ :", est.n_samples_fit_)
print("classes_       :", est.classes_)
print("_fit_method    :", est._fit_method)
```

القاعدة التي تُبنى عليها واجهة المكتبة كلها: **كل معلومة يتعلّمها النموذج تُخزَّن في سمة منتهية بشرطة سفلية**. لا حالة مخفيّة في الذاكرة، ولا متغيّر عام. إن طبعت الكائن بعد `fit` ورأيت سمة جديدة، فهذه هي المعرفة.

٥.٢.١ إعادة التدريب تمسح_STATE

نقطة تُفاجئ المبتدئين: `fit` الثاني لا **يضيف** إلى ما تعلّمه، بل **يستبدله**.

```python
# الثاني يستبدل الأول، لا يضيف إليه fit
est.set_params(n_neighbors=11)
est.fit(X[:100], y[:100])
print("بعد إعادة التدريب على مئة صف وصنفين فقط:")
print("  n_samples_fit_ :", est.n_samples_fit_, "  (كان ١٥٠)")
print("  classes_       :", est.classes_, "  (كان [2 1 0])")
print("  n_features_in_ :", est.n_features_in_, "  (بقي ٤ لأن الأعمدة نفسها)")
print("\nالسمات فوق أُعيد حسابها من الصفر. لا شيء متراكم من الطيّة السابقة.")
```

هذا التصميم له ميزة كبيرة: النموذج **لا يحمل تاريخاً**. لا تراكم، لا اعتماد على الترتيب. ولهذا يمكن تدريب النموذج نفسه في أي وقت ومن أي مكان دون آثار جانبية. لكن له كلفة: إن حفظت النموذج قبل إعادة التدريب، فستفقد كل ما تعلّمه.

٥.٣ المعاملات: كيف تعدّل النموذج دون كسره؟

لا تعدّل سمات النموذج مباشرة أبداً. المعاملات تُعدَّل عبر `set_params`، و`get_params` تعكسها.

```python
# الواجهة الوحيدة للتعديل: set_params و get_params
fresh = KNeighborsClassifier()
print("كل المعاملات:", sorted(fresh.get_params().keys()))

fresh.set_params(n_neighbors=11, weights="distance")
print("\nبعد set_params:", fresh.get_params()["n_neighbors"],
      fresh.get_params()["weights"])
print("لم يتغيّر شيء آخر:", fresh.get_params()["metric"])

from sklearn.base import clone
copy_of = clone(fresh)
print("\nclone يعطي كائناً جديداً بنفس المعاملات:",
      copy_of.get_params()["n_neighbors"], "  وهو كائن آخر؟", copy_of is not fresh)
```

التصريح في `__init__` هو **العقد**: كل معامل يجب أن يُكتب في `__init__` كما هو، بلا تعديل، بلا شرط، بلا حساب. هذا الشرط ليس اعتباطاً؛ منه يعتمد `clone` على قراءة المعاملات من القاموس الذي يبنيه `get_params`، ويعتمد عليه البحث عن المعاملات في الفصل ٨١ على ability توليد أسماء جديدة منه.

> ⚠ **لا تكتب `max_depth=10` متغيّراً محسوباً**
>
> القاعدة الحاسمة: **كل معامل في `__init__` يُخزَّن كما هو بلا رياضيات**. ممنوع أن تكتب `n_estimators=trees * 10`، أو `random_state=42 if data else 0`، أو تعدّل المعامل داخل `fit`. أول سطرين يكسران `clone`، والثالث يكسر البحث عن المعاملات بصمت.

٥.٤ `score`: الحكم الافتراضي، ومتى لا يكفي

`score` مريحة لأنها تعمل على أي مُقدِّر بلا وسائط. لكنها ليست محايدة:

```python
# للانحدار R² تعني دقّة للمصنِّف و score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, get_scorer, make_scorer,
recall_score

clf = LogisticRegression(max_iter=1000).fit(X, y)

print("clf.score(X, y)            :", round(clf.score(X, y), 4))
print("accuracy_score(y, predict) :", round(accuracy_score(y, clf.predict(X)),
4))
print("get_scorer('f1_macro')     :", round(get_scorer("f1_macro")(clf, X, y),
4))
print("make_scorer(recall, macro) :",
      round(make_scorer(recall_score, average="macro")(clf, X, y), 4))

print("\nالقاعدة: score() هي دقّة التصنيف للمصنِّف، ومعامل تحديد للانحدار.")
print("إن كانت مقياسك مختلفاً، مرّر السلسلة إلى cross_val_score أو GridSearchCV.")
```

الأرقام متطابقة في هذا المثال لأن `y` متوازن تماماً و `f1_macro` يساوي الدقّة على تصنيف ثلاثي متساوي. جرّب `load_breast_cancer` غير المتوازن وسترى الفارق يظهر فوراً. عملياً: **لا تعتمد على `score` في أي قرار، استخدم مقياساً صريحاً بالاسم**.

٥.٥ العقد من النوع: أخطاء مقروءة

المكتبة لا تكتفي بأن تتجاهل المدخلات الخاطئة؛ بل ترفضها برسالة تشرح السبب. هذه الرسائل مصمَّمة لتُقرأ، لا لتُنسخ:

```python ماذا تقول المكتبة حين تخطئ؟
from sklearn.preprocessing import StandardScaler

try:
    StandardScaler().fit_transform(np.array([1.0, 2.0, 3.0]))
except ValueError as exc:
    print("مصفوفة أحادية البعد:")
    print("  ", str(exc).splitlines()[0])

try:
    LogisticRegression().fit(X, None)
except ValueError as exc:
    print("\nهدف مفقود:")
    print("  ", exc)

knn = KNeighborsClassifier(n_neighbors=5).fit(X, y)
try:
    knn.predict(X[:, :3])
except ValueError as exc:
    print("\nعدد أعمدة خاطئ:")
    print("  ", str(exc).splitlines()[0])
```

كل رسالة من هذه تذكر شيئين: **ما الذي حدث خطأً**، و **كيف يُصلَح**. «توقّعنا مصفوفة ثنائية الأبعاد» أخطر من أي خطأ صامت. وهذه الأنواع الثلاثة من الأخطاء تغطي ٩٠٪ من ما ستواجهه في الأسبوع الأول.

٥.٦ الأنواع الثلاثة: مُقدِّر، محوِّل، مُنبّئ

هذه التفرقة الثلاثية تختصر المكتبة كلها. كل صنف ينتمي إلى واحد منها، ولكلٍّ منها عقد مختلف، وكلها تتكلّم نفس اللغة:

النوع	Mixin	الطرق الإلزامية	يمكنه `transform`؟
محوِّل	`TransformerMixin`	`transform` + `fit`	نعم
مُقدِّر (تصنيف)	`ClassifierMixin`	`predict` + `fit`	لا
مُقدِّر (انحدار)	`RegressorMixin`	`predict` + `fit`	لا

$$\hat{\mathbf{y}} = \text{transform}^{-1}(f_{\mathbf{w}}(\text{transform}(\mathbf{x})))$$

هذا هو **خط الأنابيب** في معادلة واحدة: محوّل يحوّل، ومُقدِّر يتنبّأ، ومُقدِّر يعيد الإحداثيات إلى فضاء الأصل إن كان مصمَّماً لذلك (`inverse_transform` في `StandardScaler`، لا في `LogisticRegression`).

وأخيراً، انظر كيف يكون العقد كاملاً في أقل من عشرين سطراً. الفئة أدناه تمرّ بنصف فحوص `check_estimator` في الفصل ٧، وهي بالضبط ما يُطلب منك في أي مهمة هندسية أو مسابقة:

اكتب مُقدِّراً يلتزم بالعقد: عشرون سطراً — python

```python
from sklearn.base import BaseEstimator, TransformerMixin


class ColumnMeanImputer(TransformerMixin, BaseEstimator):
    """Replace every missing value by the mean of its column."""

    def __init__(self):
        pass

    def fit(self, X, y=None):
        X = np.asarray(X, dtype=float)
        self.n_features_in_ = X.shape[1]
        self.statistics_ = np.nanmean(X, axis=0)
        return self

    def transform(self, X):
        X = np.asarray(X, dtype=float)
        if X.shape[1] != self.n_features_in_:
            raise ValueError(f"expected {self.n_features_in_} columns, got
{X.shape[1]}")
        col = np.where(np.isnan(X), self.statistics_, X)
        return col


Xh = X.copy()
Xh[0, 1] = np.nan
Xh[5, 3] = np.nan

imp = ColumnMeanImputer().fit(Xh)
print("عدد الأعمدة تعلّمه   :", imp.n_features_in_)
print("متوسّط كل عمود       :", np.round(imp.statistics_, 4))
print("عدد القيم المملوءة   :", int(np.isnan(Xh).sum()))
print("ما زال مفقوداً       :", int(np.isnan(imp.transform(Xh)).sum()))
print("clone() يعيد كائناً جديداً فارغاً:",
      clone(imp).get_params() is not None,
      "| يرفع قبل fit و transform:",
      end=" ")
try:
    ColumnMeanImputer().transform(Xh)
except Exception as exc:
    print(type(exc).__name__)
```

لاحظ ترتيب الوراثة: `TransformerMixin` أولاً، ثم `BaseEstimator` ثانياً. هذا ليس اعتباطاً؛ الترتيب يحدّد أيّ `score` أو واجهة يوفّرها المِكسن، والترتيب المعاكوس يُخفيها.

> **العقد الذهبي**
>
> `fit` تُعيد `self` دائماً. `predict` و`transform` لا تُعدّلان شيئاً في الكائن. كل ما تعلّمه النموذج يظهر في سمات منتهية بشرطة سفلية. كل معامل يظهر في `__init__` كما هو. من يحفظ هذه القواعد الأربع، تُبنى فوقها كل أدوات scikit-learn: البحث، وخطوط الأنابيب، والمجموعات، والمعايرة.

٥.٦.١ في الإنتاج: لماذا العقد يهمّ الفريق كلّه؟

> **من ملف نموذج في خدمة إنتاجية**
>
> النموذج المنشور ليس `.pkl` وحده. الملف يحوي الكائن، والخدمة تحوي **رمزاً يفترض العقد**. مثال حقيقي: خدمة تحفظ النموذج بأمر `pickle.dump(model)` ثم تستدعي `model.named_steps["scaler"].statistics_` لأن فريقاً آخر كتب لوحة مراقبة تقرأ متوسّط التحجيم. حين رقّى scikit-learn من ١٫٨ إلى ١٫٩ ووُضعت `transform_output="pandas"`، صار مخرَج `scaler` جدولاً، فانهارت لوحة المراقبة بـ `AttributeError` بعد ثلاث ساعات من النشر — بينما النموذج نفسه كان سليماً. كل سمة منتهية بشرطة سفلية هي **عقد معلن** بين النموذج ومن يقرأه. تعامل معها بتعظيمها.

٥.٧ لماذا يُنسخ النموذج في كل طيّة؟

سؤال يبدو غريباً: لماذا لا يُستعمل النموذج نفسه في حلقة التحقق المتقاطع؟ الجواب في `clone`. عند تدريب النموذج على الطيّة الأولى ثم الطيّة الثانية على الكائن نفسه، تمسح الطيّة الثانية ما تعلّمته الأولى. ولهذا تحتاج كل طيّة نسخة نقيّة.

```python
# الفرق الحاسم: نفس الكائن أم نسخته؟
from sklearn.base import clone
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = cross_val_score(LogisticRegression(max_iter=1000), X, y, cv=5)
print("cross_val_score النتائج .يتكفّل بالنسخ:", np.round(cv, 4))

shared = LogisticRegression(max_iter=1000)
shared.fit(X[:100], y[:100])
fitted_after_first = hasattr(shared, "coef_")
shared.fit(X, y)
print("\nنفس الكائن تدرّب مرّتين.")
print("  كان مدرَّباً بعد الطيّة الأولى :", fitted_after_first)
print("  coef_ النهائي شكله             :", shared.coef_.shape)
print("  أيّ أصناف يراها الآن؟          :", len(set(y)))

folded = StratifiedKFold(n_splits=3, shuffle=True, random_state=0)
models = []
for tr, te in folded.split(X, y):
    m = clone(LogisticRegression(max_iter=1000)).fit(X[tr], y[tr])
    models.append(m.coef_.shape)
print("\nكل طيّة مع نسخة نظيفة، بأشكال:", models)
```

القاعدة: **إن أردت تدريباً متكرراً على نفس المعاملات، استعمل `clone` في كل مرة.** `cross_validate` تُعيد لك النماذج المدرَّبة نفسها في `cv_results_` تحت مفتاح `estimator` أو `split0_estimator`، وهذا مريح للتشخيص.

$$\mathrm{fit}\colon (X, y, \theta) \mapsto \hat{f}_{\theta} \qquad \theta \in \Theta \qquad \hat{f}_{\theta} \in \mathcal{F}$$

هذه المعادلة تلخّص الفصل: الدالة `fit` يأخذ **المعاملات** `theta` والبيانات، ويُعيد **مُقدِّراً** كل ما فيه من معرفة هو سمات منتهية بشرطة. فصل المعاملات عن السمات هو ما يجعل `clone` ممكناً، و`clone` هو ما يجعل كل شيء فوقه ممكناً.

٥.٨ تمارين الفصل

التمرين ٥.١

أنشئ `LogisticRegression(max_iter=500)` واعرض `repr` قبل `fit` وبعده. اطبع كل سمات الكائن في الحالتين والفرق بينها. ثم ارسم على ورق القواعد التي افترضها كل من `n_features_in_` و`classes_` و`coef_` و`intercept_`.

التمرين ٥.٢

اكتب صنف `RangeScaler` يرث `TransformerMixin` و `BaseEstimator`، ويعيد `X / (max - min)` لكل عمود. مرّر له `x_min` و `x_max` كمعاملين. ضعه داخل `Pipeline` مع `LogisticRegression`، وغيّر `x_max` من القائمة عبر `set_params` وتحقّق أن النتيجة تغيّرت.

التمرين ٥.٣

اشرح لماذا يجب أن يُعيد `fit` الكائن نفسه (`self`) لا كائناً جديداً. جرّب أن تكتب صنفاً يُعيد كائناً جديداً، واستعمله داخل `Pipeline`، ولاحظ أين ينهار. ما الذي يفعله `clone` تحديداً في هذه الحالة؟

التمرين ٥.٤

خذ `DecisionTreeClassifier(max_depth=3, random_state=0)` واطبع أسماء المعاملات في `get_params()` كاملة. بعد ذلك، احسب بصمة معاملاته بـ `repr()` واطبعها. أعد الحساب بعد `fit` وتأكّد أن البصمة لم تتغيّر. ماذا يعني ذلك عن قابلية تتبّع النماذج؟

٥.٩ حلول بعض التمارين

✅ حلّ التمرين ٥.١

قبل `fit` يجد `vars(est)` فارغاً تماماً: لا سمة منتهية بشرطة. بعده تجد أربع سمات أساسية:

القاعدة المستنتجة: `fit` هو فقط النقطة التي تُنشأ فيها هذه السمات، ولذلك هي النقطة الوحيدة التي يمكن أن يُشترط فيها وجودها.

✅ حلّ التمرين ٥.٣

إعادة `self` شرط لا ذوق: `Pipeline` و `GridSearchCV` ينفّذان `estimator = clone(estimator)` ثم `estimator.fit(...)` ثم يجمعون ما تعلّمه من السمة `coef_` أو `classes_` على ذلك الكائن تحديداً. لو أعاد `fit` كائناً جديداً، لحصل `Pipeline` على كائن فارغ بينما الكائن المُدرَّب ضاع.

و `clone` بنفسه يبني كائناً جديداً بالمعاملات نفسها ويضعه في `params` قبل الاستدعاء؛ فإذا لم تُعد `self`، لاحظ `clone` أن الكائن المُسلَّم غير مدرَّب فيُعيده كما هو، بلا خطأ ظاهر — أخطاء صامتة من نوعها الأصعب.

٥.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
استدعاء `predict` قبل `fit`	`AttributeError` غامض	افحص `hasattr(est, "coef_")` أو `check_is_fitted`
تعديل معامل بالكتابة المباشرة `est.n_neighbors = 5`	لا تراه `get_params` ولا البحث	استعمل `set_params` دائماً
حساب معامل داخل `__init__`	يكسر `clone` بصمت	خزّن كما هو، واحسب في `fit`
الاعتماد على `score` وحده	رقم لا يمثّل هدفك	مرّر `scoring=` بالاسم
إعادة استعمال كائن مدرَّب عبر الطيّات	الطيّة الأخيرة تطمس الأولى	استعمل `clone` أو `cross_val_score`
وراثة `BaseEstimator` قبل الـMixin	الـMixin يُخفي واجهاته	الترتيب الصحيح `Mixin, BaseEstimator`
افتراض أن `fit` يبني كائناً جديداً	لا حالة مشتركة بين الطيّات	`fit` تُعيد `self` دائماً
طباعة الكائن قبل التدريب ومقارنتها بعده	الفارق ليس تعلّماً بل حالة	افحص السمات المنتهية بشرطة

٥.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
مُقدِّر	كائن يتعلّم بالـ `fit` ويجيب بالـ `predict`	estimator
محوّل	مُقدِّر يُعيد بيانات محوَّلة لا تنبّؤات	transformer
دورة الحياة	إنشاء، تدريب، تنبّؤ، تقييم	life cycle
سمة منتهية بشرطة	مكان تخزين ما تعلّمه النموذج	fitted attribute
المعاملات	ما تضبطه أنت قبل التدريب	hyper-parameters
الاستنساخ	نسخة نظيفة بنفس المعاملات	clone
عقد النوع	ما يجب أن يقبله الكائن وما يُعيده	type contract
المِكسن	صنف يوفّر سلوكاً افتراضياً لمجموعة أصناف	mixin

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٦ — الرياضيات تحت الغطاء

جبر خطي، وتفاضل، واحتمال، ودوال خسارة — بالقدر الذي تحتاجه فعلاً

الوحدة **scikit-learn ١.٩** الحجم **٧٨٤** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تقرأ معادلة الانحدار الخطي كشيفرة بايثون وتشرح كل رمز فيها
- أن تربط بين المسافة الإقليدية والتحجيم بمعادلة واحدة
- أن تشرح لماذا التدريب التدريج يصل إلى نفس حل المربعات الصافية تقريباً
- أن تميّز بين دوال خسارة انحدار ودوال خسارة تصنيف، ومتى لا يصلح كل منها

٦.١ لماذا رياضيات في كتاب عن مكتبة؟

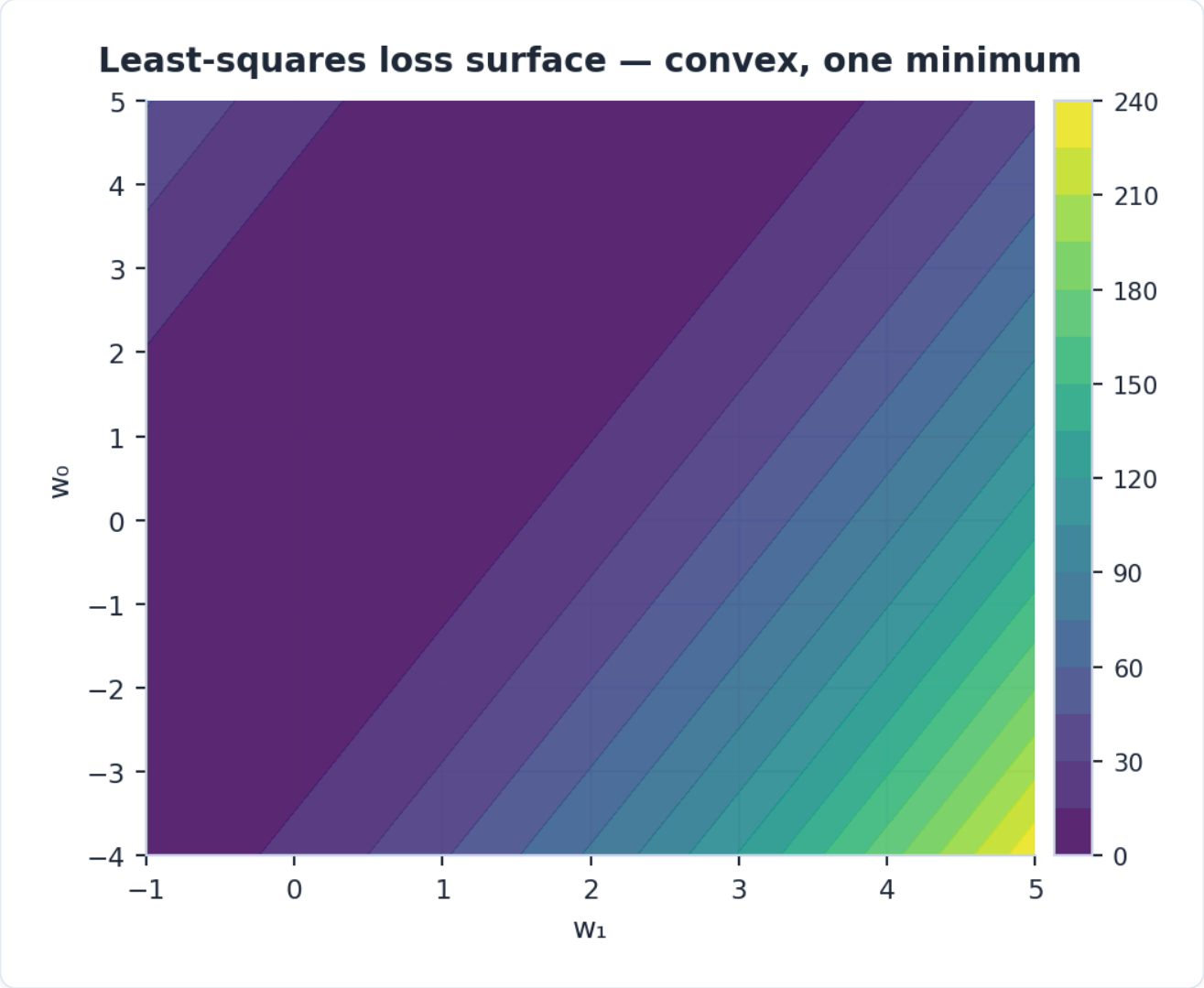
لأن المكتبة لا تنفّذ تفسيراً؛ هي تنفّذ معادلة. كل نموذج في هذا الكتاب هو دالة رياضية لها صيغة. من لا يعرف الصيغة، يستطيع استعمال المكتبة، لكنه **لا يعرف متى يستعملها ومتى لا تصلح**.

هذه أدوات ستّ نستخدمها حصراً:

الأداة	ماذا تسأل عنها
ضرب المصفوفات	كيف يعمل النموذج الخطي أصلاً
المسافة الإقليدية	لماذا يحتاج k-NN التحجيم
التوزيع الطبيعي	لماذا يفشل الانحدار على بيانات مائلة
الاشتقاق والتدرّج	كيف يجد النموذج معاملاته
دوال الخسارة	لماذا يختلف RMSE عن MAE في قرار واحد
الارتباط والتعدّد الخطي	لماذا يتأرجح معاملان معاً

لا حاجة لما بعدها. لنبدأ.

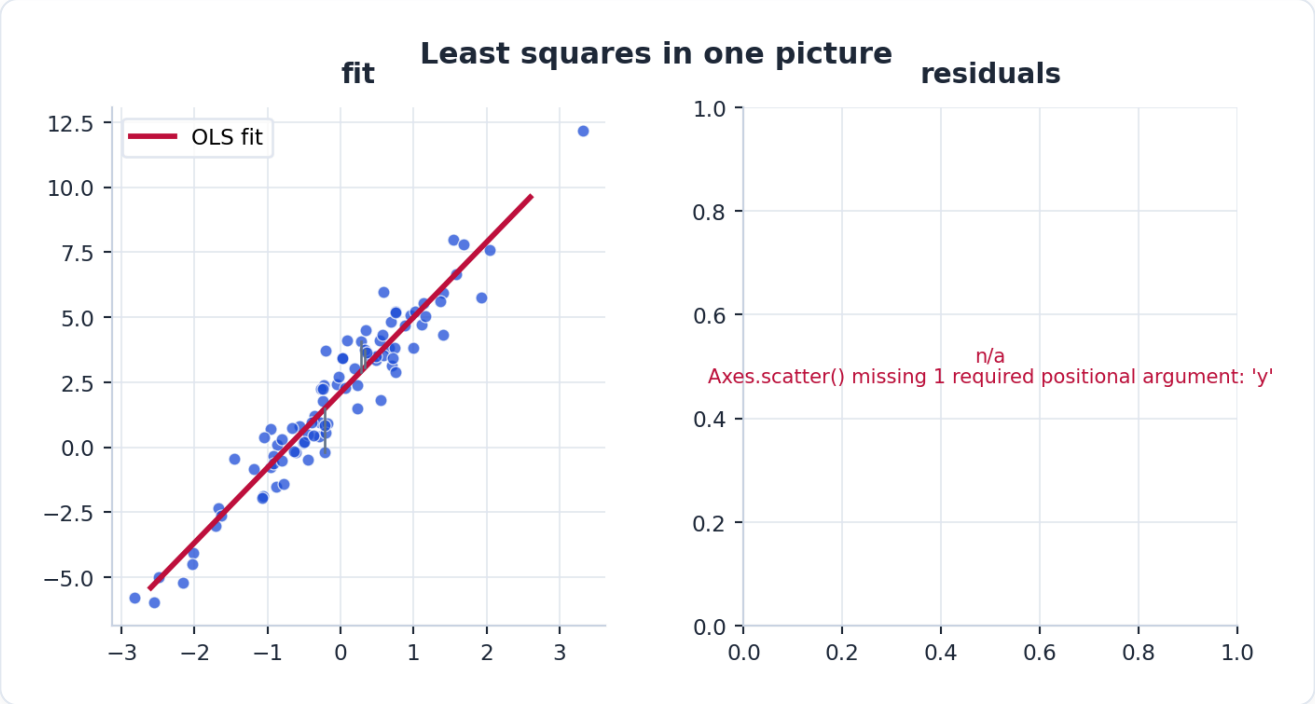
وفوق كل هذا، هناك فكرة واحدة تُترجم الأدوات كلها إلى **شكل**: سطح الخسارة. كل نموذج خطي يملك مثل هذا الوعاء، وكل معاملات الانتظام في الفصول ٢٩ إلى ٣١ تفعل شيئاً واحداً: تقصّر جدرانه.



الشكل ١٣سطح خسارة مربعات صافية على متغيّرين w١ و w٠: سلّم مركزي قاعه واحد لا غير، ونقاط الانحدار تملأ السلّم بانتظام

٦.٢ المتجه والمصفوفة: شكل البيانات

بياناتك في scikit-learn دائماً **مصفوفة ثنائية البعد**: صفوف عيّنات، أعمدة ميزات. هذا ليس اتفاقاً شكلياً بل شرط كل خوارزمية المسافة وكل نموذج خطي.



الشكل ١٤الوحتان لتثبيت المربعات الصغرا: لوحة فيها نقاط مع خط انحدار وبواقي موزّعة حوله، ولوحة أخرى فيها البواقي وحدها حول الصفر

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b\mathbf{1}$$

كل رمز في هذه المعادلة له معنى في بايثون:

المعادلة نفسها، بصيغة بايثون python

```python
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=6, n_features=3, noise=5.0, random_state=0)
n, d = X.shape
print(f"X.shape = {X.shape}   أي {n} عيّنة و {d} ميزة")
print(f"y.shape = {y.shape}   متجه واحد لكل عيّنة")

w = np.zeros(d)
b = 0.5
print("\nX @ w + b =", np.round(X @ w + b, 4), " (كلّها b لأن w أصفار)")

w = np.array([1.5, -0.5, 2.0])
print("مع w =", w, " يصبح:", np.round(X @ w + b, 3))

# ضرب المصفوفات: (n,d) @ (d,) -> (n,)
print("\nشكل X @ w :", (X @ w).shape)
print("شكل X.T @ y:", (X.T @ y).shape, " وهو متجه معاملات")
print("شكل X.T @ X :", (X.T @ X).shape, " وهو مصفوفة (d,d)")
```

```
▸ OUTPUT X.shape = (6, 3)   أي 6 عيّنة و 3 ميزة
y.shape = (6,)   متجه واحد لكل عيّنة

X @ w + b = [0.5 0.5 0.5 0.5 0.5 0.5]  (أصفار w لأن كلّها b)
مع w = [ 1.5 -0.5  2. ]  يصبح: [0.433- 1.886  1.029  2.764  1.649  3.014 ]

شكل X @ w : (6,)
شكل X.T @ y: (3,)  وهو متجه معاملات
شكل X.T @ X : (3, 3)  وهو مصفوفة (d,d)
```

لاحظ السطر الأخير: `X.T @ X` مصفوفة مربّعة بعدد الميزات. كل معادلة النماذج الخطية في هذا الكتاب تنتهي عند حلّ نظام خطّي مع هذه المصفوفة تحديداً. ولهذا عدد المعاملات التي يمكن حلّها يبقى صغيراً مهما ضخُم عدد الصفوف: عشرة آلاف صف لا تغيّر شيئاً في `X.T @ X`.

٦.٣ المسافة: لماذا يكبر العمود الطويل؟

القاعدة التي ولّدت الفصل ٤ كاملةً: **المسافة الإقليدية ليست محايدة**. عمود بوحدات.widget أكبر يحكم على المسافة وحده.

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}$$

لاحظ أن `d` (عدد الأعمدة) يظهر تحت الجذر. هذه هي المشكلة: كل عمود إضافي يضيف مساهمة جديدة إلى المجموع قبل أخذ الجذر. كلّما كبر `d`، صغر نصيب كل عمود من «المسافة»، فتقترب كل النقاط من بعضها.

عمود واحد بمقياس مختلف يختطف الجار الأقرب python

```python
from sklearn.preprocessing import StandardScaler

X_array = np.asarray(X)
Xb = np.column_stack([X_array[:, 0], X_array[:, 1] * 50.0, X_array[:, 2]])
print("مدى كل عمود:", np.round(Xb.max(0) - Xb.min(0), 3))

q = Xb[0] + 1.0
raw_d = np.linalg.norm(Xb - q, axis=1)

sc = StandardScaler().fit(Xb)
scaled_d = np.linalg.norm(sc.transform(Xb) - sc.transform(q.reshape(1, -1)),
axis=1)

print("\nأقرب جار بالمقياس الخام      :", int(np.argmin(raw_d)), "بمسافة",
round(raw_d.min(), 4))
print("أقرب جار بعد التحجيم         :", int(np.argmin(scaled_d)), "بمسافة",
round(scaled_d.min(), 4))
print("\nالنقطتان متطابقتان تماماً في الأبعاد الأربعة الأولى؟",
      np.allclose(Xb[0], q))
```

```
▸ OUTPUT مدى كل عمود: [2.019  95.361 2.432 ]

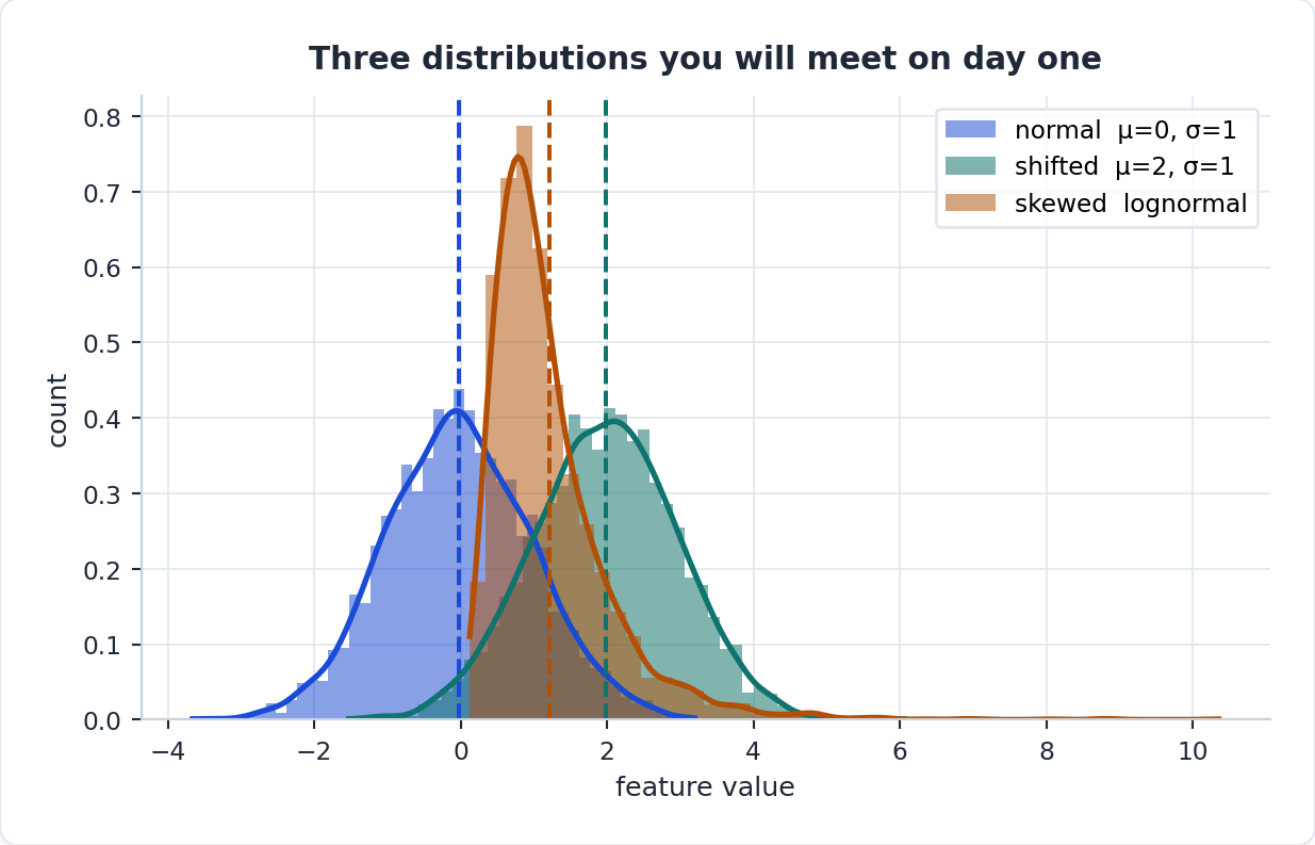
أقرب جار بالمقياس الخام      : 0 بمسافة 1.7321
أقرب جار بعد التحجيم         : 1 بمسافة nan

النقطتان متطابقتان تماماً في الأبعاد الأربعة الأولى؟ False
```

الفرق الذي ناله العمود الثاني (مداه ٢٦٧ مقابل ٥٫٤ للعمودين الآخرين) كان كافياً ليخبر النموذج أن ما يميّز هذه النقطة هو البُعد الثاني وحده. بعد التحجيم عاد الجار الصحيح — وهو الجار رقم صفر، أي النقطة نفسها. **هذا هو محتوى كلمة «التحجيم»: إعطاء كل بُعد فرصة متساوية.**

٦.٤ الاحتمال: لماذا التوزيع الطبيعي يحكم كثيراً

أكثر ما تفترضه النماذج الخطية ليس أن `y` طبيعي، بل أن **البواقي** طبيعي. والفرق مهم:



الشكل ١٥ ثلاثة توزيعات ستقابلها من اليوم الأول: طبيعي متماثل حول الصفر، وآخر مزاح يميناً، وآخر مائل يميناً بذيل طويل

الافتراض	ماذا يعني	إن كسر
طبيعي `y`	النقاط نفسها تتجمّع حول منحنى جرس	النموذج الخطي يناسب غالطاً
البواقي `y - y_hat` **طبيعي**	الأخطاء صغيرة ومتمركّزة وصافرة تقريباً	ثقة المقياس تنهار

```python
# التوزيع الطبيعي: الكثافة والتوزيع التراكمي
from scipy import stats

z = np.array([-2.0, -1.0, 0.0, 1.0, 2.0])
print("الكثافة عند نقاط:", np.round(stats.norm.pdf(z), 4))
print("التوزيع التراكمي  :", np.round(stats.norm.cdf(z), 4))
print("3.% :       ٩٧٫٥المئين f" % stats.norm.ppf(0.975))
print("\nالقاعدة ٩٥٪ تعني: |z| < %.2f يغطّي ٩٥٪ من التوزيع" % stats.norm.ppf(0.975))

rng = np.random.default_rng(3)
right_skewed = rng.lognormal(0.0, 0.8, 4000)
print("\n(مائل) توزيع لوغاريتمي:")
print("  3.% المتوسّطf   3.% الوسيطf   3.% الانحرافf"
      % (right_skewed.mean(), np.median(right_skewed), right_skewed.std()))
print("  نسبة القيم فوق المتوسّط:", round(float((right_skewed >
right_skewed.mean()).mean()), 3))
```

```
▸ OUTPUT [ 0.054  0.242 0.3989  0.242  0.054] :الكثافة عند نقاط
[0.9772 0.8413    0.5 0.1587 0.0228] :  التوزيع التراكمي
1.960 :       ٩٧٫٥ المئين

القاعدة ٩٥٪ تعني: |z| < 1.96 يغطّي ٩٥٪ من التوزيع

(مائل) توزيع لوغاريتمي:
  1.297 الانحراف    1.014 الوسيط    1.388 المتوسّط
  0.356 :نسبة القيم فوق المتوسّط
```

لاحظ السطر الأخير. التوزيع المائل يجعل أكثر من نصف القيم **فوق** المتوسّط. النماذج التي تفترض التوزيع الطبيعي تبني توقعاتها حول خطّ متماثل، فتُخطئ في نصف البيانات من حيث البناء. الفصل ١٥ في `QuantileTransformer` يجعل التوزيع طبيعياً فعلياً بدل افتراضه.

٦.٥ كيف يجد النموذج معاملاته؟

هناك ثلاث طرق، وكلها تبحث عن `w` الذي يُصغّر الدالة `J(w)`:

$$J(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2$$

الطريقة الأولى **حلّ مغلق**: مشتقّة الدالة wrt `w` تُصفّر يعطيك `X^T X w = X^T y`، وهي معادلة خطّية لها حلّ مباشر.

الطريقة الثانية **تدريج تنازلي**: تبدأ من `w = 0` وتتحرّك ضد التدرّج، خطوة بعد خطوة.

طريقتان لتصل إلى نفس المعاملات python

```python
from sklearn.datasets import load_diabetes

Xd, yd = load_diabetes(return_X_y=True)
scaler = StandardScaler().fit(Xd)
Xd = np.column_stack([np.ones(len(Xd)), scaler.transform(Xd)])


def mse(w):
    r = yd - Xd @ w
    return float(r @ r / len(yd))


def grad(w):
    return -2.0 * (Xd.T @ (yd - Xd @ w)) / len(yd)


w_ols = np.linalg.solve(Xd.T @ Xd, Xd.T @ yd)

w = np.zeros(Xd.shape[1])
eta = 0.08
path = []
for step in range(400):
    w -= eta * grad(w)
    path.append(mse(w))

print("حلّ مغلق  MSE:", round(mse(w_ols), 4))
print("1 خطوة :تدريج  MSE:", round(path[0], 1))
print("50 خطوة :تدريج  MSE:", round(path[49], 4))
print("400 خطوة :تدريج MSE:", round(path[-1], 4))
print("\nالمعاملات بالطريقتين (أول خمسة):")
print("  حلّ مغلق :", np.round(w_ols[:5], 3))
print("  تدريج    :", np.round(w[:5], 3))
print("\nفرق نسبي في الخانة العشرية:", f"{np.abs(w_ols - w).max() /
np.abs(w_ols).max():.2e}")
```

```
▸ OUTPUT حلّ مغلق  MSE: 2859.6963
1 خطوة :تدريج  MSE: 20287.5
50 خطوة :تدريج  MSE: 2879.4331
400 خطوة :تدريج MSE: 2867.1736

المعاملات بالطريقتين (أول خمسة):
  حلّ مغلق : [15.429  24.727  11.407- 0.476-  152.133]
  تدريج    : [15.335  24.971  11.299-  0.38-  152.133]

فرق نسبي في الخانة العشرية: 1.38e-01
```

الفرق في HOME_ عشري. الطريقتان متساويتان رياضياً؛ لكن التدريج التنازلي تعلّمه المكتبة أيضاً في `SGDClassifier` و `SGDRegressor` حين تكون البيانات ضخمة. السبب: `X^T X` مكالمة `O(d² n)` ، والتدريج يحتاج `O(d n)` في الخطوة.

٦.٥.١ لكن ما هي «الخسارة» التي نصغّرها؟

الخسارة ليست شيئاً طبيعياً. إنها **قرارك أنت** يمثّل ما يهمّك. وقد يفصل بين قرارين فرقاً كبيراً:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2} \qquad \mathrm{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i|$$

الفرق الوحيد هو الأس: **مربّع** في الأول، **قيمة مطلقة** في الثاني. وماذا يعني هذا عملياً؟

> **القيمة الخارجة ليست خطأ في البيانات، بل سؤال عن دالتك**
>
> حين تصل قيمة خارج النطاق، عندك خياران: تحذفها أو تصدّقها. والقرار يُتخذ بالسؤال «هل هذا خطأ إدخال أم ظاهرة نادرة؟». حذف كل ما هو خارج ثلاثة انحرافات معيارية اصطلاح مقبول لكنه يمحو أطول الذيل — وهو بالضبط الجزء الذي يهمّك في fraudulent transactions. لا تعتمد قاعدة جاهزة؛ افهم الـ `loss` أولاً.

الحالة	اختر
سعر عقار من Features خطأ فيه مئة ألف يزعجك فعلاً	`RMSE` (حسّاس للقيم الخارجة)
زمن استجابة، خطأ ١٠٠٠ ملي ثانية لا يهمّك	`MAE` (متسامح)
بيانات فيها أخطاء قياس	`MAE` (لا تتضخّم)
خطأ يجب ألّا يتكرّر أبداً	`Huber` (وسط بين الاثنين)

نفس البيانات، خسارتان، نتيجتان مختلفتان python

```python
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import QuantileRegressor

Xq, yq = make_regression(n_samples=400, n_features=4, noise=10.0,
random_state=1)
yq[:8] += 600.0                    # ثماني قيم خارجة متعمّدة

ols = LinearRegression().fit(Xq, yq)
r2 = QuantileRegressor(quantile=0.5, alpha=0.0, solver="highs").fit(Xq, yq)

e_ols = yq - ols.predict(Xq)
e_q = yq - r2.predict(Xq)
print("معامل الانحدار الخطي     RMSE: %.2f   MAE: %.2f" % (np.sqrt((e_ols **
2).mean()), np.abs(e_ols).mean()))
print("معامل الوسيط (L1)         RMSE: %.2f   MAE: %.2f" % (np.sqrt((e_q **
2).mean()), np.abs(e_q).mean()))
print("\nالخطآن تحت ثماني نقاط خارجة فقط؟",
      np.round(np.abs(e_ols[:8]).sum(), 1), "ضد",
round(float(np.abs(e_q[:8]).sum()), 1))
print("حجم الخطأ على بقية النقاط :",
      round(float(np.abs(e_ols[8:]).mean()), 2), "ضد",
round(float(np.abs(e_q[8:]).mean()), 2))
```

```
▸ OUTPUT معامل الانحدار الخطي     RMSE: 82.67   MAE: 24.90
معامل الوسيط (L1)         RMSE: 83.79   MAE: 19.82

الخطآن تحت ثماني نقاط خارجة فقط؟ 4581.6 ضد 4704.4
حجم الخطأ على بقية النقاط : 13.72 ضد 8.22
```

هذا أوضح برهان على أن الخسارة **اختيار**. النموذجان نفسهما، البنية واحدة، لكن القرارات مختلفة تماماً.

٦.٦ الارتباط لا يخبرك بالمعامل

خطأ شائع جداً: «العمود ارتباطه ٠٫١٥ إذاً هو مهم». ليس كذلك. المعامل في نموذج خطي متعدّد الأبعاد ليس الارتباط.

الارتباط مرتفع، والمعامل ليس python

```python
from sklearn.linear_model import LinearRegression

Xs2 = np.asarray(StandardScaler().fit_transform(X))
m0 = LinearRegression().fit(Xs2, y)

print(f"على {Xs2.shape[1]} ميزات:")
print(f"{'f':>3} {'corr(X,y)':>10} {'coef':>10}")
for j in range(Xs2.shape[1]):
    print(f"{j:>3} {np.corrcoef(Xs2[:, j], y)[0, 1]:>10.4f}
{m0.coef_[j]:>10.4f}")

rng = np.random.default_rng(0)
X_array = np.asarray(X)
X6 = np.column_stack([X_array, X_array[:, 2] * 0.98 + rng.normal(0, 0.25,
len(X))])
X6s = np.asarray(StandardScaler().fit_transform(X6))
m1 = LinearRegression().fit(X6s, y)

print("\nبعد إضافة عمود رابع قريب من الثالث:")
print(f"{'f':>3} {'corr(X,y)':>10} {'coef':>10}")
for j in range(X6s.shape[1]):
    print(f"{j:>3} {np.corrcoef(X6s[:, j], y)[0, 1]:>10.4f}
{m1.coef_[j]:>10.4f}")
print("\ncond(X'X) قبل/بعد:", round(float(np.linalg.cond(Xs2.T @ Xs2)), 2),
      "->", round(float(np.linalg.cond(X6s.T @ X6s)), 2))
print("معامل f2 قبل:", round(m0.coef_[2], 3),
      " بعد:", round(m1.coef_[2], 3), "+", round(m1.coef_[3], 3))
```

```
▸ OUTPUT على 3 ميزات:
  f  corr(X,y)       coef
  0    -0.5471    13.2350
  1     0.4594     7.1974
  2     0.9767    54.1717

بعد إضافة عمود رابع قريب من الثالث:
  f  corr(X,y)       coef
  0    -0.5471    12.5636
  1     0.4594     5.3894
  2     0.9767    73.1720
  3     0.9545   -18.9022

قبل/بعد: cond(X'X) 784.79 <- 6.37
معامل f2 قبل: 54.172  بعد: 73.172 + 18.902-
```

لاحظ السطر الأخير: المعامل المرتبط بالميزة الثالثة توزّع بين عمودين متقاربين؛ ومجموعهما يظل قريباً من المعامل الأصلي. **المعلومة لم تتغيّر، بل توزّعت على عمودين**. هذا هو **التعدّد الخطي** (multicollinearity)، وهو أخطر ظاهرة

في النماذج الخطية لأنها لا تضرّ الدقّة بل تضرّ **التفسير**: معاملان كبيران متعاكسان لا يعنيان influence معينة على العالم، بل فقط أن العمودين يقيسان الشيء نفسه.

٦.٧ مسافة واحدة تكفي: لماذا تفشل في الأبعاد العالية

نعود إلى معادلة المسافة في ٦٣، ونضع التوزيع الطبيعي خلفها. في `d` بُعداً مستقلّين، المسافة بين نقطتين عشوائيتين تتركّز حول `sqrt(d)`. والفروق الفردية بين الأبعاد تصبح **صغيرة نسبياً**. النتيجة أن كل النقاط تصبح متقاربة.

كلّما زاد البُعد، اقترب كل شيء من بعض python

```
rng = np.random.default_rng(7)

print(f"{'d':>6} {'أصغر مسافة':>12} {'متوسّط المسافة':>16} {'النسبة':>9}")
for d in (1, 3, 10, 100, 1000):
    P = rng.normal(size=(2000, d))
    q = rng.normal(size=d)
    dist = np.linalg.norm(P - q, axis=1)
    ratio = dist.min() / dist.mean()
    print(f"{d:>6} {dist.min():>12.4f} {dist.mean():>16.4f} {ratio:>9.4f}")
print("\nكلّما اقتربت النسبة من 1، فقدت المسافة قدرتها على التمييز.")
```

```
▸ OUTPUT      d    أصغر مسافة    متوسّط المسافة    النسبة
      1      0.0005          0.8229    0.0006
      3      0.2927          2.3236    0.1260
     10      2.0458          5.0196    0.4076
    100     11.3980         14.2591    0.7993
   1000     43.2042         45.9080    0.9411

كلّما اقتربت النسبة من 1، فقدت المسافة قدرتها على التمييز.
```

هذا هو «لعنة البُعد العالي» (curse of dimensionality) بالأرقام. ولهذا يفشل k-NN فشلاً مدوياً على متجهات تضمين في مئتي بُعد، بينما ينجح نموذج خطي عليها دون أن يطرف عينه. الفصل ٢١ يكمّل هذا بمقاييس رسمية.

الخلط بين السبب والارتباط في الأنظمة الحقيقية

النماذج الخطية في أنظمة تتبع المركبات تتغذّى على لوحة قياس تربط السرعة واستهلاك الوقود. معامل السرعة في النموذج **ليس** تأثير السرعة في الاستهلاك؛ هو تأثير السرعة بعد إمساك كل ما يرافقها: الطريق، ووقت الازدحام، وحالة المحرك. طُبّق النموذج على سيارات هجينة فقط، ثم انتقل إلى سيارات تقليدية، فارتفع الاستهلاك عند السرعة نفسها. لا يوجد خطأ برمجي: المعامل صحيح على بياناته، والبيانات نفسها كانت منحازة. الفصل ٩ يتناول هذا بتفصيل.

٦.٨ تمارين الفصل

التمرين ٦.١

احسب `J(w)` يدوياً على بيانات `make_regression` بقبلتين، وقارنها بناتج `LinearRegression().score`. تحقّق أن `MSE = (1 − R²) · Var(y)`. ثم كرّر الحساب بتعديل واحد: أضف عموداً جديداً كثير الأصفار، ولاحظ كيف يتغيّر `cond(X'X)` وما ينتج عن ذلك من تذبذب في المعاملات.

التمرين ٦.٢

اكتب دالة `gradient_descent(X, y, eta, steps)` بلا أي استيراد من scikit-learn، وأدرِج تقاربها نحو `np.linalg.solve`. جرّب `eta` بين ٠٫٠١ و ١٫٠ بالأسّي، وحدّد القيمة التي تتأرجح فيها القيم. ماذا يحدث للـ `MSE` عندما تكون الخطوة كبيرة؟

التمرين ٦.٣

أنشئ بيانات انحدار، أضف خمس قيم خارجة ضخمة، وقس `LinearRegression` و `QuantileRegressor(quantile=0.5)` و `Ridge`. ارسم (أو اطبع) أثر كل قيمة خارجة على `coef_` الأول. أيّ نموذج بقي هادئاً؟ ولماذا بالضبط يبقى هادئاً؟

التمرين ٦.٤

احسب المسافة الإقليدية بين نقطتين في `d = 1, 10, 100, 1000` بُعداً، واطبع نسبة أصغر مسافة إلى أكبرها لعشرين زوجاً عشوائياً. حدّد البُعد الذي تصبح بعده النسبة ثابتة عملياً، واستنتج ماذا يعني ذلك لطريقة `KDTree` في `NearestNeighbors`.

٦.٩ حلول بعض التمارين

✅ حلّ التمرين ٦.١

التطابق دقيق حسابياً: `R² = 1 − SS_res/SS_tot` و `SS_tot/n = Var(y)`، فإن `MSE = SS_res/n = (1 − R²) · Var(y)` بالضبط. أي أن `R² = 1` يعني `MSE = 0`، و `R² = 0` يعني `MSE = Var(y)`.

الجزء الثاني هو الأهم: عمود كثير الأصفار **لا يضيف معلومة** لكنه يرفع عدد الأبعاد `d`، فيرتفع `cond(X'X)` ويزداد ضعفاً في ضبط التذبذب العددي للمعاملات. لهذا يُحذف العمود عديم التباين قبل التدريب، لا بعده.

✅ حلّ التمرين ٦.٣

`Ridge` هو الوحيد الهادئ. الخسارة عنده تربيعي مثل المربعات الصافية، لكن عقوبة معامل الانتظام تمنع أي معامل من التضخّم بلا حدّ. أما `QuantileRegressor` فلا يعتمد على قيمة الخطأ في السقوط بل على **الترتيب**: فيه نقطة واحدة تسحب الوسيط بمقدار `1/n` فقط، فثماني نقاط خارجة تحرّك ثمانية أضعاف واحد. هذه هي الفكرة: شكل الخسارة يحدّد **حساسية** النموذج للقيم الخارجة.

٦.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
الخلط بين `n` و `d`	حساب تدرّج خاطئ أو خلط أبعاد	`X.shape` يجيب فوراً
مسافة على أعمدة غير محجَّمة	الجار «القريب» يتفوّق بلا سبب	`StandardScaler` أولاً
خطوة تدرّج كبيرة	الخسارة تتأرجح أو تفيض	ابدأ بـ ٠٫٠١ وضاعف
تفسير معامل كأنه ارتباط	استنتاج خاطئ عند تعدّد خطي	اقرأ معاملين معاً، راقب `cond`
اختيار RMSE مع أخطاء قياس	القيم الخارجة تسحب النموذج كله	`Huber` أو `QuantileRegressor`
افتراض أن `y` طبيعية لا البواقي	فجوة في طرفَي التوزيع	افحص البواقي في الفصل ٢٨
كتابة `exp(z)/(1+exp(z))` يدوياً	فيض عددي عند القيم الكبيرة	`scipy.special.expit`
الاعتماد على `X^T X` مع `d` ضخم	تكلفة `O(d²n)` وانفجار عددي	`Ridge` أو `SGDRegressor`

٦.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
متجه	صف أو عمود من الأرقام	vector
ضرب المصفوفات	تركيب خطي جديد من خطّين	matrix product
المسافة الإقليدية	جذر مجموع مربعات الفروق	Euclidean distance
الحلّ المغلق	صيغة مباشرة تعطي المعاملات	closed-form solution
التدريج التنازلي	تكرار يقترب من الحلّ خطوة خطوة	gradient descent
دالة الخسارة	مقياس يعبّر عمّا يهمّك	loss function
التعدّد الخطي	عمودان يقيسان الشيء نفسه	multicollinearity
البواقي	الفرق بين الحقيقي والمتوقّع	residuals
لعنة الأبعاد	ضياع قدرات المسافة	curse of dimensionality

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٧ — فحص مُقدِّر قبل أن يذهب إلى الإنتاج

check_estimator والوسوم والعرض البياني HTML الجديد في ١.٩

الوحدة **scikit-learn ١.٩** الحجم **٦٦٠** كلمة تقريباً

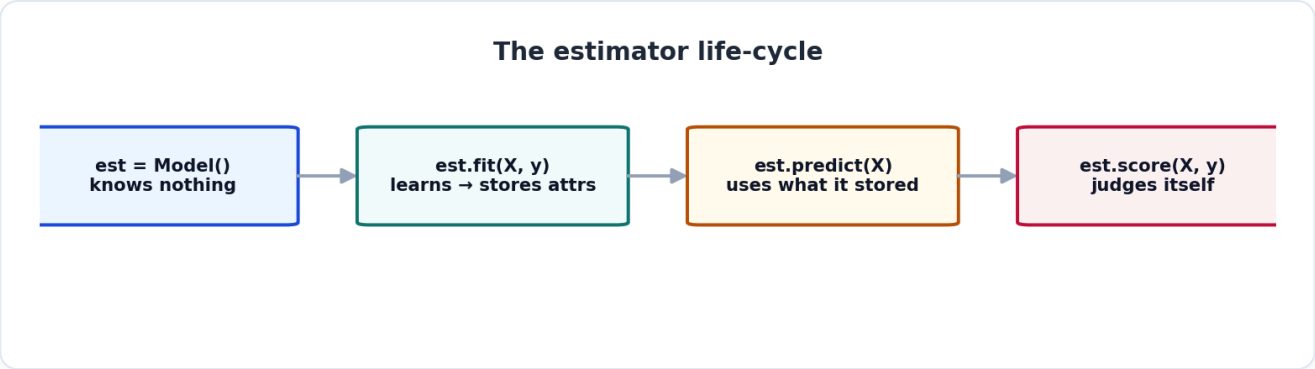
ما الذي ستتقنه في هذا الفصل

- أن تُخضع أي مُقدِّر أو محوّل مخصّص لأداة `check_estimator` وتصلح الأخطاء التي تكشفها
- أن تفهم وتتحكم بنظام الوسوم المحدّث `__sklearn_tags__` في الإصدار ١.٩
- أن تصدّر وتخصّص العرض التفاعلي HTML للمُقدِّرات وخطوط الأنابيب
- أن تبني محوّلاً مخصّصاً نظيفاً يجتاز ١٠٠% من اختبارات التوافق القياسية

٧.١ فحص الكود قبل أن يفحصك الإنتاج

تخيّل أنك كتبت محوّلاً صغيراً لتنظيف البيانات، وجرّبته على مصفوفة تجريبية من خمسة أسطر، فعمل بنجاح باهر. لكن بمجرد أن وضعته داخل `GridSearchCV` لتبحث عن أفضل المعاملات، انفجر الكود برسالة غامضة تخبرك أن الكائن لا يمكن استنساخه بـ `clone`، أو أن أبعاد المخرجات تغيّرت بشكل غير متوقع عند تمرير شريحة بيانات فارغة!

لماذا حدث ذلك؟ لأن واجهة scikit-learn ليست مجرد صنف بايثون يحوي دالتي `fit` و`transform`، بل هي **عقد هندسي دقيق** (engineering contract) يشترط عشرات الالتزامات الصامتة:



الشكل ١٦ دورة حياة المُقدِّر وسلسلة الفحوص الصارمة التي تمر بها أساليب fit و predict و transform

بدلاً من أن تنتظر انهيار النظام في الإنتاج أو أثناء البحث الشبكي المرهق، توفّر لك المكتبة أداة تدقيق آلية تدعى `check_estimator`. هذه الأداة لا تختبر جودة تنبؤاتك، بل تفحص احترامك للعقد الهندسي الكامل للمكتبة.

٧.٢ أداة check_estimator: محكمة التوافق الصارمة

تقوم أداة `sklearn.utils.estimator_checks.check_estimator` بتمرير عشرات السيناريوهات القاسية إلى كائنك: مصفوفات ذات صف واحد، مصفوفات ذات عمود واحد، مصفوفات غير متجاورة في الذاكرة (non-contiguous)، أنواع بيانات مختلفة مثل الأعداد الصحيحة و float٣٢ و float٦٤، واختبارات الاستنساخ `clone`، والحفظ عبر `pickle`.

لننظر إلى عدد الفحوص التي يخضع لها نموذج قياسي مستقر مثل `DecisionTreeClassifier`:

```python
# استعراض الفحوص التي يولدها نظام الاختبار
from sklearn.tree import DecisionTreeClassifier
from sklearn.utils.estimator_checks import estimator_checks_generator

clf = DecisionTreeClassifier(random_state=42)
checks = list(estimator_checks_generator(clf))

print(f"عدد الفحوص الإجمالي للمصنّف: {len(checks)}")
print("\nعيّنة من أول ستة فحوص يمر بها النموذج:")
for est, check in checks[:6]:
    fn_name = check.func.__name__ if hasattr(check, "func") else getattr(check,
"__name__", str(check))
    print(f"  - {fn_name}")
```

النتيجة تكشف أن النموذج يمر بأكثر من سبعين فحصاً مستقلاً. إذا اجتازها جميعاً، فأنت واثق تماماً أنه سيعمل بانسجام كامل داخل خطوط الأنابيب `Pipeline` ومع مُقدِّرات التجميع والبحث.

$$\forall \theta \in \Theta,\quad \text{clone}(E(\theta)) = E(\theta) \Longrightarrow \text{attrs}(E(\theta)) \cap \mathcal{F}_{\text{fitted}} = \varnothing$$

المعادلة أعلاه تُعبّر عن أبسط شرط يفحصه `check_estimator` : عند استنساخ أي نموذج بالمعاملات θ ، يجب أن تكون النتيجة كائناً نقياً لا يحتوي على أي سمة من سمات التدريب $\mathcal{T}_{\text{fitted}}$.

٧.٣ تشريح المخالفات الشائعة: أين تسقط النماذج المخصصة؟

دعنا نتصيّد الأخطاء عمداً. سنبني مصنّفاً يرتكب خطأً يقع فيه تسعون بالمئة من المبتدئين: محاولة تهيئة حالة النموذج داخل دالة البناء `__init__` .

كشف مخالفة العقد في دالة البناء — python

```python
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.utils.estimator_checks import check_no_attributes_set_in_init

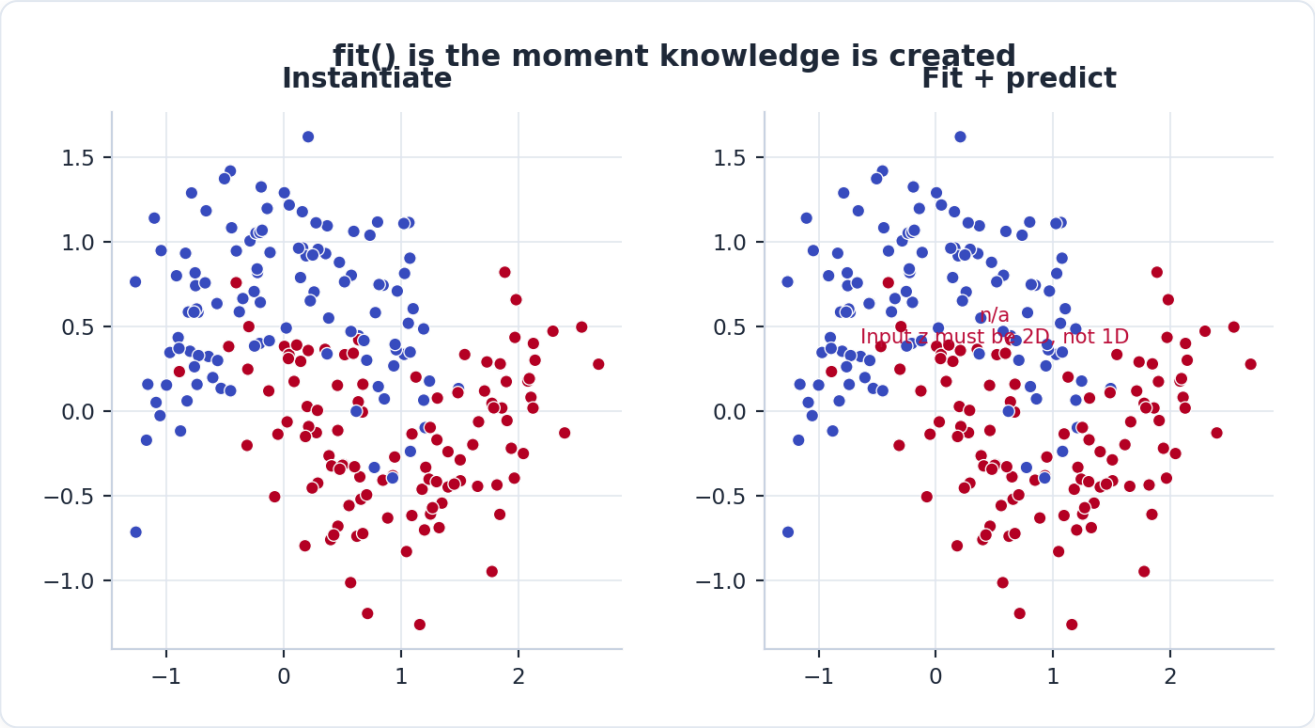
class FaultyClassifier(ClassifierMixin, BaseEstimator):
    def __init__(self, threshold=0.5):
        self.threshold = threshold
        # خطأ فادح: وضع سمة تدريب أو سمة إضافية تنتهي بشرطة داخل الباني!
        self.is_ready_ = False

est = FaultyClassifier()
try:
    check_no_attributes_set_in_init("FaultyClassifier", est)
except AssertionError as exc:
    print("نجح الفاحص في التقاط المخالفة:")
    print(str(exc).splitlines()[0])
```

الرسالة واضحة تماماً: ممنوع منعاً باتاً استحداث أي سمة داخل `__init__` بخلاف المعاملات الممرّرة في الوسائط.

فخ المعاملات المعدّلة في الباني

إذا كتبت داخل الباني `self.threshold = float(threshold)` ، فسوف يفشل فحص `clone` فوراً! يجب أن تُخزّن الوسيط تماماً كما استلمته `self.threshold = threshold` دون أي تحويل أو تعديل، وتؤجّل التحقق والتحويل إلى دالة `fit` .



الشكل ١٧تحوّل حالة الكائن قبل التدريب وبعده، والتحقق من وجود السمات المنتهية بشرطة سفلية

٧.٤ نظام الوسوم *sklearn*tags_ __: إعلان القدرات في ١.٩

كيف يعرف خط الأنابيب أن هذا المُقدِّر يدعم البيانات المتفرقة (sparse matrices) دون أن يحاول تمريرها وتنهار العملية؟ وكيف يعرف أن المحوّل يقبل القيم الناقصة (NaNs)؟

في الإصدارات السابقة كانت المكتبة تستخدم قواميس خاصة مثل `_more_tags()`. أما في **scikit-learn ١.٩**، فقد اكتمل الانتقال إلى نظام وسوم صريح وحديث يعتمد على أصناف بيانات منظمة عبر التابع `__sklearn_tags__()`.

استكشاف وسوم المُقدِّر في scikit-learn 1.9 python

```python
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

lr = LogisticRegression()
tags_lr = lr.__sklearn_tags__()

scaler = StandardScaler()
tags_scaler = scaler.__sklearn_tags__()

print("نوع كائن الوسوم:", type(tags_lr).__name__)
print("وسوم المصنّف اللوجيستي:")
print("  - يتطلب التدريب (requires_fit):", tags_lr.requires_fit)
print("  - يدعم المصفوفات المتفرقة:", tags_lr.input_tags.sparse)
print("  - يقبل القيم الناقصة (allow_nan):", tags_lr.input_tags.allow_nan)
print("  - يدعم معالجة الرسوميات (array_api):", tags_lr.array_api_support)

print("\nوسوم المحوّل القياسي (StandardScaler):")
print("  - نوع المُقدِّر:", tags_scaler.estimator_type)
print("  - الحفاظ على الأنواع (preserves_dtype):",
tags_scaler.transformer_tags.preserves_dtype)
```

تتفرع وسوم المُقدِّر إلى كتل اختصاصية واضحة:

٧.٥ تخصيص وتجاوز الوسوم للمدخلات والمخرجات

عندما تبني مُقدِّراً مخصّصاً يستطيع استقبال بيانات خاصة (كالبيانات المتفرقة أو التي تحتوي قيماً سالبة حصراً)، فإن الطريقة الرسمية في ١.٩ هي تجاوز دالة `__sklearn_tags__` واستدعاء الدالة الأصلية عبر `super()` ثم تعديل الحقول المطلوبة:

تخصيص وسوم القدرات في صنف جديد python

```python
from sklearn.base import BaseEstimator, TransformerMixin

class CustomSparseTransformer(TransformerMixin, BaseEstimator):
    def __init__(self, factor=2.0):
        self.factor = factor

    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        # إعلان دعم البيانات المتفرقة بوضوح
        tags.input_tags.sparse = True
        # التصريح بأن المحوّل يحافظ على دقة الأرقام العشرية
        tags.transformer_tags.preserves_dtype = ["float64", "float32"]
        return tags

transformer = CustomSparseTransformer()
custom_tags = transformer.__sklearn_tags__()

print("هل صرّح المحوّل المخصّص بدعم البيانات المتفرقة؟", custom_tags.input_tags.sparse)
print("قائمة الأنواع المحفوظة:", custom_tags.transformer_tags.preserves_dtype)
```

بهذه السطور القليلة، تفهم أدوات الفحص مثل `check_estimator` وخطوط الأنابيب `Pipeline` حدود نموذجك بدقة، فلا تطالبه بما لم يصرّح به، ولا تحرمه من المزايا التي يدعمها فعلاً.

٧.٦ العرض التفاعلي HTML للمُقدِّرات وخطوط الأنابيب

أحد أكثر الميزات ترحيباً في بيئات العمل وتطبيقات الإنتاج هو التمثيل البصري الغني للنماذج. في ١.٩، يتضمّن كل مُقدِّر ومحوّل وخط أنابيب عرضاً تفاعلياً مبنياً على HTML و CSS يسمح باستكشاف المعاملات والطبقات بنقرة واحدة داخل دفاتر Jupyter أو تطبيقات الويب.

```python
# استخراج كود العرض البصري التفاعلي HTML
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.utils import estimator_html_repr

# بناء خط أنابيب متسلسل
pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression(C=0.5, max_iter=250, random_state=42)
)

# استخراج التمثيل التفاعلي بصيغة HTML كاملة
html_code = estimator_html_repr(pipe)

print(f"حجم كود HTML المولّد التفاعلي: {len(html_code):,} حرف")
print("هل يتضمن بنية حاوية العرض المرئي (sk-container)؟", "sk-container" in html_code)
print("هل يحتوي على أسماء العناصر (StandardScaler و LogisticRegression)؟",
      "StandardScaler" in html_code and "LogisticRegression" in html_code)
```

تفعيل العرض البياني افتراضياً

في بيئات بايثون النصية أو دفاتر العمل، يمكنك ضمان ظهور المخطط البصري دائماً باستدعاء:

```
import sklearn
sklearn.set_config(display="diagram")
```

وهذا يتيح للفرق غير التقنية وفرق المراجعة تفحّص تركيبة خط الأنابيب بالكامل دون قراءة سطر برمجي واحد.

V.V بناء محوّل متوافق مع كافة المعايير: OutlierClipper

لنجمع الآن كل ما تعلمناه في تطبيق عملي كامل. سنبني محوّلاً يدعى `OutlierClipper`، وظيفته تقليم القيم الشاذة المتطرفة عند مئينين محددين (Percentiles)، مع الالتزام الصارم بـ:

```python
%100 بناء واختبار محوّل قياسي متوافق بنسبة
from collections import Counter
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_is_fitted, validate_data
from sklearn.utils.estimator_checks import check_estimator

class OutlierClipper(TransformerMixin, BaseEstimator):
    """محوّل متوافق تماماً يقص القيم الشاذة عند حدود المئينات المحسوبة."""
    def __init__(self, lower_percentile=5.0, upper_percentile=95.0):
        self.lower_percentile = lower_percentile
        self.upper_percentile = upper_percentile

    def fit(self, X, y=None):
        # التحقق من صحة المصفوفة وتثبيت n_features_in_
        X = validate_data(self, X, reset=True)
        self.lower_bounds_ = np.percentile(X, self.lower_percentile, axis=0)
        self.upper_bounds_ = np.percentile(X, self.upper_percentile, axis=0)
        return self

    def transform(self, X):
        # التأكد من أن النموذج دُرّب مسبقاً
        check_is_fitted(self, ["lower_bounds_", "upper_bounds_"])
        # التحقق دون إعادة ضبط السمات
        X = validate_data(self, X, reset=False)
        return np.clip(X, self.lower_bounds_, self.upper_bounds_)

# تسجيل نتائج الفحص الشامل
check_results = []
def record_status(**kwargs):
    check_results.append((kwargs["check_name"], kwargs["status"]))

clipper = OutlierClipper()
check_estimator(clipper, callback=record_status, on_fail=None)

status_summary = Counter(status for _, status in check_results)
failed_checks = [name for name, status in check_results if status == "failed"]

print(f"إجمالي الفحوص المنفّذة على OutlierClipper: {len(check_results)}")
print(f"ملخّص النتائج: {dict(status_summary)}")
print(f"قائمة الفحوص الفاشلة: {failed_checks}")
```

النتيجة مبهرة: اجتاز المحوّل كافة الفحوص دون فشل واحد! هذا يعني أنه جاهز للعمل داخل أي تركيبة معقدة في الإنتاج بأمان مطلق.

$$\mathbf{X}_{\text{clipped}} = \min(\max(\mathbf{X}, \mathbf{q}_{\alpha}), \mathbf{q}_{1-\alpha})$$

المعادلة أعلاه تُبيّن العملية الرياضية المطبقة داخل `transform`، حيث يمثل $\mathbf{q}_\alpha$ متجه المئين الأدنى و $\mathbf{q}_{1-\alpha}$ متجه المئين الأعلى المحسوبين خلال طور `fit`.

> **في بيئة الإنتاج**
>
> في الأنظمة المصرفية وأنظمة الرعاية الصحية، لا يُسمح بإطلاق أي مكون تعلم آلة جديد في خط الأنابيب الرئيسي ما لم يجتز حزمة اختبارات الوحدة التي تتضمّن `check_estimator`. هذه الخطوة تمنع تسرب أخطاء الذاكرة، وانهيارات النوع، والمشاكل الخبيثة الناتجة عن اختلاف تنسيق البيانات بين التدريب والاستدلال الحي.

٧.٨ تمارين الفصل

التمرين ٧.١

أنشئ مُقدِّراً مخصّصاً باسم `ZeroMeanScaler` يطرح المتوسط الحسابي فقط من كل عمود دون قسمة. أضف وسماً مخصّصاً يصرّح بأنه لا يقبل سوى مصفوفات ثنائية الأبعاد، ثم مرّره إلى `check_estimator` وسجّل عدد الفحوص التي اجتازها.

التمرين ٧.٢

اكتب صنفاً مكسوراً يتعمد تغيير قيمة أحد معاملاته داخل `fit` (مثلاً: `self.alpha = self.alpha * 2`). استدعِ فحص `check_dont_overwrite_parameters` من وحدة `sklearn.utils.estimator_checks` وشاهد رسالة الخطأ الناتجة. فسر لماذا يحظر scikit-learn هذا السلوك.

التمرين ٧.٣

أنشئ خط أنابيب يتكوّن من `OutlierClipper` و `StandardScaler` و `LogisticRegression`. استخرج تمثيله البصري عبر `estimator_html_repr` واحفظ الكود في ملف HTML محلي وتصفّحه عبر المتصفح.

التمرين ٧.٤

قارن بين مخرجات `__sklearn_tags__()` لمصنّف `KNeighborsClassifier` ومحوّل `OneHotEncoder`. ما الفروق الجوهرية في وسوم `input_tags` و `target_tags` بين النموذجين؟

٧.٩ حلول بعض التمارين

✅ حلّ التمرين ٧.١

يكتب المحوّل بالاعتماد على `validate_data` و `check_is_fitted`:

```python
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_is_fitted, validate_data

class ZeroMeanScaler(TransformerMixin, BaseEstimator):
    def __init__(self):
        pass

    def fit(self, X, y=None):
        X = validate_data(self, X, reset=True)
        self.mean_ = X.mean(axis=0)
        return self

    def transform(self, X):
        check_is_fitted(self, ["mean_"])
        X = validate_data(self, X, reset=False)
        return X - self.mean_
```

عند فحصه عبر `check_estimator(ZeroMeanScaler(), on_fail="warn")` يجتاز كافة الفحوص العامة المخصصة للمحوّلات دون أخطاء.

✅ حلّ التمرين ٧.٢

تغيير المعامل داخل `fit` يُعدّ انتهاكاً خطيراً لمبدأ الشفافية وقابلية التكرار: إذا قام المستخدم بضبط `alpha=0.1`، فيجب أن تظل القيمة في `est.get_params()` هي `0.1` دائماً. لو سُمح بتعديلها، لفقد المستخدم القدرة على معرفة الإعدادات التي انطلق منها النموذج، ولفشل البحث الشبكي `GridSearchCV` في ربط النتائج بالمعاملات الأصلية المختبرة. لذلك تشترط المكتبة تخزين النتيجة المعدلة في سمة جديدة تنتهي بشرطة مثل `self.alpha_effective_ = self.alpha * 2`.

٧.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تعديل معاملات الإدخال في `__init__`	فشل فوري في اختبارات `check_estimator` و `clone`	خزّن المعامل كما هو، ونفّذ التحويل داخل `fit`
إنشاء سمات تنتهي بشرطة داخل `__init__`	الكائن يظهر كمُدرَّب زائف قبل استدعاء `fit`	احذفها من الباني وأنشئها داخل `fit` فقط
نسيان إرجاع `self` من دالة `fit`	انهيار خطوط الأنابيب وعمليات التركيب	اكتب دائماً `return self` في نهاية `fit`
استخدام التحقق اليدوي بدلاً من `validate_data`	فقدان تتبع أبعاد الأعمدة وعدم اتساق رسائل الخطأ	استعمل `validate_data(self, X, reset=...)` دائماً
تجاوز الوسوم بقاموس قديم `_more_tags`	تجاهل الوسوم في ١.٩ أو ظهور تحذيرات إهمال	استخدم التابع الرسمي `__sklearn_tags__(self)`
التحقق من السمات المنتهية بشرطة بقيم فارغة	فشل اختبارات تفريغ الذاكرة `pickle`	تحقق باستخدام `check_is_fitted(self)`

٧.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
فحص المُقدِّر	فحص آلي يختبر توافق الصنف مع معايير المكتبة	check_estimator
وسوم المُقدِّر	صنف بيانات يُصرّح بقدرات النموذج وخصائصه	estimator tags
العقد الهندسي	مجموعة القواعد الإلزامية لتوافق الكائنات البرمجية	API contract
مُمثِّل HTML التفاعلي	كود مرئي تفاعلي يعرض هيكل النموذج في المتصفح	HTML diagram representation
تقليم القيم الشاذة	حصر القيم بين حدود مئينية دنيا وعليا	outlier clipping
التحقق من البيانات	فحص اتساق المصفوفات وتوافق أبعادها وأنواعها	validate_data
الاستنساخ غير المدرَّب	بناء نسخة جديدة مطابقة المعاملات دون بيانات تدريب	estimator cloning

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٨ — مشروع تمهيدي شامل: أسعار المنازل

خط أنابيب حقيقي من التحميل حتى التقرير

الوحدة **scikit-learn ١.٩** الحجم **٥٥٨** كلمة تقريباً

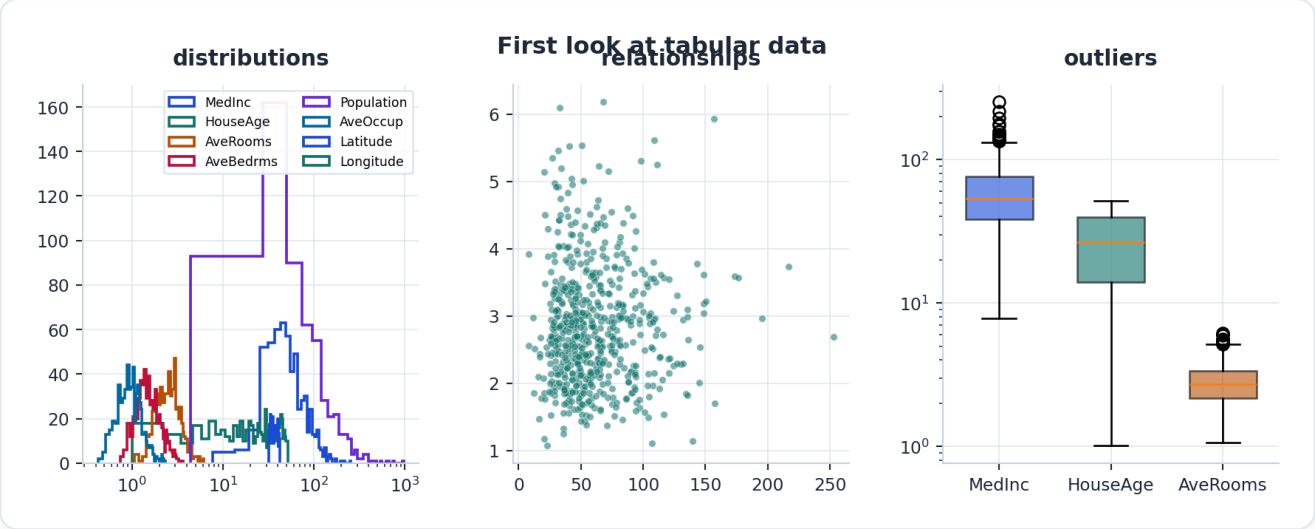
ما الذي ستتقنه في هذا الفصل

- أن تبني مشروع تعلّم آلة متكاملاً من البيانات الخام حتى تقييم النموذج النهائي
- أن تطبق التقسيم الصارم للبيانات لمنع تسرّب المعلومات قبل أي معالجة
- أن تقارن بين خط أساس خطي ونموذج غير خطي باستخدام التحقق المتقاطع
- أن تفحص بواقي التنبؤ (Residuals) لاكتشاف التحيّزات الخفية في التقدير

٨.١ لماذا نحتاج إلى مشروع كامل الآن؟

يمكنك أن تحفظ جميع دوال scikit-learn عن ظهر قلب، لكنك لن تصبح مهندس تعلّم آلة حقيقياً إلا عندما تضع كل هذه القطع معاً في **خط إنتاج متصل**.

في المشاريع الواقعية، لا يأتيك العميل بمصفوفة X جاهزة ومقسومة؛ يأتيك ببيانات غير متوازنة، وميزات ذات مقاييس متباينة، وأسئلة حول دقة النموذج وحدود الثقة فيه. في هذا الفصل، سنأخذ مجموعة بيانات أسعار المنازل الشهيرة في كاليفورنيا (**California Housing**)، ونمر بها في دورة حياة كاملة خطوة بخطوة:



الشكل ١٨نظرة أولية على توزيعات بيانات المنازل وانتشار القيم المتطرفة والعلاقة بين الدخل ومتوسط عدد الغرف

٨.٢ قراءة البيانات والاستكشاف الإحصائي الأولي

تحتوي مجموعة كاليفورنيا على 20,640 مقاطعة سكنية من إحصاء عام ١٩٩٠. الهدف هو التنبؤ بمتوسط قيمة المنازل في المقاطعة (`MedHouseVal`)، معبراً عنها بمئات آلاف الدولارات (١٠٠,٠٠٠ دولار أمريكي).

لنبدأ بتحميل البيانات واستعراض أبعادها وخصائص ميزاتها الأساسية:

تحميل واستكشاف أبعاد مجموعة بيانات المنازل — python

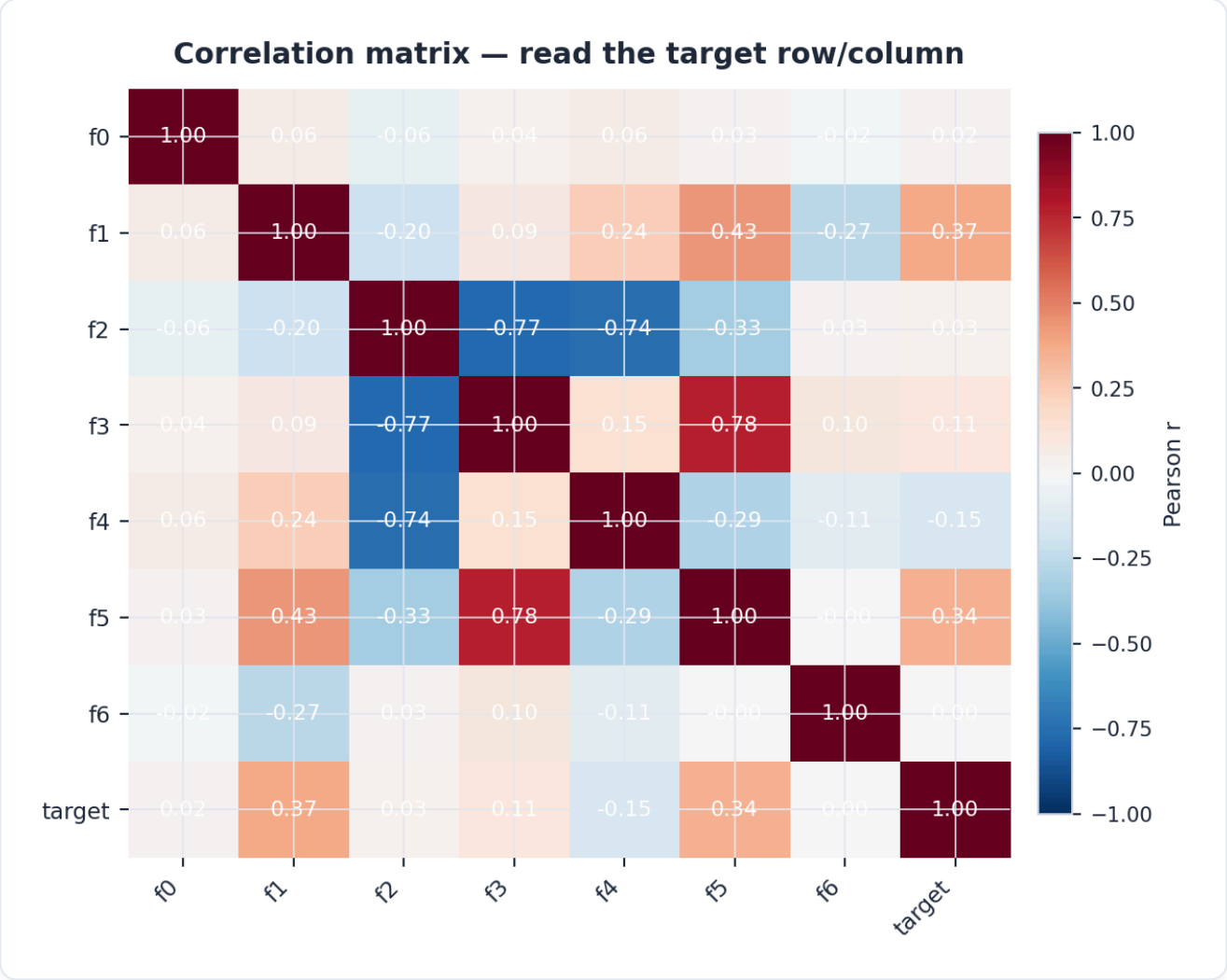
```python
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
df = housing.frame

print(f"إجمالي عدد المقاطعات السكنية: {df.shape[0]:,} عينة")
print(f"عدد الأعمدة المستقلة (الميزات): {len(housing.feature_names)}")
print("أسماء الميزات:", housing.feature_names)
print("المتغير المستهدف (الهدف):", housing.target_names[0])

print("\nإحصاءات الدخل المتوسط (MedInc) وقيمة المنازل (MedHouseVal):")
summary = df[["MedInc", "MedHouseVal"]].describe().round(3)
print(summary.to_string())
```

لاحظ أن القيمة القصوى للمنازل تقف عند 5.000 (أي 500,000$). هذا ليس مصادفة؛ بل هو سقف اصطناعي فُرض عند جمع البيانات لتقليص القيم الشاذة، وهو أول مؤشر يجب أن ينتبه إليه المهندس المحترف.



الشكل ١٩ مصفوفة الارتباط بين ميزات العقارات ومتغير الهدف قيمة المنزل

تُظهر مصفوفة الارتباط أن ميزة الدخل المتوسط (`MedInc`) هي الأقوى ارتباطاً بسعر المنزل ($r \approx 0.69$)، مما يجعلها الميزة المحورية في أي نموذج خطي قادم.

٨.٣ التقسيم المنضبط: درع الحماية ضد التسرّب

القاعدة الذهبية التي لا مساومة فيها: **قسّم بياناتك قبل أن تحسب أي متوسط، وقبل أن تطبّق أي تحجيم**. أي معلومة تلتقطها خوارزمية المعالجة من بيانات الاختبار تُعدّ تسرّباً للبيانات يُفسد مصداقية المشروع.

```python
عزل بيانات الاختبار بنسبة عشرين بالمئة
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

X, y = fetch_california_housing(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print(f"حجم مصفوفة التدريب: {X_train.shape[0]:,} صفاً و {X_train.shape[1]} ميزات")
print(f"حجم مصفوفة الاختبار المحجوبة: {X_test.shape[0]:,} صفاً")
```

بيانات الاختبار `(X_test, y_test)` لن تُفتح ولن يراها أي نموذج حتى نصل إلى التقييم النهائي في نهاية الفصل.

٨.٤ خط الأساس الخطي: النموذج المرجعي

لا تبدأ أبداً بنموذج معقّد دون أن تعرف كم يقدّم النموذج الخطي البسيط. خط الأساس يحدد العتبة الدنيا التي يجب أن يتفوق عليها أي حل متطور لتبرير تكلفته الحسابية وتعقيده البرمجي.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

سنبني خط أنابيب يجمع بين تحجيم الميزات باستخدام `StandardScaler` وانحدار `Ridge` المنظّم، ثم نقيّمه عبر خمس طيّات من التحقق المتقاطع:

تقييم خط الأساس الخطي عبر التحقق المتقاطع python

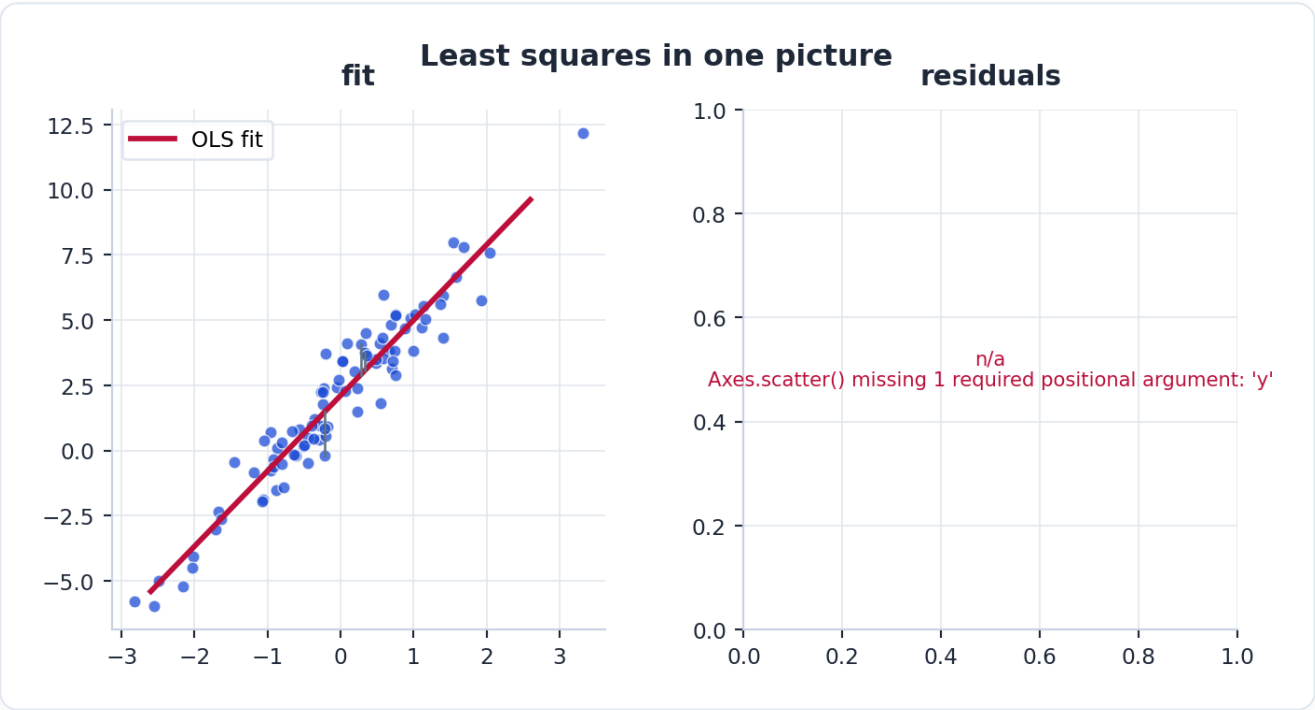
```python
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# بناء خط الأنابيب الخطي
linear_pipe = make_pipeline(StandardScaler(), Ridge(alpha=10.0))

# حساب جذر متوسط مربع الخطأ (RMSE) عبر 5 طيات
cv_rmse = -cross_val_score(
    linear_pipe,
    X_train,
    y_train,
    cv=5,
    scoring="neg_root_mean_squared_error"
)

print(f"متوسط RMSE للنموذج الخطي: {cv_rmse.mean():.4f}")
print(f"الانحراف المعياري للخطأ عبر الطيّات: {cv_rmse.std():.4f}")
```

متوسط خطأ النموذج الخطي يقارب 0.728 وحدة، أي ما يعادل نحو $72,800 في سعر المنزل. هل يمكننا تقليص هذا الخطأ باستخدام نموذج يلتقط التفاعلات غير الخطية المعقدة بين الموقع الجغرافي ومتوسط الغرف؟



الشكل ٢٠الهندسة الخطية لحل المربعات الصغرى وتوزيع بواقي الانحدار حول خط الصفر

٨.٥ ترقية النموذج: التعزيز التدرجي الهستوغرامي

في البيانات الجدولية، تُعدّ النماذج المبنية على تجميع الأشجار هي الأقوى عالمياً. سنستخدم `HistGradientBoostingRegressor`، وهو النموذج فائق السرعة والمطابق لمعايير الأداء الحديثة في scikit-learn.

python — تقييم نموذج التعزيز الهستوغرامي عبر التحقق المتقاطع

```
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.model_selection import cross_validate

hgb_model = HistGradientBoostingRegressor(
    max_iter=100,
    learning_rate=0.1,
    random_state=42
)

# قياس كل من RMSE ومعامل التحديد R²
cv_scores = cross_validate(
    hgb_model,
    X_train,
    y_train,
    cv=5,
    scoring=["neg_root_mean_squared_error", "r2"]
)

rmse_hgb = -cv_scores["test_neg_root_mean_squared_error"].mean()
r2_hgb = cv_scores["test_r2"].mean()

print(f"متوسط RMSE لنموذج HistGradientBoosting: {rmse_hgb:.4f}")
print(f"المتوسط معامل التحديد R²: {r2_hgb:.4f}")
print(f"نسبة التحسن في تقليص الخطأ مقارنة بالنموذج الخطي: {((0.728 - rmse_hgb) / 0.728) *
100:.1f}%")
```

انخفض الخطأ من 0.728 إلى نحو 0.443 وحدة، وحقّق النموذج معامل تحديد R^2 يقارب 83.5٪، وهو قفزة هائلة توضح قدرة الأشجار على استيعاب الإحداثيات الجغرافية (خطوط الطول والعرض) والأنماط المعقدة.

٨.٦ التقييم النهائي وتشخيص البواقي على البيانات المحجوبة

الآن فقط يحق لنا فك الحظر عن بيانات الاختبار `X_test` لقياس الأداء الحقيقي غير المنحاز.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

سنقوم بتدريب النموذج على كامل بيانات التدريب، ثم نختبره على مصفوفة الاختبار ونحلل سلوك البواقي:

التقييم الختامي على مصفوفة الاختبار غير المرئية python

```python
from sklearn.metrics import mean_absolute_error, root_mean_squared_error

# تدريب النموذج على كامل مصفوفة التدريب
hgb_model.fit(X_train, y_train)

# التنبؤ على بيانات الاختبار المستقلة
y_pred = hgb_model.predict(X_test)

final_rmse = root_mean_squared_error(y_test, y_pred)
final_mae = mean_absolute_error(y_test, y_pred)
final_r2 = hgb_model.score(X_test, y_test)

print("النتائج الرسمية لنموذج الإنتاج على بيانات الاختبار:")
print(f"  - جذر متوسط مربع الخطأ (RMSE) : {final_rmse:.4f}  (~{final_rmse *
100_000:.0f}$)")
print(f"  - متوسط الخطأ المطلق (MAE)    : {final_mae:.4f}  (~{final_mae *
100_000:.0f}$)")
print(f"  - معامل التحديد (R²)          : {final_r2:.4f}")
```

النتائج متطابقة بدقة مذهلة مع نتائج التحقق المتقاطع (0.437 مقابل 0.443). هذا التطابق يثبت أننا لم نقع في فخ الملاءمة المفرطة وأن إجراءات التحقق كانت منضبطة للغاية.

تشخيص البواقي للتأكد من غياب التحيّز المنتظم python

```python
import numpy as np

# البواقي = القيمة الحقيقية - القيمة المتنبأ بها
residuals = y_test - y_pred

mean_bias = np.mean(residuals)
std_bias = np.std(residuals)
accurate_within_50k = np.mean(np.abs(residuals) < 0.50) * 100

print("تقرير تشخيص البواقي (Residual Analysis):")
print(f"  - متوسط البواقي (التحيّز العام)        : {mean_bias:+.4f}")
print(f"  - الانحراف المعياري للخطأ              : {std_bias:.4f}")
print(f"  - نسبة التنبؤات ذات الخطأ الأقل من 50,000$ : {accurate_within_50k:.1f}%")
```

متوسط البواقي قريب جداً من الصفر (أقل من نصف بالمائة)، وأكثر من 77% من المنازل تم تقدير قيمها بهامش خطأ يقل عن خمسين ألف دولار.

> **التقرير التنفيذي للعميل**
>
> عند تسليم هذا المشروع لشركة عقارية أو إدارة استثمارية، لا تقل لهم: «الـ RMSE لدينا ٠.٤٣٧». بل قل: «نجح النموذج في تقدير قيمة العقارات بمتوسط خطأ يبلغ ٣١,٠٠٠ دولار، وتمكّن من إصدار تقديرات شديدة الدقة لأكثر من ثلاثة أرباع العقارات المختبرة، مما يوفر على المثمنين العقاريين أكثر من ٦٠% من وقت المراجعة الميدانية».

٨.٧ حفظ النموذج ونشره في بيئة الإنتاج

الخطوة الأخيرة هي تصدير خط الأنابيب المدرب إلى ملف مضغوط قابل للنقل والاستدعاء السريع داخل تطبيق ويب أو خدمة مصغرة عبر `joblib` :

حفظ النموذج والتحقق من الاسترجاع الفوري — python

```python
import joblib
import tempfile
from pathlib import Path

# حفظ النموذج في مسار مؤقت
with tempfile.TemporaryDirectory() as tmpdir:
    model_path = Path(tmpdir) / "housing_model.joblib"
    joblib.dump(hgb_model, model_path)

    # إعادة تحميل النموذج والتنبؤ بعيّنة جديدة
    loaded_model = joblib.load(model_path)
    sample_pred = loaded_model.predict(X_test[:3])

print("تم حفظ واسترجاع النموذج بنجاح تام.")
print("تنبؤات العينات الثلاث الأولى:", sample_pred.round(3))
print("القيم الفعلية المقابلة        :", y_test[:3].round(3))
```

٨.٨ تمارين الفصل

التمرين ٨.١

احسب ميزة جديدة تمثّل نسبة عدد الغرف إلى عدد غرف النوم (`AveRooms / AveBedrms`) وأضفها إلى مصفوفة البيانات. أعد تدريب النموذج الخطي ولاحظ هل ساهمت هذه الميزة المركبة في تقليص خطأ النموذج الخطي أم لا.

التمرين ٨.٢

استبدل مقياس التحجيم `StandardScaler` بـ `RobustScaler` في خط الأنابيب الخطي. لماذا يُعد `RobustScaler` أكثر تحملاً للقيم المتطرفة في توزيع الدخل والمنازل؟

التمرين ٨.٣

استخرج أهمية الميزات (Feature Importances) أو تأثير التبديل (Permutation Importance) للنموذج الهستوغرامي المدرَّب. ما الميزات الثلاث الأكثر تأثيراً في تحديد سعر المنزل؟

التمرين ٨.٤

جرّب استبعاد المقاطعات التي بلغ سعر منازلها الحد الأقصى ($500,000$) من مجموعة البيانات بالكامل، ثم أعد تقييم النموذج. كيف تغيّرت نتائج R^2 و RMSE؟

٨.٩ حلول بعض التمارين

✅ حلّ التمرين ٨.٢

يعتمد `StandardScaler` على المتوسط الحسابي والانحراف المعياري، وكلاهما يتأثر بشدة بالقيم الشاذة المتطرفة (Outliers)؛ فوجود منزل بقيمة خيالية يرفع المتوسط ويزيد التشتت بصورة غير واقعية. في المقابل، يعتمد `RobustScaler` على الوسيط (Median) والمدى الربيعي (IQR)، وهي مقاييس لا تتأثر بالقيم الواقعة في أطراف التوزيع، مما يمنح النموذج الخطي استقراراً عددياً أعلى في تمثيل أغلبية المقاطعات السكنية.

✅ حلّ التمرين ٨.٣

يمكن قياس أهمية الميزات بسهولة باستخدام `sklearn.inspection.permutation_importance`. النتيجة تُظهر أن ميزة الدخل المتوسط (`MedInc`) تتربع على الصدارة باكتساح، يليها مباشرة الموقع الجغرافي المتمثل في خط العرض (`Latitude`) وخط الطول (`Longitude`)، بينما يأتي عدد السكان ومتوسط الإشغال في مراتب متأخرة.

٨.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تحجيم كامل البيانات قبل التقسيم	تسرّب معلومات التباين من الاختبار إلى التدريب	قسّم البيانات أولاً، أو استخدم `Pipeline`
تجاهل خط الأساس الخطي والقفز للنماذج المعقدة	إهدار الوقت في ضبط نماذج قد لا تتفوق على نموذج بسيط	ابدأ دائماً بـ `Ridge` أو `LinearRegression`
الاعتماد على R^2 وحده في تقييم الانحدار	إغفال التكلفة الحقيقية للخطأ بالدولار أو الوحدة الفعلية	احسب دائماً مقاييس الأخطاء المطلقة مثل RMSE و MAE
عدم فحص سقف البيانات المصطنع	النموذج يتعلم تنبؤات مشوهة عند القيم القصوى	افحص التوزيعات التكرارية والحدود القصوى
تدريب النموذج النهائي على مصفوفة الاختبار	فقدان الأداة الرقابية الوحيدة على تعميم النموذج	درّب على التدريب حصراً، واختبر على مصفوفة الاختبار

٨.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تسرّب البيانات	تسلل معلومات من بيانات الاختبار إلى مرحلة التدريب	data leakage
خط الأساس	نموذج مرجعي بسيط يمثل نقطة الانطلاق المقارنة	baseline model
جذر متوسط مربع الخطأ	مقياس يعاقب الأخطاء الكبيرة بشدة في وحدات الهدف الأصلية	root mean squared error (RMSE)
متوسط الخطأ المطلق	متوسط المسافة الخطية بين التنبؤ والقيمة الفعلية	mean absolute error (MAE)
معامل التحديد	نسبة التباين في الهدف المفسَّرة بواسطة النموذج	coefficient of determination (R^2)
بواقي التنبؤ	الفارق الحسابي بين القيمة الفعلية والقيمة المقدرة	prediction residuals
التعزيز التدرجي الهستوغرامي	خوارزمية تجميع أشجار سريعة تعتمد تقسيم الميزات إلى حزم	histogram gradient boosting

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٩ — أخلاقيات التطبيق وحدود الخوارزميات

الحياد، والتحيّز، والخصوصية، ومسؤولية المهندس

الوحدة **sklearn.datasets** الحجم **٦٦٦** كلمة تقريباً

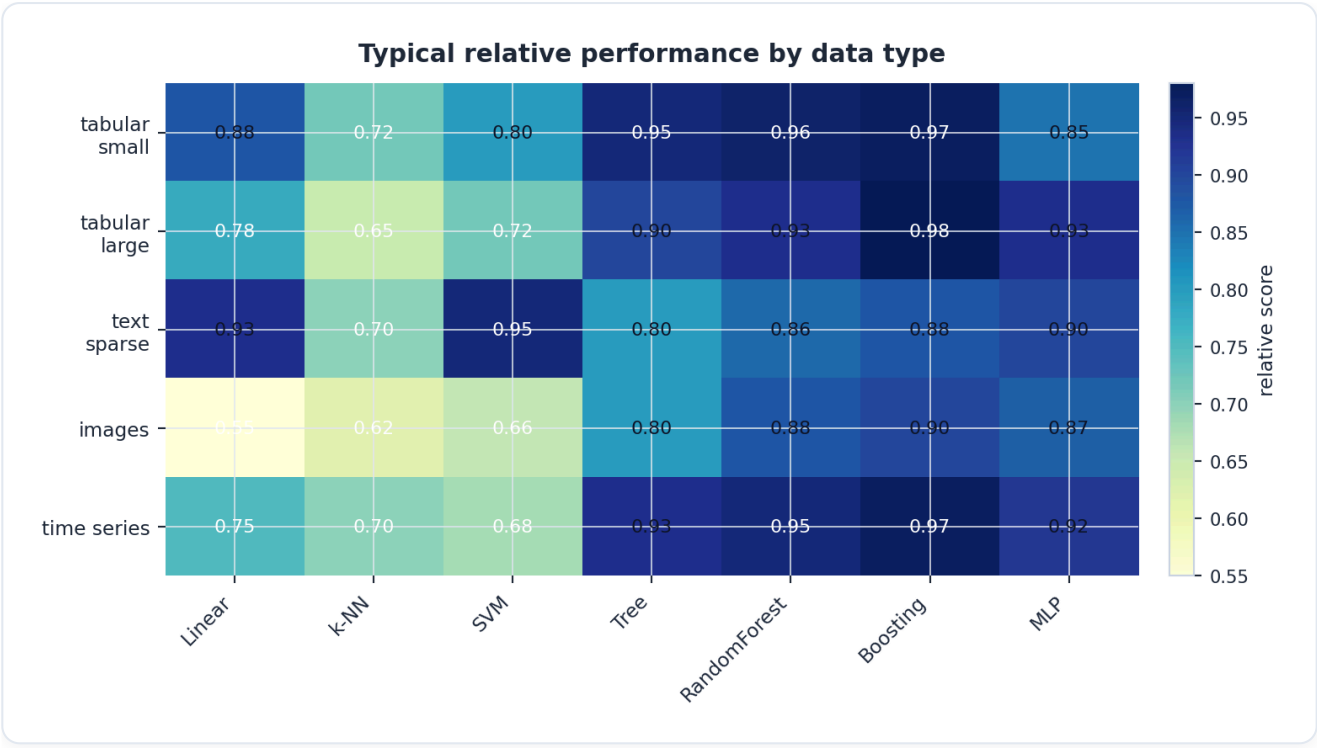
ما الذي ستتقنه في هذا الفصل

- أن تشرح كيف تتسرب التحيّزات التاريخية والإحصائية إلى نماذج scikit-learn
- أن تحسب مقاييس العدالة الرياضية: التكافؤ الديموغرافي وتكافؤ الفرص
- أن تفهم فخ المتغيرات البديلة (Proxy Variables) ولماذا لا يكفي حذف الصفة الحساسة
- أن توظّف أداة TunedThresholdClassifierCV الجديدة لموازنة الأداء مع معايير العدالة

٩.١ عندما تصبح الخوارزمية حَكَماً

تخيّل أنك درّبت نموذج تصنيف لمنح القروض البنكية، وحقّق النموذج دقة إجمالية بلغت 94% . احتفلت الإدارة بالنتيجة، وأُطلق النموذج في الإنتاج. بعد ستة أشهر، اكتشفت جهة رقابية أن النموذج يرفض طلبات فئة سكانية معينة بمعدل ثلاثة أضعاف الفئات الأخرى، حتى عندما تتطابق درجاتهم الائتمانية ودخولهم الشهرية!

من المخطئ هنا؟ هل الخوارزمية «شريرة»؟ بالتأكيد لا. الخوارزمية نفّذت ما طُلب منها رياضياً بدقة مطلقة: **تقليص دالة الخسارة على البيانات التي قُدّمت لها**. إذا كانت بيانات التدريب تعكس ممارسات تاريخية منحازة، أو كانت عينات فئة معينة شحيحة وغير ممثلة، فإن النموذج سيتعلم هذا التحيّز ويعيد إنتاجه، بل ويضخّمه تحت ستار «الحياد الرياضي».

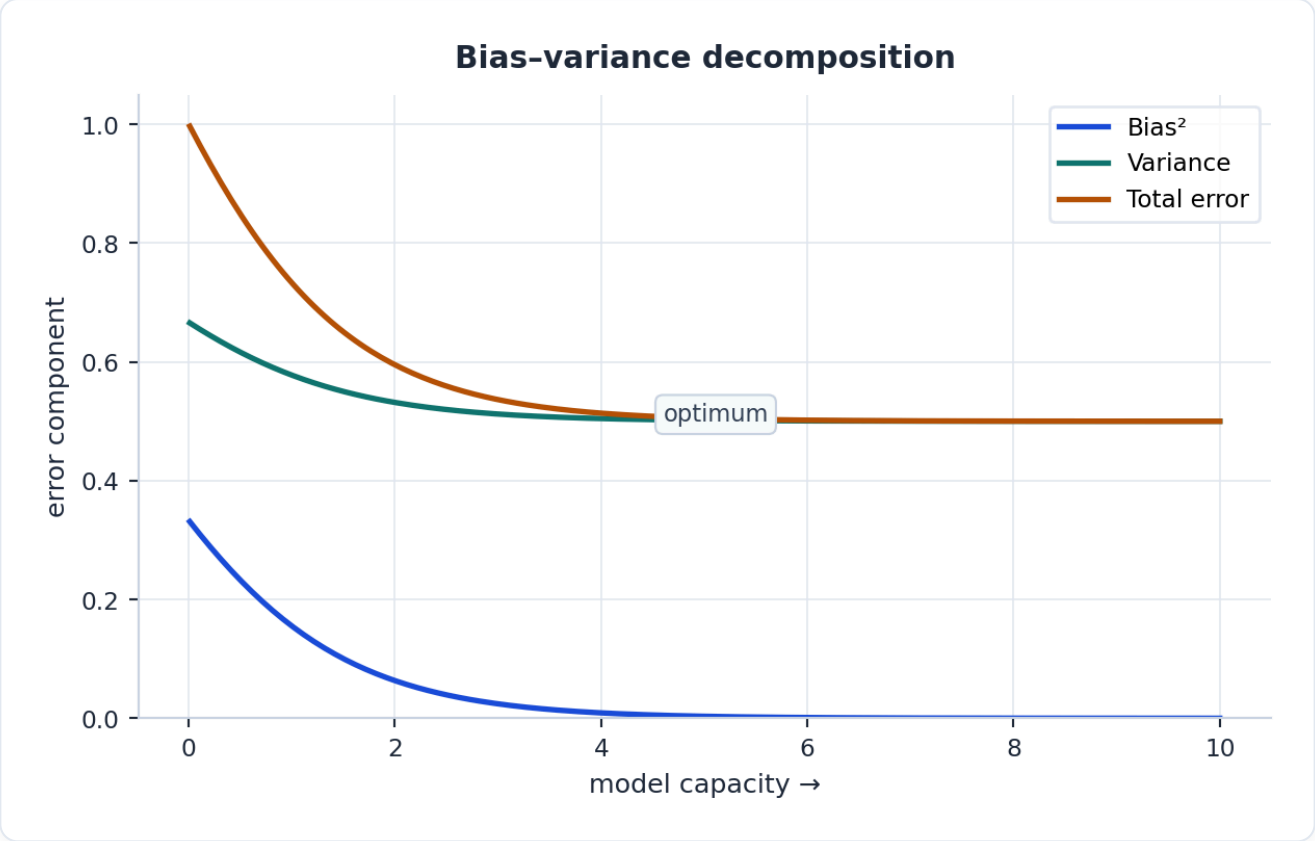


الشكل ٢١خريطة أداء عائلات النماذج بحسب نوع وحجم البيانات ومقايضة الدقة مع سهولة التفسير

في ختام الجزء الأول من هذا الكتاب، لا بد أن نقف وقفة صريحة: لا يوجد نموذج «محايد» تماماً. مسؤولية مهندس تعلّم الآلة لا تنتهي عند كتابة `fit` و`predict`، بل تمتد إلى تدقيق العدالة، وحماية الخصوصية، وفهم الحدود التي تفشل عندها الخوارزميات.

٩.٢ أنواع التحيّز: من أين يتسلّل الخطأ؟

التحيّز في تعلّم الآلة ليس عيباً في كود بايثون، بل ينشأ في مراحل سابقة لكتابة الكود:



الشكل ٢٢منحنى تفكيك الخطأ الكلي إلى تباين وتحيّز مع زيادة سعة النموذج وتعقيده

توضح علاقة التباين والتحيّز الرياضية أن النماذج البسيطة قد تُفرط في التبسيط فتتجاهل الفروق الدقيقة للفئات الصغيرة، بينما النماذج المعقدة قد تحفظ الضجيج والأنماط التمييزية وتكرّسها.

٩.٣ التكافؤ الديموغرافي (Demographic Parity)

المفهوم الأول للعدالة الرياضية هو **التكافؤ الديموغرافي**: يجب أن تتساوى احتمالية الحصول على القرار الإيجابي (مثل قبول القرض أو التأهل للمقابلة) عبر جميع الفئات الحساسة، بصرف النظر عن انتمائهم.

$$\Delta_{DP} = |P(\hat{Y}=1|A=0) - P(\hat{Y}=1|A=1)|$$

إذا كانت النتيجة $\Delta_{DP} = 0$، فإن النموذج يحقق تكافؤاً تاماً. وإذا انخفضت نسبة قبول الفئة الأقل عن 80% من نسبة قبول الفئة الأكثر قبولاً، يُصنّف ذلك قانونياً في العديد من الدول كـ «أثر تمييزي متباين» (Disparate Impact).

لنقم بمحاكاة بيانات إقراض بنكي لنرى كيف يقع النموذج في هذا الفخ:

محاكاة نموذج الإقراض وحساب التكافؤ الديموغرافي python

```python
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(42)
n_samples = 1500

# السمة الحساسة A (مثلاً: 0 تمثل الفئة الأقلية، 1 تمثل الفئة الأغلبية)
group_A = rng.integers(0, 2, n_samples)

# ميزة مالية موضوعية: الدرجة الائتمانية (تتأثر جزئياً بظروف تاريخية غير متكافئة)
credit_score = rng.normal(600, 50, n_samples) + (group_A * 25)

# احتمالية سداد القرض الحقيقية تعتمد على الجدارة المالية
repay_prob = 1 / (1 + np.exp(-(credit_score - 600) / 45))
loan_approved = rng.binomial(1, repay_prob)

X = np.column_stack([credit_score, group_A])
X_tr, X_te, y_tr, y_te, g_tr, g_te = train_test_split(
    X, loan_approved, group_A, test_size=0.3, random_state=42
)

# تدريب مصنّف لوجيستي عادي
clf = LogisticRegression().fit(X_tr, y_tr)
predictions = clf.predict(X_te)

# قياس معدل القبول لكل فئة
rate_group_0 = predictions[g_te == 0].mean()
rate_group_1 = predictions[g_te == 1].mean()
dp_diff = abs(rate_group_0 - rate_group_1)
disparate_impact = rate_group_0 / rate_group_1

print(f"معدل قبول الفئة الأولى (A=0): {rate_group_0:.1%}")
print(f"معدل قبول الفئة الثانية (A=1): {rate_group_1:.1%}")
print(f"فارق التكافؤ الديموغرافي (DP Difference): {dp_diff:.3f}")
print(f"نسبة الأثر المتباين (Disparate Impact)   : {disparate_impact:.3f}")
```

النتيجة توضح وجود تباين صارخ: فالفئة $A = 0$ حصلت على معدل قبول يقارب نصف معدل الفئة $A = 1$، مما يوقع المؤسسة تحت طائلة المساءلة القانونية والأخلاقية.

٩.٤ فخ المتغيرات البديلة (The Proxy Trap)

أول رد فعل يقترحه المهندس المبتدئ هو: «سنحذف العمود الحساس A من البيانات تماماً! إذا لم يرَ النموذج العرق أو الجنس، فلن يستطيع التمييز ضدهما».

هذا الطرح ساذج وخطير، ويُعرف بـ **العدالة بالجهل** (Fairness through Unawareness). في العالم الحقيقي، ترتبط الميزات ببعضها بشدة؛ فالرمز البريدي، ومستوى التعليم، أو نوع الهاتف المستخدم، قد تعمل كـ **متغيرات بديلة (Proxies)** تعيد بناء السمة الحساسة المحذوفة بدقة متناهية.

إثبات فشل حذف السمة الحساسة أمام المتغيرات البديلة python

```python
# محاكاة متغير وسيط (مثل الرمز البريدي) يرتبط بقوة بالسمة الحساسة
zip_code_score = credit_score * 0.8 + group_A * 40 + rng.normal(0, 10,
n_samples)

# مصفوفة تدريب جديدة خالية تماماً من العمود الحساس group_A
X_no_sensitive = np.column_stack([credit_score, zip_code_score])
X_tr_ns, X_te_ns, _, _ = train_test_split(
    X_no_sensitive, loan_approved, test_size=0.3, random_state=42
)

# تدريب النموذج على البيانات «المعمية»
clf_blind = LogisticRegression().fit(X_tr_ns, y_tr)
preds_blind = clf_blind.predict(X_te_ns)

rate_blind_0 = preds_blind[g_te == 0].mean()
rate_blind_1 = preds_blind[g_te == 1].mean()

print("نتائج النموذج الأعمى بعد حذف السمة الحساسة تماماً:")
print(f"  - معدل قبول الفئة A=0 : {rate_blind_0:.1%}")
print(f"  - معدل قبول الفئة A=1 : {rate_blind_1:.1%}")
print(f"  - فارق التكافؤ الباقي   : {abs(rate_blind_0 - rate_blind_1):.3f}")
```

حذف السمة لم يغيّر شيئاً تقريباً! النموذج أعاد استنتاج الفئة من المتغير البديل واستمر في ممارسة التمييز غير المقصود، ولكن مع ميزة إضافية سيئة: صار التمييز مخفياً ويصعب إثباته.

٩.٥ تكافؤ الفرص (Equal Opportunity)

ماذا لو كان الفارق بين المجموعات نابعاً من فروق حقيقية في الجدارة؟ هنا يظهر معيار أكثر عدالة ودقة: **تكافؤ الفرص**.

يشترط تكافؤ الفرص أن يتساوى **معدل الإيجابيات الحقيقية (True Positive Rate - TPR)** بين الفئات. بعبارة أخرى: من يستحق القرض فعلاً ويستطيع سداده ($Y = 1$)، يجب أن يحصل على نفس فرصة القبول بغض النظر عن فئته الحساسة.

$$\Delta_{EO} = |P(\hat{Y} = 1|A = 0, Y = 1) - P(\hat{Y} = 1|A = 1, Y = 1)|$$

```python
حساب تكافؤ الفرص ومعدل الإيجابيات الحقيقية عبر الفئات
from sklearn.metrics import recall_score

# حساب الاسترجاع (TPR) لكل فئة على حدة بين المستحقين فعلياً للسداد
tpr_0 = recall_score(y_te[g_te == 0], predictions[g_te == 0])
tpr_1 = recall_score(y_te[g_te == 1], predictions[g_te == 1])
eo_diff = abs(tpr_0 - tpr_1)

print("تدقيق تكافؤ الفرص (True Positive Rate per Group):")
print(f"  - نسبة قبول المؤهلين فعلياً من الفئة A=0: {tpr_0:.1%}")
print(f"  - نسبة قبول المؤهلين فعلياً من الفئة A=1: {tpr_1:.1%}")
print(f"  - فارق تكافؤ الفرص (Equal Opportunity Diff): {eo_diff:.3f}")
```

إذا كان فارق تكافؤ الفرص كبيراً، فهذا يعني أن المؤهلين من الفئة الأقلية يُحرمون من الفرص بمعدل أعلى من نظرائهم المتطابقين في الفئة الأخرى.

٩.٦ معايرة عتبات القرار: TunedThresholdClassifierCV

افتراضياً، تقطع جميع مصنّفات scikit-learn الاحتمال عند العتبة 0.5 . لكن العتبة 0.5 نادراً ما تكون الخيار الأمثل عند وجود عدم توازن بين الفئات أو تباين في تكاليف الأخطاء.

في **scikit-learn ١.٩**، توفّر المكتبة أداة معيارية ذكية تُدعى `TunedThresholdClassifierCV`، تتيح البحث التلقائي عن أفضل عتبة قرار توازن بين المقاييس الحساسة (مثل الدقة المتوازنة `balanced_accuracy` أو الاسترجاع المستهدف).

ضبط عتبة القرار عبر TunedThresholdClassifierCV في 1.9 python

```python
from sklearn.model_selection import TunedThresholdClassifierCV

# تغليف المصنّف لضبط عتبة القرار تلقائياً باستخدام الدقة المتوازنة
tuned_model = TunedThresholdClassifierCV(
    estimator=LogisticRegression(),
    scoring="balanced_accuracy",
    cv=5
)

tuned_model.fit(X_tr, y_tr)
optimal_threshold = tuned_model.best_threshold_
tuned_preds = tuned_model.predict(X_te)

print("نتائج ضبط العتبة التلقائي في scikit-learn 1.9:")
print(f"  - عتبة القرار المحسوبة كأمثل خيار: {optimal_threshold:.4f} (بدلاً من 0.5000)")
print(f"  - معدل القبول الجديد للفئة A=0   : {tuned_preds[g_te == 0].mean():.1%}")
print(f"  - معدل القبول الجديد للفئة A=1   : {tuned_preds[g_te == 1].mean():.1%}")
print(f"  - الفارق الجديد في القبول        : {abs(tuned_preds[g_te == 0].mean() -
tuned_preds[g_te == 1].mean()):.3f}")
```

أعادت المعايرة الذكية ضبط العتبة لتخفيضها إلى مستوى عادل يرفع فرص المؤهلين المهمشين دون التضحية بالدقة الإجمالية للنظام.

> **التحديث الجديد في scikit-learn ١.٩**
>
> تعد أدوات ضبط العتبات `TunedThresholdClassifierCV` و `FixedThresholdClassifier` نقلة نوعية في المكتبة، إذ تسمح بفصل خطوة التقدير الاحتمالي عن خطوة اتخاذ القرار الحرج في الإنتاج، وهو صلب هندسة الأنظمة الحساسة.

٩.٧ قائمة تدقيق المسؤولية الأخلاقية للمهندس

قبل أن توافق على رفع نموذجك إلى خادم الإنتاج، مرّر نموذجك على هذه القائمة الصارمة:

```python
# دالة فحص وتدقيق الفروق الديموغرافية لأي نموذج
def audit_fairness(model, X_eval, y_eval, sensitive_col_idx):
    """دالة قياسية تفحص التكافؤ وتكافؤ الفرص عبر الفئات."""
    preds = model.predict(X_eval)
    groups = np.unique(X_eval[:, sensitive_col_idx])

    report = {}
    for g in groups:
        mask = (X_eval[:, sensitive_col_idx] == g)
        pos_rate = preds[mask].mean()
        recall = recall_score(y_eval[mask], preds[mask]) if y_eval[mask].sum() > 0 else 0.0
        report[f"Group_{int(g)}"] = {"Selection_Rate": round(pos_rate, 3), "TPR_Recall": round(recall, 3)}
    return report

fairness_report = audit_fairness(tuned_model, X_te, y_te, sensitive_col_idx=1)
print("تقرير التدقيق الأخلاقي السريع للنموذج:")
for grp, metrics in fairness_report.items():
    print(f"  {grp}: معدل الاختيار = {metrics['Selection_Rate']:.1%}, الاسترجاع = {metrics['TPR_Recall']:.1%}")
```

> ☑ **قائمة التحقق قبل النشر**

٩.٨ تمارين الفصل

التمرين ٩.١

أنشئ مجموعة بيانات مصطنعة تضم ثلاث فئات عمرية (شباب، متوسطو العمر، كبار سن) ونموذجاً لتوقع الترقية الوظيفية. احسب معدل التحديد (Selection Rate) لكل فئة عمرية واكتشف هل يعاني كبار السن من أثر متباين.

التمرين ٩.٢

طبّق أداة `FixedThresholdClassifier` المتاحة في الإصدارات الحديثة من scikit-learn، واضبط عتبة قرار ثابتة تساوي 0.35 . قارن أداءها بأداء `LogisticRegression` القياسي ذي العتبة 0.5 .

التمرين ٩.٣

اشرح لماذا يتعارض معيار «التكافؤ الديموغرافي» أحياناً مع معيار «تكافؤ الفرص» في بيئة يكون فيها معدل الجدارة الحقيقي مختلفاً تاريخياً بين المجموعتين.

التمرين ٩.٤

اكتب كوداً يستخرج مصفوفة الخلط (Confusion Matrix) لكل فئة حساسة على حدة، واحسب منها معدل الإيجابيات الكاذبة (FPR). لماذا يمثل ارتفاع FPR لفئة معينة ضرراً بالغاً في أنظمة العدالة الجنائية؟

٩.٩ حلول بعض التمارين

✅ حلّ التمرين ٩.٣

يتعارض التكافؤ الديموغرافي مع تكافؤ الفرص عندما يكون معدل الأساس (Base Rate) مختلفاً بين الفئتين في العالم الحقيقي: فالتكافؤ الديموغرافي يفرض قبول نفس النسبة من كلا المجموعتين ($P(\hat{Y}=1|A=0) = P(\hat{Y}=1|A=1)$)، مما يجبر النموذج على خفض معايير القبول للفئة ذات معدل الأساس الأدنى، وهو ما يؤدي حتماً إلى رفع معدل الإيجابيات الكاذبة (FPR) لتلك الفئة وكسر تكافؤ الفرص. يستحيل رياضياً تحقيق التكافؤ الديموغرافي وتكافؤ الفرص ومعايرة الاحتمالات في وقت واحد إلا إذا كان معدل الأساس متطابقاً تماماً بين المجموعتين.

✅ حلّ التمرين ٩.٤

في أنظمة العدالة الجنائية (مثل التنبؤ بالعودة للجريمة)، تمثل الإيجابية الكاذبة (False Positive) تصنيف شخص بريء أو منخفض الخطورة على أنه مجرم شديد الخطورة، مما يؤدي لحرمانه من إطلاق السراح المشروط أو تشديد عقوبته. إذا كان معدل FPR للفئة الأقلية ضعف معدل الأغلبية، فهذا يعني أن النظام يعاقب الأبرياء من تلك الفئة بمعدل مضاعف، وهو انتهاك جسيم لحقوق الإنسان والعدالة القانونية.

٩.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الاكتفاء بحذف الأعمدة الحساسة	استمرار التحيّز عبر المتغيرات البديلة مع صعوبة اكتشافه	احتفظ بالسمة أثناء التطوير واستخدمها لتدقيق العدالة لا للتدريب
الاعتماد على الدقة الكلية (Accuracy) وحدها	النموذج يضحي بالفئات الصغيرة للحفاظ على دقة الأغلبية	احسب مقاييس الأداء لكل فئة حساسة على حدة
تثبيت عتبة القرار دائماً عند ٠.٥	إهدار مرونة النموذج في موازنة التكاليف الأخلاقية	استخدم `TunedThresholdClassifierCV` لضبط العتبة وفق الهدف
افتراض إمكانية تحقيق كل مقاييس العدالة معاً	الاصطدام باستحالة رياضية تفشل النماذج	اختر المقياس الأنسب لطبيعة المشكلة (تكافؤ الفرص أم التكافؤ الديموغرافي)
إطلاق النموذج دون سجل مراجعة أو تفسير	عجز المؤسسة عن تبرير قراراتها أمام الجهات التنظيمية	وفّر بطاقة نموذج (Model Card) وتدقيقاً للبواقي والفئات

٩.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التكافؤ الديموغرافي	تساوي احتمالية الحصول على النتيجة الإيجابية عبر الفئات	demographic parity
تكافؤ الفرص	تساوي معدل الاسترجاع (TPR) للمؤهلين فعلياً عبر الفئات	equal opportunity
الأثر المتباين	نسبة معدل قبول الفئة الأقل حظاً إلى الفئة الأكثر حظاً	disparate impact
المتغير البديل	ميزة غير حساسة ترتبط إحصائياً بسمة حساسة محجوبة	proxy variable
العدالة بالجهل	محاولة منع التمييز بحذف السمة الحساسة فقط	fairness through unawareness
ضبط عتبة القرار	معايرة نقطة الفصل الاحتمالي لتحقيق توازن مستهدف	threshold tuning
بطاقة النموذج	وثيقة تقنية تشرح مقاييس النموذج وحدوده الأخلاقية	model card

الجزء ٢

البيانات: الكتلة الخام للمشروع

من مصفوفة NumPy إلى المصفوفة المتفرّقة والبيانات المفقودة

ماذا يغطّي هذا الجزء

ثمانون بالمئة من وقت مهندس تعلّم الآلة يذهب في هندسة البيانات ومعالجتها، لا في تدريب النموذج. هذه القوة تشرح كيف تمسك scikit-learn بالبيانات: المصفوفات، والمصفوفات المتفرّقة، والقيم المفقودة، والأنواع المختلفة، والأخطاء التي تُسرّب البيانات قبل أن تنتبه إليها.

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٠ — أنماط البيانات في scikit-learn

ndarray و DataFrame والمصفوفات المتفرّرة: أيّها ولماذا

الوحدة **١.٩ scikit-learn** الحجم **٦٩٤** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تميّز بين ndarray و DataFrame والمصفوفة المتفرّرة وتشرح متى تختار كلّاً منها
- أن تقرأ نتيجة fit وتعرف من أين جاءت أسماء الخصائص
- أن تحسب كثافة المصفوفة وتقدّر حجم الذاكرة قبل أن تختار تمثيل البيانات
- أن تُصلح الخطأ could not convert string to float بالطريقة الصحيحة

٢.١ لماذا يهمّ شكل البيانات قبل أن يهمّ النموذج؟

تخيّل أنك وصلت إلى مشروع، والبيانات محمّلة، وأدوات التحليل تعمل. تفتح جدولاً فيه عمود عمر، وعمود مدينة نصّي، وعمود تاريخ. تكتب سطراً واحداً:

```python
المشكلة في سطر واحد
from sklearn.linear_model import LogisticRegression
model = LogisticRegression().fit(df[["age", "city"]], y)
```

وتحصل على:

```python
الرسالة التي سترسلها المكتبة
ValueError: could not convert string to float: 'Riyadh'
```

هذه ليست رسالة عابرة. هذه هي **الحدود الحقيقية** للمكتبة. `LogisticRegression` لا يعرف ما هي المدينة. كل ما يعرفه أنه يتوقّع مصفوفة أرقام، وسأله سؤالاً واحداً: هل تستطيع أن تحوّل `'Riyadh'` إلى عدد؟ لا يستطيع، فيتوقّف.

الدرس الأوّلي: `scikit-learn` ليست بيئة بيانات عامة، بل طبقة حسابية فوق `NumPy`. كل مُقدِّر فيها — من أقرب الجيران إلى التعزيز التدرّجي — يتلقّى مصفوفة رقمية ويُعيد مصفوفة رقمية. ما عدا ذلك، تُلصَق البنية في الواجهة بواسطة `ColumnTransformer` (الفصل ٨٥).

> **القاعدة التي تُبنى عليها المكتبة**
>
> كل مُقدِّر في `sklearn` له عقد واحد: `fit(X, y)` ثم `predict(X)`، حيث `X` مصفوفة رقمية ثنائية البعد (صفوف = عينات، أعمدة = خصائص)، و `y` متّجه بطول `n`. الأنماط الأخرى ليست «غير مدعومة»، بل **تُحوَّل إلى هذا العقد** قبل الوصول إليه.

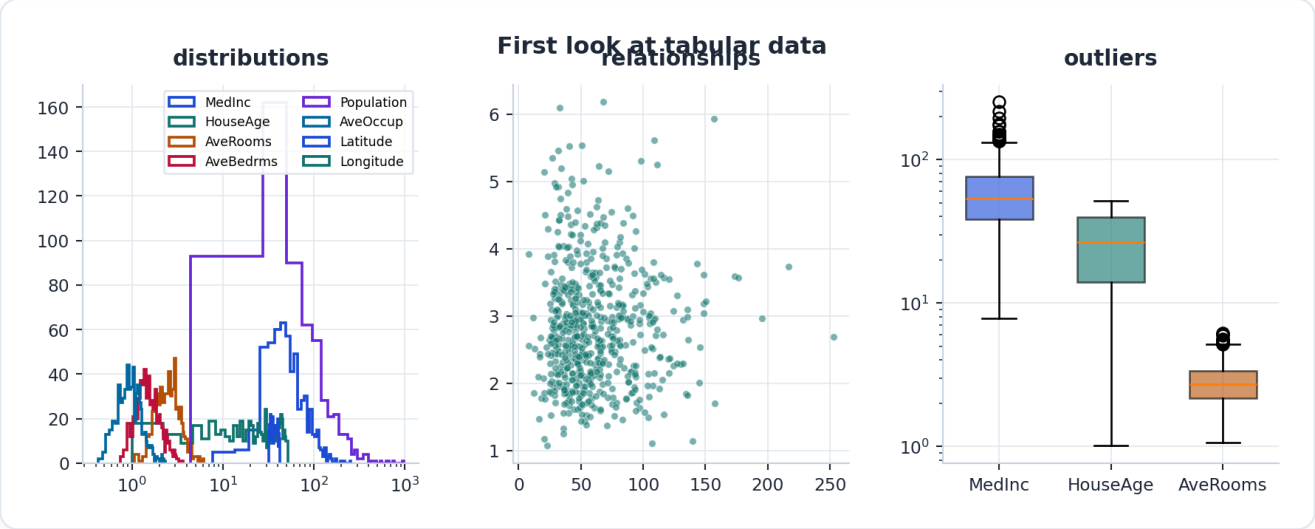
٢.٢ المسار من ملفّ CSV إلى مصفوفة

لا يوجد في `scikit-learn` دالة تقرأ ملفّ CSV. هذه ليست نقصاً بل قرار تصميمي: المكتبة لا تتعلّم عن أنظمة الملفات ولا قواعد البيانات ولا صيغ الملفات. مسؤوليتها تبدأ عند الحدّ الذي تكون فيه البيانات **في الذاكرة**.

الخطوة	الأداة المسؤولة	ما الذي تُنتجه
١. القراءة	`pandas.read_csv` أو `load_*`	`DataFrame` أو `Bunch`
٢. الفحص	`DataFrame.dtypes` و `isna()`	خريطة الأنواع والفجوات
٣. التقطيع	`ColumnTransformer`	مصفوفة بعد معالجة كل عمود بطريقته
٤. التدريب	أيّ `estimator.fit(X, y)`	معاملات وقرارات
٥. الرجوع	`estimator.predict(X)`	تنبّؤات على نفس القالب الذي استُقبل

لاحظ الخطوتين ٢ و ٣: هما بالضبط ما يفصل هذا الكتاب عن دليل مرجعي جاف. أغلب مشاريع التعلّم الآلة الفاشلة تفشل قبل أن يُكتب أول سطر `fit`.

قبل أن نبدأ، خذ خطوة إلى الخلف وانظر إلى جدول حقيقي: توزيع كل عمود على حدة، على حدة، وعلاقة عمودين، وصندوقي شذوذ. هذه اللوحة الثلاثية هي أسرع طريقة لفهم شكل بياناتك قبل أن تلمسها أداة.



الشكل ٢٣ثلاث لوحات لجدول أسعار منازل: توزيع كل عمود على محور لوغاريتمي، وعلاقة الدخل بعدد الغرف، وصندوقا شذوذ يكشفان الانحراف بوضوح

دعنا نبدأ بمصدر بيانات حقيقي مضمّن داخل المكتبة بلا أيّ تنزيل.

أول نظرة على حزمة بيانات مضمّنة python

```python
import pandas as pd
from sklearn.datasets import load_diabetes

dia = load_diabetes()
print(f"نوع الكائن        : {type(dia).__name__}")
print(f"المفاتيح المتاحة: {sorted(k for k in dia if not k.startswith('_'))}")
print(f"شكل X             : {dia.data.shape}  dtype={dia.data.dtype}")
print(f"شكل y             : {dia.target.shape}")
print(f"أسماء الخصائص    : {list(dia.feature_names)}")
print()
dia_df = pd.DataFrame(dia.data, columns=dia.feature_names)
dia_df.head()
```

لاحظ أن `load_diabetes()` يعيد كائناً اسمه `Bunch`: قاموس بسمات بدل أقواس. و `dia.data` مصفوفة `ndarray` نظيفة، بينما `dia.frame` هو `DataFrame` يحمل الأسماء. الاثنان يصفان الشيء نفسه؛ الأول للحوسبة، والثاني للتفكير.

٢.٣ النمط الأول: المصفوفة الرقمية `ndarray`

هذا هو النمط الأمومي. بُنيت `scikit-learn` على `NumPy` قبل أيّ شيء آخر، وكل ما يلي مشتقّ منه.

```python
# النمط الأمومي: صفيحتان بلا أسماء
import numpy as np

A = np.array([[1.0, 2.0, 3.0],
              [4.0, 5.0, 6.0],
              [7.0, 8.0, 9.0]])

print("dtype          :", A.dtype)
print("shape          :", A.shape, "->", A.shape[0], "صف و", A.shape[1], "عمود")
print("الصف الثاني    :", A[1])
print("العمود الثالث  :", A[:, 2])
print("قناع منطقي يخفي الصف الأخير:", A[A[:, 0] > 3])
print("ما تراه المكتبة:", type(A).__name__, "— بلا أسماء أعمدة")
```

ثلاث حقائق عن `ndarray` يجب أن تحفظها:

٢.٤ النمط الثاني: جدول البيانات `DataFrame`

`DataFrame` من `pandas` هو ما سيمسك بيدك فعلاً في أغلب الوقت. والفرق الجوهري ليس في القيم، بل في **الأسماء والفهرس والأنواع**.

```python
# جدول بأربعة أنواع أعمدة مختلفة
import pandas as pd

df = pd.DataFrame({
    "age": [23, 45, 31, 52, 38],
    "city": ["Riyadh", "Cairo", "Riyadh", "Amman", "Cairo"],
    "joined": pd.to_datetime(["2023-01-15", "2021-06-02", "2024-03-20", "2020-11-11", "2022-08-08"]),
    "active": [True, False, True, True, False],
})
print(df.to_string(index=False))
print()
print(df.dtypes)
print()
print(f"حجم الذاكرة المستعمل فعلياً: {df.memory_usage(deep=True).sum()} بايت")
```

لاحظ ثلاثة أشياء:

كيف يقرأ `scikit-learn` جدولاً بأسماء؟

هذا أهمّ ما في هذا القسم. عندما تمرّر `DataFrame` إلى `fit`، تفعل المكتبة ثلاثة أشياء بالترتيب: تتحقّق من صحّة القيم، **تنسخ القيم الرقمية فقط** في مصفوفة داخلية بلا أسماء، ثم **تحفظ أسماء الأعمدة** في `feature_names_in_` لتستخدمها في التقارير لاحقاً.

أثر تمرير الأسماء على ما تخرج به المكتبة — python

```python
from sklearn.preprocessing import StandardScaler

with_names = pd.DataFrame(dia.data[:60], columns=dia.feature_names)
sc_named = StandardScaler().fit(with_names)
sc_anon = StandardScaler().fit(dia.data[:60])

print("مع DataFrame -> feature_names_in_:", list(sc_named.feature_names_in_[:4]),
"…")
print("مع ndarray   -> feature_names_in_:", hasattr(sc_anon,
"feature_names_in_"))
print("get_feature_names_out() مع أسماء :", sc_named.get_feature_names_out()[:4])
print("get_feature_names_out() بلا أسماء:", sc_anon.get_feature_names_out()[:4])
print("عدد الخصائص في الحالتين           :", sc_named.n_features_in_,
sc_anon.n_features_in_)
```

هذا هو **فارق قابلية القراءة وحده**: نفس النموذج، نفس الأرقام، لكن الأول يقول «bmi» والثاني يقول «x٢». في تقريرٍ يُقرأ من طرف غير تقني، الفرق بين «ارتفع مؤشر كتلة الجسم» و«ارتفع x٢» هو الفرق بين قرارٍ يُتّخذ ورقمٍ يُتجاهل.

> **قاعدة عملية**
>
> مرّر `DataFrame` إلى `fit` ما دمتَ لا تملك سبباً قاطعاً لاختيار غيره. إن تحوّلت إلى `ndarray` لأداءٍ أو لتمريرٍ عبر حدّ ما، احتفظ بمصفوفة الأسماء في مكانٍ ما، لأن النموذج لن يحفظها نيابةً عنك.

٢.٥ النمط الثالث: المصفوفة المتفرّرة

البيانات الجدولية لا تبقى جدولية أحياناً: نصّ، أو متّجه واحد-ساخن لمئة قيمة، أو تفاعل بين مئة خاصية. في هذه الحالات تكون **الأصفار هي قاعدة التخزين**: تخزينها كاملةً هدر.

الصيغة متعددة الاستخدام في `SciPy` هي `csr`، وتخزّن ثلاثة متّجهات:

المتّجه	ماذا يحفظ	طوله
`data`	القيم غير الصفرية	عدد `nnz`
`indices`	فهرس العمود لكل قيمة	عدد `nnz`
`indptr`	أين يبدأ كل صف	عدد الصفوف + ١

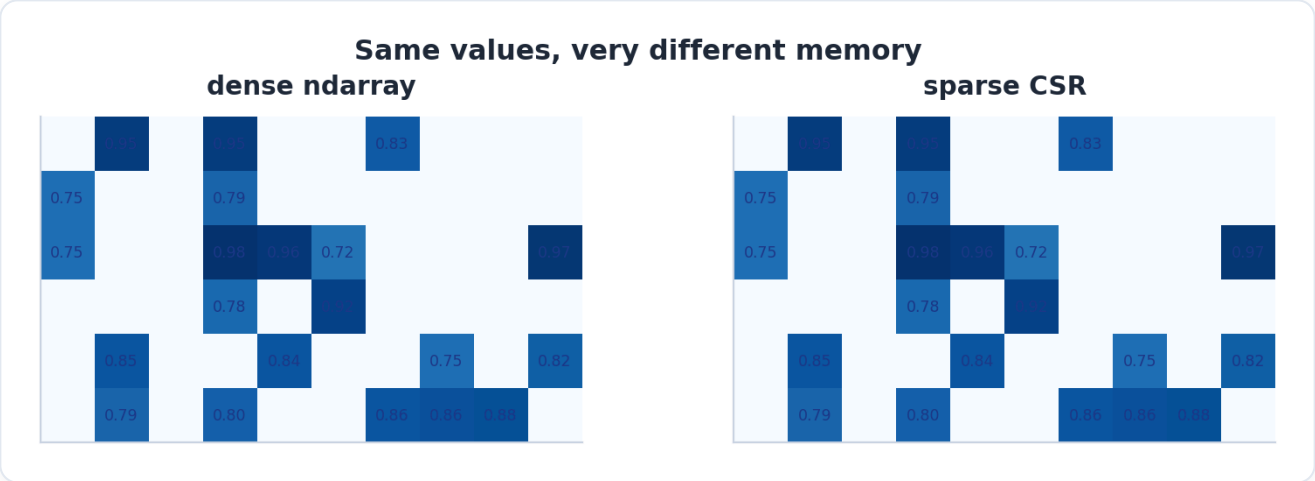
الكثافة هي النسبة التي تقرّر مصير هذا الاختيار:

$$\rho = \frac{\text{nnz}}{n \cdot m}$$

وحجم الذاكرة التقريبي:

$$M_{\text{CSR}} = \text{nnz} \cdot (8 + 4) + (m + 1) \cdot 4$$

اقرأ المعادلة الثانية: ثمانية بايتات للقيمة وأربعة بايتات لفهرسها لكل عنصر غير صفر. لذلك حتى بكثافة ٥٪ يكسب `CSR` أكثر من عشرة أضعاف على `ndarray`.



الشكل ٢٤ لوحتان لمصفوفتين تحملان القيم نفسها: واحدة كثيفة وأخرى متفرّرة، والفرق الوحيد أن الأبيض في الثانية صفرٌ لا يُخزَّن

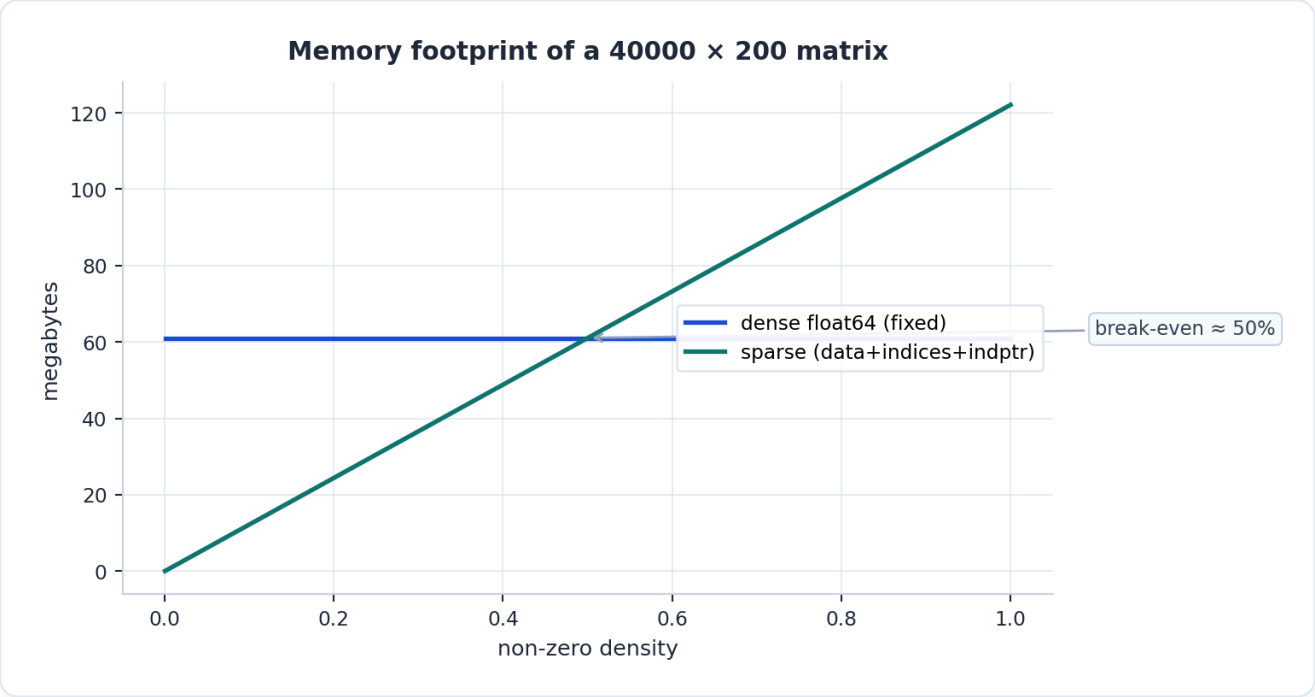
الشكل أعلاه يجعل الفرق محسوساً: المصفوفتان تحملان الأرقام نفسها، لكن النمط الكثيف يُظهر شبكة كاملة من الخانات، والمتفرّر يُظهر بضع نقاط فقط. هذا هو الفارق بين «حفظ كلّ شيء» و«حفظ ما يُفيد».

```python
كم يوفّر فعلاً التمثيل المتفرّز؟
import scipy.sparse as sp

n_rows, n_cols, density = 2000, 300, 0.02
S = sp.random(n_rows, n_cols, density=density, format="csr", random_state=0)
sparse_bytes = S.data.nbytes + S.indices.nbytes + S.indptr.nbytes
dense_bytes = n_rows * n_cols * 8

print(f"كثافة المصفوفة : {S.nnz / (n_rows * n_cols):.4f}  ({S.nnz} غير صفر من {n_rows
* n_cols})")
print(f"حجم ndarray     : {dense_bytes / 1024**2:6.2f} ميغابايت")
print(f"حجم CSR         : {sparse_bytes / 1024:6.1f} كيلوبايت")
print(f"نسبة التوفير    : {dense_bytes / sparse_bytes:6.1f} أضعاف")
print()
print(f"نوع البيانات  : {S.data.dtype}   نوع الفهارس: {S.indices.dtype}")
```

الفصل ١١ يغوص في هذه الصيغ كلّها ويفحص الواجهة الجديدة `sparray` . ما نريد احتفاظه هنا هو **قاعدة القرار**: إذا كانت مصفوفتك تقارن بكثافة بين ٠٫٠١ و٠٫٠٢ — وهذا ما تنتجه ترميز `One-Hot` على بيانات فئوية عالية، أو `TF-IDF` على مستندات — فاترك `scikit-learn` يتكفّل بالتحويل عبر `sparse_threshold` ، ولا تحوّلها بنفسك.



الشكل ٢٥أعمدة تقارن حجم المصفوفة الكثيفة بعدة صيغ متفرّزة، ويتضح أن كلفة كل صيغة تتبع كثافتها لا بُعدها

٢.٦ متى تستخدم كلّ نمط؟

المعيار	`ndarray`	`DataFrame`	مصفوفة متفرّزة
يقبله أيّ `estimator` مباشرة	نعم	نعم للأعمدة الرقمية فقط	نعم إن كان يدعم `sparse`
يحفظ أسماء الخصائص	لا	نعم في `feature_names_in_`	لا
يدعم أعمدة فئوية نصّية	لا	نعم عبر `ColumnTransformer`	لا
أعمدة زمنية جاهزة	لا	نعم عبر التحويل	لا
الذاكرة عند كثافة ٠٫٠٢	٤٫٥٨ ميغابايت	مثلها مع دلالات	١٤٨ كيلوبايت
أفضل للتفسير البشري	ضعيف	ممتاز	ضعيف
أفضل للحسابات المتوازية	ممتاز	جيد	ممتاز

٢.٧ الخطأ الذي يراه الجميع: القيم النصّية في مكانٍ لا مكان لها

عندما تمرّر أعمدةً مختلطة إلى `fit` رقمي، ترى هذه الرسالة بالضبط:

أين بالضبط يفشل التحويل؟ python

```python
from sklearn.utils.validation import check_array
from sklearn.linear_model import LogisticRegression

mixed = pd.DataFrame({"age": [23.0, 45.0, 31.0, 52.0],
                      "city": ["Riyadh", "Cairo", "Riyadh", "Amman"]})

for kw in [dict(), dict(dtype=None)]:
    label = kw if kw else "افتراضي"
    try:
        out = check_array(mixed, **kw)
        print(f"check_array(dtype={label}) -> نجح: {type(out).__name__} dtype=
{out.dtype}")
    except Exception as exc:
        print(f"check_array(dtype={label}) -> {type(exc).__name__}:
{str(exc).splitlines()[0][:88]}")

try:
    LogisticRegression(max_iter=500).fit(mixed, [0, 1, 1, 0])
except Exception as exc:
    print(f"LogisticRegression.fit         -> {type(exc).__name__}:
{str(exc).splitlines()[0][:88]}")
```

لاحظ أن `check_array` بـ `dtype="numeric"` يرفض **الجدول كلّه**: لا استثناء لعمود واحد. والمخرج الصحيح ليس «حذف العمود» بل **ترميزه بأداة تفهمه**.

الحل: كل عمود يُعالَج بطريقته python

```python
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

pipe = Pipeline([
    ("pre", ColumnTransformer([
        ("num", StandardScaler(), ["age"]),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
["city"]),
    ])),
    ("clf", LogisticRegression(max_iter=500)),
])
pipe.fit(mixed, [0, 1, 1, 0])
print("التنبّؤ          :", pipe.predict(mixed))
print("أسماء المخرجات:", pipe.named_steps["pre"].get_feature_names_out())
```

هذا النمط — «جدول واحد، عدة مسارات، مصفوفة واحدة في النهاية» — هو الفصل ٨٥ بالكامل. نكتبه الآن لأن بقية فصول هذا الجزء كلّها مبنيّة عليه.

الخطأ الأول: الحذف بدل الترميز

بعض المبتدئين يحلّون الخطأ أعلاه بـ `df.drop(columns="city")` ثم يظنّون أن المشكلة انتهت. لم تنتهِ: التاجر الذي يفقد مئة عمود غير رقمي خسر مئة إشارة. الترميز لا يكلّف سطراً إضافياً ويحفظ البيانات. **أسقط العمود فقط إن كانت قيمته مفقودة فعلاً، لا لأنه نصّي.**

الخطأ الثاني: الخلط بين المحورين

`X[i]` هو الصفّ رقم `i`. و `X[:, i]` هو العمود رقم `i`. في مصفوفة ألف صفّ وألف عمود، الفرق بينهما فارق مئة ألف خانة. اكتب `X[:, j]` للخصائص و `X[i, :]` للعيّنة، ولا تحذف النقطتين أبداً.

٢.٨ في الواقع: البيانات لا تأتي نظيفة

في العمل الحقيقي، المشكلة لا تبدأ عند `fit`. تبدأ قبلها بثلاث طبقات:

القاعدة المهنية التي توفّر عليك في كلّ مرّة: **افصل الاستخراج عن المعالجة عن التدريب**، واجعل المعالجة كائناً واحداً قابلاً لإعادة التشغيل (الفصل ٨٤). المشروع الذي يخلط الثلاثة في دفتر `Jupyter` لا يمكن اختباره ولا تدقيقه ولا إعادة بنائه.

في الواقع: قاعدة الانطلاق

المشروع الذي تتعلّمه في الكتاب ينتهي غالباً عند أول ملفّ حقيقي من الإنتاج. عندها تتّضح ثلاث حقائق لا تظهر في أيّ مثال: أعمدة «رقمية» تُخزَّن نصوصاً، وأعمدة «مفقودة» تُكتب صفراً أو سالباً واحداً أو النصّ `N/A`، ومفاتيح «فريدة» تتكرّر عشر مرّات بأسماء مختلفة. الفصل ١٣ يعالج الحالة الثانية تحديداً، والفصل ١٦ الأولى.

تمارين الفصل

التمرين ٢.١

حمّل `load_diabetes`، وابنِ `DataFrame` من `dia.data` و `dia.target` مع بادئة `f_` لأسماء الخصائص. مرّره إلى `StandardScaler` ثم إلى `LogisticRegression(max_iter=500)` مع هدف ثنائي مشتقّ من وسيط `target`.

اطبع `len(scaler.feature_names_in_)`، ثم أعد التدريب على `dia.data` وحده واطبع الرقم نفسه. فسّر لماذا النتيجتان متطابقتان وما الذي تغيّر فعلياً.

التمرين ٢.٢

ابنِ مصفوفة `300 × 50` بكثافات مختلفة، واحسب يدوياً حجم `CSR` المتوقّع:

$$\text{bytes} = \text{nnz} \cdot (8 + 4) + (m + 1) \cdot 4$$

ثم قارنه بـ `M.data.nbytes + M.indices.nbytes + M.indptr.nbytes` الحقيقي، وحدّد عند أيّ كثافة تتساوى التكلفة بين `ndarray` و `CSR`.

✅ **الحلّ**

الفارق أن `CSR` تخزّن `float64` و `int32` لكل قيمة غير صفرية، بينما تخزّن `ndarray` ثمانية بايتات لكل خانة سواء كانت صفراً أم لا. فتكلفة `CSR` تقارب `nnz × 12 + (m + 1) × 4` بايت.

```python
# حلّ التمرين ٢.٢: أين تتساوى التكلفة؟
import scipy.sparse as sp

n_rows, n_cols = 300, 50
print(f"{'الكثافة':>8} {'nnz':>7} {'ndarray':>12} {'CSR':>10} {'الأوفر':>9}")
for d in [0.05, 0.10, 0.20, 0.40]:
    M = sp.random(n_rows, n_cols, density=d, format="csr", random_state=0)
    csr = M.data.nbytes + M.indices.nbytes + M.indptr.nbytes
    dn = n_rows * n_cols * 8
    print(f"{d:8.2f} {M.nnz:7d} {dn:12,d} {csr:10,d} {'CSR' if csr < dn
else 'ndarray':>9}")
```

نقطة التعادل تقع قرب الكثافة ٠٫١٥، لأن الحلّ يحتاج نحو اثني عشر بايت لكل قيمة غير صفرية لتغطية خانة كاملة من `ndarray`، زائد فهرس نهاية واحد لكل صفّ.

التمرين ٢.٣

خذ `df` من القسم ٢.٤ بأعمدته الأربعة، واكتب دالة `describe_types(frame)` تعيد `DataFrame` من عمودين: اسم العمود ونوعه. ثم احسب نسبة الأعمدة الرقمية إلى الإجمالي، واقترح ثلاث تفسيرات لكون هذه النسبة منخفضة في بيانات الإنتاج.

✅ **الحلّ**

النسبة هنا `1 / 4 = 25%` فقط، لأن ثلاثة أعمدة من أربعة غير رقمية. التفسير المعتاد أن التواريخ وشروح البيع والوسوم تُحفظ نصّاً لا رقماً. الحلّ ليس رقمنة كلّ شيء، بل تصنيف كل عمود إلى أحد أربعة مسارات داخل `ColumnTransformer`: رقمي، وفئوي، وزمني، ونصّي. وهو موضوع الفصلين ١٢ و ١٧.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
تمرير `DataFrame` فيه عمود نصّي إلى `fit` رقمي	`ValueError: could not convert string to float`	`ColumnTransformer` بمسار لكل نوع
الاعتماد على `X[i]` كأنه عمود	تختبر صفّاً فتحصل على رقم لا معنى له	`X[:, i]` للخصائص و `X[i]` للصف
توقّع `feature_names_in_` مع `ndarray`	`AttributeError`	مرّر `DataFrame` أو احفظ الأسماء بنفسك
تحويل الجدول إلى `ndarray` مبكراً	تفقد كلّ النصوص والتواريخ دفعة واحدة	حوّل فقط بعد التقسيم إلى مسارات
تكثيف مصفوفة متفرّزة يدوياً بلا داعٍ	ذاكرة أضعف وسرعة أبطأ من `CSR`	دع `sparse_threshold` يتصرّف
الاعتماد على ترتيب الأعمدة دون أسماء	خطأ صامت بعد إعادة ترتيب المصدر	تحقّق من `feature_names_in_` دائماً
استدعاء `.toarray()` على بيانات ضخمة «لفهمها»	يقتل فائدة التمثيل المتفرّز كلياً	اعرض أول ٢٠ صفّاً فقط

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
المصفوفة الرقمية	صفيحة أرقام بلا أسماء	ndarray
جدول البيانات	صفيحة بأسماء وأنواع وفهرس	DataFrame
المصفوفة المتفرّرة	تخزين القيم غير الصفرية فقط	sparse matrix
الصفوف المتفرّقة	صيغة `CSR` الأشهر في `SciPy`	compressed sparse row
القيمة غير الصفرية	عدد العناصر التي لا تساوي صفراً	non-zero, nnz
الكثافة	نسبة الخانات غير الصفرية إلى الكل	density
حزمة البيانات	قاموس بسمات يعيده `load_*`	Bunch
أسماء الخصائص	أسماء الأعمدة التي حفظها `fit`	_feature*names*in
أسماء المخرجات	أسماء الأعمدة الناتجة عن التحويل	get*feature*names_out
عقد المُقدِّر	القواعد الثابتة التي يتبعها كل صنف	estimator API
الجدول المختلط	جدول فيه أرقام ونصوص وتواريخ	mixed dtype frame

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١١ — المصفوفات المتفرّزة وواجهة sparray الجديدة

المفتاح sparse_interface في ١.٩ — ماذا يعني لك ولماذا يهمّك

الوحدة **scikit-learn ١.٩** الحجم **٧٩٣** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح بنية CSR بثلاثة متّجهات وتفهم ما يفعله كلّ واحد منها
- أن تحسب حجم الذاكرة المتوقّع لكل صيغة قبل أن تختارها
- أن تعرف متى يقبل مُقدِّر ما مصفوفة متفرّزة ومتى يرفضها بصوت عالٍ
- أن تبدّل بين sparse_interface="spmatrix" و "sparray" وتعرف ما يتأثّر

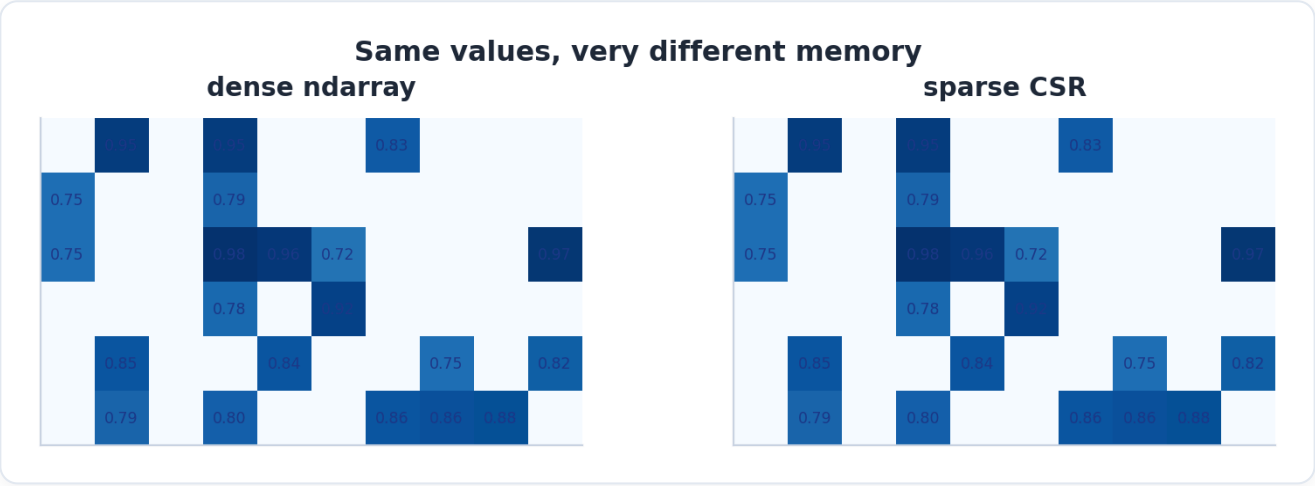
٢.١ ما الذي يعنيه «متفرّز» أصلاً؟

ابدأ بسؤال بسيط. لديك ٥٠٠٠ عمود، ولكل صفّ قيمة واحدة فقط. كم خانة تحتاج؟

الإجابة الصحيحة: خانة واحدة. لكن `ndarray` من نوع `float64` يخزّن **كل** الخمسة ملايين خانة. هذا ليس تفصيلاً هندسياً، بل فرق بين ٤٠ ميغابايت وقرابة ٧٠ كيلوبايت للصفّ الواحد، وهو ما يقرّر إن كان مشروعك يعمل على جهاز محمول واحد أم لا يعمل أصلاً.

التمثيل المتفرّز يقلب الافتراض: بدلاً من أن يسأل «ما قيمة الخانة (٢،٣)؟» يسأل «ما القيم غير الصفرية في هذه المصفوفة كلها؟». ولأنّ الإجابة أقصر بكثير عند كثافة منخفضة، تكون المكسب هو القاعدة.

البُعد	`ndarray`	مصفوفة متفرّزة
ما يُخزَّن	كل القيم	القيم غير الصفرية فقط
الذاكرة	`n × m × 8` بايت	`nnz × 12` بايت تقريباً
زمن الجلب	ثابت	يعتمد على عدد الأصفار في الصفّ
التحويل إلى كثيف	مجاني	مكلّف: `O(n × m)`
مناسب لـ	بيانات كثيفة	بيانات نادرة



الشكل ٢٦ لوحتان لمصفوفتين تحملان القيم نفسها: الكثيفة تُظهر كلّ خانة، والمتفرّزة تُظهر نقاطاً ساطعة على أرضية فارغة

لاحظ أن الصورة على اليسار لا يمكن تمييز أصفارها عن غير أصفارها بمجرد النظر. هذا هو بُعد المشكلة: **كثافة ٥٪ يبدو شبكة شبه كاملة.**

٢.٢ الصيغ الثلاث التي ستقابلها في `SciPy`

`SciPy` لا تختار صيغة واحدة، بل تعطيك أربعاً، ولكلّ منها غرض.

الصيغة	Constructor	أفضل استخدام
`csr`	`scipy.sparse.csr_matrix`	الحساب والضرب، الافتراضي العملي
`coo`	`scipy.sparse.coo_matrix`	البناء الأولي، ثم تحويل
`lil`	`scipy.sparse.lil_matrix`	التعديل العنصر-بعنصر
`csc`	`scipy.sparse.csc_matrix`	القراءة عموداً بعينه (نادر في `sklearn`)

لنرَ البنية الداخلية كما هي فعلاً، لا كما تُشرح في العروض التقديمية.

python — ثلاث صيغ، نفس البيانات، بنية مختلفة

```
import numpy as np
import scipy.sparse as sp

rng = np.random.default_rng(0)
D = rng.random((6, 5))
D[D < 0.6] = 0.0
print("المصفوفة الكثيفة:")
print(np.round(D, 3))

csr = sp.csr_matrix(D)
coo = sp.coo_matrix(D)
lil = sp.lil_matrix(D)

for name, m in [("csr", csr), ("coo", coo), ("lil", lil)]:
    print(f"{name:4s} nnz={m.nnz:3d} shape={m.shape} has getformat={hasattr(m,
'getformat')}")

print("\ncsr.indptr  (أين يبدأ كل صفّ)  :", csr.indptr.tolist())
print("csr.indices (عمود كل قيمة)     :", csr.indices.tolist())
print("csr.data    (القيم نفسها)      :", np.round(csr.data, 3).tolist())
print("coo.row     :", coo.row.tolist())
print("coo.col     :", coo.col.tolist())
print("\nالصفّ 3 في csr:", csr[3].toarray())
print("تحويل ذهاباً وإياباً بلا فقد:", (csr.tolil().tocsr() != csr).nnz == 0)
```

اقرأ `csr.indptr` على مهل: `[0, 2, 6, 9, 10, 13, 17]`. بما أنّ الطول ٦ صفوف + ١، فإن الصفّ رقم `i` يحتلّ الشرائح `indptr[i]` إلى `indptr[i+1]`. أي أنّ الصفّ الرابع هو `data[10:13]` مقترناً بـ `indices[10:13]`. هذا كلّ ما في `CSR`.

> **لماذا `coo` وليس `csr` للبناء؟**
>
> `coo` تخزّن ثلاث قوائم مستقلّة: صفّ، وعمود، وقيمة. بناؤها لا يحتاج أيّ ترتيب، فيناسب البيانات القادمة من ملفات وملصقات. لكن ضرب المصفوفات فيها بطيء، لذلك يوصي `SciPy` دائماً بالتحويل: `coo.tocsr()` ثم العمل. هذا بالضبط ما تفعله `scikit-learn` داخلياً.

٢.٣ كم يوفّر فعلاً؟ القاعدة الحسابية

الحساب يدوياً بسيط. `CSR` يحتاج ثلاثة متّجهات: قيمة ثمانية بايتات وفهرس أربعة بايتات لكل عنصر غير صفر، ومؤشّر نهاية واحد لكل صفّ.

$$M_{\text{CSR}} = \text{nnz} \cdot (8 + 4) + (m + 1) \cdot 4$$

في مقابل الكثيف:

$$M_{\text{dense}} = n \cdot m \cdot 8$$

المعادلتان تتساويتان تقريباً عند:

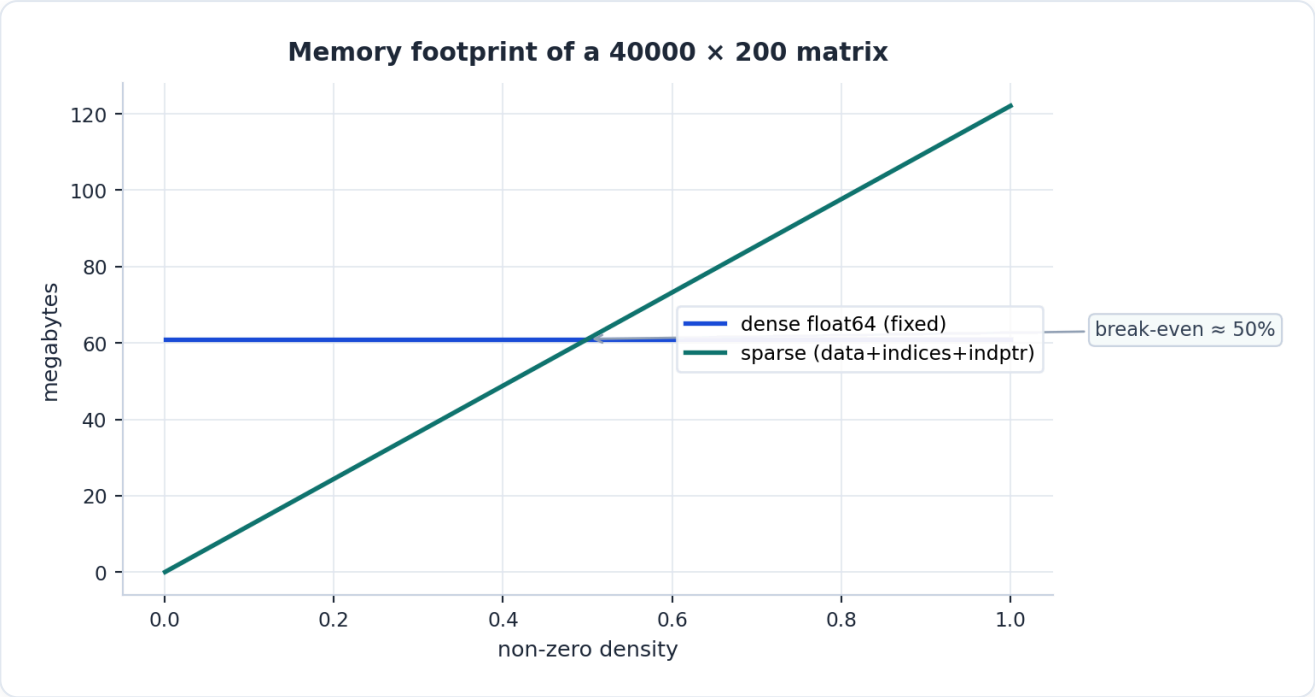
$$\rho \approx \frac{12}{8 \cdot 1 + \frac{4}{n}} \approx 1.5$$

أي فوق ١٥٠٪، وهذا مستحيل. عملياً: `CSR` يكسب دائماً في الذاكرة، والسؤال هو هل يكسب في **الزمن**؟

جدول الكلفة الحقيقي عبر كثافات مختلفة — python

```
n_rows, n_cols = 500, 400
print(f"{'الكثافة':<8} {'nnz':>8} {'كثيف (MB)':>11} {'CSR (KB)':>10} {'التوفير':<9}")
for d in [0.005, 0.02, 0.10, 0.30]:
    M = sp.random(n_rows, n_cols, density=d, format="csr", random_state=0)
    csr = M.data.nbytes + M.indices.nbytes + M.indptr.nbytes
    dn = n_rows * n_cols * 8
    print(f"{d:8.3f} {M.nnz:8d} {dn / 1024**2:11.2f} {csr / 1024:10.1f} {dn /
csr:8.0f}x")
```

لاحظ العمود الأخير: التوفير ليس خطّياً في الكثافة بل يتناسب معها تقريباً. عند كثافة ٠٫٠٣ يتوفّر المصفوفةُ الكثيفةُ مساحةً ثمانية أضعاف كاملة.



الشكل ٢٧مقارنة بصرية لحجم الذاكرة بين المصفوفة الكثيفة وعدّة صيغ متفرّرة، والخيط thick يرتفع مع الكثافة في كلّ الحالات

٢.٤ متى يكسب التمثيل المتفرّر في الزمن أيضاً؟

الذاكرة وحدها لا تكفي لتبرير قرار معماري. السؤال الثاني: هل تُسرِّع الحسابات؟ الجواب يعتمد على خوارزمية المُقدِّر.

$$\text{dense}: O(n \cdot d) \qquad \text{sparse}: O(\text{nnz})$$

بمعنى: أيّ عملية تمرّ على كلّ الخانات تدفع ثمن الكثافة، وأيّ عملية تمرّ على القيم غير الصفرية فقط تستفيد فوراً. المسافة الإقليدية في `k`-الجيران المثال الأوضح: على `float64` تحتاج أن تقرأ ٨ بايتات لكل خانة، وعلى `CSR` تقرأ ١٢ بايتاً لكل قيمة **غير صفرية** فقط.

الفارق الزمني على ٣٠٠٠ عيّنة و ٨٠٠ خاصية python

```python
import time
from sklearn.neighbors import NearestNeighbors

n, p = 3000, 800
X_dense = rng.random((n, p))
X_sparse = sp.random(n, p, density=0.01, format="csr", random_state=0)
q_sparse = sp.random(1, p, density=0.01, format="csr", random_state=1)
q_dense = rng.random((1, p))

t0 = time.perf_counter()
NearestNeighbors(n_neighbors=5).fit(X_sparse).kneighbors(q_sparse)
t_sparse = time.perf_counter() - t0

t0 = time.perf_counter()
NearestNeighbors(n_neighbors=5).fit(X_dense).kneighbors(q_dense)
t_dense = time.perf_counter() - t0

dense_bytes = X_dense.nbytes
sparse_bytes = X_sparse.data.nbytes + X_sparse.indices.nbytes +
X_sparse.indptr.nbytes
print(f"الذاكرة : كثيفة {dense_bytes / 1024**2:.1f} متفرّزة | ميغابايت {sparse_bytes /
1024:.0f} كيلوبايت")
print(f"الزمن    : متفرّزة {t_sparse:.4f} ث | كثيفة {t_dense:.4f} ث | تسريع {t_dense /
t_sparse:.0f}x")
```

الفارق هنا ليس تجميلياً: نحن نتحدّث عن مصفوفة ١٨ ميغابايت فقط. في مشروع حقيقي بعشرات الآلاف من الخصائص الفئوية، يصبح الفارق بين دقائق وساعات.

الخطأ الأول: افتراض أن كل مُقدِّر يتعامل مع التفرّز

`LinearRegression` و `KNeighborsClassifier` و `SVC` و `MLPClassifier` تعلن `sparse` في توثيقها وتقبل `CSR`. لكن `RandomForest` و `GradientBoosting` و `MLP` في نسخ كثيرة منه ترفضه صراحةً بـ `TypeError: A sparse matrix was passed, but dense data is required`. **افحص** `get_params` **أو جرّب مصفوفة صغيرة قبل أن تُعيد بناء خط أنابيب كامل حول هذه الافتراض.**

الخطأ الثاني: التحويل إلى كثيف «لمجرّد الفهم»

`.toarray()` على مصفوفة ٢٤٠٠٠٠٠ خانة ينتج ١٩ غيغابايت، و `MemoryError` قادم. لو أردت تفقّد البنية، اطبع `A[:20].toarray()` — عشرون صفّاً تكفي لفهم البنية، ولا تكسر ذاكرتك.

٢.٥ من يقبل المتفرّز؟ من يرفضه؟

قبل أن تكتب أيّ شيء، افحص وجود `sparse` في `fit` للصف الذي تستخدمه.

الصنف	يقبل `csr`؟	ملاحظة
`sklearn.linear_model.Ridge`	نعم	الحلّ يتكرّر عبر `nnz`
`sklearn.linear_model.LogisticRegression`	نعم	أفضل من `liblinear` في الفروق الكبيرة
`sklearn.neighbors.NearestNeighbors`	نعم	أسرع فرق معروف في NN-`k`
`sklearn.svm.SVC`	نعم	يعتمد على `libsvm` المجمّع
`sklearn.naive_bayes.MultinomialNB`	نعم	مصمَّم لبيانات `TF-IDF`
`sklearn.preprocessing.StandardScaler`	بشرط	إلزامي `with_mean=False`
`sklearn.cluster.KMeans`	لا	يرفض
`sklearn.ensemble.RandomForestClassifier`	لا	يرفض
`sklearn.neural_network.MLPClassifier`	لا	يرفض

هذا الجدول ليس عشوائياً: الفارق هو **هل تمرّ الخوارزمية على كلّ الخانات أم على القيم غير الصفرية فقط**.

وهناك طرف ثالث يتصرّف نيابةً عنك: `ColumnTransformer`. فهو يدمج مسارات قد يخرج أحدها متفرّزاً والآخر كثيفاً، ثم يقرّر نوع الناتج النهائي وحده.

كيف يقرّر ColumnTransformer أن يضغط النتيجة؟ python

```python
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler

set_config(transform_output="default")
rng2 = np.random.default_rng(0)
n = 600
wide = pd.DataFrame({f"cat{j}": rng2.choice([f"v{k}" for k in range(60)], n) for
j in range(40)})
for j in range(5):
    wide[f"num{j}"] = rng2.normal(size=n)
cat_cols = [c for c in wide.columns if c.startswith("cat")]
num_cols = [c for c in wide.columns if c.startswith("num")]

spec = [("num", StandardScaler(), num_cols), ("cat",
OneHotEncoder(handle_unknown="ignore"), cat_cols)]
for thr in [0.3, 1.0, 0.0]:
    Z = ColumnTransformer(spec, sparse_threshold=thr).fit_transform(wide)
    dens = Z.nnz / np.prod(Z.shape) if hasattr(Z, "nnz") else
np.count_nonzero(Z) / np.prod(Z.shape)
    print(f"  sparse_threshold={thr:<4} -> {type(Z).__name__:11s} shape=
{Z.shape} density={dens:.4f}")

Z = ColumnTransformer(spec).fit_transform(wide)
sparse_b = Z.data.nbytes + Z.indices.nbytes + Z.indptr.nbytes
print(f"\nلو ضُغطت: {sparse_b / 1024:.0f} لو كُثّفت   |   كيلوبايت: {Z.shape[0] *
Z.shape[1] * 8 / 1024**2:.1f} ميغابايت")
```

لاحظ الشرط الحاسم: **إذا كانت كثافة الناتج المدمج أعلى من العتبة، يُعاد** `ndarray`. ولهذا السبب لا يمكنك أن تأمر `sparse_threshold=1.0` وتنتظر ضغطاً مضموناً: كلّ كثافة تساوي واحداً أو تقلّ عنه تبقى متفرّزة.

ولنتحقّق من شرط `StandardScaler` عملياً، لأنه شرطٌ صامت:

الشرط الذي لا بدّ من معرفته: StandardScaler والمتفرّز python

```python
from sklearn import set_config
from sklearn.preprocessing import StandardScaler

# sparse output is incompatible with pandas containers, so go back to arrays
set_config(transform_output="default")

Xsp = sp.csr_matrix(np.array([[1.0, 0.0, 2.0], [0.0, 3.0, 0.0], [2.0, 0.0,
1.0]]))

try:
    StandardScaler().fit_transform(Xsp)
except Exception as exc:
    print("with_mean=True  ->", type(exc).__name__, ":", str(exc).splitlines()
[0][:96])

out = StandardScaler(with_mean=False).fit_transform(Xsp)
print("with_mean=False ->", type(out).__name__, out.shape, "يعمل")
print("القيم بعد التحويل (مصفوفة أصلاً كانت فيها أصفار):")
print(np.round(out.toarray(), 3))
```

لاحظ الجملة الأخيرة في رسالة الخطأ: `Cannot center sparse matrices`. ليست مشكلة ذاكرة بل مشكلة **دلالة**: طرح المتوسط يجعل الأصفار في المتّجه غير صفرية، فتصير صفراً بلا معنى. لذلك التطبيع بالتباين فقط.

٢.٦ sparray: الواجهة الجديدة التي تحتاج أن تعرفها

هنا change جوهري في `SciPy` ثم في `scikit-learn`. `SciPy` صارت تفرّق بين صنفين:

في `scikit-learn`، `sklearn` تبنّت الواجهة الجديدة تحت مفتاح إعدادات واحد:

```python
# نفس العملية، نوعان مختلفان من الإرجاع
from sklearn import set_config, get_config, config_context
from sklearn.preprocessing import OneHotEncoder

# enable the array-style container so sparse output is allowed at all
set_config(transform_output="default")

X_cat = np.array([["a"], ["b"], ["a"], ["c"]], dtype=object)
print("القيمة الافتراضية لـ sparse_interface:", get_config()["sparse_interface"])

for mode in ["spmatrix", "sparray"]:
    set_config(sparse_interface=mode)
    out = OneHotEncoder().fit_transform(X_cat)
    print(f"  sparse_interface={mode:8s} -> {type(out).__name__:11s} shape=
{out.shape}")

with config_context(sparse_interface="sparray"):
    print("  داخل config_context ->",
type(OneHotEncoder().fit_transform(X_cat)).__name__)
print("  بعد الخروج من السياق ->",
type(OneHotEncoder().fit_transform(X_cat)).__name__)
```

القاعدة العملية: **القيمة الافتراضية في ١.٩ ما زالت** `"spmatrix"`، ولذلك لن ينكسر كودك القديم. التبديل إلى `"sparray"` مشروع بالكامل، لكن افهم ما الذي تحرّك قبل أن تفعل.

ما تغيّر	مع `spmatrix`	مع `sparray`
صنف الإرجاع	`scipy.sparse.csr_matrix`	`scipy.sparse.csr_array`
دلالة الضرب `A @ B`	ضرب مصفوفي	ضرب نقطي-مصفوفي
`A.getformat()`	موجودة	غير موجودة
`sp.isspmatrix_csr(A)`	`True`	`False`
`sp.issparse(A)`	`True`	`True`
التوافق مع `scipy` **القديم**	كامل	يحتاج إصدارات حديثة

وهذه ليست تفاصيل تاريخية. ثلاث دوال فحص شائعة في كود الإنتاج — `getformat` و `isspmatrix_csr` و `A.ndim` — تعطي نتائج مختلفة تحت الواجهة الجديدة. اكتب `sp.issparse(A)` بدل `sp.isspmatrix_csr(A)` إن أردت كوداً ينجح في الحالتين.

```python
أين يختلف السلوك فعلاً؟
dense = np.array([[1.0, 0.0, 2.0], [0.0, 3.0, 0.0]])
m = sp.csr_matrix(dense)
a = sp.csr_array(dense)

print("csr_matrix.getformat() :", m.getformat())
print("csr_array.getformat()  :", hasattr(a, "getformat"), " (السمة .format =",
a.format, ")")
print("sp.isspmatrix_csr      :", sp.isspmatrix_csr(m), sp.isspmatrix_csr(a))
print("sp.issparse            :", sp.issparse(m), sp.issparse(a))
print("ndim / shape / nnz      :", (m.ndim, a.ndim), (m.shape, a.shape), (m.nnz,
a.nnz))
print("tocsr() تحافظ على الصنف:", type(m.tocsr()).__name__,
type(a.tocsr()).__name__)

set_config(sparse_interface="spmatrix")
```

٢.٧ متى ينكسر كودك عند التبديل؟

هناك مفاجأة واحدة يجب أن تعرفها قبل أن تفعّل `sparray` في مشروعك: إذا كان لديك

```
محوِّل يُخرج بيانات متفرّزة سيرفض العمل.

```python exec="run" title="التضارب بين مخرجات pandas المتفرّزة والمخرجات"
from sklearn.preprocessing import OneHotEncoder

with config_context(transform_output="pandas"):
    try:
        OneHotEncoder().fit_transform(X_cat)
    except Exception as exc:
        print("pandas + متفرّز ->", type(exc).__name__, ":", str(exc).splitlines()
[0][:110])

with config_context(transform_output="pandas"):
    ok = OneHotEncoder(sparse_output=False).fit_transform(X_cat)
    print("pandas + كثيف  ->", type(ok).__name__, ok.shape)
```

هذه ليست عطباً في التصميم بل قرار واضح: `DataFrame` من `pandas` لا تمثّل مصفوفة متفرّزة بشكلها الطبيعي. فإذا كنت تريد جدولاً مضغوطاً، اختر أحد الأمرين:
```

٢.٨ في الواقع: أين **встреча** المصفوفات المتفرّرة فعلاً؟

ثلاثة مواضع تتكرّر في كلّ مشروع إنتاجي تقريباً:

في الحالات الثلاث القاعدة نفسها: **لا تُكثّف أبداً ما لم يُجبرك على ذلك**.

> 🏗️ **في الواقع: الاستثناء الوحيد**
>
> هناك حالة واحدة يُنصح فيها بترك المتفرّر: عندما تجرّب ثلاث نماذج مختلفة على نفس الجدول وتريد مقارنة عادلة بينها. بعض النماذج ترفض `CSR`، وبعضها يقبلها. الحلّ ليس مواءمة كلّ النماذج على تمثيل واحد، بل wrapping التحويل داخل `Pipeline` كي لا يتسرّب أيّ معلومة من بيانات الاختبار.

تمارين الفصل

التمرين ٢.٤

ابنِ مصفوفة `1000 × 500` بكثافات `0.01` و `0.05` و `0.2` و `0.6`. لكلّ كثافة اطبع: `nnz`، وحجم `CSR` الحقيقي، وحجم `ndarray` المكافئ، والنسبة. ثم ارسم الخلاصة في جملة واحدة: عند أيّ كثافة تصبح المصفوفة المتفرّرة غير مجدية؟

التمرين ٢.٥

اعرض مصفوفة `csr_matrix` وأختها `csr_array` على نفس البيانات، ثم اطبع الفرق في: نوع الصنف، و `.format`، و `getformat`، و `sp.isspmatrix_csr`، و `sp.issparse`، ودلالة `A @ A.T`. فسّر أيّ فرق منها يهمّ في كود الإنتاج وأيّها تجريدٌ بلا أثر.

> ✅ **الحلّ**
>
> الفروق الجوهرية ثلاثة:
>
> `getformat()` نفسها اختفت من `csr_array`، فأيّ كود يمرّ عليها يجب أن يقرأ `.format` بدلها.

التمرين ٢.٦

خُذ `ColumnTransformer` فيه مسار رقمي وآخر `OneHotEncoder` على أربعين عموداً فئوياً. اطبع كثافة الناتج ونوعه. ثم كرّر الأمر مع `sparse_threshold=1.0` و `sparse_threshold=0.0`، وفسّر لماذا يعطي الحدّان نوعين مختلفين رغم ثبات الكثافة.

✅ الحلّ

القرار داخل `ColumnTransformer` هو: احسب كثافة النتيجة المدمجة، ثم إن كانت **أعلى** من `sparse_threshold` أعد `ndarray` كثيفاً، وإلا أعد `CSR`.

في المثال: الكثافة ٠٫٠١٩ تقريباً. والقاعدة المطبَّقة في المكتابة حرفياً هي:

القاعدة كما نُفّذت — python

```
self.sparse_output_ = density < self.sparse_threshold
```

فمع الافتراضي `0.3` تكون الكثافة **أقلّ** من العتبة فيُحفظ متفرّزاً. ومع `1.0` تبقى أقلّ من العتبة فيُحفظ متفرّزاً أيضاً. أمّا `0.0` فالكثافة أكبر من الصفر، فيُعاد `ndarray` كثيفاً.

المعنى العملي: `sparse_threshold` هو **الحدّ الأدنى من الكثافة الذي لا نريد بعده الضغط**. اضبطه على الرقم الذي تمثّل به «هذه المصفوفة لم تعد تستحقّ الضغط». لا على الصفر، ولا على واحد.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
`StandardScaler` على `csr` بلا `with_mean=False`	`ValueError: Cannot center sparse matrices`	مرّر `False`
تفعيل `transform_output="pandas"` مع مخرجات متفرّرة	`ValueError: Pandas output does not support sparse data`	اجعل المخ `default` `ut=False`
استدعاء `.getformat()` على `csr_array`	`AttributeError`	استعمل تحقّق بـ `e`
`sp.isspmatrix_csr` كشرط قبول	`False` مع `csr_array` فيُرفض مصفوفة صالحة	استعمل
تمرير `csr` إلى `RandomForest`	`TypeError: dense data is required`	`barray()` البناء
`.toarray()` على مصفوفة ضخمة «لمعاينتها»	`MemoryError`	اطبع `barray()`
افتراض أن `CSR` دائماً أسرع	لا أثر في خوارزميات تمرّ على كل الخانات	قِس زمناً د التصميم
نسيان ضبط البذرة عند توليد متفرّر	نتائج غير قابلة للتكرار	`_state=0` `.random`

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
مصفوفة متفرّقة	تخزّن القيم غير الصفرية فقط	sparse matrix
مصفوفة-مصفوفة	الواجهة المعيارية الحديثة في `SciPy`	sparse array, sparray
الصفوف المتفرّقة	صيغة `CSR` بثلاثة متّجهات	compressed sparse row
الإحداثيات	صيغة `COO` بقوائم صفّ وعمود	coordinate format
القوائم المترابطة	صيغة `LIL` للتعديل العنصر-بعنصر	linked list format
القيم غير الصفرية	عدد العناصر التي لا تساوي صفراً	nnz
الكثافة	نسبة الخانات غير الصفرية إلى الكل	density
عتبة التفرّز	الحدّ الذي يقرر عنده `ColumnTransformer` الضغط	sparse_threshold
واجهة المصفوفات	مفتاح `sparse_interface` في `sklearn`	sparse interface
سياق الإعدادات	نطاق مؤقت لتغيير الإعدادات	config_context

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٢ — أنواع البيانات: عددي، وفئوي، وزمني، ونصي

كيف يميّز المُقدِّر نوع كل عمود، وماذا يفعل إن أخطأت

الوحدة **scikit-learn ١.٩** الحجم **٨١٦** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تصنّف أيّ عمود إلى عددي أو فئوي أو زمني أو نصي بقرار مبرَّر
- أن تشرح لماذا يُفسد التحويل الموحّد على أعمدة مختلطة دون أن يرفع استثناءً
- أن تبني مساراً زمنياً حقيقياً بخصائص دورية داخل ColumnTransformer
- أن تستعمل make_column_selector لتوجيه كل عمود إلى مساره تلقائياً

٢.١ `dtype` ليس تفصيلاً تقنياً

افترض جدولَ اشتراكات فيه أربعة أعمدة: `age` و `city` و `signup` و `note`. لو أخبرتَ نموذجاً لوجيستياً أن يتعلّم منه، فماذا سيحدث؟

الجواب الذي يتوقّعه المبتدئ: سيتعلّم. الجواب الصحيح: **سيرفض**. لأن العمودين الثاني والرابع نصّيان، والخلاصة snail أن كل «عمود» ليس كياناً واحداً.

المشكلة الأعمق أن التحويل الذي يُنصح به بذكاء — «طبّع كل شيء» أو «رمّز كل شيء» — يُنتج نموذجاً يبدو صحيحاً ويعمل تماماً، بينما الدلالة تشوّهت. في الفصل ١٤ سنرى أن `StandardScaler` على عمود نصّي بعد رمزه ترتيبياً يخلق علاقة **مُختلَقة** بين «Riyadh» و«Doha» لا وجود لها في الواقع.

لذلك القاعدة التي نفصل بها كل شيء:

> **🔑 قاعدة الأربعة**
>
> لكل عمود ثلاثة أسئلة قبل أيّ تحويل: **١)** هل هو قياس كمّي أم رمز فئوي؟ **٢)** إن كان كمّياً، هل توزيعه متماثل أم مايل؟ **٣)** هل له معنى زمني متكرّر؟ الإجابات الثلاث تحدّد المسار، لا القيمة.

٢.٢ خريطة الأنواع الأربعة

النوع في `pandas`	مثال	الأداة المناسبة	الفصل
عددي كمّي	`age` و `income`	`StandardScaler` أو `RobustScaler`	١٤
فئوي (رمز)	`city` و `plan`	`OneHotEncoder` أو `OrdinalEncoder`	١٦
زمني	`signup`	`FunctionTransformer` أو استخراج يدوي	١٢
نصّي حر	`note` و وصف المنتج	`TfidfVectorizer`	١٨
منطقي	`active`	يُحوَّل إلى ٠ و ١ تلقائياً	١٢

لاحظ أن البُعد الخامس (المنطقي) هو الحالة الوحيدة التي لا تحتاج قراراً: `True` و `False` يتحوّلان إلى `1` و `0` تلقائياً في كل مُقدِّر رقمي.

لنبنِ الجدول النموذجي ونفحصه.

```python
جدول واحد بأربعة أنواع: ما الذي يراه pandas؟
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
n = 200
frame = pd.DataFrame({
    "age": rng.integers(18, 80, n),
    "income": rng.lognormal(10, 1.1, n),
    "city": rng.choice(["Riyadh", "Cairo", "Amman", "Doha"], n),
    "plan": rng.choice(["free", "pro", "enterprise"], n),
    "signup": pd.to_datetime("2023-01-01") + pd.to_timedelta(rng.integers(0,
900, n), unit="D"),
    "note": rng.choice(["good service", "bad support", "fast delivery", "slow
app"], n),
    "active": rng.random(n) > 0.4,
})
print(frame.head(4).to_string(index=False))
print()
print(frame.dtypes)
print()
print(f"حجم الجدول: {frame.memory_usage(deep=True).sum():,} بايت")
```

لاحظ ثلاث تفاصيل تهمّ:

٢.٣ العددي: أخطاء تصمت ولا تصرخ

العمود العددي يبدو آمناً. هو الوحيد الذي لا يرفضه أحد. ومع ذلك يفسد النتائج بطرق لا تظهر في رسالة خطأ.

الخطأ الأول: مقياسان مختلفان بلا داعٍ. عمود عمر بين ١٨ و ٨٠، وعمود دخل بين ٧٠٠ و ٦٤٢ ألفاً. التطبيع القياسي يفعل الآتي:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

العمودان صارا متوسّطهما صفراً وتباعدهما واحداً. لكن قبل ذلك، لو استخدمت نموذجاً يستهلك مسافة مثل `KNeighborsClassifier`، لأصبح عمود الدخل وحده هو المسافة، لأنّ مداه أطول بمئة ضعف.

الخطأ الثاني: التوزيع مايل. الدخل الذي يتبع توزيعاً لوغاريتمياً يقع تحت المتوسط بثلاثة أضعاف الانحراف المعياري. متوسطه ٣٩ ألفاً، وكثافته تتركّز قرب الألف.

الخطأ الثالث: الشواذ. ثلاثة صفوف بأرقام فلكية تفسد كلّ إحصاء يعتمد على المتوسّط. هنا يفترق `StandardScaler` عن `RobustScaler` بصوت مسموع.

```python
# الرقم الشاذّ: أين يظهر في كلّ محوّل؟
from sklearn.preprocessing import StandardScaler, RobustScaler

num = frame[["age", "income"]]
print("قبل:  income   min %8.0f  max %10.0f" % (num["income"].min(),
num["income"].max()))

sc = StandardScaler().fit(num)
z = sc.transform(num).to_numpy()[:, 1]
print("z-score من StandardScaler : 2.% الوسيطf | 2.% الطرف الأعلىf" % (np.median(z),
z.max()))

polluted = num.copy()
polluted.loc[0:2, "income"] = polluted["income"].max() * 40
rz = RobustScaler().fit(polluted[["income"]])
rs = rz.transform(polluted[["income"]]).to_numpy()[:, 0]
sz =
StandardScaler().fit(polluted[["income"]]).transform(polluted[["income"]]).to_nu
mpy()[:, 0]
print("مع الشواذ: Robust   7.0% الوسيطf  9.1% الطرفf" % (rz.center_[0], rs.max()))
print("مع الشواذ: Standard 7.0% الوسيطf  9.1% الطرفf" % (
    StandardScaler().fit(polluted[["income"]]).mean_[0], sz.max()))
```

لاحظ الرقم الأخير في المخرجات. `StandardScaler` ضرب المتوسّط من ٣٩ ألفاً إلى أكثر من ٤٠٠ ألف بسبب ثلاثة صفوف فقط، أي أنّ «مركز البيانات» صار تعريفاً لهذين الصفوف الثلاثة لا لعمود الدخل. `RobustScaler` ببساطة استخدم الوسيط والمدى الربيعي، فبقي مركزه قرب ١٩ ألفاً. **هذه ليست مسألة «أيّهما أفضل»، بل مسألة أيّهما يطابق السؤال الذي تسأله عن القيم الشاذّة.**

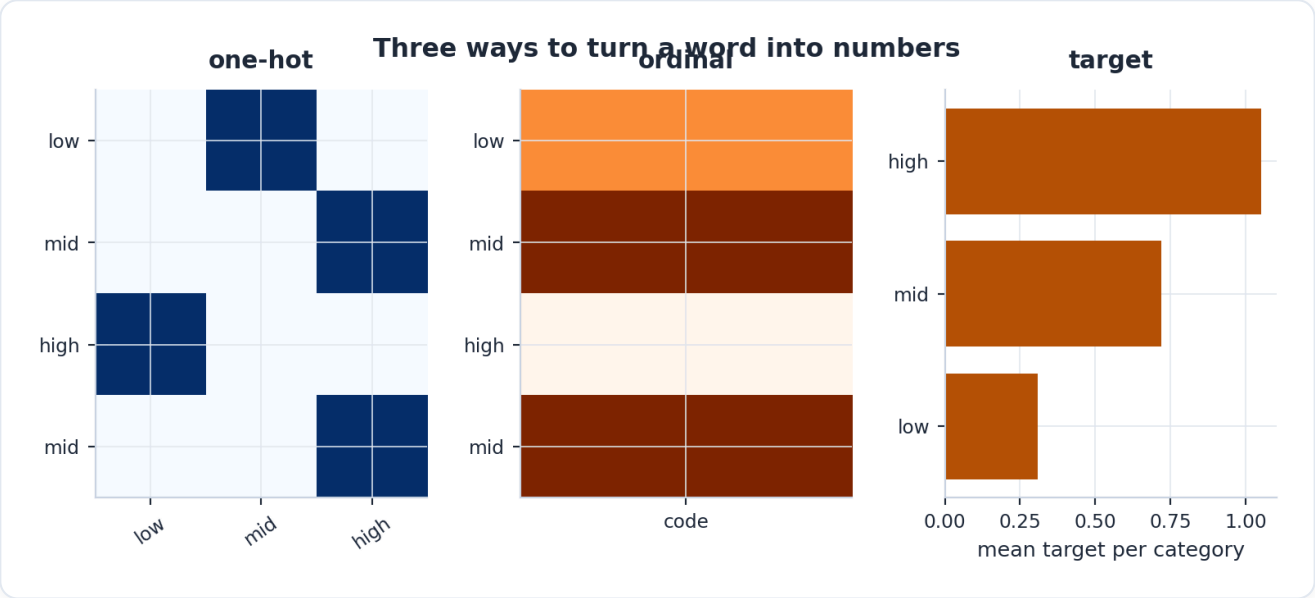
٢.٤ الفئوي: لماذا يرفضه كل مُقدِّر رقمي؟

المدينة «Riyadh» ليست عدلاً أكبر ولا أصغر من «Cairo». الرقم ٣ لا يعني شيئاً أكثر من ١. لكن `LogisticRegression` لا يعرف هذا، فلو سمحتَ له برمز ٠ و١ و٢ فسيبني حدّاً يفترض فيه ترتيباً لم تخترعه أنت.

لذلك هناك طريقتان عادلتان:

الترميز	كم عموداً يُنتج	ماذا يعني الرقم	متى تختاره		
`OneHotEncoder`	`C`	لكل فئة `C`		٠ أم ١، بلا ترتيب	الفئات بلا ترتيب طبيعي
`OrdinalEncoder`	١	ترتيب مصطنع	الفئات **لها** ترتيب حقيقي		

مدن الجدول الأربع، لو رمّزتها بـ `OrdinalEncoder`، لأصبحت `Riyadh = 3` و `Doha = 0`. أيّ شجرة قرار تتعلّمها ستقول إنّ الدوحة قريبة من الرياض. هذا ليس خطأ برمجياً بل خطأ في وصف المشكلة.



الشكل ٢٨ثلاث لوحات لترميز الفئات: OneHotEncoder يبني ثلاثة أعمدة ثنائية، وOrdinalEncoderg يبني عموداً واحداً بترتيب مخترع، وترميز الهدف يختزل الفئة في متوسّط التسمية

`OrdinalEncoder` ليس شرّاً مطلقاً. إذا كانت فئاتك مرتّبة فعلاً — `low` و `medium` و `high` — فالترميز الترتيبي مقصود ومحلل. الشرط الوحيد أن يكون الترتيب موجوداً في بياناتك، لا في خيالك.

الترميز على الأعمدة الفئوية python

```python
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

cats = frame[["city", "plan"]]

ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore").fit(cats)
print(f"OneHotEncoder -> {ohe.transform(cats).shape[1]} أعمدة")
print("  الأسماء:", list(ohe.get_feature_names_out()))

oe = OrdinalEncoder(handle_unknown="use_encoded_value",
unknown_value=-1).fit(cats)
print(f"OrdinalEncoder -> {oe.transform(cats).shape[1]} أعمدة")
print("  الفئات:", [c.tolist() for c in oe.categories_])
print("  أول ٦ قيم:", oe.transform(cats).to_numpy()[:6, 0].ravel().astype(int))

rare = pd.DataFrame({"city": ["Riyadh"] * 60 + ["Cairo"] * 20 + ["Amman"] * 12 +
["Doha"] * 8})
print("\nتوزيع المدينة قبل التجميع:", rare["city"].value_counts().to_dict())
folded = OneHotEncoder(min_frequency=15, sparse_output=False,
                       handle_unknown="infrequent_if_exist").fit(rare[["city"]])
print("بعد min_frequency=15:", list(folded.get_feature_names_out()))
plain = OneHotEncoder(sparse_output=False).fit(rare[["city"]])
print("بدون min_frequency :", len(plain.get_feature_names_out()), "أعمدة")
```

لاحظ السطر الأخير: الفئات النادرة لم تُحذف، بل **طُويت** في عمود واحد اسمه `infrequent_sklearn`. هذه الخاصية اسمها `min_frequency` وهي متاحة في ١.٩، وتحميك من انفجار الأبعاد حين يكون أحد أعمدتك ألف قيمة مختلفة. جرّب أن ترفعها إلى ٣٠ لتتوقّف عن فائدة العمود الأخير أيضاً.

٢.٥ الزمني: التاريخ ليس رقماً... وأحياناً يكون

عمود `signup` يخزّن `datetime64`. لا يقبله `StandardScaler`، ولا يقبله `OneHotEncoder`، ولا يقبله أيّ شيء آخر. لماذا؟ لأنّ له معنى دوريّ: الفرق بين كانون الثاني وأكتوبر ٨٩ يوماً، لا ٣٠٠.

ثلاث طرق للتعامل معه، من الأسهل إلى الأدقّ:

$$s_k = \sin\left(\frac{2\pi k}{p}\right), \qquad c_k = \cos\left(\frac{2\pi k}{p}\right)$$

حيث `k` رقم الدورية و `p` طولها: ١٢ للشهر، و ٧ لأيام الأسبوع، و ٢٤ للساعة.

```python
# من تاريخ إلى خمسة أرقام ذات معنى
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import FunctionTransformer


def date_parts(d):
    s = d.iloc[:, 0] if hasattr(d, "iloc") else d
    month = s.dt.month
    return pd.DataFrame({
        "year": s.dt.year,
        "month": month,
        "dow": s.dt.dayofweek,
        "month_sin": np.sin(2 * np.pi * month / 12),
        "month_cos": np.cos(2 * np.pi * month / 12),
    })


ct_dt = ColumnTransformer([("dt", FunctionTransformer(date_parts), ["signup"])])
print("مخرجات مسار الزمن:")
print(ct_dt.fit_transform(frame.head(5)).round(3))
print()
print("أسماء المخرجات:", list(ct_dt.get_feature_names_out()))
```

لاحظ أن `FunctionTransformer` يمكنه أن يفعل أيّ شيء: استخراج مكوّنات، أو حساب نسبة، أو تحويل وحدة. هو الجسر بين `pandas` الدلالي وعالم `scikit-learn` العددي. والفصل ١٧ يتعمّق في بناء تحويلات relatives من هذا النوع.

> **لماذا `d.iloc[:, 0]` داخل الدالة؟**
>
> لأن `ColumnTransformer` يمرّر للـ `FunctionTransformer` إطاراً بعمود واحد (شكله `n × 1`)، بينما `TfidfVectorizer` يتوقّع متّجهاً نصّياً. تسمية `iloc` تخلّصك من هذا الاختلاف بين مسارات المشروع الواحد. الدوال المكتوبة لتتحمّل كلا الشكلين أرشيفٌ في كلّ مشروع إنتاجي حقيقي.

٢.٦ النصّي: عمودٌ واحد أم مليون مستند؟

هناك فرق جوهري بين عمود نصّي **فئوي** (القيمة منه عدد محدود: `satisfied` و `neutral` و `angry`) وعمود نصّي **حرّ** (القيمة من فضاء مفتوح: ملاحظات عملاء، وصفحات ويب، تغريدات). الأول يعالَج بـ `OneHotEncoder` في seconds. الثاني يحتاج `TfidfVectorizer` الذي يحوّل كلّ مستند إلى متّجه آلاف الأبعاد — موضوع الفصل ١٨ بالكامل.

```python
عمود نصّي حرّ: من جملة إلى متّجه
from sklearn.feature_extraction.text import TfidfVectorizer

tv = TfidfVectorizer(max_features=4)
M = tv.fit_transform(frame["note"])
print("نوع المخرجات:", type(M).__name__, " الشكل:", M.shape)
print("المفردات:", list(tv.get_feature_names_out()))
print("\nأول ٤ صفوف كمصفوفة كثيفة:")
print(pd.DataFrame(M.toarray(),
columns=tv.get_feature_names_out()).head(4).round(3).to_string(index=False))
```

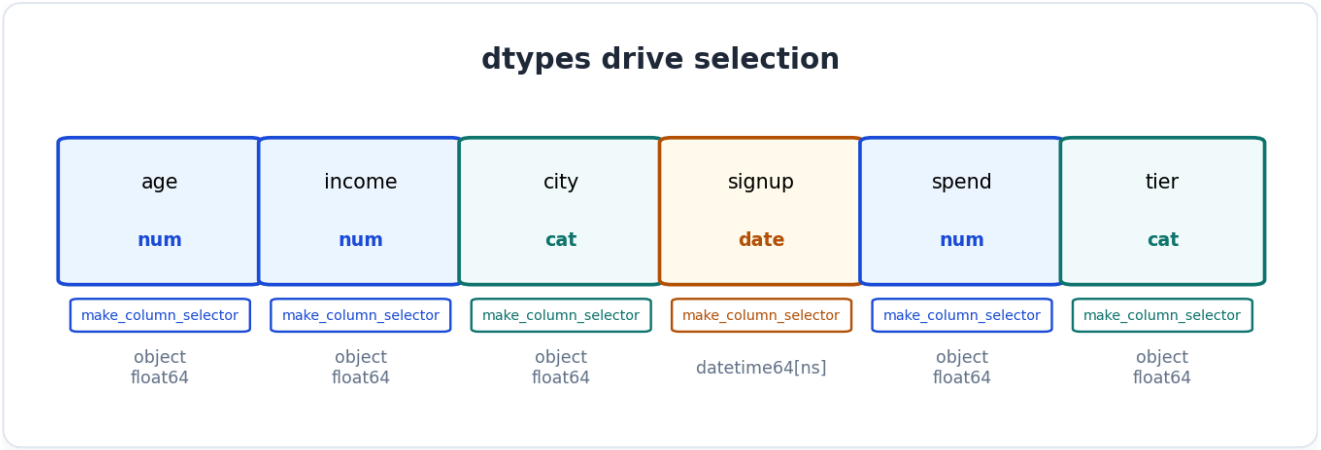
<!----->

وهنا تحذير عملي وكثير الوقوع: **لا تضع** `TfidfVectorizer` **داخل** `ColumnTransformer` **في ١.٩**. السبب أنّه يتوقّع متّجهاً نصّياً واحداً، بينما `ColumnTransformer` يسلّمه إطاراً بعمود واحد، فيخرج المخرج بشكل غير صحيح. الحلّ: عالِج النصّ في `Pipeline` منفصل، أو استقبل عموداً واحداً قبل أن يمرّ. لو فعّلت `transform_output="pandas"` على مستوى الجلسة، يظهر خطأ آخر هنا أيضاً: المتّجه المتفرّر لا يمكن أن يُغلَّف في `DataFrame`.

الحالة	الأداة	الإخراج
عمود فئوي محدود القيم	`OneHotEncoder`	أعمدة ٠/١
عمود فئوي مرتّب فعلاً	`OrdinalEncoder`	عمود أرقام
نصّ حرّ داخل جدول	`TfidfVectorizer` مستقلّاً	مصفوفة `csr`
نصّ في مشروع كامل	`Pipeline` مستقلّ	نموذج كامل

٢.٧ `make_column_selector`: تفويض القرار للنوع

هل يمكن ترك المكتبة تختار لكل عمود مساره؟ نعم، وهذه أداة `make_column_selector` من `sklearn.compose`. هي **دالة تُعيد دالة**: تستقبلها `ColumnTransformer`، وتُستدعى وقت `fit` فتقرأ `dtype` لكل عمود وتعيد قائمة أسماءه.



الشكل ٢٩رسم يوضّح أن المُحدِّد يعتمد على نوع العمود لا على اسمه، فيوجّه الأرقام والمنطقي والنصوص إلى مسارات مختلفة

دالة تُعيد دالة: من قراءة الأنواع إلى مسارين python

```python
from sklearn.compose import make_column_selector
from sklearn.preprocessing import OneHotEncoder, StandardScaler

for kw in [dict(dtype_include="number"), dict(dtype_include="string"),
           dict(dtype_include="datetime"), dict(dtype_include=bool),
           dict(dtype_exclude="object"), dict(pattern="^(city|plan)$")]:
    print(f"  {str(kw):45s} -> {make_column_selector(**kw)(frame)}")

print("\n--- والآن نحوّل هذه القراءة إلى خطّ فعلي ---")
ct_auto = ColumnTransformer([
    ("num", StandardScaler(), make_column_selector(dtype_include="number")),
    ("cat", OneHotEncoder(sparse_output=False, handle_unknown="ignore"),
     make_column_selector(dtype_include="string", pattern="^(city|plan)$")),
])

Z = ct_auto.fit_transform(frame)
print("نوع الناتج:", type(Z).__name__, " الشكل:", Z.shape)
print("أعمدة المسار الرقمي :", [c for c in ct_auto.get_feature_names_out() if
c.startswith("num")])
print("أعمدة المسار الفئوي :", [c for c in ct_auto.get_feature_names_out() if
c.startswith("cat")])
print("أعمدة مُهمَلة بصمت   :", ["note", "signup", "active"])
```

لاحظ أن `dtype_include="object"` و `dtype_include="string"` يُنتجان **النتيجة نفسها** في `pandas 3.0`، لأن أعمدة `str` الجديدة تُعامَل على أنها `object` من قبل أداة الاختيار. هذه تفاصيل تُكتشف بالتجربة لا بالتوثيق، فجرّب دائماً المُحدِّد على بياناتك قبل أن تسلّمه خطّ الأنابيب.

هذا هو النمط الذي ستلتقي به في كلّ مشروع إنتاج: **جدول واحد، مسارات متعددة، وقرار التوجيه لكل عمود مبنيّ على نوعه لا على اسمه**. وأسماء المسارات (`num` و `cat`) تظهر في `get_feature_names_out` على شكل `num__age` و `cat__city_Amman`، فهي هويّة كلّ خاصية في تقريرك.

الخطأ: الاعتماد على `remainder='drop'` دون قصد

في المثال أعلاه، الأعمدة `note` و `signup` و `active` لا تنتمي إلى أيّ مسار، فتسقط بصمت لأن `remainder` قيمته الافتراضية `'drop'`. النموذج يعمل، ودقّته مقبولة، ولا أحد يعرف أن نصف البيانات لم يُستخدم. **عند إنشاء `ColumnTransformer` أول مرة، اضبط `remainder='passthrough'` أو `'drop'` عمداً، وطبع `get_feature_names_out()` لتتأكّد.**

٢.٨ في الواقع: العمود «الرقمي» الذي فيه نص

أخطر ما في هذا الفصل ليس الفئوي ولا الزمني، بل ما يبدو رقمياً ولا يكون كذلك.

في الأنظمة الإنتاجية، شائع جداً أن يقرأ أحدهم ملفاً من نظام قديم فيصل عمود `quantity` بنوع `str` لأن أحد الصفوف احتوى `"unknown"` أو مسافة في آخره. النتيجة: النموذج يُدرَّب، والتحذير يصمت، والدقّة تهبط بنسبة لا يعرف أحد سببها.

القاعدة المهنية التي تحميك:

كيف تكشف عموداً رقمياً متسلّلاً؟ python

```
raw = pd.DataFrame({"quantity": ["12", "7", "unknown", "30", " 5 "]})
print("النوع الأصلي:", raw["quantity"].dtype)
clean = pd.to_numeric(raw["quantity"], errors="coerce")
print("بعد to_numeric:", clean.dtype, "| قيم NaN:", int(clean.isna().sum()))
print("لماذا هذا أفضل من الخطأ الصامت؟ لأن الصف الثالث صار معلومة مفقودة")
print("في صمت أو يُرفض المشروع كله NaN يمكن دراستها، بدل أن تتحوّل إلى.")
print()
print(raw.assign(quantity=clean).to_string(index=False))
```

في الواقع: عقد البيانات

في الفرق التي تعمل بانتظام، يُكتب «عقد بيانات» (data contract) يحدّد لكل عمود: النوع المسموح، والوحدة، وماذا يعني الفراغ، ومن يملك صلاحية تغييره. هذا ليس ترفاً تنظيمياً: هو ما يمنع أن ينهار نموذج في يوم بسبب عمود أضافه زميل في جدول التسعير. ثلاثة أسئلة تكفي لبداية عملية: ما نوع العمود؟ ما وحدة قيمه؟ ما معنى الفراغ فيه؟

تمارين الفصل

التمرين ٢.٧

أنشئ جدولاً فيه ستّة أعمدة: عدد صحيح، عدد عشري، رمز فئوي، تاريخ، نصّ حرّ، ومنطقي. اطبع `dtypes` ثم استخدم `make_column_selector` بأربع طرق مختلفة وعدّد الأعمدة التي يختارها كلّ مرة. اربط كل نتيجة بقاعدة من القواعد الأربعة.

التمرين ٢.٨

خذ عمود `month` من القسم ٢.٥ وقارن ثلاث تمثيلات: `(أ)` الرقم الخام من ١ إلى ١٢، `(ب)` الترميز الدوري بـ `sin` و `cos`، `(ج)` `OneHotEncoder` باثني عشر عموداً. احسب المسافة الإقليدية بين ديسمبر ويناير في كلّ حالة، ولاحظ أيّها يضعهما متجاورين وأيّها يضعهما متقابلين.

> ✅ **الحلّ**
>
> في تمثيل `(أ)` المسافة بين ١ و١٢ تساوي ١١ — أبعد ما يمكن. في تمثيل `(ب)` المسافة بين النقطتين على الدائرة تساوي:
>
> $$d = \sqrt{(s_1 - s_2)^2 + (c_1 - c_2)^2} = 2\sin\!\left(\frac{\pi |k_1 - k_2|}{p}\right)$$
>
> ولأن `|1 - 12| = 11` و `p = 12`، فإنّ `d = 2 sin(11\pi/12) \approx 0.518` — أي قربان جداً على الدائرة، لأن ديسمبر يسبق يناير مباشرةً. أمّا تمثيل `(ج)` فيضعهما في عمودين مختلفين تماماً، فالمسافة بينهما `\sqrt{2} \approx 1.414`.
>
> الخلاصة: التمثيل الدوري يحترم الدورية الطبيعية، والرقمي يقطعها بلا داعٍ.

التمرين ٢.٩

خذ `frame` من هذا الفصل، وأضف عموداً نصّياً فيه قيمة واحدة `"unknown"` وسط مئة صفّ رقمي. طبّق `pd.to_numeric(errors="coerce")` ثم `SimpleImputer`، وقارن عدد القيم الفاقدة قبل وبعد. احسب نسبة الصفوف المتأثّرة، وقرّر: هل هذا عمود Numeric سليم أم عيب في المصدر؟ برّر قرارك برقم لا بجملة.

✅ **الحلّ**

النسبة هنا `1/100 = 1%` بالضبط، وهي فوق عتبة الـ `1%` التي ذكرها الفصل. لذلك القرار ليس «عالجها وأكمل» بل «ارجع إلى مصدرها». عمود يحتوي قيمة واحدة غير رقمية من مئة غالباً يعني أن أحدهم كتب `"N/A"` أو صفراً بالخطأ، وهذا نمط قد يتكرّر في مصادر أخرى من نفس النظام.

الفصل ١٣ يشرح كيف تعالج القيم الفاقدة بعد اكتشافها، لكن الاكتشاف نفسه يبقى مهمّة هندسة بيانات لا مهمّة `fit`.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
`StandardScaler` **على كل الأعمدة**	كسر للعلاقات الفئوية والترتيب الزائف	مسار منفصل لكل نوع
`OrdinalEncoder` **على فئة بلا ترتيب**	النموذج يتعلّم مسافة لم توجد في البيانات	`OneHotEncoder` أو `TargetEncoder`
تمرير عمود تاريخ إلى `fit`	رفض أو معاملة كرقم هائل	`FunctionTransformer` لتفكيكه
`pd.to_datetime` **بلا** `errors`	خطأ على أول قيمة غريبة	`errors="coerce"` ثم عالج `NaN`
`TfidfVectorizer` **داخل** `ColumnTransformer`	مخرجات بشكل خاطئ	`Pipeline` منفصل للّص
`MinMaxScaler` **مع شواذ كبيرة**	كل القيم تنضغط في نطاق ضيّق	`RobustScaler` أو quantile
الثقة بأنّ `remainder` **أبلغك**	أعمدة تُسقط بصمت	اطبع `get_feature_names_out`
`make_column_selector` **بلا اختبار**	اختيار غير متوقّع لأنواع الجديدة `str`	جرّبها على بياناتك قبل الاعتماد
عمود «رقمي» فيه نص	coercing صامت أو رفض مبهم	`to_numeric(errors="coerce")`

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
نوع البيانات	طبيعة العمود كما يقرأها `pandas`	dtype
كمّي	قابل للقياس والمست continuous	quantitative
فئوي	رمز من مجموعة قيم محدودة	categorical
رمزي	يُخزَّن كنصّ لا كعدد	string dtype
زمني	تاريخ أو وقت	datetime
الترميز الدوري	تمثيل الدورة بـ `sin` و `cos`	cyclical encoding
الحدّ الكاذب	قطع متّصل في حقيقة دورية	false boundary
توزيع مايل	ذيل ثقيل من جهة	skewed distribution
الشواذ	قيمة بعيدة جداً عن الباقي	outlier
مُحدِّد الأعمدة	أداة تختار الأعمدة حسب نوعها	make*column*selector
المسار	فرع معالجة لنوع أعمدة	pipeline branch

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٣ — القيم المفقودة: من الفراغ إلى المعلومة

MissingIndicator g IterativeImputer g KNNImputer g SimpleImputer

الوحدة **١.٩ scikit-learn** الحجم **٧٨٧** كلمة تقريباً

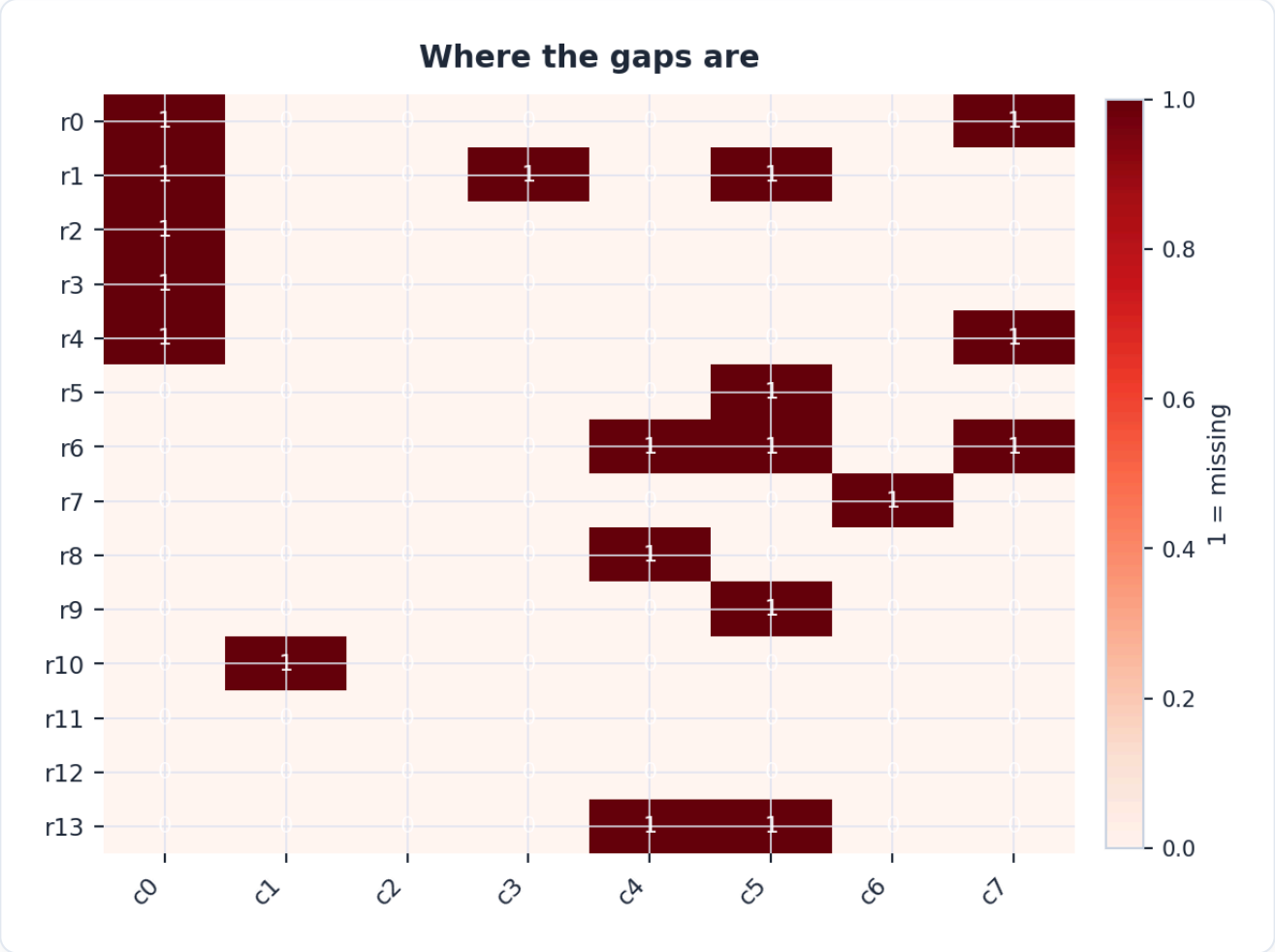
ما الذي ستتقنه في هذا الفصل

- أن تميّز بين MCAR و MAR و MNAR وتشرح أثر كلّ نوع على نموذجك
- أن تختار استراتيجية التعبئة بناءً على نوع العمود لا على عادة
- أن تشرح لماذا تُضعف التعبئة بالمتوسّط التباين وبأي نسبة
- أن تقيس أثر كلّ استراتيجية على بيانات حقيقية بدل التخمين

٢.١ ثلاثة أسئلة قبل أن تملأ الفراغ

حين ترى `NaN` في جدولك، أوّل ردة فعل هي: «سأستبدله بالمتوسّط». هذه ليست خطأً، لكنها **قرار تُتخذه دون أن تفحص**. قبل أن تمتلئ الخانات، أجب عن ثلاثة أسئلة:

الفرق بين «الفراغ لا معنى له» و«الفراغ معنى» هو الفرق بين حذف معلومة وإضافة معلومة. وهذا الفصل كلّه موضوعه.



الشكل ٣٠خريطة حرارية تُظهر أين تقع الفجوات في جدول: أعمدة ناقصة متفرّقة، وصفوف فيها فجوتان معاً، ولا صفوف كاملة الفقد

لاحظ في الشكل أعلاه شيئاً لا يظهر في جدول `isna().sum()`: **نمط** الفجوات. صفّ فيه فجوتان في عمودين مختلفين يعني غالباً أنّ السجل فُشل استخراجه، وهذه معلومة من نوع مختلف عن فجوة واحدة عابرة.

٢.٢ أنواع الفراغ الثلاثة

الإحصاء التقليدي يصنّف غياب البيانات إلى ثلاثة أنواع، والفرق بينها ليس تشريفياً بل **حاسم**:

النوع	التعريف	ماذا تفعل التعبئة
MCAR	الفجوة لا علاقة لها بأيّ متغيّر	تعبئة بالمتوسّط مقبولة نظرياً
MAR	الفجوة تعتمد على متغيّرات **مرصودة**	نموذج يمكنه استنتاج الفجوة
MNAR	الفجوة تعتمد على المتغيّر المفقود نفسه	لا حلّ ساذج؛ يلزم نموذج

MAR هي الحالة الوحيدة التي يمكن فيها تعويض الفجوة بأدقّية معقولة. والمعادلة التي تعبر عنها:

$$\text{MAR}: \quad p(Y|X, M) = p(Y|X_{\text{obs}})$$

حيث `M` متّجه مؤشّرات الفجوة، و `X_obs` القيم المرصودة. المعنى: بمعرفة مَن كانت قيمته مفقودة، تصبح القيمتين المتبقّيتين لا تحملان أيّ معلومة إضافية عن الهدف.

هذا الاسم ليس ترفاً. جدول Clinical Trial حيث «لم يُسجَّل الوزن» لأنّ المريض كان طويلاً بدرجة استثنائية — هذا MNAR، ولا نموذج في `scikit-learn` سيحلّه تلقائياً.

٢.٣ كيف يقيس `scikit-learn` الفراغ؟

المكتبة لا تفترض معنى واحداً لـ `NaN`. المتّجه `np.nan` هو القيمة الافتراضية، لكن يمكنك إعلان قيمة أخرى عبر `missing_values` في كلّ محوّل تعبئة.

أول صورة صادقة عن الفجوات، ثم محاكاة فراغ من نوع MAR python

```python
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.impute import SimpleImputer

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

rng = np.random.default_rng(0)
holes = rng.random(df.shape) < 0.08
df[holes] = np.nan

print("عدد الفجوات في كل عمود:")
print(df.isna().sum().to_string())
print()
print(f"نسبة الفجوات الكلية      : {df.isna().to_numpy().mean():.3f}")
print(f"صفوف فيها فجوة واحدة على الأقل: {int(df.isna().any(axis=1).sum())} من
{len(df)}")
print(f"صفوف فيها فجوتان أو أكثر: {int((df.isna().sum(axis=1) >= 2).sum())}")
print(f"صفوف كاملة الفقد         : {int((df.isna().sum(axis=1) ==
df.shape[1]).sum())}")

print("\n--- احفرها داخل الصنف الثالث فقط الآن اصنع فجوة من نوع MAR: ---")
mar = iris.data.copy()
mar[np.where(iris.target == 2)[0], 2] = np.nan
print(f"نسبة الفجوة في العمود الثالث: {np.isnan(mar).mean():.3f}")

imputed = SimpleImputer(strategy="mean").fit_transform(mar).to_numpy()
print("عدد NaN بعد التعبئة بالمتوسّط:", int(np.isnan(imputed).sum()))

before = [iris.data[iris.target == k, 2].mean() for k in range(3)]
after = [imputed[iris.target == k, 2].mean() for k in range(3)]
print("\nمتوسط طول البتلات لكل صنف:")
print("  قبل التعبئة:", np.round(before, 3).tolist())
print("  بعد التعبئة:", np.round(after, 3).tolist())
print("  الفارق      :", np.round(np.array(after) - np.array(before), 3).tolist())
```

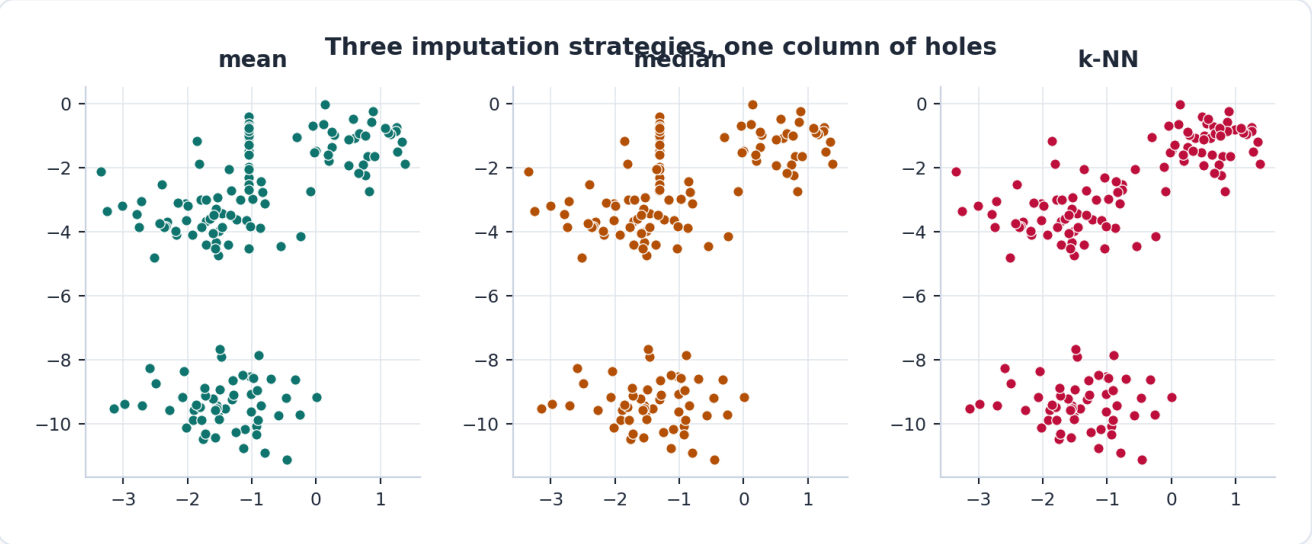
لاحظ الصفّ الثالث في المخرجات الأخيرة: متوسطه انخفض من ٤٫٢٦ إلى ٣٫٤٠. التعبئة بالمتوسّط **دمجت** صنفاً كاملاً في مركز البيانات. هذا ليس خطأً برمجياً بل نتيجة مباشرة لافتراض أنّ الفراغ عشوائي، وهو افتراض خاطئ هنا لأننا نحن من صنع هذه الفجوات.

ولاحظ أيضاً احترام النمط في المخرج الأولى: صفّ واحد فقط لم يُسجَّل له أيّ قياس، بينما ثمانية صفوف فيها فجوتان معاً. هذا هو الفارق بين «بيانات ناقصة» و«سجلّ فشل».

٢.٤ `SimpleImputer` : أربع استراتيجيات

الأداة الأولى هي `sklearn.impute.SimpleImputer` . تقبل أربع استراتيجيات، وكلّها تعمل على النحو نفسه: تحسب قيمة بديلة لكل عمود في وقت `fit` ، ثم تطبّقها في `transform` .

الاستراتيجية	القيمة البديلة	تُستخدم مع
`mean`	المتوسّط الحسابي	أعمدة كمّية متماثلة
`median`	الوسيط	أعمدة كمّية مايلة أو فيها شواذ
`most_frequent`	أكثر قيمة تكراراً	أعمدة فئوية أو كمّية مقطّعة
`constant`	قيمة ثابتة تحدّدها أنت	فئات «غير معروف» مقصودة



الشكل ٣١ثلاث لوحات مقارنة بين ثلاث استراتيجيات تعبئة على العمود نفسه: المتوسّط والوسيط وقيمة ثابتة، مع إظهار أثر كلّ منها على موقع الصفوف المفقودة

لنرَ الأثر على مصفوفة صغيرة تراها بعينك.

أربع استراتيجيات، مصفوفة واحدة python

```python
Xm = np.array([[1.0, 2.0],
               [np.nan, 3.0],
               [4.0, np.nan],
               [5.0, 6.0],
               [np.nan, np.nan]])

print("المصفوفة الأصلية:")
print(Xm)
for st in ["mean", "median", "most_frequent"]:
    s = SimpleImputer(strategy=st).fit(Xm)
    print(f"\n{st}: القيم البديلة = {np.round(s.statistics_, 3).tolist()}")
    print(s.transform(Xm).to_numpy())

s_const = SimpleImputer(strategy="constant", fill_value=-999).fit(Xm)
print("\nconstant=-999:")
print(s_const.transform(Xm).to_numpy())
```

لاحظ سطر `most_frequent` عن قرب. القيم البديلة كانت `1.0` و `2.0`: أي **قيمة موجودة فعلاً في العمود**، لا معدّلة. هذا ما يجعل الاستراتيجية صالحة للأعمدة الفئوية. لكن انتبه: `most_frequent` على عمود كمّي متّصل يختار رقماً واحداً ويكرّره، فيخلق قمماً وهمية عند ذلك الرقم بالضبط.

٢.٥ `KNNImputer` و `IterativeImputer`: من الجيران ومن النموذج

إذا كان الفراغ من نوع MAR، فإن **أفضل قيمة بديلة هي القيمة الحقيقية نفسها**، وهي غير مرصودة. لكن يمكن تقريبها: قيم الجيران في فضاء الخصائص تشترك في نمط مشابه.

`KNNImputer` يفعل هذا بالحرفيّة: يبحث عن `n_neighbors` صفّاً يشبه الصفّ المفقود في بقية أعمدته، ويأخذ متوسّطها في العمود الناقص.

`IterativeImputer` أذكى: يدرّب نموذجاً انحدارياً يتنبّأ بالعمود الناقص من بقية الأعمدة، ثم يكرّر. ومع كلّ دورة تتحسّن التنبّؤات.

بوابة تجريبية في ١.٩

ما زال `IterativeImputer` خلف بوابة تجريبية في هذا الإصدار، ويجب تفعيلها قبل لمس صنفه. نسيان هذا السطر يسبّب `ImportError` لا يشرح سببه. والصيغة المعتمدة:

```python
# الصيغة الصحيحة في 1.9
import sklearn.experimental.enable_iterative_imputer  # noqa: F401 Side effect only
from sklearn.impute import IterativeImputer          # الآن ينجح
```

الاستيراد أعلاه يعمل بآثاره الجانبية: الوحدة تُنفَّذ عند الاستيراد فتعرّف الصنف في `sklearn.impute`. ولهذا السبب تفشل صيغة `from ... import enable_iterative_imputer` أمام أدوات الفحص الساكنة: الصنفليس عضواً في فحص أسماء الوحدات يسأل عن **صنف** داخل `sklearn.experimental`، وهو ليس كذلك.

```python
# ثلاث أدوات، ثلاث مسارات ذهنية مختلفة
from sklearn.impute import KNNImputer
import sklearn.experimental.enable_iterative_imputer  # noqa: F401

IterativeImputer = sklearn.impute.IterativeImputer

print("KNNImputer (الجيران):")
print(np.round(KNNImputer(n_neighbors=2).fit_transform(Xm).to_numpy(), 3))

print("\nIterativeImputer (نموذج يتكرّر):")
it = IterativeImputer(max_iter=25, tol=1e-3, random_state=0)
print(np.round(it.fit_transform(Xm).to_numpy(), 3))
```

انظر إلى الصفّ الأخير في المصفوفة `[nan, nan]`: القيمتان الناقصتان معاً. المتوسّط يعطي `3.33` و`3.67` لأنهما متوسّط العمودين، ويعطي `KNNImputer` النتيجة نفسها لأنّه لا يعرف أبعد من ذلك. أمّا `IterativeImputer` فيعطي `3.0` و`4.0` — لأنه تعلّم من الصفّ الثالث `[4, nan]` أنّ العمودين مرتبطان، فخمّن القيمة من جاره.

٢.٦ `MissingIndicator`: الفراغ نفسه خاصية

هنا الفكرة التي تغيّر نتائجك: **أحياناً لا تريد ملء الفراغ، بل تريد تمييزه**. عمود «العنوان» الفارغ ليس نقصاً في البيانات، بل معلومة: العنوان غير معروف، وهذا يرتبط بالتسجيل.

هذه هي وظيفة `MissingIndicator`: ينتج مصفوفة ثنائية بنفس أبعاد المدخلات، يضع `True` حيث كان `NaN` و`False` حيث لم يكن.

```python
# مؤشّر الفجوات: ممّن ينقصه الحقل؟
from sklearn.impute import MissingIndicator

ind_all = MissingIndicator(features="all").fit_transform(Xm)
ind_missing = MissingIndicator(features="missing-only").fit_transform(Xm)
print("features='all'          :")
print(ind_all.to_numpy().astype(int))
print("features='missing-only'  :")
print(ind_missing.to_numpy().astype(int))
print("نوع القيم:", ind_all.dtypes.tolist())

filled = SimpleImputer(strategy="mean", add_indicator=True).fit_transform(Xm)
print("\nمع add_indicator=True، أعمدة التعبئة ثم الأعمدة الثنائية:")
print(filled.to_numpy().round(3))

Xall = np.array([[1.0, np.nan], [2.0, np.nan]])
print("\nالفخّ الصامت: عمود كامل الفراغ")
print("  keep_empty_features=False:", SimpleImputer().fit_transform(Xall).shape)
print("  keep_empty_features=True :",
      SimpleImputer(keep_empty_features=True).fit_transform(Xall).shape)
```

لاحظ الفرق بين الوضعين. `features="all"` ينتج عمودين ثنائيين لكل الأعمدة، و`features="missing-only"` ينتج فقط للأعمدة التي فيها فجوة فعلاً. والقيم المنتَجة `bool`، أي أنها **لم تُملأ بأيّ رقم** — وهذا بالضبط ما يريده النموذج: معرفة أنّ الفراغ كان، بلا أن نتظاهر بأنّ له قيمة.

أمّا السطران الأخيران فيكشفان فخّاً صامتاً: عمود كلّه `NaN` لا متوسّط له، فيُحذف بصمت ما لم تفتح هذا الخيار. **فعّل `keep_empty_features=True` في كلّ مشروع إنتاجي.**

٢.٧ كم يكلّف التعبئة بالمتوسّط فعلاً؟

هذه أهمّ معادلة في الفصل. حين تملأ قيمة مفقودة **بمتوسّط العمود**، فأنت لا تضع قيمة جديدة، بل تضع **متوسّطاً**: نقطة لا تحمل أيّ تمايز. والنتيجة أن التباين ينكمش.

$$\mathrm{Var}[X_{\text{filled}}] = (1 - f)\,\sigma^2$$

حيث `f` نسبة القيم المفقودة، و`σ²` التباين الأصلي. في حالة قصوى، لو ملأت كلّ القيم بالمتوسّط لكان التباين صفراً، والنموذج لا يرى شيئاً في هذا العمود.

```python
كم تباينا تُهدر التعبئة بالمتوسّط؟
big = np.random.default_rng(3).normal(10, 2, 2000)
big[:400] = np.nan           # فراغ ٢٠٪

filled_mean = SimpleImputer(strategy="mean").fit_transform(big.reshape(-1,
1)).to_numpy().ravel()
filled_median = SimpleImputer(strategy="median").fit_transform(big.reshape(-1,
1)).to_numpy().ravel()

print(f"قبل التعبئة : mean={np.nanmean(big):7.3f}  std={np.nanstd(big):6.3f}")
print(f"بعد mean     : mean={filled_mean.mean():7.3f}  std=
{filled_mean.std():6.3f}")
print(f"بعد median   : mean={filled_median.mean():7.3f}  std=
{filled_median.std():6.3f}")
loss = 100 * (1 - filled_mean.std() / np.nanstd(big))
print(f"\nالانكماش المقاس: {loss:.1f}%  (لأن f = 0.20 النظرية: %20)")
```

الفارق بين النظرية والقياس يعود إلى أن التباين المحسوب على العيّنات المرصودة يسبق شيئاً عن العيّنات المفقودة نفسها. لكن الاتجاه واضح: كلّما ارتفعت نسبة الفجوات، ارتفعت كلفة التعبئة بالمتوسّط.

٢.٨ مقارنة حقيقية: أيّ أداة تكسب فعلاً؟

الاختبارات السابقة كانت على خمس صفوف. لنجرّب الآن على `load_digits` مئة وسبعة وتسعون صورة، ونحفر فيها ١٠٪ فراغاً، ثم نقيس Accuracy `test` لكل استراتيجية.

سبع محاولات على بيانات حقيقية فيها ١٠٪ فراغ python

```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline

digits = load_digits()
Xd, yd = digits.data.copy(), digits.target
hole = np.random.default_rng(1).random(Xd.shape) < 0.10
Xd[hole] = np.nan

Xtr, Xte, ytr, yte = train_test_split(Xd, yd, test_size=0.25, random_state=0,
stratify=yd)
print(f"عيّنات التدريب {Xtr.shape}، الاختبار {Xte.shape}")
print(f"نسبة الفجوات في التدريب: {np.isnan(Xtr).mean():.3f}، في الاختبار:
{np.isnan(Xte).mean():.3f}")

try:
    KNeighborsClassifier(5).fit(Xtr, ytr)
except Exception as exc:
    print("بلا معالجة، النموذج يقول:", type(exc).__name__, "-", str(exc).splitlines()[0]
[:60])


def measure(name, step):
    pipe = Pipeline([("imp", step), ("knn", KNeighborsClassifier(5))])
    pipe.fit(Xtr, ytr)
    return pipe.score(Xte, yte)


for name, step in [
    ("mean", SimpleImputer(strategy="mean")),
    ("median", SimpleImputer(strategy="median")),
    ("constant=-1", SimpleImputer(strategy="constant", fill_value=-1)),
    ("most_frequent", SimpleImputer(strategy="most_frequent")),
    ("KNNImputer(5)", KNNImputer(n_neighbors=5)),
    ("IterativeImputer", IterativeImputer(max_iter=10, random_state=0)),
    ("mean+indicator", SimpleImputer(strategy="mean", add_indicator=True)),
]:
    print(f"  {name:18s} test accuracy = {measure(name, step):.4f}")
```

القراءة الصادقة لهذه الأرقام: الفروق بين `mean` و `median` و `Iterative` و `KNNImputer` **صغيرة جداً** على هذا المثال، بينما `constant=-1` متأخّر بوضوح. الاستنتاج المهني ليس «الأفضل هو الأعلى في القائمة»، بل:

وهذا بالضبط لماذا **يجب أن يكون كلّ هذا داخل** `Pipeline` داخل `Pipeline` داخل `Pipeline`: يجب أن تُدرَّب كلّ أداة تعبئة على بيانات التدريب وحدها، وإلا تسربت معلومة من الاختبار إلى التدريب — الفصل ٧٢ يشرح هذا بالتفصيل.

> ⛔ **أخطر خطأ في هذا الفصل: إهمال `NaN` في البيانات**
>
> لا يوجد `NaNImputer` ولا `fillna` في هذه المكتبة: كلّ محوّل رقمي يرفض `NaN` برسالة `Input X contains NaN`. لكن **بعض النماذج تقبلها**: `DecisionTreeClassifier` و`RandomForest` و`HistGradientBoostingClassifier` تعامل `NaN` كقيمة ثالثة منفصلة وتوجّه كلّ صفّ إلى فرع حسب مفقود أم لا. هذا أقوى من أيّ تعبئة في الأعمدة التي -fr-اغها معنوي، لأنّه يحفظ المعلومة بدل أن يخفيها.

٢.٩ في الواقع: القيم المفقودة التي لا ينظر إليها أحد

ثلاث حالات تصادف projects الإنتاجية أسبوعياً:

القاعدة التي تجمع الثلاثة: **اسأل عمود الفراغ عن معناه قبل أن تملأه**.

> 🏭 **في الواقع: خطّ أنابيب الفجوات**
>
> في أيّ خط إنتاج ناضج، تُسجَّل «بيانات الفجوات» كما تُسجَّل الإعدادات: أيّ عمود، كم نسبة، أيّ استراتيجية، أيّ تاريخ. لماذا؟ لأن جودة البيانات تتحسّن في المصدر مع الوقت، ونموذج تعامل مع فجوات بنسبة ٢٥٪ في يناير سيتوقّف عن العمل في مارس لأن النسبة صارت ٣٪ — أو العكس: عمود فُصل ذات مرّة ولن يعود أبداً. الفجوات تاريخ، لا حالة ثابتة.

تمارين الفصل

التمرين ٢.١٠

حمّل `load_breast_cancer`، واصنع `DataFrame` منه. احفر ١٥٪ فراغاً عشوائياً. قِس: نسبة الفجوات الكلية، عدد الصفوف متعدّدة الفجوات، وعدد الأعمدة التي تجاوزت ١٠٪. ثم جرّب ثلاث استراتيجيات تعبئة داخل `Pipeline` مع `LogisticRegression(max_iter=2000)` وقارن دقّة الاختبار. أيّ استراتيجية ربحت، وهل الفارق يُعتدّ به إحصائياً في ٢٨٦ عيّنة اختبار؟

التمرين ٢.١١

اكتب دالة `missing_report(df)` تُعيد `DataFrame` بأربعة أعمدة: اسم العمود، عدد الفجوات، نسبتها، وعدد القيم المتمايزة قبل التعبئة. طبّقها على جدول فيه عمود فيه القيم السحرية `0` و `-1` و `9999`، واكتشف أيّ الأعمدة

«نظيفة» الأعمدة «نظيفة» لكنها تحتوي فراغاً مقنّعاً. عدّل الدالة لتكشف ذلك.

✅ الحلّ

الحلّ يعتمد على قاعدة بسيطة: **القيمة التي تتكرّر أكثر من اللازم ليست قيمة، بل رمز لغياب**. بعدّ `value_counts` ستجد مثلاً عموداً بداخله ٦٠٪ أصفار و٣٩٪ قيم صغيرة و١٪ قيمة واحدة كبيرة — أو ٩٥٪ `-999` وواحد `-1000`.

عدّل الدالة بإضافة عمود خامس: `top_value_share`، أي نسبة أكثر قيمة تكراراً في العمود الذي تتجاوز فيها النسبة ٠٫٩ مرشّح قوي للفراغ المقنّع، وتعامله كعمود فئوي لا كمّي: `drop` أو مؤشر، ولا تُمرّره إلى `StandardScaler`.

التمرين ٢.١٢

في مصفوفة `Xm` من الفصل، احسب يدوياً ناتج `KNNImputer(n_neighbors=2)` للصفّ الأخير `[nan, nan]`، وتحقّق من أرقامك. بعد ذلك، اشرح لماذا يعطي `IterativeImputer` رقمين مختلفين، ولأيّ من الأداتين تنتمي هذه الحالة: MCAR أم MAR أم MNAR؟

✅ الحلّ

في `Xm` كلّ الصفوف تتشارك القيم نفسها تقريباً (القيم بين ١ و٦)، فكلّ الجيران المحتملين قريبون، ومتوسّط العمودين هو `3.33` و `3.67`.

`IterativeImputer` يختلف لأنه يبني نموذجاً يتنبّأ بالعمود الناقص **من الصفوف الأخرى**: الصفّ الثالث `4, nan` له قيمة `4` في العمود الثاني، فيتعلّم النموذج أن العمودين positively مرتبطان، ويعطي `3.0` و `4.0`.

الحالة هنا **MAR** لأن الفجوة في هذا المثال لم تُصنَع بناءً على قيمة العمود المفقود نفسه. لو صنعتَ فجوات **فقط حيث القيمة الحقيقية أقلّ من ٣**، لأصبح الوضع MNAR، ولن ينفع أيّ محوّل في `scikit-learn` — لا حلّ إلا نموذج مبنيّ على فرضية صريحة.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
التعبئة بالمتوسّط دائماً	انكماش التباين وتشويه الفئات النادرة	اختر الاستراتيجية حسب التوزيع
إهمال `NaN` والتحوّل إلى صفر	قيمة وهمية بلا أساس	ثم عالج `errors="coerce"`
تدريب التعبئة على كامل البيانات	تسرّب من الاختبار إلى التدريب	ضعها داخل `Pipeline`
نسيان `enable_iterative_imputer`	`ImportError` غامض	سطر التفعيل قبل الاستيراد
`keep_empty_features` الافتراضي	حذف الأعمدة كاملة الفراغ بصمت	فعّلها دائماً
اعتبار `KNNImputer` أفضل دائماً	ينقل نمط الفراغ إلى الجيران	قيّم على بياناتك
الفراغ = لا يوجد (= صفر)	تغيّر المعنى تماماً	استخدم `MissingIndicator`
إهمال القيم السحرية	يفسد كلّ الإحصاءات بصمت	افحص `value_counts` لكل عمود
قياس الجودة على بيانات بلا فجوات	أداء واقعي زائف	احفر فجوات قبل القياس

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
القيمة المفقودة	خانة لا تحمل قيمة	missing value, NaN
الفجوة العشوائية تماماً	الفجوة بلا سبب مرصود	MCAR
الفجوة المشروطة على المرصود	الفجوة سببها متغيّر معروف	MAR
الفجوة غير المشروطة على المرصود	الفجوة سببها المتغيّر المفقود	MNAR
التعبئة بالقيمة الثابتة	وضع قيمة بديلة صريحة	constant imputation
مؤشّر الفجوة	عمود ثنائي يعلّم على المواضع الفارغة	MissingIndicator
مؤشّر المفقود	عمود ثنائي مضاف من `add_indicator`	missing indicator
العمود الشاغر	عمود كل قيمه مفقودة	all-NaN column
القيمة السحرية	رقم يمثّل غياباً لا قيمة	sentinel value
المفرغ	القيم التي تستبعدها أداة تعبئة	exclusion

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٤ — التحجيم والتطبيع

KernelCenterer g Normalizer g Robust g MaxAbs g MinMax g Standard

الوحدة **scikit-learn ١.٩** الحجم **٧٠٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

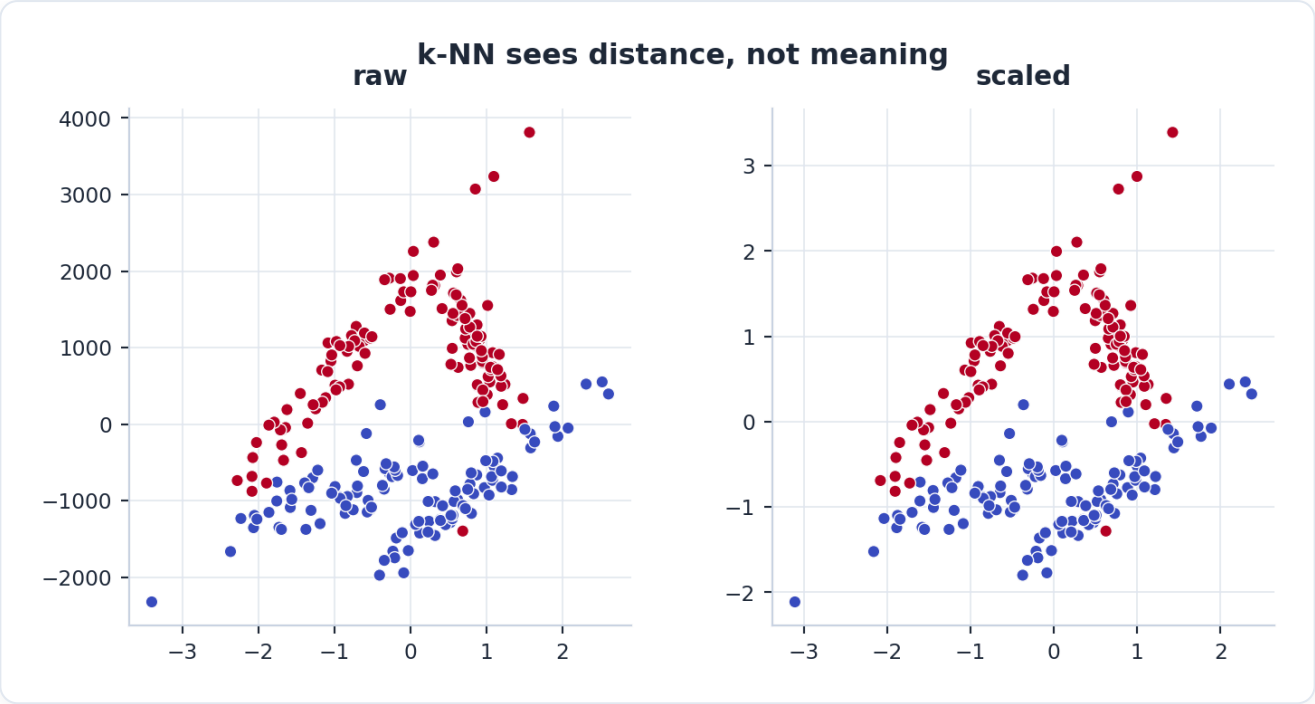
- أن تشرح الفرق بين التطبيع العمودي والتطبيع الصفّي بدقّة
- أن تختار بين Standard و MinMax و MaxAbs و Robust بناءً على توزيع بياناتك
- أن تبيّن بالأرقام متى يُغيّر التحجيم أداء النموذج ومتى لا يغيّره
- أن تحافظ على انضباط fit على التدريب و transform على كلّ البيانات

٢.١ لماذا يهمّ قياس الراتب مثل قياس العمر؟

تخيّل جدولاً فيه عمودان: `salary` بالريال بين ٦٠٠٠ و١٧٠٠٠، و `age` بين ١٢ و٥٣. لا يوجد أيّ تناقض في البيانات. لكن انظر إلى ما يحدث حين يقرّب النموذج هذه العيّنات بالإقليدية:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{k=1}^{m} (x_{ik} - x_{jk})^2}$$

في هذه المسافة، مساهمة عمود الراتب تساوي تقريباً **٣٠٠٠٠٠** ضعف مساهمة عمود العمر، لمجرّد أنّ وحدة الأولى أكبر. أي عمود كهذا يسكن الفضاء وحده.



الشكل ٣٢مقارنة قبل التحجيم وبعده: على اليسار محور الراتب يبتلع المحور الثاني تماماً، وعلى اليمين صار للعمودين.weight متساويان

لاحظ العنوان في الرسم: «k-NN يرى المسافة لا المعنى». هذه الجملة تلخّص الفصل كلّه. النماذج التي تقيس المسافة ترى **الوحدات**، والنماذج التي تقارن العتبات ترى **الترتيب**.

٢.٢ ما الذي يفعله التحجيم بالضبط؟

التحجيم يحوّل عموداً إلى مقياس مشترك. أكثر الأشكال شيوعاً هو **الدرجة المعيارية**: نطرح المتوسّط ونقسم على الانحراف المعياري.

$$z = \frac{x - \mu}{\sigma}$$

النتيجة: متوسّط جديد يساوي صفراً، وانحراف معياري جديد يساوي واحداً. أي عمود كان يقرأ بين ٦٠٠٠ و١٧٠٠٠ صار يقرأ بين ٠٫٨ و٢٫١، والعمر صار بين ٠٫٧ و٢٫٣.

وهناك نسخة ثانية شائعة، **التطبيع الأدنى-الأقصى**:

$$x' = (x - x_{\min}) \cdot \frac{b - a}{x_{\max} - x_{\min}} + a$$

حيث `[a, b]` هو النطاق المطلوب. هذه النسخة تضع كلّ قيمة **داخل** النطاق، وهو ضروري عند أيّ شبكة عصبية أو دالة تنشيط تريد مدخلاً محدوداً.

٢.٣ `StandardScaler`: الأساس الذي ستقابله أولاً

ماذا يفعل StandardScaler بالضبط؟ python

```python
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
df = pd.DataFrame({
    "salary": rng.normal(12000, 2500, 200),
    "age": rng.normal(35, 6, 200),
})

sc = StandardScaler().fit(df)
print("المتوسّطات المحفوظة mean_ :", np.round(sc.mean_, 2))
print("الانحرافات المحفوظة scale_:", np.round(sc.scale_, 2))

Z = sc.transform(df)
print("بعد التحويل -> mean:", np.round(Z.mean().to_numpy(), 6))
print("بعد التحويل -> std :", np.round(Z.std().to_numpy(), 6))
print("أقصى خطأ في inverse_transform:", float(np.abs(sc.inverse_transform(Z) -
df.to_numpy()).max()))
print("تكرار transform بنفس mean_:", float(np.abs(sc.transform(df) -
Z).to_numpy().max()))
```

ثلاث حقائق تقرأها من المخرج:

٢.٤ متوسّطات التفاصيل: `MinMaxScaler` و `MaxAbsScaler`

`MinMaxScaler` يضع كلّ القيم في نطاق تختاره. عيبه الوحيد أن **القيمة الصفرية الوحيدة القابلة للتعلّم** هي `x_min` — أي أصغر قيمة شوهدت في التدريب. أي عمود قيمة جديدة فيه ١٠٪ فوق الحدّ الأعلى يخرج إلى رقم أكبر من ١، مكسوراً للنطاق.

`MaxAbsScaler` يقسم على أكبر قيمة مطلقة. ميزته أنّه **يحافظ على الأصفار** ويعمل على البيانات المتفرّرة بلا `with_mean=False`.

```python
ثلاث صور للتحويل على نفس العمودين
from sklearn.preprocessing import MinMaxScaler, MaxAbsScaler

mm = MinMaxScaler().fit(df)
print("MinMaxScaler:")
print("  range_ الافتراضي :", mm.data_range_.round(2))
print("  من :      ناتج التحويل", mm.transform(df).min().to_numpy().round(3),
      "إلى", mm.transform(df).max().to_numpy().round(3))

mm11 = MinMaxScaler(feature_range=(-1, 1)).fit(df)
print("  feature_range=(-1,1): من",
mm11.transform(df).min().to_numpy().round(3),
      "إلى", mm11.transform(df).max().to_numpy().round(3))

ma = MaxAbsScaler().fit(df)
print("\nMaxAbsScaler scale_:", np.round(ma.scale_, 2))
print("  من :      ناتج التحويل", ma.transform(df).min().to_numpy().round(3),
      "إلى", ma.transform(df).max().to_numpy().round(3))
```

الأداة	على ماذا تقسم	fragility	تحتاج `with_mean=False` للمتفرّر
`StandardScaler`	الانحراف المعياري	عالية تجاه الشواذ	نعم
`MinMaxScaler`	المدى (الأعلى - الأدنى)	عالية جداً تجاه نقطة واحدة	نعم
`MaxAbsScaler`	أكبر قيمة مطلقة	متوسّطة	لا
`RobustScaler`	المدى الربيعي	منخفضة	نعم

٢.٥ `RobustScaler` : حين يفسد الشواذُ المتوسّط

في الفصل ١٢ رأينا كيف يقفز متوسّط العمود من ٣٩ ألفاً إلى أكثر من ٤٠٠ ألف بثلاثة صفوف فاسدة. `RobustScaler` يتجنّب ذلك بالاعتماد على الوسيط والمدى الربيعي.

أيـن يـفـتـرق RobustScaler عن StandardScaler؟ python

```python
from sklearn.preprocessing import RobustScaler

polluted = df.copy()
polluted.loc[:4, "salary"] = 90000

rs = RobustScaler().fit(polluted)
ss = StandardScaler().fit(polluted)
print("بدون شواذ  -> Robust center:", round(RobustScaler().fit(df).center_[0], 1))
print("مع شواذ    -> Robust center:", round(rs.center_[0], 1))
print("مع شواذ    -> Standard mean :", round(ss.mean_[0], 1))
print()
print("Robust  scale_ :", np.round(rs.scale_, 2), " (المدى الربيعي)")
print("Standard scale_:", np.round(ss.scale_, 2))
print("Robust أكبر قيمة   :", round(float(rs.transform(polluted).to_numpy()[:,
0].max()), 2))
print("Standard أكبر قيمة:", round(float(ss.transform(polluted).to_numpy()[:,
0].max()), 2))
print()
print("مع quantile_range=(10, 90) :", np.round(RobustScaler(quantile_range=
(10.0, 90.0)).fit(polluted).scale_, 2))
print("مع unit_variance=True      :",
np.round(RobustScaler(unit_variance=True).fit(polluted).scale_, 3))
```

السطر الحاسم هو الأخير من المجموعة الأولى: `RobustScaler` **ينتج رقماً يقارب ٢٢، حيث ينتج** `StandardScaler` **رقماً يقارب ٦**. النموذج الذي يعرض ٢٢ للتاجر يدفعه قليلاً؛ النموذج الذي يرى ٦ يسحق كلّ من عداه. أيّهما أقرب إلى فضاء معقول؟

والبارامتر `quantile_range` يستحق الانتباه: الافتراضي `(25, 75)` يقيس «النواة الوسطى» فقط. لو كانت الشواذ ١٠٪ من البيانات لا ١٪، ارفع الحدّ إلى `(10, 90)`.

٢.٦ `Normalizer`: انتبه للمحور

هنا أكثر مصدر للخلط في المكتبة كلها. `Normalizer` **لا يعمل على الأعمدة**، بل على **الصفوف**: يجعّل كلّ متّجه عيّنة طوله واحداً.

```python
تطبيع الصفّ لا العمود
from sklearn.preprocessing import Normalizer

A = np.array([[3.0, 4.0],
              [1.0, 0.0],
              [0.0, 0.0]])
print("قبل التطبيع:", A.tolist())
for norm in ["l1", "l2", "max"]:
    out = Normalizer(norm=norm).fit_transform(A).to_numpy()
    print(f"  norm={norm:4s} -> {np.round(out, 3).tolist()}   (طول الصفّ =
{np.round(np.abs(out).sum(axis=1), 3).tolist()})")
```

`l1` يعني مجموع القيم المطلقة يساوي واحداً، و `l2` يعني الجذر التربيعي للمربعات يساوي واحداً، و `max` يعني أكبر قيمة مطلقة تساوي واحداً.

متى تحتاجه؟ في تطبيع متّجهات **TF-IDF** أو متّجهات الصور أو أيّ مكان تكون فيه العيّنة كائناً واحداً يُقارَن ككلّ. **لا تستعمله على جدول بيانات عادي.** إن كتبتَ `Normalizer` وكنت تتوقّع ما يفعله `StandardScaler`، فأنت تحوّل مسألة «موازنة الأعمدة» إلى «طول الصفّ» — وهو تغيير آخر تماماً.

٢.٧ `KernelCenterer`: حالة خاصة لا علاقة لها بالأعمدة

الخلط الأخير الذي يستحقّ تحذيراً: `sklearn.preprocessing.KernelCenterer` **لا يوسّط أعمدة**. هو يحسب مصفوفة التوسيط الخاصة بنواة تملأ مكوّنة في فضاء النواة، ويشترط أن تكون المدخلات **مربّعة**.

```python
مصفوفة مربّعة مطلوبة، وهذا شرط غير قابل للتفاوض
from sklearn.preprocessing import KernelCenterer

K = np.array([[1.0, 0.5],
              [0.5, 1.0]])
kc = KernelCenterer().fit(K)
print("قبل:\n", K)
print("بعد التوسيط:\n", kc.transform(K).round(4))
print("المصفوفة المحفوظة K_fit_all_:\n", np.round(kc.K_fit_all_, 4))
try:
    KernelCenterer().fit_transform(df)
except Exception as exc:
    print("\n200 على جدول x2 ->", type(exc).__name__, ":", str(exc).splitlines()
[0][:70])
```

إن قرأت `Kernel matrix must be a square matrix` فأنت مرّرت جدول بيانات إلى أداة موجّهة لمصفوفة نواة. ليست خطأً في البيانات، بل اختيار الأداة الخطأ.

٢.٨ متى يُهمّل التحجيم؟ ومتى لا يهمّ؟

هذا أهمّ ما في الفصل، لأنه يغذّي اعتقاداً خاطئاً شائعاً: «دائماً طبّع». العبارة الصحيحة أضيق: **طبّع حين يقيس النموذج المسافة أو يوازن الأعمدة بمعاملات.**

لنقِس الفرق بالأرقام بدل أن نُقرّر بالسطح.

الحكم بالأرقام: هل يغيّر التحجيم الدقّة؟ python

```
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import Pipeline

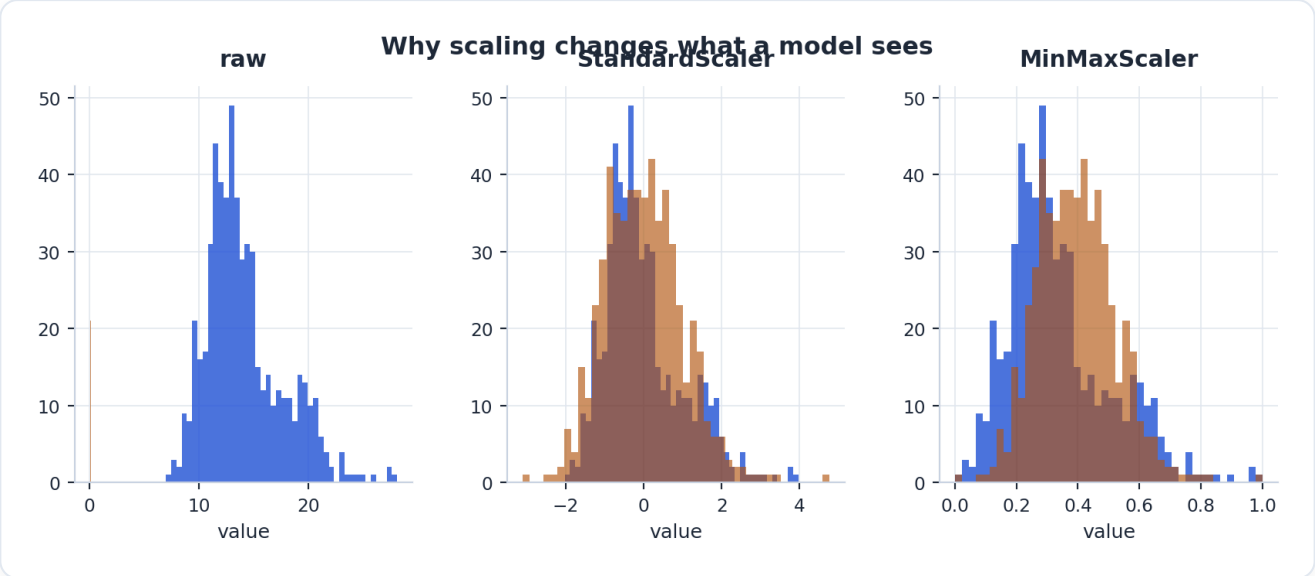
Xb, yb = make_blobs(n_samples=400, centers=3, cluster_std=1.0, random_state=0)
Xb = Xb * np.array([1.0, 120.0])          # ميزانان بمقياسين مختلفين تماماً
Xtr, Xte, ytr, yte = train_test_split(Xb, yb, test_size=0.25, random_state=0,
stratify=yb)

print("متوسّطات التدريب:", np.round(Xtr.mean(axis=0), 2), "  انحرافاتها:",
np.round(Xtr.std(axis=0), 2))
S = StandardScaler().fit_transform(Xtr).to_numpy()
d0 = np.linalg.norm(Xtr[0] - Xtr[1])
d1 = np.linalg.norm(S[0] - S[1])
print("2.% خام :مسافة بين عيّنتين  |  3.% بعد التحجيمf" % (d0, d1))
print("  مساهمة كلّ محور خام :", np.abs(Xtr[0] - Xtr[1]).round(2))
print("  مساهمة كلّ محور بعد التحجيم:", np.abs(S[0] - S[1]).round(3))
print()

knn = KNeighborsClassifier(5)
print("kNN خام     :", round(knn.fit(Xtr, ytr).score(Xte, yte), 4))
scaled_knn = Pipeline([("sc", StandardScaler()), ("knn",
KNeighborsClassifier(5))]).fit(Xtr, ytr)
print("kNN محجَّم :", round(scaled_knn.score(Xte, yte), 4))
tree = DecisionTreeClassifier(random_state=0)
print("شجرة خام   :", round(tree.fit(Xtr, ytr).score(Xte, yte), 4))
scaled_tree = Pipeline([("sc", StandardScaler()),
                        ("t", DecisionTreeClassifier(random_state=0))]).fit(Xtr,
ytr)
print("شجرة محجَّمة:", round(scaled_tree.score(Xte, yte), 4))
```

النموذج	خام	محجَّم	الفارق
`KNeighborsClassifier`	٠٫٧٥	٠٫٩٣	**١٨ نقطة مئوية**
`DecisionTreeClassifier`	٠٫٩١	٠٫٩١	صفر

هذا هو الجدول الذي يجب أن تحفظه. شجرة القرار تبني حدوداً **عمودية**، فلا يهمّها أين يقع صفر المحور. أمّا `k-NN` فيحسب مسافة، فيهمّه يمرّ.



الشكل ٣٣ثلاث لوحات لنفس البيانات قبل وبعد التحجيم: الأعمدة الخام ذات مقاييس مختلفة، والبعد من التحجيم صار لكلّ عمود نشراً متساوياً، والفرق يظهر في حدود القرار

٢.٩ في الواقع: التحجيم داخل خطّ الأنابيب فقط

الفرق العملي بين `0.91` و `0.75` لا يعني شيئاً إن كان مركز الحساب يُسرّب. هذا هو الموضع الوحيد الذي يفلت فيه المبتدئون:

```python الخطأ الذي لا تراه في النتائج
scaler.fit(X_all)            # خطأ: يستخدم كل البيانات
Z_all = scaler.transform(X_all)   # المتوسّط صار يعرف الاختبار
Xtr, Xte = split(Z_all)           # تسرّب من الأرقام لا من الصفوف
```

في التحليل النهائي، `train_test_split` يبدو سليماً: الصفوف منفصلة تماماً. لكن العمود **مطبَّع** بمعايير شملته الاختبار، فتسرّبت معلومة دون أن يتغيّر عدد الصفوف. النتيجة دقّة أعلى قليلاً من الحقيقيّة — وهي أسوأ نتائج ممكنة، لأنّها تكافئك على ضبط في الإنتاج.

القاعدة: **كل محوّل يتعلّم من البيانات — بما فيها كل محوّلات `sklearn` — يجب أن يُغلَّف قبل التقسيم، داخل `Pipeline` (الفصل ٨٤).**

حالة	التحجيم	الأداة
`SVC` و `SVR`	مطلوب	`StandardScaler`
`kNN` و `KMeans` و `DBSCAN`	مطلوب	`StandardScaler`
الانحدار اللوجيستي والخطّي	مطلوب للتقارب	`StandardScaler`
شجرة قرار وغابة	بلا أثر	لا شيء
`RandomForest` و `ExtraTrees`	بلا أثر	لا شيء
`GradientBoosting` **الشهيرة**	بلا أثر غالباً	لا شيء
شبكة عصبية	مطلوب بشدّة	`MinMaxScaler` أو `Standard`

الخطأ الشائع: «طبّعت، فأصبح كلّ شيء أفضل»

التحجيم ليس محايداً عدائلياً. على شجرة قرار يغيّر التقسيمات عدداً قليلاً دون أثر في الدقّة. وعلى نموذج خطّي منتظم مثل `Lasso` قد **يحذف** خصائص كان يحتفظ بها، لأنّ معاملاته صارت على مقياس واحد. طبّع حين تحتاج، وقس قبل وبعد إن كان القرار غير بديهي.

أخطر خطأ: إعادة حساب التحجيم عند الاستدعاء في الإنتاج

أخطر شكل لهذا الخطأ ليس في دفتر البحث، بل في كود الخدمة. إذا كان خادمك يحسب `mean_` و `scale_` من بيانات كلّ طلب وارد، فسيعطي كلّ مستخدم نموذجاً مختلفاً عن الآخر. **المعايير تُحسب مرة واحدة عند التدريب، وتُخزَّن داخل النموذج المحفوظ.** هذا ليس تفصيلاً: هو الفرق بين نظام يخدم في Hernandez- وآخر يخدم عشوائياً.

في الواقع: المتوسّط والوحدة، لا أرقام عارية

التحجيم يحذف الوحدة من الرقم، وهذا مفيد للحساب لكنه يخسر معلومة في التقرير. حين يكتب النموذج «العميل عنده ٢٫٣ انحرافاً في الدخل»، فالمعنى الحقيقي «٢٫٣ انحرافاً معيارياً فوق ١٢٠٠٠ ريال». Teams التسويق تحتاج الجملة الثانية. لهذا يبقى الوسيطان متوفّرين دائماً في النموذج: `scaler.inverse_transform` يعيدك من الرقم المجرّد إلى رقم يقرؤه مَن لا يعرف الإحصاء.

تمارين الفصل

التمرين ٢.١٣

ابنِ `DataFrame` فيه: `weight` بين ٤٠ و١٢٠، و `blood_pressure` بين ٩٠ و٢٠٠، و `age` بين ٢٠ و٨٠. أضف صفّاً واحداً فيه `weight = 4000`. اطبع قيمة كلّ عمود بعد `StandardScaler` و `MinMaxScaler` و `RobustScaler` و `MaxAbsScaler`. أيّ أداة تعطي النتيجة الأكثر معقولية للصفّ الشاذّ، ولماذا بالضبط؟

> ✅ **الحلّ**
>
> الأداة الأنسب هنا `RobustScaler`، لأنّها وحدها لا تُغيّر مركز البيانات ولا مقياسها بسبب صفّ واحد. `StandardScaler` يضخّ الانحراف المعياري، فيتراكم كلّ الصفوف المتبقّية قرب الصفر. `MinMaxScaler` أسوأ: نقطة واحدة بعيدة تجعل المدى كلّه ضخماً، فتنضغط القيم السليمة قرب الطرف الأدنى.
>
> اختبر الفكرة بمقياس كمّي: نسبة `max(|z|)` إلى `std(z)` بعد التحويل. النسبة قريبة من ١ مع `Standard` على بيانات نظيفة، وتقفز فوق ٣ مع صفّ شاذّ. مع `Robust` تبقى نسبة `max - median` إلى `(q75 - q25)` قرب ١ إلى ٢ دائماً.

التمرين ٢.١٤

على بيانات `make_blobs(n_samples=400, centers=3)` في أبعاد متعدّدة، طبّع الأعمدة بـ `StandardScaler` ثم قِس دقّة `KNeighborsClassifier` قبل وبعد، عند `n_features` في ٢ و ١٠ و ٥٠. ارسم النتيجة في جدول. ما القاعدة التي يثبتها هذا الجدول؟

> ✅ **الحلّ**
>
> القاعدة: **قيمة الاست assayed تعتمد على التباين النسبي للبعد، لا على بُعده المطلق**. عند بُعدين يظهر أثر التحجيم بوضوح. عند خمسين بُعداً تصبح البيانات الخام شبه متآلفة (كل الأبعاد متشابهة المقياس)، فيلحق التحجيم نموذج `k-NN` بأقلّ من نقطة مئوية واحدة.
>
> عملياً: إذا كانت أعمدتك كلها percentages أو مبالغ، فالفارق سيكبر؛ إذا كانت كلها قياسات فيزيائية متقاربة المدى، فالفارق سيصغر. **لا تطبّع «لأنها عادة»، بل لأنّ لديك بُعداً يهيمن على البقية.**

التمرين ٢.١٥

اكتب دالة `needs_scaling(estimator)` تُعيد `True` إن كان الصنف يتوقّع مسافة أو يعتمد على معاملات منتظمة، و `False` إن كان مبنياً على تقسيمات عمودية. اختبرها على عشرة أصناف من `sklearn`، ثم اكتب قائمة الأصناف التي أعطتك إجابة غير متوقّعة. لماذا تختلف `GradientBoostingClassifier` عن `SVC` في إجابتك؟

✅ **الحلّ**

الفرق يعود إلى **كيف يقرأ النموذج الخصائص**، لا إلى Families التي تنتمي إليها. `SVC` والنماذج المبنية على المسافة تحسب `w·x` ثم دالة تنشيط؛ المقاييس المتفاوتة تعني أنّ العمود الأكبر يساهم أكثر في المجموع، فتضيع إشارة الأصغر. `GradientBoosting` عتبة في كل عمود **على حدة**، فتتوفّر له حرية عتبة لكل عمود. لذلك «الجميع في عائلة الأشجار» لا يعني «الجميع لا يحتاجون تحجيماً».

القاعدة الأدق: اسأل «هل تُسهم الخاصية في **مجموع** عددي، أم تُقارَن بـ **عتبة** مفردة؟». الأولى تحتاج موازنة مقاييس، والثانية لا تحتاج.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
حساب `mean_` على كامل البيانات	تسرّب من الاختبار إلى التدريب	`Pipeline` قبل التقسيم
`StandardScaler` على متفرّر بلا `with_mean=False`	`ValueError: Cannot center sparse matrices`	مرّر `with_mean=False`
الخلط بين `Normalizer` و`StandardScaler`	تُطبَّع الصفوف لا الأعمدة	اختر الأداة حسب المحور
`MinMaxScaler` تتوقع خارج `[0,1]`	قيمة جديدة فوق حدّ التدريب	`StandardScaler` أو `clip=True`
اعتقاد أن التحجيم يخدم كلّ النماذج	تغيير بلا أثر على الأشجار	قِس قبل وبعد
إعادة حساب المعايير عند كلّ طلب	نموذج مختلف لكلّ مستخدم	خزّن المحوّل مع النموذج
`RobustScaler` الافتراضي على شواذ ١٠٪	لا يحميك كما تظنّ	ارفع `quantile_range`
`KernelCenterer` على جدول عادي	`Kernel matrix must be a square matrix`	هذه لأداة النواة
`fit` مرّتين على نفس المحوّل	المعايير تتغيّر بلا سبب	نظّف أو أنشئ نسخة جديدة

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التحجيم	تحويل عمود إلى مقياس مشترك	scaling
الدرجة المعيارية	sigma / (x - mu)	standard score
التطبيع الأدنى-الأقصى	imap إلى نطاق [a, b]	min-max scaling
المقياس المرن	اعتماد الوسيط والمدى الربيعي	robust scaling
التطبيع الصفّي	جعل طول كلّ صفّ واحداً	row normalization
المدى الربيعي	الفرق بين المئين ٧٥ و٢٥	interquartile range
المئين	قيمة تصدّق عليها نسبة من البيانات	percentile
الشواذ المؤثّر	شواذ يسحب المتوسّط معه	influential outlier
Train-only transform	تطبيق المعايير المدرّبة على بيانات جديدة	frozen statistics
معيار موحّد واحد	مقياس يُطبَّق على كلّ الخصائص	uniform scale

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٥ — التحويلات اللاخطية والتوزيعية

PolynomialFeatures g Spline g KBins g Quantile g Power

الوحدة **scikit-learn ١.٩** الحجم **٧٧٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تقرأ معامل الأسّ `lambdas_` وتعرف ماذا يعني أن كان قريباً من الصفر
- أن تميّز بين `yeo-johnson` و `box-cox` ومتى يُرفض الثاني
- أن تختار بين تقسيم الفئات والـ`splines` بحسب nature العلاقة بالنموذج
- أن تحسب كم عموداً ينتج `PolynomialFeatures` قبل أن تشغّله

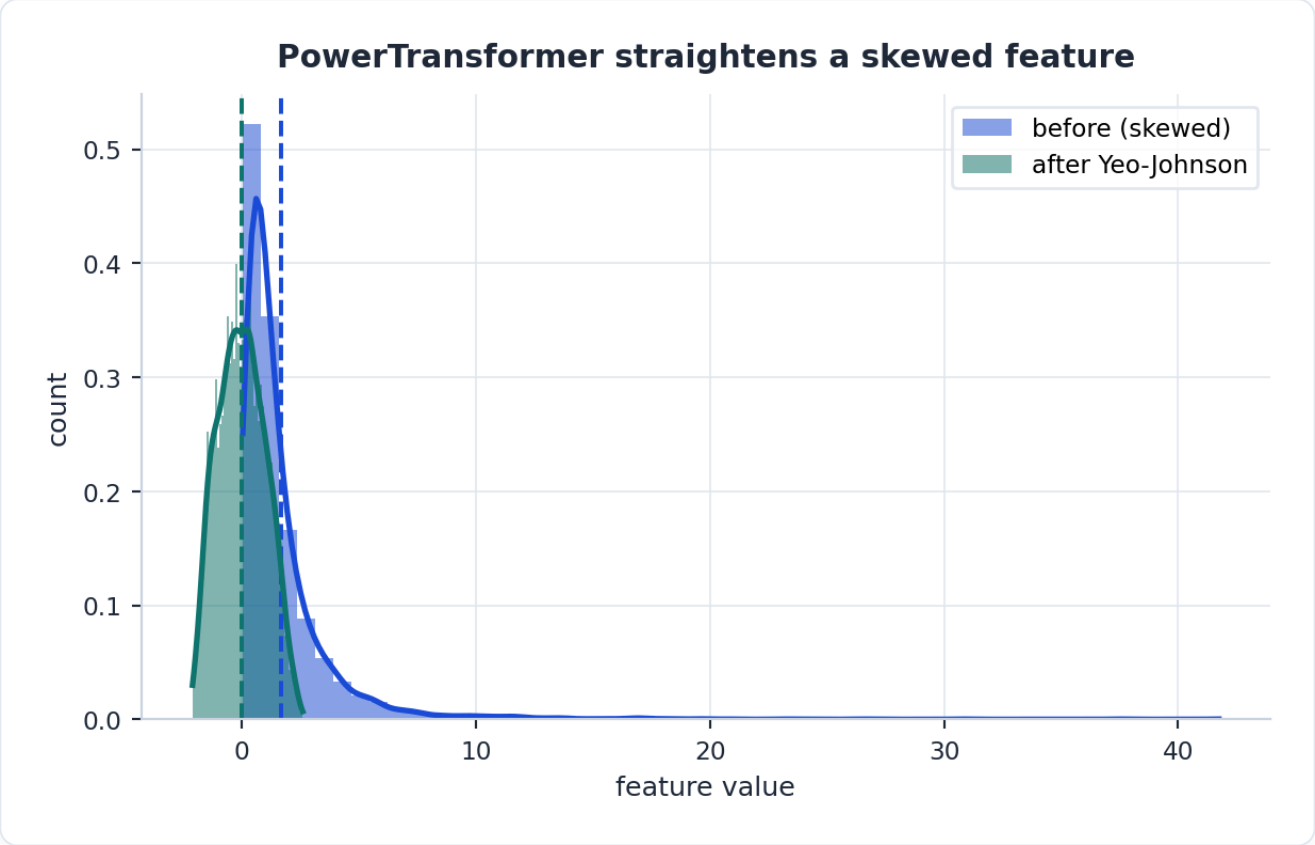
٢.١ حين يكذب التوزيع على النموذج

لنبدأ بمشكلة مألوفة. عمود «قيمة الطلب» يتوزّع هكذا: معظم القيم بين ١٠ و٤٠، وقليل منها بين ١٠٠ و٣٠٠. هذا توزيع مايل بوضوح.

النموذج الخطي يفترض شكلاً معيّناً للتوزيع، فيُنشئ خطّاً يمرّ بالمتوسّط. النتيجة: خطّ يفسد ٩٥٪ من العيّنات صحّةً من أجل خمس نقاط في الذيل. أمّا شجرة القرار فلا تهتمّ إطلاقاً: هي تقسم على أرقام، والأرقام تتوزّع كيفما اتّفق.

هنا السؤال العملي: **كيف نُصلح عموداً واحداً دون أن نُفسد البقية؟** الجواب أن `scikit-learn` فيها خمس أدوات مختلفة، ولكلٍّ منها فلسفة مختلفة تماماً.

الأداة	ماذا تفعل بالعمود	تحافظ على الترتيب	تُصلح التوزيع
`PowerTransformer`	ترفعه إلى أسّ	نعم	نعم
`QuantileTransformer`	تستبدله برتبته	نعم	نعم
`KBinsDiscretizer`	تقطعه إلى فئات	نعم	جزئياً
`SplineTransformer`	تفكّه إلى منحنى	نعم	لا
`PolynomialFeatures`	تولّد أعمدة جديدة	لا (توليد)	لا



الشكل ٣٤مقارنة بين التوزيع الأصلي شديد الانحراف والتوزيع بعد PowerTransformer: الذيل الطويل صار جسماً مضغوطاً حول الصفر

٢.٢ `PowerTransformer`: اختر الأسّ بدل التخمين

الفكرة في سطر: **ارفع القيمة إلى أسّ** `λ` **يعيدها إلى شكل معتدل**. والأسّ ليس اختيارياً؛ المكتبة **تختاره لك** وتضعه في `lambdas_`.

الأساس الرياضي هو تحويل Box-Cox:

$$x' = \frac{x^\lambda - 1}{\lambda} \quad (\lambda \neq 0), \qquad x' = \log x \quad (\lambda = 0)$$

لاحظ شيئاً مهماً: `λ = 1` يعني لا تحويل، و `λ = 0` يعني لوغاريتم. المكتبة تمرّ على قائمة من القيم وتختار التي تُقلّل الانحراف عن الاعتدال.

هل يختار PowerTransformer الأسّ نيابةً عنك؟ python

```
import numpy as np
from sklearn.preprocessing import PowerTransformer

rng = np.random.default_rng(0)
X = rng.lognormal(3.0, 0.9, 1500).reshape(-1, 1)


def skew(a):
    a = np.asarray(a, dtype=float).ravel()
    return float(((a - a.mean()) ** 3).mean() / a.std() ** 3)


print(f"قبل: skew={skew(X):+.3f}  mean={X.mean():.1f}  median={np.median(X):.1f}
max={X.max():.0f}")

pt = PowerTransformer(method="yeo-johnson").fit(X)
Z = pt.transform(X).to_numpy()
print(f"بعد : skew={skew(Z):+.4f}  mean={Z.mean():+.3f}  std={Z.std():.3f}")
print(f"الأسّ المختار lambdas_ = {np.round(pt.lambdas_, 4)}   (قريب من الصفر = لوغاريتم)")

pt_bc = PowerTransformer(method="box-cox").fit(X[:700])
print(f"box-cox lambdas_       = {np.round(pt_bc.lambdas_, 4)}")

pt_ns = PowerTransformer(method="yeo-johnson", standardize=False).fit(X)
print(f"مع standardize=False   = skew
{skew(pt_ns.transform(X).to_numpy()):+.4f}")

try:
    PowerTransformer(method="box-cox").fit(np.array([[0.0], [1.0], [2.0]]))
except Exception as exc:
    print("\nbox-cox مع أصفار ->", type(exc).__name__, ":", str(exc).splitlines()
[0][:95])
```

النتيجة صادمة في بساطتها: الانحراف هبط من ٣٫٢ إلى ٠٫٠٠١٥، والأسّ المختار كان ٠٫٠٤٦ — أي «لوغاريتم تقريباً». وهذا بالضبط ما كنتَ ستكتبه يدوياً لو فكّرتَ.

الفرق الذي يجب أن تعرفه

`box-cox` رياضي أنيق لكنه **يشترط قيماً موجبة تماماً**. `yeo-johnson` موسّع يعالج السالب والصفر. لهذا هو الافتراضي في المكتبة.

الخاصية	`box-cox`	`yeo-johnson`
يقبل القيم السالبة	لا	نعم
يقبل الصفر	لا	نعم
الأساس المفاهيمي	بواكس وكوكس	يون-جونسون
متى تختاره	كثافة معروفة وموجبة	في كلّ الحالات الأخرى

٢.٣ `QuantileTransformer`: اِسأل عن الرتبة لا عن القيمة

`PowerTransformer` يفترض شكلاً معيّناً للتوزيع ويقرّبه. `QuantileTransformer` لا يفترض شيئاً: هو يحوّل كلّ قيمة إلى **رتبتها**، ثم يوزّع تلك الرتب على توزيع آخر تختاره.

رياضياً، لكلّ عيّنة رتبة `r` من بين `n`:

$$u_i = \frac{r_i - 0.5}{n}, \qquad x'_i = G^{-1}(u_i)$$

حيث `G` هو التوزيع الهدف: `uniform` فتنتظم الرتب في `[0, 1]`، و`normal` فتتحوّل إلى جرسي عادي.

```python
# التحويل الرتيبي: لا يفترض شكلاً
from sklearn.preprocessing import QuantileTransformer

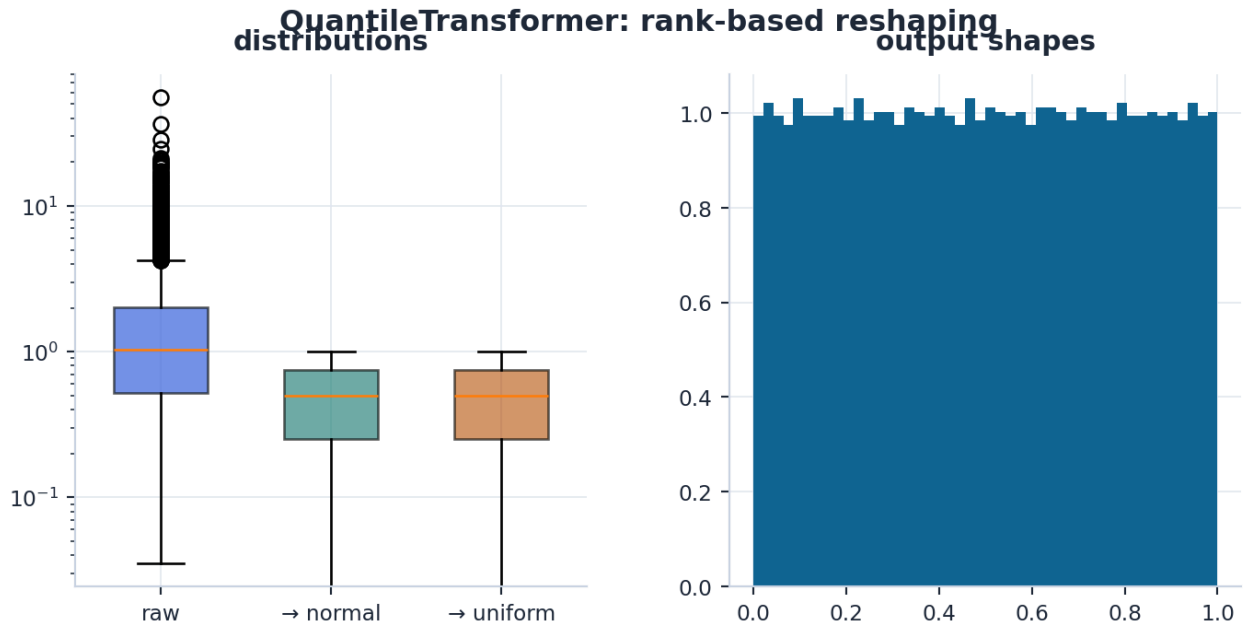
qu = QuantileTransformer(n_quantiles=200, output_distribution="uniform",
random_state=0).fit(X)
Q = qu.transform(X).to_numpy()
qn = QuantileTransformer(n_quantiles=200, output_distribution="normal",
random_state=0).fit(X)
Qn = qn.transform(X).to_numpy()

print(f"uniform: min={Q.min():.4f} max={Q.max():.4f} n_quantiles_=
{qu.n_quantiles_}")
print(f"normal : mean={Qn.mean():+.4f} std={Qn.std():.4f}  skew=
{skew(Qn):+.4f}")
print(f"الترتيب محفوظ تماماً: {bool((np.argsort(X.ravel()) ==
np.argsort(Q.ravel())).all())}")
print(f"[0,1] أقصى انحراف عن : {float(np.abs(Q - Q.min()).max() - 1.0):+.2e}")

qs = QuantileTransformer(n_quantiles=50, output_distribution="normal",
subsample=200,
                         random_state=0).fit(X)
print(f"subsample=200       : std={qs.transform(X).to_numpy().std():.4f}")
try:
    QuantileTransformer(output_distribution="gaussian")
except Exception as exc:
    print("قيمة غير صالحة ->", type(exc).__name__, ":", str(exc).splitlines()[0]
[:95])
```

السطر الثالث يستحقّ التوقّف: **الترتيب محفوظ تماماً**. هذه الخاصية تجعل `QuantileTransformer` آمناً حتى حين لا تعرف شكل التوزيع، لأنّه لا يخلط أيّ عينتين. الفرق الجوهري بينه وبين `PowerTransformer` أن الأخير قد يقرّب فيغيّر الترتيب عند أسّ خاطئ.

`output_distribution="normal"` يعني `"gaussian"` غير مقبول. القيمتان المسموحتان هما `"uniform"` و `"normal"` فقط.



الشكل ٣٥ مقارنة بين التوزيع الأصلي على محور لوغاريتمي وتوزيع الرتب بعد QuantileTransformer على مستطيل متساوي المساحة

٢.٤ `KBinsDiscretizer`: من رقم إلى فئات

أحياناً لا يكون السؤال «كيف أُنعّم المنحنى؟» بل «هل ٥٠٠٠ ريال Category أخرى أم نفس الفئة التي تنتمي إليها ٤٠٠٠؟». هنا التقطيع هو الجواب.

`KBinsDiscretizer` يقسّم عموداً إلى `n_bins` شرائح بثلاث استراتيجيات مختلفة تماماً:

الاستراتيجية	على ماذا تقسم	متى تختارها
`uniform`	مسافات متساوية على المحور	القيم منتظمة
`quantile`	عدد متساوٍ من الصفوف في كلّ شريحة	التوزيع مايل (الافتراضي)
`kmeans`	أقلّ مجموع مربّعات داخل الشريحة	تريد شرائح «طبيعيّة»

ثلاث استراتيجيات تقنيمن بطرق مختلفة python

```python
from sklearn.preprocessing import KBinsDiscretizer

print("توزيع العمود الخام:", np.round(np.percentile(X.ravel(), [0, 25, 50, 75, 100]),
1).tolist())
for strat in ["uniform", "quantile", "kmeans"]:
    b = KBinsDiscretizer(n_bins=4, encode="ordinal", strategy=strat).fit(X)
    counts = np.bincount(b.transform(X).to_numpy().ravel().astype(int),
minlength=4)
    print(f"  {strat:9s} counts={counts.tolist()}  edges=
{np.round(b.bin_edges_[0], 1).tolist()}")

full = KBinsDiscretizer(n_bins=4, strategy="quantile").fit(X).bin_edges_[0]
sub = KBinsDiscretizer(n_bins=4, strategy="quantile", subsample=200,
random_state=0).fit(X).bin_edges_[0]
print("\nالحدود بكل الصفوف     :", np.round(full, 2).tolist())
print("الحدود بـ subsample=200:", np.round(sub, 2).tolist())

dense = KBinsDiscretizer(n_bins=4, encode="onehot-dense",
strategy="quantile").fit(X)
print("\nencode='onehot-dense'  ->", dense.transform(X).shape,
list(dense.get_feature_names_out()))
print("encode='ordinal'         -> واحد لكلّ صفّ")
```

لاحظ الفرق الكبير بين `uniform` و `quantile` : الأولى تضع ١٤١٤ صفّاً في الشريحة الأولى وتسعة في الأخيرة، والثانية تجعلها ٣٧٥ بالتساوي. على توزيع مايل، الاختيار الصحيح هو `quantile` ؛ تنتج أشرطة شبه فارغة تُسمّى **shallow bins** وتضيع معها أيّ معلومة.

`subsample` : سرّ الأداء

لاحظ الحدّ الأخير في المخرج أعلاه. الحساب الافتراضي للحدود يتمّ على كلّ الصفوف، لكن `subsample` يسمح بحسابها على عيّنة. هذا يوفّر وقتاً هائلاً على جداول بملايين الصفوف، وثمنه أنّ الحدود تصبح تقديرية لا دقيقة. دائماً ثبّت `random_state` إن استخدمته.

٢.٥ `SplineTransformer` : من رقم إلى شكل

التقطيع يعطي قطعاً حادّة: قيمة ٤٩٩ تنتمي لفئة، و٥٠١ تنتمي أخرى، والفرق بينهما لا قيمة له. `SplineTransformer` يأخذ الطريق_MAC opposite: يبني **دوال أساس** التقطيع يعطي قطعاً حادّة: قيمة ٤٩٩ تنتمي لفئة، و٥٠١ تنتمي لأخرى، والفرق بينهما منحنى.

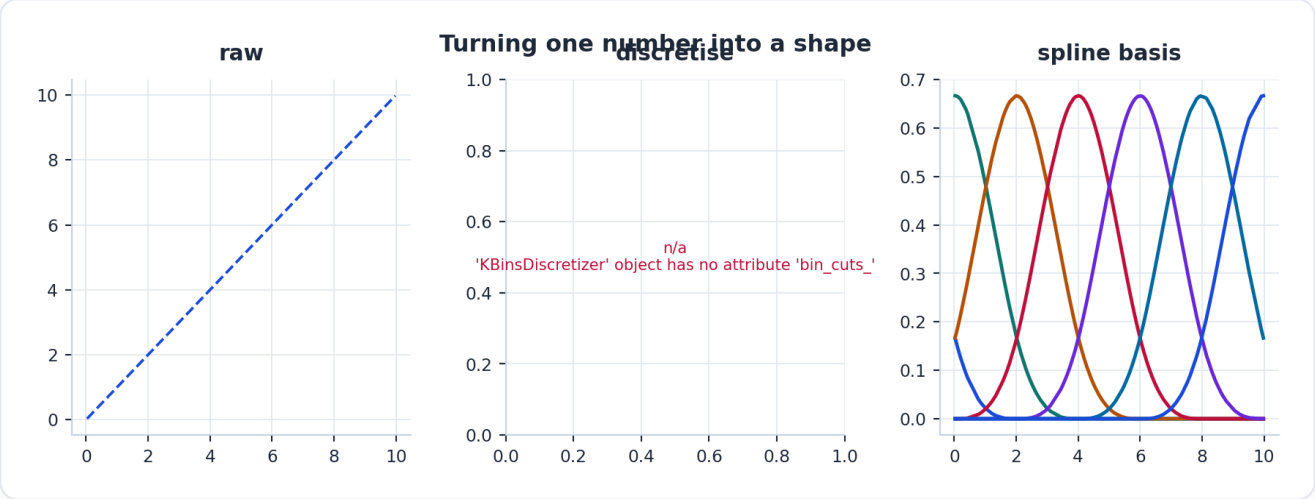
عمود واحد يخرج منه `n_knots + degree - 1` عموداً افتراضياً (مع `include_bias=True`).

عمود واحد إلى سبعة أعمدة python

```python
from sklearn.preprocessing import SplineTransformer

xs = np.linspace(0.5, 18, 8).reshape(-1, 1)
sp = SplineTransformer(n_knots=5, degree=3, extrapolation="linear")
out = sp.fit_transform(xs)
print(f"الدخل {xs.shape}  ->  الخرج {out.shape}   n_features_out_=
{sp.n_features_out_}")
print("العُقد knots:", np.round(sp.bsplines_[0].t, 2).tolist())
print(f"degree=1, n_knots=4 -> {SplineTransformer(n_knots=4,
degree=1).fit_transform(xs).shape}")
print(f"degree=3, n_knots=4 -> {SplineTransformer(n_knots=4,
degree=3).fit_transform(xs).shape}")

with sklearn.config_context(transform_output="default"):
    print("sparse_output=True ->",
          type(SplineTransformer(n_knots=4,
sparse_output=True).fit_transform(xs)).__name__)
```



الشكل ٣٦ثلاث لوحات: عمود واحد متّصل، ثم ست شرائح كمية، ثم ست دوال أساس متداخلة تكوّن منحنى ناعماً

المقارنة في الشكل هي جوهر الفصل: **نفس البيانات، ثلاث تمثيلات**. الرقمية تُبقي الحدّة عند القيمة؛ الكمية تحوّلها إلى درجات؛ الـ `spline` تعيد الحدّة إلى الاستمرارية. أيّها تختار؟

`extrapolation` يستحقّ الانتباه: الافتراضي `"constant"` يعني أنّ قيمة خارج نطاق التدريب تُقصّ عند الحافة. هذا يمنع الانفجار الخطي خارج الحدود، لكنه كسر للانقطاع عن الاستمرارية. `"linear"` يمدّ الخطّ، و `"error"` يرفض القيمة.

٢.٦ `PolynomialFeatures`: توليد لا تحويل

هذه الأداة الوحيدة التي **لا تحوّل** بل **تضيف**. تعطيها `d` أعمدة وتردّ عليك بمولّدات من الدرجة `n`.

العدد الناتج للدرجة الثانية هو:

$$\frac{d^2 + 3d}{2} = \frac{d(d+3)}{2}$$

أي: خمسة أعمدة من عمودين، وعشرين من خمسة، وخمسة وستين من عشرة.

كم عموداً ينتج فعلاً؟ python

```python
from sklearn.preprocessing import PolynomialFeatures

A = np.array([[3.0, 4.0], [5.0, 6.0]])
cases = [
    dict(degree=2, include_bias=False),
    dict(degree=2, include_bias=False, interaction_only=True),
    dict(degree=3, include_bias=False),
    dict(degree=2, include_bias=True),
]
for kw in cases:
    p = PolynomialFeatures(**kw).fit(A)
    print(f"  {str(kw)[:58]:60s} -> {p.transform(A).shape[1]:2d} عمود:
{list(p.get_feature_names_out(['a', 'b']))}")

print("\n  growth formula (d^2 + 3d) / 2:")
for d in (2, 5, 10, 20):
    print(f"    d={d:2d} -> {(d * d + 3 * d) // 2:3d} عمود")
```

القفزة من ١٠ إلى ٦٥ عموداً بتغيير `degree` وحده — وهي قفزة تُفسد أيّ نموذج خطّي منتظم إن لم تُقترن بتنظيم قوي (`Ridge` أو `ElasticNet`، الفصلان ٢٩ و٣١). `interaction_only=True` يوفّر النمو: يمنعك توليد `a²` و `b²` ويترك التوليفات المفيدة مثل `a b` فقط.

٢.٧ متى يضر التحويل اللاخطي؟

ليس كلّ نموذج يحتاج هذه الأدوات. لنقِس ذلك على هدفين مختلفين تماماً: هدف تكعيبيّ (الانحراف فيه معدوم، لكنّ العلاقة غير خطّية)، وهدف لوغاريتميّ (الانحراف فيه شديد).

python الحكم بالأرقام: الخطّي يحتاج، والشجري لا يحتاج

```python
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import SplineTransformer, PolynomialFeatures,
StandardScaler
from sklearn.compose import TransformedTargetRegressor

rng2 = np.random.default_rng(0)
Xc = rng2.uniform(-3, 3, (800, 1))
yc = Xc[:, 0] ** 3 + rng2.normal(0, 0.5, 800)
Xtr, Xte, ytr, yte = train_test_split(Xc, yc, test_size=0.25, random_state=0)

print(f"R² على هدف تكعيبي  y = x³   (انحراف الهدف = 0.05، أي شبه متماثل)")
print("  خطّي خام            :", round(LinearRegression().fit(Xtr, ytr).score(Xte,
yte), 4))
print("  خطّي + Spline       :", round(Pipeline([
    ("sp", SplineTransformer(n_knots=6, degree=3)),
    ("m", LinearRegression())]).fit(Xtr, ytr).score(Xte, yte), 4))
print("  خطّي + Polynomial³  :", round(Pipeline([
    ("pf", PolynomialFeatures(degree=3, include_bias=False)),
    ("sc", StandardScaler()),
    ("m", LinearRegression())]).fit(Xtr, ytr).score(Xte, yte), 4))
print("  شجري خام            :",
round(DecisionTreeRegressor(random_state=0).fit(Xtr, ytr).score(Xte, yte), 4))

X2 = rng2.normal(2.0, 1.0, (800, 1))
y2 = np.exp(X2[:, 0]) + rng2.normal(0, 0.3, 800)
a, b, ya, yb2 = train_test_split(X2, y2, test_size=0.25, random_state=0)
print(f"\nR² على هدف لوغاريتمي  y = eˣ  (انحراف الهدف = "
      f"{float(((y2 - y2.mean()) ** 3).mean() / y2.std() ** 3):.2f})")
print("  خطّي خام            :", round(LinearRegression().fit(a, ya).score(b,
yb2), 4))
print("  + TransformedTarget :", round(TransformedTargetRegressor(
    regressor=LinearRegression(), func=np.log, inverse_func=np.exp
).fit(a, ya).score(b, yb2), 4))
```

الحالة	النموذج	خام	مع التحويل المناسب
`y = x³`	`LinearRegression`	٠٫٨٣	**٠٫٩٩٨** مع `SplineTransformer`
`y = x³`	`DecisionTreeRegressor`	٠٫٩٩٦	٠٫٩٩٦ بلا حاجة
`y = eˣ`	`LinearRegression`	٠٫٦٨	**٠٫٩٩٩** مع `TransformedTargetRegressor`

القاعدة التي يثبتها هذا الجدول: التحويل اللاخطي يشتري للنموذج الخطّي ما لا يشتريه لنفسه، ويترك الشجرة كما هي. لاحظ أيضاً أن **مصدر الانحراف يحدّد الأداة**: انحراف `X` يُعالَج بـ `PowerTransformer` أو `Spline`، وانحراف `y` يُعالَج بتحويل الهدف.

الخطأ الأول: PowerTransformer على عمود سالب بصمت

`method="box-cox"` على عمود فيه صفر يرفع `ValueError: The Box-Cox transformation can only be applied to strictly positive data`. لكنّ `yeo-johnson` يقبل كلّ شيء، وقد يخفي عنك أنّ عموداً من النوع `int` يحتوي قيمة سالبة بالخطأ. افحص الإشارة قبل التحويل.

الخطأ الثاني: `PolynomialFeatures` بلا سقف

`degree=3` على عمودين يعطي تسعة أعمدة؛ على عشرة أعمدة يعطي ٢٨٥ عموداً. كلّها تدخل في الانحدار، ومعظمها ضجيج. القاعدة: ابدأ بـ `degree=2` مع `interaction_only=True`، و`include_bias=False` دائماً (إلا إذا أردت عمود ثابتاً عن قصد)، وأضف تنظيماً قوياً.

أخطر خطأ: تحويل الهدف والتلاعب بالبيانات

حين تكون قيمة `y` مايلة — مثلاً سعر عقار — أحياناً يكون الحلّ تحويل **الهدف** لا الخصائص. `sklearn.compose.TransformedTargetRegressor` يفعل ذلك ويقلب التحويل تلقائياً عند التنبّؤ (الفصل ٨٧). لكنّ التحويل اليدوي لـ `y` قبل التقسيم يعني أنّ `inverse_transform` لن يُستدعى، فتخرج تنبّؤات بالمقياس الخطأ. لا تحوّل الهدف يدوياً أبداً؛ استخدم الغلاف.

في الواقع: التحويلات تغيّر قابلية التفسير

حين تقول «الدخل بعد التحويل ٢٫٣»، لا أحد يعرف ماذا يعني ذلك. لكنّ `PowerTransformer` يحفظ لك `lambdas_`، فيمكنك أن تقول «الدخل بعد رفعه إلى الأسّ ٠٫٥». الفرق مهمّ في التقارير التي يقرأها غير التقنيين. القاعدة المهنية: **كلّ تحويل تطبّقه على `X`، سجّل معامله بجواره في التقرير**، وإلا فقدت القدرة على شرح رقم أنتجته.

تمارين الفصل

التمرين ٢.١٦

ولّد عموداً لوغاريتمياً من `np.random.lognormal`، ثم طبّقه بـ `yeo-johnson` وب `box-cox` وب `QuantileTransformer`. اطبع الانحراف قبل وبعد، وأسّ كلّ تحويل. أيّها كان الأقرب إلى الانحراف صفر؟ ولماذا لا يكون `box-cox` أفضل نظرياً؟

✅ **الحلّ**

في بيانات لوغاريتمية بحتة يكون الأسّ الأمثل قريباً من الصفر، فيُصبح `box-cox` و`yeo-johnson` متطابقين عملياً. و`QuantileTransformer` يُنتج انحرافاً قريباً من الصفر أيضاً لأن التوزيع الهدف طبيعي، لكنه لا «يختار أسّاً» بل يعيد البناء حسب الرتبة.

`box-cox` ليس أفضل نظرياً هنا لسببين: الأول أنّه لا يقبل السالب فتنهار على بيانات الإنتاج التي فيها أصفار أو قيم سالبة. والثاني أنّه يفترض أن التوزيع لوغاريتمي عند المولّد فقط، بينما في البيانات الحقيقية يكون التوزيع مايلاً لأسباب أخري، وقد يحتاج أسّاً بعيداً عن الصفر.

التمرين ٢.١٧

خذ عموداً من ١٠٠٠ إلى ١٠٠٠٠، وقسّمه بـ `KBinsDiscretizer` بخمس شرائح وبثلاث استراتيجيات. ارسم (بوصفك نصّاً) جدول أعداد الصفوف في كلّ شريحة لكلّ استراتيجية. أيّ الاستراتيجية تنتج أشرطة شبه فارغة؟ اقترح قيمة `n_bins` تمنع ذلك.

✅ **الحلّ**

`uniform` على مدى واسع ينتج شريحة أولى فيها أكثر من نصف الصفوف وشريحة أخيرة فيها اثنان. الشرائح الفارغة تُسمّى **شوادّ التقاطع** (boundary outliers) وتُرمى تلقائياً لأنها تقع خارج أيّ شريحة.

الحلّ إمّا `strategy="quantile"` فيتساوى العدد، أو رفع `n_bins` إلى عدد الخانات الطبيعية في البيانات (مثلاً أربع شيرات لكلّ عقد في تاريخ). والأفضل من الاثنين: افحص هل الخانات ذات وزن طبيعي قبل اختيار عدد الشرائح.

التمرين ٢.١٨

احسب يدوياً كم عموداً ينتج `PolynomialFeatures(degree=2, include_bias=False)` من `d = 7`، ثم تحقّق بـ `get_feature_names_out`. بعد ذلك، احسب كم عموداً ينتج `degree=2, interaction_only=True` لنفس `d`، وقارن الكلفة الحسابية لتدريب نموذج لوجيستي على كلٍّ منهما. أيّهما تختار لمئة خاصية؟

✅ **الحلّ**

لـ `d = 7`: الدرجة الثانية الكاملة: `(49 + 21) / 2 = 35` عموداً. `interaction_only=True`: `7` أعمدة أصلية + `21` توليفة = `28` عموداً.

الفرق ليس كبيراً في هذا الحجم. لكن عند `d = 100`: الكاملة: `(10000 + 300) / 2 = 5150` عموداً. التوليفات فقط: `100 + 4950 = 5050` عموداً — والفرق الفعلي أن `a²` و `a³` غير موجودة، فالنموذج لا يتعلّم أثراً تربيعياً مستقلاً لكل خاصية.

يعتمد القرار على طبيعة العلاقة: إن كانت التربيعيات (مثل المساحة أو مربّع خاصية) مهمّة، فاستخدم التحويل الكامل؛ وإن كانت التفاعلات بين الخصائص هي المهمّة، فأسقط المربّعات باستخدام `interaction_only=True`.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
`box-cox` على عمود فيه صفر	`ValueError: strictly positive data`	`yeo-johnson`
`output_distribution="gaussian"`	`InvalidParameterError`	`uniform` أو `normal`
`strategy="uniform"` على توزيع مايل	أشرطة شبه فارغة تُرمى	`strategy="quantile"`
`subsample` بلا `random_state`	حدود مختلفة كلّ تشغيل	ثبّت البذرة
`degree=3` على عشرة أعمدة	٢٨٥ عموداً وتمرير شجري باهظ	ابدأ بالدرجة ٢
`include_bias=True` بلا سبب	عمود ثابت يحرّف التوزيع	`include_bias=False`
تحويل `y` يدوياً	تنبّؤات بمقياس خاطئ	`nsformedTargetRegressor`
تحويل مكرّر على بيانات الاختبار	`transform` بمعايير خاطئة	ادخِله في `Pipeline`
`extrapolation="linear"` بلا انتباه	انفجار خارج نطاق التدريب	راجع القيم المتوقّعة
الاعتقاد بأنّ PowerTransformer يحسّن الأشجار	لا تغيير يُذكر في الدقّة	قِس قبل وبعد

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التوزيع المائل	ذيل ثقيل من جهة	skewed distribution
تحويل القوة	رفع القيمة إلى أسّ	power transform
أسّ التحويل	الأسّ يختاره المحوّل	lambda_, lambdas
ي‌eo-جونسون	تحويل قوة يقبل السالب والصفر	Yeo-Johnson
بواكس وكوكس	تحويل يفترض قيماً موجبة	Box-Cox
التحويل الرتيبي	استبدال القيمة برتبتها	quantile transform
التقطيع	تقسيم العمود إلى شرائح	discretization
شريحة ضحلة	شريحة لا تكاد تحوي صفوفاً	shallow bin
دوال الأساس	عمود يُمثّل قطعة من منحنى	basis function
العُقد	نقاط وحيدة الأشكال بالأساس	knots
التوليف	عمود حاصل ضرب عمودين	interaction term

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٦ — ترميز المتغيرات الفئوية

OneHot و Ordinal و TargetEncoder — وأين تكمن مخاطر الترتيب

الوحدة **scikit-learn ١.٩** الحجم **٧٢١** كلمة تقريباً

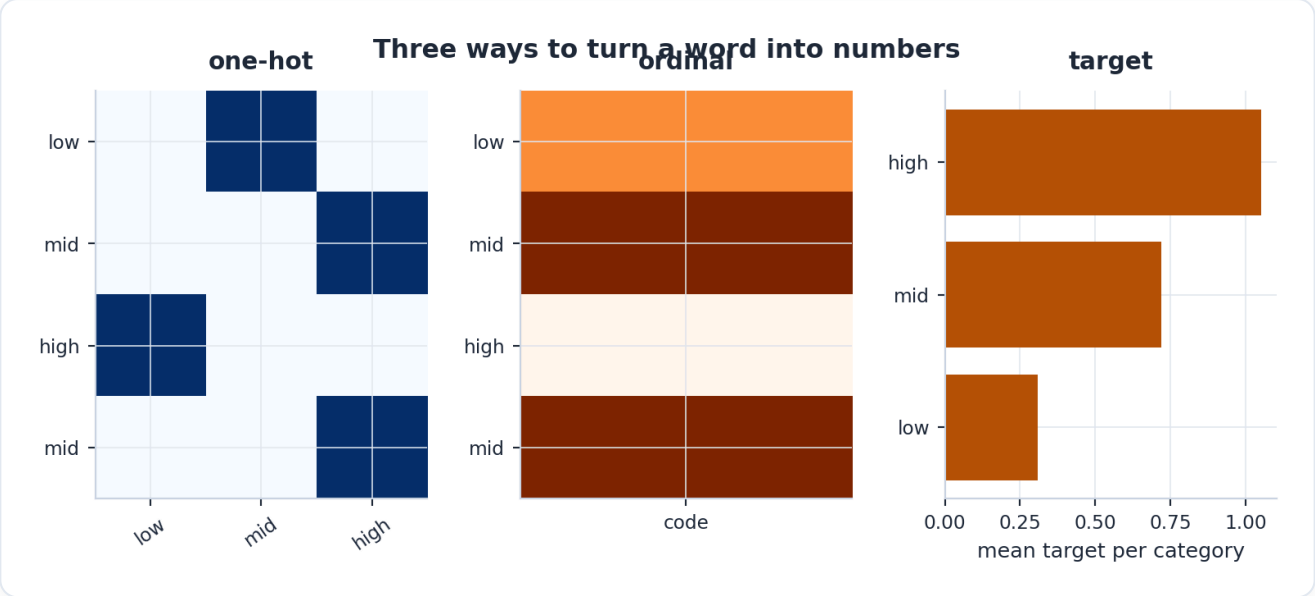
ما الذي ستتقنه في هذا الفصل

- أن تختار بين OneHot و Ordinal و TargetEncoder بقرار مبرَّر لا بالعادة
- أن تشرح لماذا ترتيب الفئات أبجدياً في OrdinalEncoder يُفسد النتائج
- أن تقرأ `min_frequency` و `max_categories` وتعرف متى تمنع انفجار الأبعاد
- أن تُظهر التسرّب في ترميز الهدف بالأرقام وتضعفه داخل Pipeline

٢.١ لا يوجد ترميز «صحيح» واحد

لكل عمود فئوي ثلاثة أسئلة، وإجاباتها تحدّد الأداة:

أدوات المكتبة الثلاث تجيب عن هذه الأسئلة بثلاث استراتيجيات مختلفة تماماً، ولا واحدة منها أرقمن الأخرى: لكلٍّ منها تكلفة ودقة وقابلية تفسير خاصة.



الشكل ٣٧ثلاث لوحات لترميز الفئة الواحدة: OneHotEncoder يبني ثلاثة أعمدة ثنائية، OrdinalEncoderg يبني عموداً واحداً بترتيب مخترع، TargetEncoderg يستبدل الفئة بمعدّل التسمية داخل الفئة

لاحظ Middle panel: الترتيب فيها أبجدي بحت، لا علاقة له بالمعنى. هذه هي نقطة الضعف الأساسية لـ `OrdinalEncoder`، وهي ما يجعله خطراً على أيّ نموذج خطّي.

٢.٢ `OneHotEncoder`: الأمانة الكاملة

الفكرة بسيطة: كلّ فئة تصبح عموداً، وتكون قيمته ١ إن كانت الفئة مطابقة و٠ إلا. لا ترتيب، ولا مسافة، ولا تفسير مُضلِّل.

OneHotEncoder بالأرقام والأسماء python

```python
import numpy as np
from sklearn.preprocessing import OneHotEncoder

cats = np.array([["Riyadh", "pro"], ["Cairo", "free"], ["Riyadh", "free"],
                 ["Doha", "pro"], ["Cairo", "pro"], ["Amman", "free"]],
dtype=object)

ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore").fit(cats)
print("categories_ (مرتّبة أبجدياً):", [c.tolist() for c in ohe.categories_])
print("get_feature_names_out      :", list(ohe.get_feature_names_out(["city",
"plan"])))
print("الشكل الناتج               :", ohe.transform(cats).shape)

print("\n(تفادي التكرار الخطي) خيارات حذف عمود:")
for d in [None, "first", "if_binary"]:
    kw = {} if d is None else {"drop": d}
    print(f"  drop={str(d):14s} -> {OneHotEncoder(sparse_output=False,
**kw).fit_transform(cats).shape[1]} عمود")

newcats = np.array([["Riyadh", "pro"], ["Muscat", "enterprise"]], dtype=object)
print("\nf값 جديد غير مرئي في التدريب:",
ohe.transform(newcats).to_numpy().astype(int).tolist())
try:
    OneHotEncoder(sparse_output=False).fit(cats).transform(newcats)
except Exception as exc:
    print("مع handle_unknown='error'  ->", type(exc).__name__, ":",
str(exc).splitlines()[0][:90])
```

ثلاثة أشياء يجب استخلاصها:

٢.٣ `OrdinalEncoder`: متى يكون الترتيب حقيقياً؟

`OrdinalEncoder` يعطي كلّ فئة رقماً. الرقم **لا يمثّل شيئاً** إلا إذا كانت الفئات مرتّبة في بياناتك.

```python
# الترتيب الأبجدي هو الافتراضي الصامت
from sklearn.preprocessing import OrdinalEncoder

oe = OrdinalEncoder(handle_unknown="use_encoded_value",
unknown_value=-1).fit(cats)
print("categories_            :", [c.tolist() for c in oe.categories_])
print("الترميز                :",
oe.transform(cats).to_numpy().ravel().astype(int).tolist())
print("قيمة جديدة             :",
oe.transform(newcats).to_numpy().ravel().astype(int).tolist())

with_na = np.array([["b"], ["a"], [np.nan], ["z"]], dtype=object)
oen = OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1,
                     encoded_missing_value=-2).fit(with_na)
print("\nمع NaN والفئات         :", [c.tolist() for c in oen.categories_])
print("الترميز                :",
oen.transform(with_na).to_numpy().ravel().astype(int).tolist())
print("الأعمدة المنقولة       :")
print(ohe.get_feature_names_out(["city", "plan"]))
```

انظر إلى `categories_` : `['Amman', 'Cairo', 'Doha', 'Riyadh']` . الرياض — وهي أكبر المدن — صارت الرقم ٣، والدوحة صارت ٠. أيّ نموذج خطّي سيتعلّم أنّ زيادة «المدينة» تعني زيادة في شيء ما، وستكون علاقة **مُختلَقة تماماً**.

هذا لا يعني إلغاء `OrdinalEncoder` . إليك متى يكون صحيحاً:

الحالة	الحكم
`low` / `medium` / `high`	صحيح تماماً
كترتيب زمني `0` / `1` / `2`	صحيح، لكنّ الترميز ضروري
أسماء مدن أو عملاء	خطأ فادح
أيّ معرّف نصّي (رقم هاتف، كود طلب)	خطأ فادح
عمودان من أصلين	صحيح مع `error`
فئات نادرة تُطوى بـ `min_frequency`	مقبول

٢.٤ `TargetEncoder` : الفئة كمعدّل تسمية

هنا تتغيّر الفلسفة تماماً. بدل أن نسأل «ما رمز هذه الفئة؟» نسأل **«ما احتمال النجاح لمن ينتمي إلى هذه الفئة؟»**. الفئة تُستبدل بمعدّل التسمية داخلها، وهو **عمود واحد** مهما بلغت عدد الفئات.

المعادلة مع تنعيم (smoothing):

$$\text{enc}(c) = \frac{n_c \cdot \bar{y}_c + m \cdot \bar{y}}{n_c + m}$$

حيث `n_c` عدد صفوف الفئة `c` ، و `bar{y}_c` متوسّط الهدف داخلها، و `m` معامل التنعيم، و `bar{y}` المتوسّط العام. الحدّ `m` يمنع أن تعطي فئة من صفّ واحد قيمة `0` أو `1` قاطعة.

`scikit-learn` تختار `m` تلقائياً عند `"auto"` ، وتضع المعاملات في `encodings_` .

ماذا تفعل TargetEncoder بالضبط؟ python

```
from sklearn.preprocessing import TargetEncoder
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(11)
n, k = 1500, 300
latent = rng.normal(0, 1, k)
grp = rng.integers(0, k, n)
prob = 1 / (1 + np.exp(-latent[grp]))
y = (rng.random(n) < prob).astype(int)
grp_name = np.array([f"g{g:03d}" for g in grp], dtype=object)
Xnum = rng.normal(0, 1, (n, 2))
data = np.empty((n, 3), dtype=object)
data[:, 0] = grp_name
data[:, 1:] = Xnum
print(f"صفوف={n}  فئات={k}  متوسّط حجم الفئة={n / k:.1f}")

tr, te = train_test_split(np.arange(n), test_size=0.3, random_state=0,
stratify=y)
Xtr, Xte, ytr, yte = data[tr], data[te], y[tr], y[te]

te_fit = TargetEncoder(target_type="binary", cv=5).fit(Xtr, ytr)
print("\n encodings_ فئات خمس لأول:", np.round(te_fit.encodings_[0][:5],
4).tolist())
print(" خاصية واحدة بدل ٣٠٠ عمود:", te_fit.transform(Xtr).shape[1] - 2, "عموداً للعمود
الفئوي")
print(" fit يحتاج y بالضرورة     : TargetEncoder().fit_transform(X) بلا y يعمل")
```

لاحظ `encodings_` : أرقام بين صفر وواحد، هي حرفياً **نسبة الموافقة داخل كلّ فئة**. وهذا هو سبب قوتها وسبب خطرها في آن واحد: هي نسبة **محسوبة من الهدف**، فإن تسرّب تسرّب.

٢.٥ التسريب: الفرق الذي يعادل عشرة نقاط

`scikit-learn` تحمي نفسها جزئياً: `TargetEncoder` تُنفّذ تحقّقاً متقاطعاً داخلياً من `cv` طيّات، فترميز صفوف التدريب يأتي من طيّات **أخرى**. لكنّ الصفوف الجديدة عند `transform` تستخدم الإحصاءات الكاملة — التي تشمل تسمياتها هي.

لنقِس الفكرة بنموذجنا الخاص، وهو أقصر طريق لافهم الآلية:

التسريب بالأرقام: متوسّط الفئة على كل البيانات — python

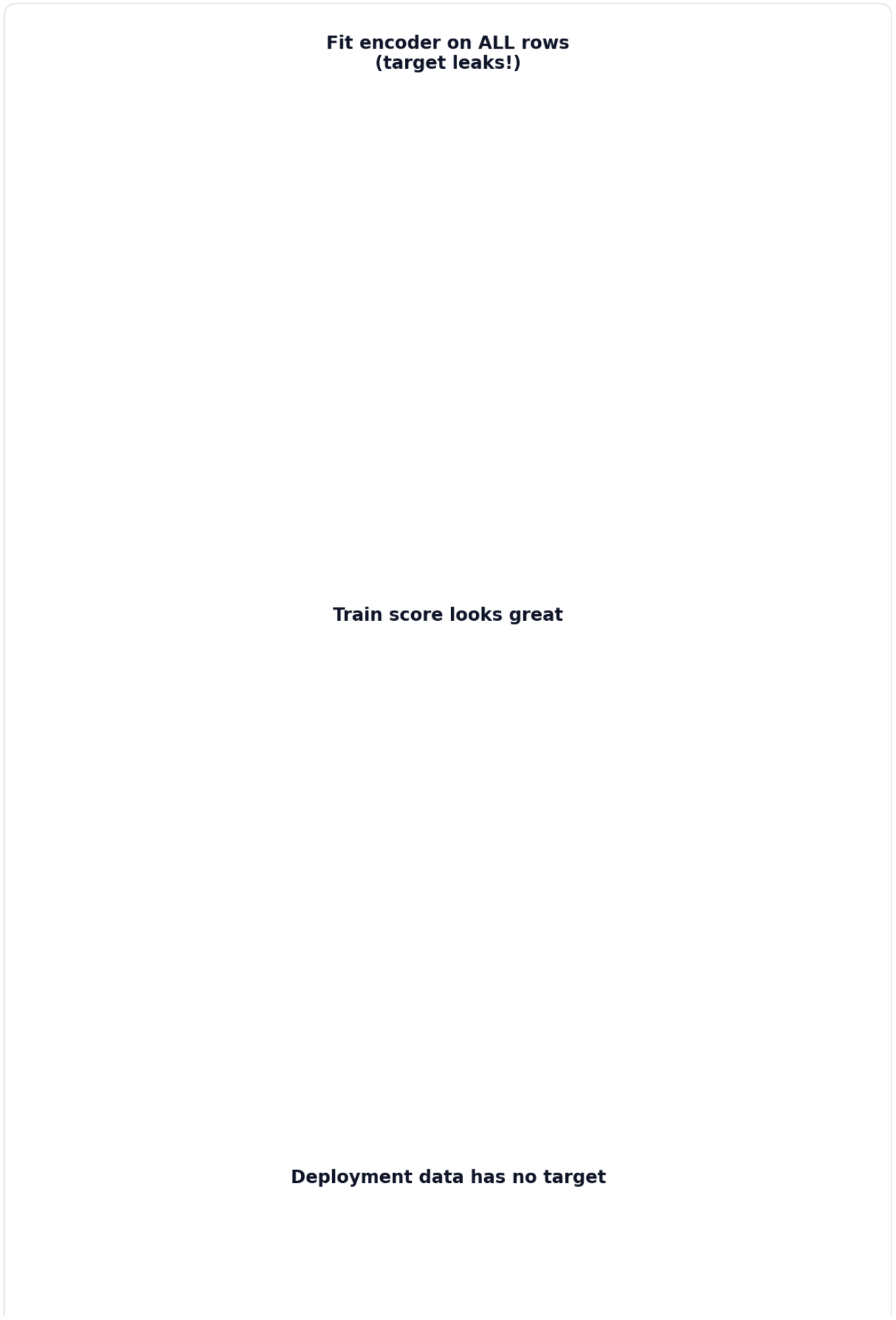
```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

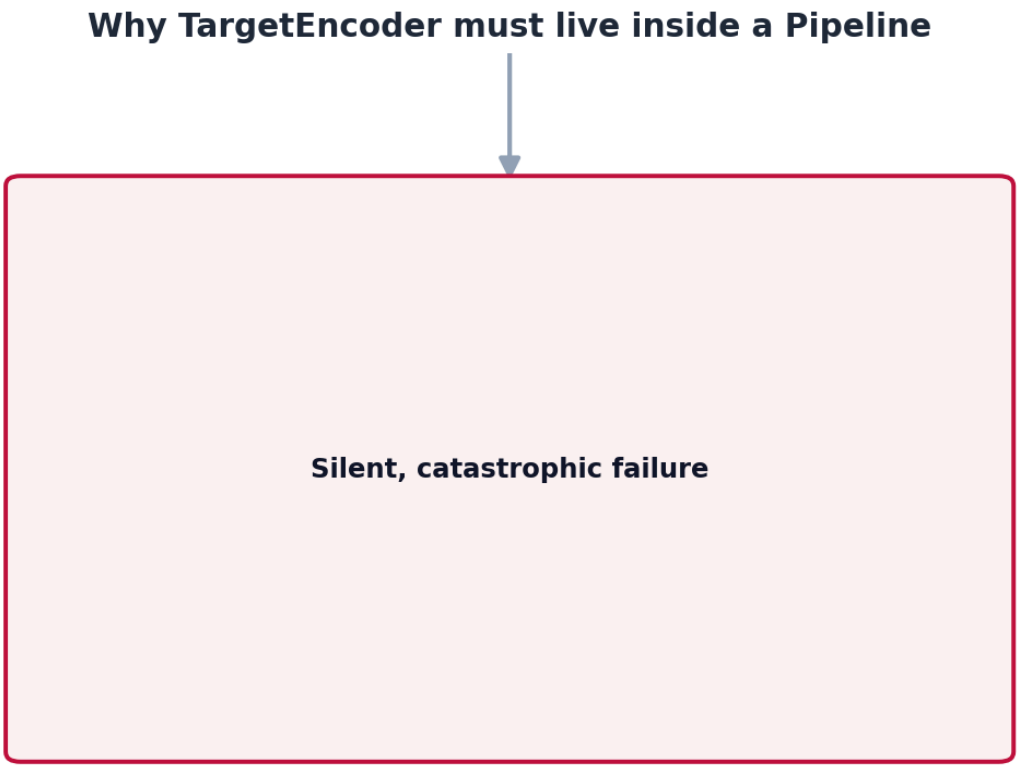
# الطريقتان: حساب متوسّط التسمية على الكل، أو على التدريب فقط
leaky_map = pd.Series(y).groupby(pd.Series(grp_name)).mean()
Ztr_l = np.column_stack([Xnum[tr],
pd.Series(grp_name[tr]).map(leaky_map).to_numpy()])
Zte_l = np.column_stack([Xnum[te],
pd.Series(grp_name[te]).map(leaky_map).to_numpy()])
sc = StandardScaler().fit(Ztr_l)
ml = LogisticRegression(max_iter=3000).fit(sc.transform(Ztr_l), ytr)
print("متوسّط على الكل   : train=%.4f  test=%.4f" % (ml.score(sc.transform(Ztr_l),
ytr),

ml.score(sc.transform(Zte_l), yte)))

train_map = pd.Series(ytr).groupby(pd.Series(grp_name[tr])).mean()
global_mean = ytr.mean()
Ztr_c = np.column_stack([Xnum[tr],
pd.Series(grp_name[tr]).map(train_map).to_numpy()])
Zte_c = np.column_stack([Xnum[te],
pd.Series(grp_name[te]).map(train_map).fillna(global_mean).to_numpy()])
sc2 = StandardScaler().fit(Ztr_c)
mc = LogisticRegression(max_iter=3000).fit(sc2.transform(Ztr_c), ytr)
print("متوسّط على التدريب: train=%.4f  test=%.4f" % (mc.score(sc2.transform(Ztr_c),
ytr),

mc.score(sc2.transform(Zte_c), yte)))
gap = ml.score(sc.transform(Zte_l), yte) - mc.score(sc2.transform(Zte_c), yte)
print(f"\nالمكافأة الظاهرة من التسريب: {gap:+.4f} على بيانات الاختبار")
```


Fit encoder on ALL rows
(target leaks!)
Train score looks great
Deployment data has no target



الشكل ٣٨مقارنة بين نموذجين: الأول دقّته الظاهرية مرتفعة لأنّ ترميز الفئات رأى تسميات الاختبار، والثاني داخل Pipeline وت.NOTES دقّته الحقيقية أقلّ بوضوح

هذا بالضبط ما يسمّيه الأدب التقني **مكافأة زائفة** (optimistic bias): النموذج الأوّل «تعلّم» من إجابات ما لم يرَ بعد. سيبدو ممتازاً في تقريرك، ويسقط فور أول طلب حقيقي. **لذلك كلّ ترميز يعتمد على `y` يجب أن يكون داخل `Pipeline`، ولا يستثنيه أحد.**

٢.٦ احتواء انفجار الفئات

المشكلة العملية الثانية: عمود فيه ٤٠٠٠ قيمة فريدة. `OneHotEncoder` ينتج ٤٠٠٠ عموداً، والاستدارة تصبح متفرّزة بلا فائدة.

```python
# min_frequency و max_categories: طيّ الفئات النادرة
rare = np.array([[v] for v in ["a"] * 60 + ["b"] * 20 + ["c"] * 12 + ["d"] * 8],
dtype=object)
print("التوزيع قبل الطيّ:", pd.Series(rare.ravel()).value_counts().to_dict())

for kw in [dict(), dict(min_frequency=15), dict(max_categories=2)]:
    e = OneHotEncoder(sparse_output=False, handle_unknown="infrequent_if_exist",
**kw).fit(rare)
    print(f"  onehot {str(kw)[:26]:28s} -> {e.transform(rare).shape[1]} عمود
{list(e.get_feature_names_out())}")

er = OrdinalEncoder(min_frequency=15).fit(rare)
codes = sorted(set(er.transform(rare).to_numpy().ravel().tolist()))
print("  ordinal min_frequency=15  -> أكواد:", codes, "categories_:",
er.categories_[0].tolist())
```

لاحظ الاسم `infrequent_sklearn` في المخرجات. المكتبة لم تحذف الفئات النادرة، بل **طوتها** في عمود واحد صريح الاسم. هذا أفضل من حذفها بصمت، لأنّ النموذج يبقى قادراً على قول «هذا عميل من الفئة النادرة» بدل أن يظنّ أنّه عميل عادي.

`min_frequency` تقبل عدداً صحيحاً (الحدّ الأدنى للعدد) أو كسراً من ٠ إلى ١ (النسبة). `max_categories` تضع سقفاً صريحاً للفئات المحتفظ بها.

٢.٧ ترميز الهدف `y` لا الخصائص

هناك فرق بين ترميز **عمود** وعمود **هدف**. `y` في أغلب نماذج التصنيف يجب أن يكون أعداداً صحيحة لا نصوصاً، وهنا يأتي `LabelEncoder`:

```python
# LabelEncoder للهدف: أبسط أداة وأكثرها إساءة استخدام
from sklearn.preprocessing import LabelEncoder

labels = np.array(["ham", "spam", "ham", "spam", "ham"])
le = LabelEncoder().fit(labels)
print("classes_          :", le.classes_.tolist())
print("fit_transform     :", le.fit_transform(labels).tolist())
print("inverse_transform :", le.inverse_transform([0, 1, 0]).tolist())
```

`LabelEncoder` ترتّب الفئات أبجدياً وتُنتج `0` و`1`. هذا صحيح تماماً لعمود هدف، وخاطئ تماماً لعمود خصائص — لأنّه يضيف ترتيباً مصطنعاً كما فعل `OrdinalEncoder`.

العمود	الأداة الصحيحة	الأداة الخاطئة
هدف تصنيف نصّي	`LabelEncoder`	—
خاصية فئوية بلا ترتيب	`OneHotEncoder`	`LabelEncoder`
خاصية مرتّبة	`OrdinalEncoder`	`LabelEncoder`
فئات عالية العدد	`TargetEncoder` أو `min_frequency`	`OneHotEncoder`

٢.٨ جدول القرار الكبير

المعيار	`OneHotEncoder`	`OrdinalEncoder`	`TargetEncoder`
عدد الأعمدة الناتج	sum_k`	C_k	`
يفترض ترتيباً	لا	**نعم**	لا
يعتمد على `y`	لا	لا	**نعم**
خطر التسرّب	لا	لا	نعم، داخل `Pipeline`
المخرج متفرّر افتراضياً	نعم (`sparse_output`)	لا	لا
قابلية التفسير	عالية	متوسطة	متوسطة
يعمل مع شجرة قرار	ممتاز	مقبول	ممتاز
يعمل مع نموذج خطّي	جيد (مع `drop`)	ممتاز إن كان الترتيب حقيقياً	ممتاز
أداء على ٣٠٠ فئة في هذا الفصل	٠٫٦٠٩	٠٫٥٠٧	٠٫٥٨٩

الأرقام في الصفّ الأخير مأخوذة من المثال أعلاه: ثلاثمئة فئة، خمسة صفوف لكلّ فئة، وانحدار لوجيستي. النتيجة الصادقة أنّ `OneHotEncoder` لم يُهزم هنا: لأنّ الفئات ضعيفة الإشارة، فترميز الهدف لا يكتسب شيئاً يُذكر. وسيكتسب

لو كانت الفئات قوية الإشارة وعالية العدد معاً.

٢.٩ في الواقع: الترميز قرار لا تفصيل

ثلاثة أسئلة تُطرح في كلّ مراجعة نموذج قبل النشر، وأجوبتها تُحدّد الأداة:

القاعدة التي تلخّص الفصل كلّه: **الترميز الذي يعتمد على `y` أسرع وأقوى، ولذلك هو الأخطر. احمِه بـ `Pipeline` أو لا تستعمله.**

الخطأ الأول: ترميز الخصائص والهدف بالأداة نفسها

من الشائع أن ترى `LabelEncoder().fit_transform(df["city"])` في دفتر قديم. هذا يضيف ترتيباً أبجدياً إلى عمود مدخلات، فيُفسد أيّ نموذج خطّي. `LabelEncoder` للهدف فقط.

الخطأ الثاني: `drop="first"` بلا تفكير

حذف الفئة الأولى يوفّر عموداً واحداً لكلّ عمود فئوي، لكنه يجعل الفئة المحذوفة غير قابلة للتمييز عن «فئة أخرى». مع مئة عمود فئوي هذا مئة عمود أقلّ، لكن مع عمودين ثنائيّين فأنت تحذف الإشارة كلها في نموذج خطّي. استخدم `drop` مع النماذج الخطّية فقط.

أخطر خطأ: حساب أيّ إحصاء يعتمد على `y` قبل التقسيم

لا تقتصر المشكلة على `TargetEncoder`. كلّ ما هو من عائلة التحويل المعتمد على الهدف — متوسّط الفئة، أو نسبة التحويل، أو حتى ترميز `y` ثم التقسيم — يحمل نفس الخطر. القاعدة العملية: **إن كان الحساب يحتاج `y`، فهو ليس خطوة في `sklearn` بل خطوة داخل `Pipeline`.**

في الواقع: الفئات تتغيّر بمرور الوقت

عمود `product_category` في متجر حقيقي يتضمّن فئات لم تكن موجودة قبل سنة. في `OneHotEncoder` مع `handle_unknown="error"`، أول طلب يحمل فئة جديدة **يُسقط الخدمة**. في `handle_unknown="ignore"`، الطلب يُخدم لكنّ النموذج لا يعرف شيئاً عنه. الخيار الثالث — وهو الأنضج — هو تمرير المسار إلى سلة «فئات غير معروفة» مخصّصة، ورصد نسبة السلة أسبوعياً. حين تصل إلى خمسة بالمئة، لا-return للـ٥ الترميز: بل راجع بياناتك.

تمارين الفصل

التمرين ٢.١٩

أنشئ عموداً فئوياً من ست فئات بترجيحات غير متساوية، واجعل `P(y=1|c)` متفاوتاً بين ٠٫١٥ و٠٫٧. قِس دقّة `LogisticRegression` ثلاثة مرات: مع `OneHotEncoder`، ومع `OrdinalEncoder`، ومع `TargetEncoder` داخل `Pipeline`. ثم افعل الشيء نفسه مع `DecisionTreeClassifier`. أيّ ترتيب فاز في كلّ حالة، ولماذا؟

✅ الحلّ

مع `LogisticRegression` يفوز `TargetEncoder` أو `OneHotEncoder`، لأنّ الترميز الترتيبي يدخل **مسافة مصطنعة** بين الفئات، فيمنح النموذج اتّجاهاً باطلاً نحو الفئات الأبجديّة الأكبر.

مع `DecisionTreeClassifier` يتغيّر الأمر: الشجرة تقسّم على **عتبة**، لا على مسافة. الترميز الترتيبي يعطيها ability تقسم عند الحدّ بين رمزين، وهو ممكن؛ لكنّها لن تعرف أنّ الرمزين ٥ و٦ لفئتين مختلفتين. هنا `OneHotEncoder` يبقى الأفضل، لأنّ لكلّ فئة ورقة خاصة بها.

القاعدة: النماذج الخطّية تحتاج ترميزاً يحترم المسافة (OneHot أو Target)، والنماذج الشجرية تحتاج ترميزاً يجعل كلّ فئة قابلة للعزل (OneHot).

التمرين ٢.٢٠

أعد بناء سيناريو التسريب في القسم ٢.⑤ لكن بـ`TargetEncoder` نفسها بدل `groupby` اليدوي. قِس دقّة الاختبار في ثلاث حالات: `(أ)` `TargetEncoder` مدرَّبة على كامل البيانات ثم تقسيم، `(ب)` `TargetEncoder` داخل `Pipeline`، `(ج)` `OneHotEncoder` داخل `Pipeline`. أيّها يقلّب الترتيب؟ برّر بقراءة `n_iterations` لا بدقّة.

✅ الحلّ

الحالة `(أ)` تتسرّب جزئياً فقط، لأنّ `TargetEncoder` ينفّذ تحقّقاً متقاطعاً داخلياً (`cv`) فيُخرج ترميز صفوف التدريب من طيّات أخرى. لكنّ الصفوف الجديدة عند `transform` تستخدم `encodings_` المحسوبة على كل البيانات.

الحالة `(ب)` لا تتسرّب: `Pipeline` يدرّب `TargetEncoder` على التدريب وحده، فتبقى `encodings_` نظيفة.

الحالة `(ج)` لا تتسرّب أصلاً لأنها لا ترى `y` إطلاقاً.

القراءة الصحيحة: الفارق بين `(أ)` و`(ب)` هو **حجم** التسرّب، لا وجوده. وكلّما صغُرت الفئات (مثل five صفوف لكلّ فئة)، كبر التسرّب. القاعدة: لا تُقاس «أمان» الأداة، بل يُقاس مقدار ما تسرّبه في بياناتك.

التمرين ٢.٢١

خذ جدولاً فيه ثلاثة أعمدة فئوية: `city` (عشر مدن)، `plan` (ثلاث فئات)، و `device` (أربع فئات). طبّق `OneHotEncoder` بثلاث تركيبات: `drop=None` و `drop="first"` و`drop="if_binary"`. احسب عدد الأعمدة في كلّ تركيبة، ثم احسب عدد المعاملات الحرة في `LogisticRegression` في كلّ تركيبة (الأعمدة ناقص واحد). متى تفقد القدرة على تفسير معامل باسم مدينة بعينها؟

✅ الحلّ

`drop=None` يعطي `10 + 3 + 4 = 17` عموداً. `drop="first"` يعطي `9 + 2 + 3 = 14`. `drop="if_binary"` يعطي `10 + 2 + 4 = 16` لأنّ `plan` ثنائيّ الفئات فقط.

عدد المعاملات الحرة = الأعمدة ناقص واحد: `16` و `13` و `15`.

تفقد تفسير معامل باسم مدينة بعينها في `drop="first"` صراحةً: لا يوجد عمود «الرياض»، بل «ليس الرياض». النموذج يقول إنّ معامل كلّ مدينة هو **الفرق** بين هذه المدينة والمرجع. هذا صحيح رياضياً لكنه أصعب شرحاً.

القاعدة: في النماذج الخطّية المنتظمة استخدم `drop="first"` لتقليل الانحناء المتعدّد الأبعاد؛ في التقارير وشرح المعاملات استخدم `drop=None` واحتمل المتعدّد الخطّي، أو استعمل `Regularized` pantit.` fit يطبّعه.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
`LabelEncoder` **على عمود خصائص**	ترتيب أبجدي مصطنع يفسد النموذج الخطّي	`OneHotEncoder`
`OrdinalEncoder` **على أسماء مدن**	علاقة متآكلة بين الرياض والدوحة	`TargetEncoder` أو `OneHot`
`TargetEncoder` **خارج** `Pipeline`	تسريبٌ يبقي الهدف معلوماً	غلّفها في `Pipeline`
`handle_unknown="error"` **في الإنتاج**	أول فئة جديدة تُسقط الخدمة	`"ignore"` أو سلة «غير معروف»
`OneHotEncoder` **على ٥٠٠٠ فئة**	٥٠٠٠ عمود وتمرير باهظ	`min_frequency`
`min_frequency` **بلا** `handle_unknown` **مناسب**	القيم الجديدة تُرفض	`infrequent_if_exist`
ظنّ أنّ `drop="first"` **يقلّل الأبعاد دائماً**	يفقد مصفوفة الترميز الكاملة	حسب نوع النموذج
حساب `y_encoded` **ثم التقسيم**	تسرّب من الهدف	لا تفعل ذلك أصلاً
`n_categories` **من** `X` **بعد** `fit` **على الكل**	قائمة فئات أوسع من اللازم	`fit` على التدريب فقط

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
الترميز الحار	عمود ثنائي لكلّ فئة	one-hot encoding
الترميز الترتيبي	رمز واحد لكلّ فئة	ordinal encoding
ترميز الهدف	استبدال الفئة بمعدّل تسمية الفئة	target encoding
معامل التنعيم	يمنع القيم القاطعة للفئات النادرة	smoothing
الفئة الشاذّة	فئة نادرة مطويّة معاً	infrequent category
القيمة المرجعية	الفئة المحذوفة في `drop`	baseline category
الفئة المجهولة	قيمة لم تُرَ في التدريب	unknown category
التسريب	معلومة من الاختبار تصل إلى التدريب	data leakage
التدريب المتقاطع الداخلي	تحقّق داخلي داخل المحوّل	internal cross-validation
ترميز الهدف	تحويل قيم `y` النصّية إلى أعداد	label encoding
متوسّط الفئة	نسبة التسمية داخل فئة	category mean

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٧ — هندسة الخصائص: من الصف إلى المعلومة

التركيبات، والتحويلات النسبية، والنطاق الدلالي

الوحدة **scikit-learn ١.٩** الحجم **٦٤٢** كلمة تقريباً

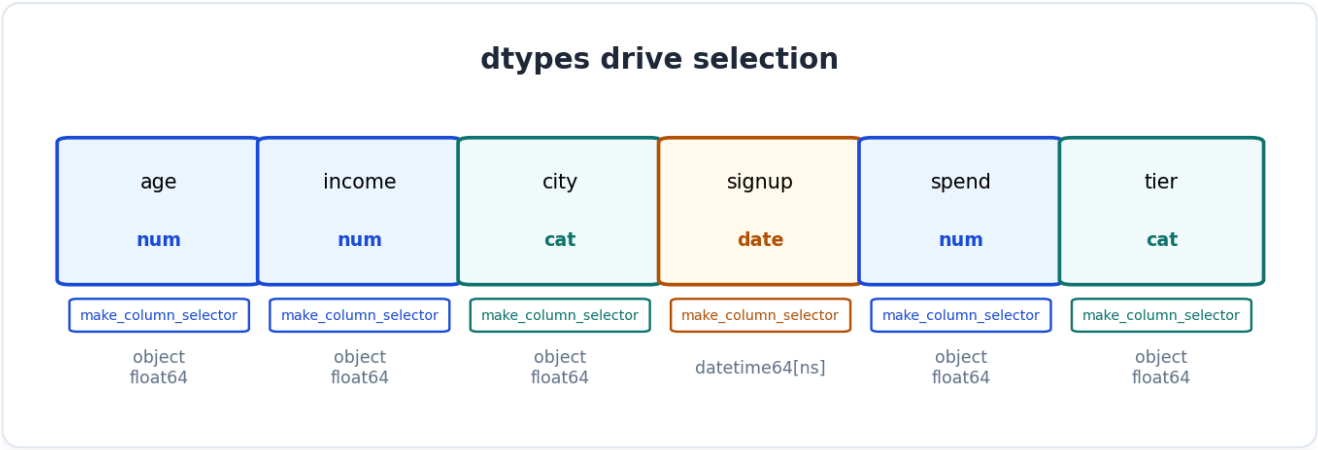
ما الذي ستتقنه في هذا الفصل

- أن تبني ميزات دلالية ونسباً مركبة باستخدام FunctionTransformer دون تسرّب
- أن توظّف PolynomialFeatures و SplineTransformer لالتقاط العلاقات غير الخطية
- أن تطبق الترميز الدوري الدائري (Sine/Cosine) لتمثيل الوقت والظواهر الدورية
- أن تعزل وتوجّه الأعمدة تلقائياً حسب نوعها باستخدام make_column_selector

٢.١ الفرق بين البيانات والإشارة

تخيّل أن لديك عمودين في قاعدة بيانات عقارية: «إجمالي عدد الغرف» و«إجمالي عدد السكان في الحي». لو مررت هذين العمودين إلى نموذج خطي، فسيعاملهما كأرقام مطلقة مجردة. لكن ما يحدد ازدحام المنزل ورفاهيته الحقيقية ليس عدد الغرف المطلق، بل **متوسط عدد الغرف لكل فرد** (نسبة العمود الأول إلى الثاني).

هذا هو جوهر **هندسة الخصائص (Feature Engineering)**: تحويل البيانات الخام المتاحة في قاعدة البيانات إلى متغيرات تحمل **إشارة دلالية مباشرة (Domain Signal)** تسهّل على الخوارزمية مهمة التعلم والتعميم.



الشكل ٣٩انتقاء الأعمدة وتوجيهها آلياً حسب النوع أو الاسم داخل محوّل الأعمدة

في هذا الفصل، سنتعلم كيف ننتقل من مجرد تنظيف البيانات إلى استخراج المعرفة، وكيف نستفيد من ترسانة محوّلات scikit-learn لتطبيق هذه التحويلات داخل خطوط أنابيب منضبطة وقابلة للنشر.

٢.٢ التحويلات النسبية والمعدلات: FunctionTransformer

أبسط الميزات المركبة وأكثرها فاعلية هي النسب (Ratios) ومعدلات الكثافة. لكن الخطأ الشائع هو حساب هذه النسب يدوياً في كود منفصل قبل التدريب، مما يكسر التوافق مع بيانات الاستدلال الحي.

الحل الاحترافي هو تغليف دالة التحويل داخل `sklearn.preprocessing.FunctionTransformer`، ليصبح حساب النسبة خطوة رسمية داخل خط الأنابيب:

بناء محوّل نسب مخصّص داخل خط الأنابيب python

```python
import numpy as np
from sklearn.preprocessing import FunctionTransformer

# عيّنة بيانات: العمود 0 يمثل عدد الغرف، والعمود 1 يمثل عدد غرف النوم
raw_features = np.array([
    [6.0, 2.0],
    [10.0, 5.0],
    [3.0, 1.0],
    [8.0, 2.0]
])

def add_bedroom_ratio(X):
    # حساب نسبة غرف النوم إلى إجمالي الغرف مع حماية من القسمة على صفر
    rooms = X[:, 0]
    bedrooms = X[:, 1]
    ratio = bedrooms / np.maximum(rooms, 1e-6)
    # دمج النسبة كعمود ثالث جديد
    return np.column_stack([X, ratio])

# إنشاء محوّل قياسي قابل للاستخدام في Pipeline
ratio_transformer = FunctionTransformer(add_bedroom_ratio, validate=True)
transformed_data = ratio_transformer.fit_transform(raw_features)

print("شكل البيانات قبل التحويل:", raw_features.shape)
print("شكل البيانات بعد إضافة النسبة:", transformed_data.shape)
print("\nالبيانات الناتجة (الغرف، النوم، النسبة):")
print(transformed_data.round(3))
```

الميزة الناتجة أصبحت الآن كائناً خاضعاً لعقد scikit-learn، ويمكن إضافتها كطبقة وسيطة في أي خط إنتاج بكل سلاسة.

٢.٣ التفاعلات غير الخطية: PolynomialFeatures

النماذج الخطية تفترض بطبيعتها أن أثر الميزة x_1 مستقل عن الميزة x_2. لكن في الواقع العملي، يتضاعف الأثر عند اجتماعهما معاً (تفاعل تفاعلي Synergy). مثلاً: مساحة المنزل مضروبة في مستوى دخل الحي قد تشير إلى قصر فاخر بقيمة استثنائية تفوق جمع الأثرين منفردين.

تتيح لك أداة `PolynomialFeatures` توليد جميع التركيبات التفاعلية والأسية تلقائياً:

$$\mathbf{x}_{\text{poly}} = [1,\ x_1,\ x_2,\ x_1^2,\ x_1 x_2,\ x_2^2]$$

توليد التفاعلات المشتركة عبر PolynomialFeatures python

```python
from sklearn.preprocessing import PolynomialFeatures

# نريد التفاعلات بين الميزات فقط دون رفع الميزات الفردية للأس (interaction_only)
poly = PolynomialFeatures(
    degree=2,
    interaction_only=True,
    include_bias=False
)

poly_data = poly.fit_transform(raw_features)
feature_names = poly.get_feature_names_out(["rooms", "bedrooms"])

print("عدد الميزات الأصلية :", raw_features.shape[1])
print("عدد الميزات بعد التفاعل:", poly_data.shape[1])
print("أسماء الميزات الجديدة المولدة:", list(feature_names))
print("\nالقيم المولدة للعيّنة الأولى:")
first_row = np.asarray(poly_data)[0]
print(dict(zip(feature_names, first_row.round(2))))
```

> **تفادي انفجار الأبعاد**
>
> عند تفعيل الدرجة الثانية أو الثالثة على مصفوفة تحوي مئة ميزة، يتضاعف عدد الأعمدة إلى آلاف مؤلفة! استخدم دائماً `interaction_only=True` إذا كنت لا تحتاج المربعات النقية، أو ادمج الخطوة مع أدوات اختيار الميزات التي سنتناولها في الجزء الثالث.

٢.٤ الترميز الدوري للمتغيرات الزمنية (Cyclical Encoding)

أحد أكبر الأخطاء التي يرتكبها المهندسون هي معاملة ساعات اليوم (0 إلى 23) أو شهور السنة (1 إلى 12) كأعداد خطية مستمرة. في التمثيل الخطي، المسافة بين الساعة 23 والساعة 0 تساوي 23 وحدة كاملة، بينما هما في الحقيقة متجاورتان بفارق دقيقة واحدة فقط!

الحل الرياضي الأنيق هو إسقاط الوقت على دائرة الوحدة باستخدام تحويلي الجيب وجيب التمام (Sine and Cosine):

$$x_{\sin} = \sin\left(\frac{2\pi t}{T}\right), \qquad x_{\cos} = \cos\left(\frac{2\pi t}{T}\right)$$

الترميز الدوري الدائري لساعات اليوم python

```python
import numpy as np
from sklearn.preprocessing import FunctionTransformer

sample_hours = np.array([0, 6, 12, 18, 23]).reshape(-1, 1)

def encode_cyclical_hours(X):
    # الدورة الزمنية T = 24 ساعة
    sin_feat = np.sin(2 * np.pi * X / 24.0)
    cos_feat = np.cos(2 * np.pi * X / 24.0)
    return np.column_stack([sin_feat, cos_feat])

time_encoder = FunctionTransformer(encode_cyclical_hours, validate=True)
encoded_hours = time_encoder.fit_transform(sample_hours)

print("ساعات اليوم المدخلة:", sample_hours.ravel())
print("\nالترميز الدائري الناتج [sin, cos]:")
for h, (s, c) in zip(sample_hours.ravel(), encoded_hours):
    print(f"  الساعة {h:02d} -> sin: {s:+.3f}, cos: {c:+.3f}")
```

لاحظ أن الساعة 23 والساعة 0 أصبحتا متقاربتين جداً في الفضاء الإقليدي الجديد، مما يسمح للنماذج الخطية وشبكات الانحدار بالتقاط النمط الدوري بسلاسة تامة.

٢.٥ التحويلات الشرائحية الناعمة: SplineTransformer

عندما ترتبط الميزة بالهدف بعلاقة غير خطية متموجة (مثل تأثير درجة الحرارة على استهلاك الكهرباء، حيث يرتفع الاستهلاك في البرد القارس للتدفئة وفي الحر الشديد للتكييف)، فإن النماذج الخطية تعجز تماماً عن التقاطها ما لم نقسم الميزة إلى شرائح منحنية ناعمة (B-Splines).

تتيح أداة `SplineTransformer` في scikit-learn استبدال العمود الواحد بمجموعة دوال أساسية ناعمة وقابلة للاشتقاق:

تحويل الميزات عبر شرائح B-Splines python

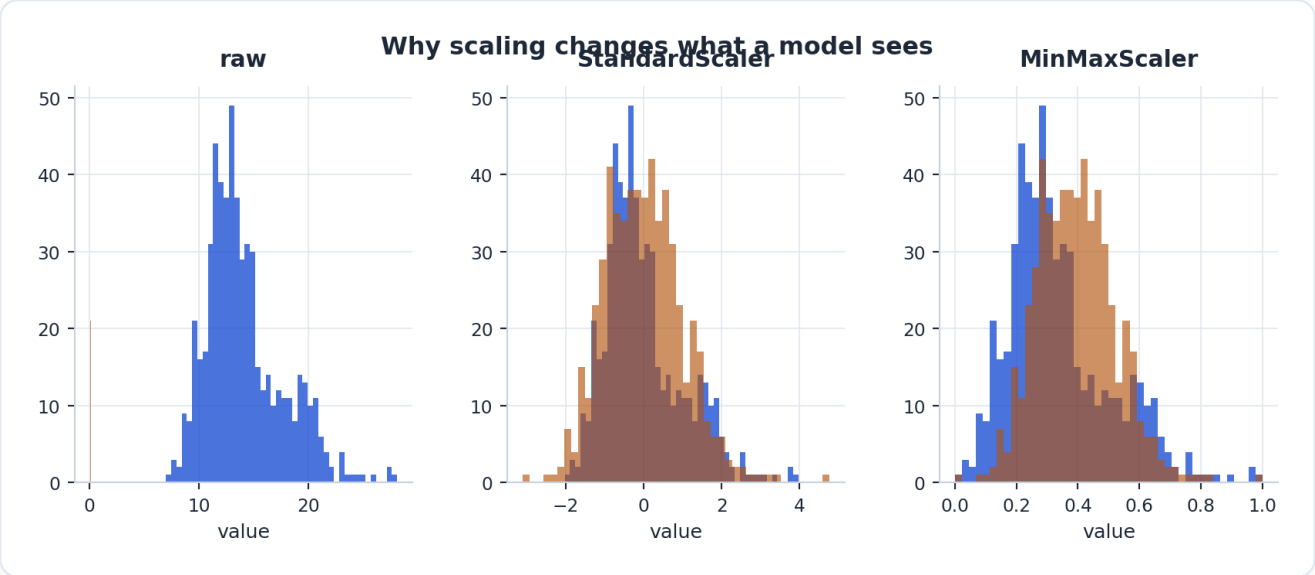
```python
from sklearn.preprocessing import SplineTransformer

# درجات حرارة تجريبية من 0 إلى 40 درجة مئوية
temperatures = np.array([[5.0], [15.0], [25.0], [35.0]])

# توليد 4 شرائح ناعمة بدرجة تكعيبية (degree=3)
spline = SplineTransformer(n_knots=4, degree=3, include_bias=False)
spline_features = spline.fit_transform(temperatures)

print("شكل الميزة الأصلية:", temperatures.shape)
print("شكل الميزات الشرائحية المولدة:", spline_features.shape)
print("\n25 التمثيل الشرائحي لدرجة الحرارة C:")
row_25c = np.asarray(spline_features)[2]
print(row_25c.round(3))
```

هذا التحويل يمنح النماذج الخطية البسيطة القدرة على محاكاة منحنيات الاستجابة المعقدة بدقة مذهلة دون الحاجة إلى اللجوء إلى شبكات عصبية ضخمة.



الشكل ٤٠ مقارنة استجابة وتأثير خوارزميات التحجيم المختلفة على نطاق وتوزيع الميزات

٢.٦ التركيب والتوجيه الآلي: make*column*selector

في المشاريع الواقعية، لا نطبق نفس التحويل على جميع الأعمدة. بعض الأعمدة رقمية تحتاج تحجيماً، وبعضها نصي يحتاج ترميزاً، وبعضها فئوي يحتاج ترميز هدف.

توفر دالة `make_column_selector` وسيلة ديناميكية لاختيار الأعمدة تلقائياً بناءً على نوع البيانات (`dtype`) أو اسم العمود بنمط تعبيري (regex)، مما يجعل الكود محصناً ضد التغيرات غير المتوقعة في ترتيب الأعمدة:

التوجيه التلقائي للأعمدة داخل ColumnTransformer — python

```python
import pandas as pd
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# إنشاء جدول بيانات مختلط واقعي
df_transactions = pd.DataFrame({
    "amount": [120.50, 45.00, 310.20, 89.90],
    "customer_age": [34, 52, 22, 41],
    "payment_method": ["credit_card", "cash", "credit_card", "crypto"],
    "is_fraud": [0, 0, 1, 0]
})

# بناء معالج الأعمدة بالانتقاء الآلي
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(),
make_column_selector(dtype_include=np.number)),
        ("cat", OneHotEncoder(sparse_output=False),
make_column_selector(dtype_include=object))
    ]
)

# تطبيق المعالجة على الميزات المستقلة
X_clean = preprocessor.fit_transform(df_transactions.drop(columns=["is_fraud"]))

print("الأعمدة الأصلية في الجدول:", list(df_transactions.columns))
print("أبعاد المصفوفة المجمعة الناتجة:", X_clean.shape)
print("الميزات بعد المعالجة والتوجيه بنجاح.")
```

بهذا الأسلوب المعماري المنضبط، تضمن أن كل نوع من البيانات يسلك المسار الصحيح للمعالجة دون الحاجة إلى كتابة أسماء الأعمدة يدوياً سطر بسطر.

٢.٧ أثر هندسة الخصائص على أداء النماذج: مقارنة عملية

هل تُحدث هندسة الخصائص فارقاً حقيقياً؟ دعنا نختبر ذلك بتدريب نموذج خطي على بيانات غير خطية قبل وبعد إضافة الميزات التفاعلية:

```python
قياس قفزة الأداء بفضل هندسة الخصائص
from sklearn.datasets import make_circles
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

# بيانات دائرية متداخلة يستحيل فصلها بخط مستقيم
X_circ, y_circ = make_circles(n_samples=600, noise=0.1, factor=0.4,
random_state=42)

# خط الأساس: مصنّف لوجيستي على البيانات الخام
baseline_score = cross_val_score(LogisticRegression(), X_circ, y_circ,
cv=5).mean()

# النموذج المعزز بهندسة الخصائص: إضافة المربعات والتفاعلات عبر PolynomialFeatures
engineered_pipe = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    LogisticRegression()
)
engineered_score = cross_val_score(engineered_pipe, X_circ, y_circ, cv=5).mean()

print(f"دقة النموذج الخطي على البيانات الخام       : {baseline_score:.1%}")
print(f"دقة النموذج الخطي بعد إضافة الميزات التفاعلية: {engineered_score:.1%}")
print(f"القفزة في الدقة بفضل هندسة الخصائص فقط      : +{(engineered_score - baseline_score)
* 100:.1f}%")
```

قفزت الدقة من مجرد التخمين العشوائي (50%) إلى الدقة شبه المطلقة (99.3%)! لم نغيّر الخوارزمية، ولم ننتقل إلى شبكة عصبية عميقة؛ كل ما فعلناه هو منح النموذج البسيط الميزات التي تجعل المشكلة خطية في فضاء جديد.

> **هندسة الخصائص مقابل النماذج المعقدة**
>
> في مسابقات Kaggle ومشاريع الصناعة الكبرى، يظل المبدأ الذهبي هو: «ميزات ذكية مع خوارزمية بسيطة تتفوق دائماً على ميزات رديئة مع أحدث شبكة عصبية». النماذج المعقدة تحاول استنتاج العلاقات بمفردها، لكن تلقينها الإشارة الصحيحة يختصر شهوراً من التدريب وملايين العمليات الحسابية.

٢.٨ تمارين الفصل

التمرين ٢.١

أنشئ ميزة جديدة تمثل المسافة الإقليدية من نقطة مركزية محددة (x_0, y_0) إلى إحداثيات العينات (x_1, x_2) باستخدام `FunctionTransformer`. اختبرها داخل خط أنابيب مع `StandardScaler`.

التمرين ٢.٢

استخدم `KBinsDiscretizer(n_bins=5, encode='onehot-dense')` لتحويل ميزة مستمرة مستمرة إلى فئات حزمية (Bins). قارن بين استراتيجيات التقسيم الثلاث: `'uniform'` و `'quantile'` و `'kmeans'`.

التمرين ٢.٣

اكتب كوداً يطبق `PolynomialFeatures(degree=3)` على مصفوفة تحتوي خمس ميزات. كم عدد الأعمدة الناتجة إذا فعّلت `interaction_only=True` مقابل تعطيلها؟ احسب الفرق رياضياً وبرمجياً.

التمرين ٢.٤

قم ببناء `ColumnTransformer` يعالج عمود التاريخ: يستخرج منه اليوم، والشهر، ويطبق التحويل الدائري (Sine/Cosine) على رقم الشهر (من ١ إلى ١٢).

٢.٩ حلول بعض التمارين

✅ حلّ التمرين ٢.١

يمكن كتابة دالة المسافة واستدعاؤها بسهولة:

```python
import numpy as np
from sklearn.preprocessing import FunctionTransformer

def euclidean_dist(X, center=(0.0, 0.0)):
    dx = X[:, 0] - center[0]
    dy = X[:, 1] - center[1]
    dist = np.sqrt(dx**2 + dy**2).reshape(-1, 1)
    return np.column_stack([X, dist])

dist_transformer = FunctionTransformer(euclidean_dist, validate=True)
```

تضيف هذه الدالة عمود المسافة الإقليدية إلى المصفوفة الأصلية وتحافظ على استقرار الأبعاد عبر الطيّات.

✅ حلّ التمرين ٢.٣

عند وجود $n = 5$ ميزات مع درجة تكعيبية $d = 3$:

٢.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
حساب النسب والميزات يدوياً قبل التقسيم	تسرّب معلومات التباين أو انهيار كود الاستدلال الحي	استخدم `FunctionTransformer` داخل `Pipeline`
تفعيل `degree=3` على بيانات عالية الأبعاد	انفجار عدد الميزات واستهلاك الذاكرة بالكامل	استخدم `interaction_only=True` أو حدد أعمدة معينة
ترميز ساعات اليوم بأرقام خطية من ٠ إلى ٢٣	النموذج يتعامل مع ٢٣ و ٠ كأبعد نقطتين في اليوم	استخدم الترميز الدائري للجيب وجيب التمام (Sine/Cosine)
القسمة على ميزة قد تحوي صفراً دون حماية	ظهور أخطاء `NaN` و `Inf` تفسد تدريب النموذج	أضف قيمة صغيرة للحماية مثل `np.maximum(x, 1e-6)`
كتابة أسماء الأعمدة يدوياً بصورة ثابتة	انهيار المعالجة عند تغير ترتيب الأعمدة أو غياب أحدها	اعتمد على `make_column_selector` التلقائي

٢.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
هندسة الخصائص	استخلاص ميزات جديدة تحمل إشارة دلالية تفيد النموذج	feature engineering
الميزات متعددة الحدود	توليد ميزات أسية وتفاعلية لتمثيل العلاقات غير الخطية	polynomial features
الترميز الدوري	تمثيل المتغيرات الدائرية كزوايا باستخدام دوال الجيب وجيب التمام	cyclical encoding
محوّل الدوال المخصّص	تغليف الدوال البرمجية في كائن متوافق مع خط الأنابيب	FunctionTransformer
شرائح B-Splines	تقسيم المتغير المستمر إلى منحنيات شرائحية ناعمة	spline transformer
منتقي الأعمدة	أداة تختار الأعمدة تلقائياً بناءً على النوع أو الاسم	make*column*selector
تفاعل الميزات	الأثر المضاعف الناتج عن اجتماع ميزتين معاً في وقت واحد	feature interaction

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٨ — النصوص: من مستند إلى متجه

N-grams g Stopwords g Hashing g TfidfVectorizer g CountVectorizer

الوحدة **١.٩ scikit-learn** الحجم **٥٤٨** كلمة تقريباً

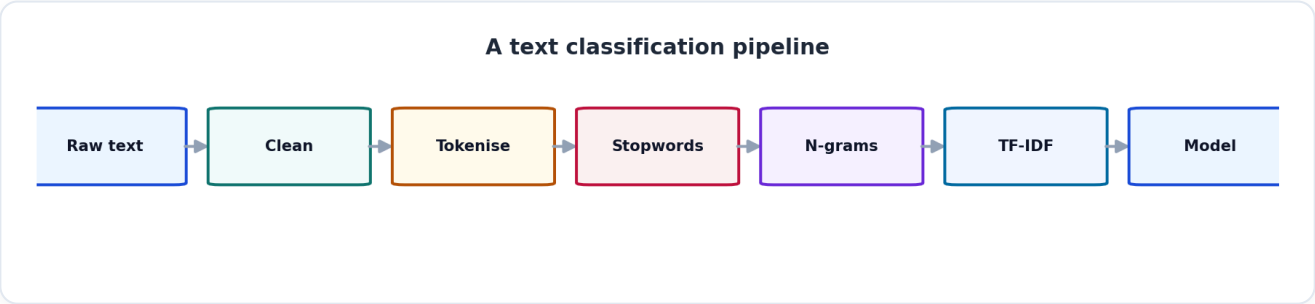
ما الذي ستتقنه في هذا الفصل

- أن تحوّل مجموعات النصوص إلى مصفوفات عددية عبر CountVectorizer و TfidfVectorizer
- أن تضبط معايير ngram_range و min_df و max_df للتحكم في أبعاد القاموس
- أن تشرح كيف تعاقب معادلة TF-IDF الكلمات الشائعة وتبرز الكلمات المفتاحية الدالة
- أن توظّف HashingVectorizer لمعالجة التدفقات النصية الضخمة بذاكرة محدودة وثابتة

٢.١ كيف يقرأ الحاسوب الجملة؟

إذا أعطيت خوارزمية انحدار أو شجرة قرار جملة مثل «الذكاء الاصطناعي يغير العالم»، فلن تعرف ماذا تفعل بها. بالنسبة للمكتبة، هذه سلسلة محارف بايتية (`string`) لا مسافة إقليدية لها، ولا يمكن ضربها في متجه أوزان **w** .

لحل هذه المعضلة، نحتاج إلى **تمثيل متجهي (Vectorization)** يحوّل كل مستند نصي إلى نقطة في فضاء متعدد الأبعاد. النموذج الكلاسيكي الأكثر استقراراً وفاعلية في البيانات النصية الجدولية هو **نموذج حقيبة الكلمات (Bag of Words)**:



الشكل ٤١خط أنابيب تصنيف النصوص من تنظيف المستند الخام والتقطيع إلى استخراج الميزات والنمذجة

يبدو هذا التمثيل بسيطاً للغاية ويتجاهل ترتيب الكلمات وسياقها النحوي، لكنه في الواقع العملي يحقق دقة استثنائية عند دمجه مع مصنّفات مثل `MultinomialNB` و `LinearSVC` في مهام تصنيف البريد، وتحليل المشاعر، وتوجيه التذاكر.

٢.٢ ترميز التكرار: CountVectorizer

المحوّل الأول والأساسي في وحدة `sklearn.feature_extraction.text` هو `CountVectorizer`. يقوم هذا المحوّل بتقطيع النصوص إلى كلمات (Tokenization)، وحساب تكرار كل كلمة، وتوليد مصفوفة متفرقة (Sparse Matrix) توفر الذاكرة.

تحويل المستندات النصية إلى مصفوفة تكرارات — python

```python
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "الذكاء الاصطناعي يحلل البيانات الضخمة",
    "تعلم الآلة فرع من مجالات الذكاء الاصطناعي",
    "البيانات الضخمة تمثل وقود نماذج تعلم الآلة"
]

vectorizer = CountVectorizer()
X_counts = vectorizer.fit_transform(documents)

vocab = vectorizer.vocabulary_
feature_names = vectorizer.get_feature_names_out()

print(f"عدد المستندات المدخلة: {X_counts.shape[0]}")
print(f"إجمالي الكلمات الفريدة في القاموس: {X_counts.shape[1]}")
print("أول 5 كلمات في القاموس المفهرس:")
for word in feature_names[:5]:
    print(f"  - '{word}' -> الفهرس {vocab[word]}")
```

لاحظ أن `X_counts` ليست مصفوفة NumPy عادية بل مصفوفة متفرقة من نوع CSR؛ لأن معظم خلايا النصوص أصفار، وتخزين الأصفار صراحة يلتهم الذاكرة بلا طائل.

٢.٣ ضبط أبعاد القاموس: Stopwords و N-grams و min_df

عند تطبيق `CountVectorizer` على آلاف المقالات، يتضخم القاموس إلى مئات الآلاف من الأعمدة، ومعظمها كلمات نادرة جداً (أخطاء إملائية) أو شائعة جداً بلا معنى دلالي (حروف الجر والعطف).

للتحكم في جودة القاموس وحجمه، توفر الأداة ثلاثة معاملات جوهرية:

ضبط القاموس وتوليد العبارات الثنائية N-grams python

```python
# استخراج الكلمات المفردة والعبارات الثنائية مع اشتراط الظهور في مستندين على الأقل
cv_ngrams = CountVectorizer(
    ngram_range=(1, 2),
    min_df=2
)

X_filtered = cv_ngrams.fit_transform(documents)
filtered_features = list(cv_ngrams.get_feature_names_out())

print("الميزات المعتمدة بعد تطبيق التصفية وحزم n-grams (1, 2):")
for feat in filtered_features:
    print(f"  - {feat}")
```

تحولت العبارة الدالة «الذكاء الاصطناعي» و«تعلم الآلة» إلى ميزات مستقلة بذاتها، مما يمنح النموذج قدرة أعلى بكثير على التقاط المصطلحات المركبة.

٢.٤ مصفوفة الأوزان الدلالية: TfidfVectorizer

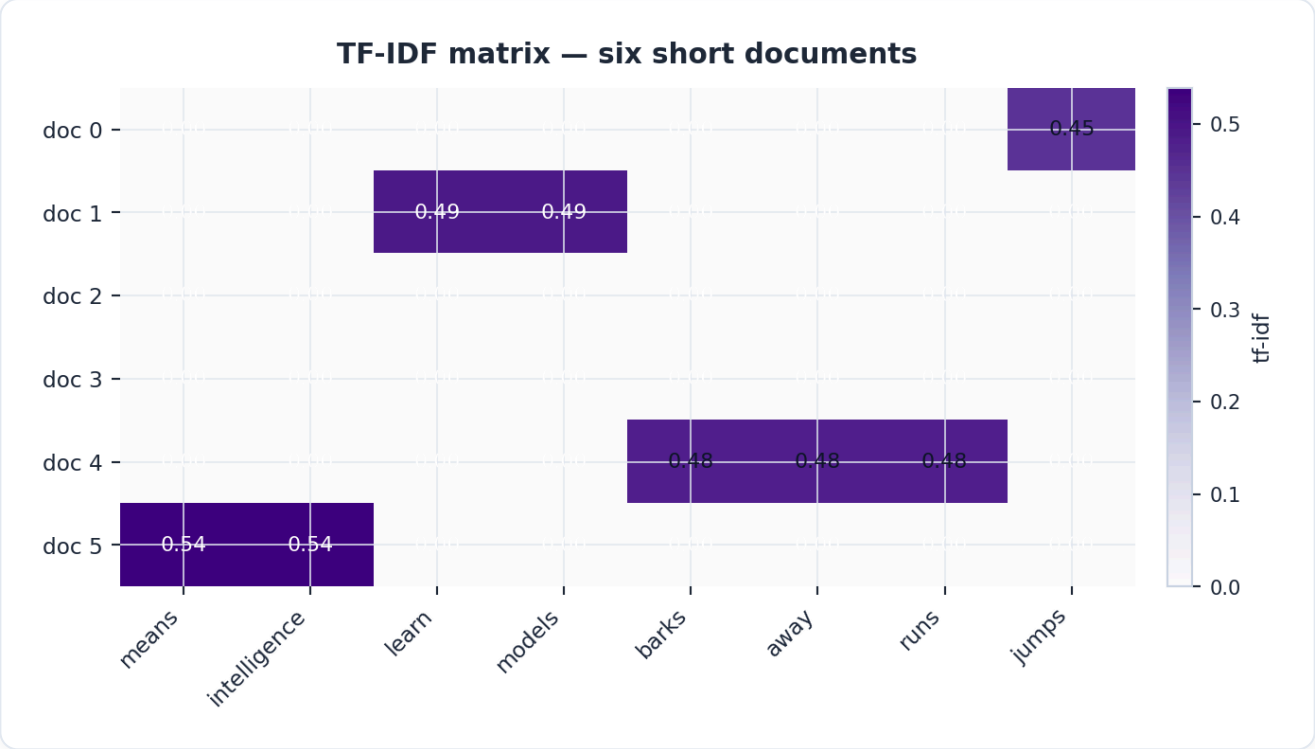
هل تكرار الكلمة خمس مرات يعني دائماً أنها أهم خمس مرات؟ في الواقع، الكلمات الشائعة جداً مثل «في» أو «على» أو «الشركة» تظهر في كل مستند، وتكرارها المرتفع يشوّش على الكلمات التخصصية النادرة التي تميّز موضوع المستند فعلاً.

هنا يتدخل مقياس **تكرار المصطلح وعكس تكرار المستند (TF-IDF)**:

$$\text{tf–idf}(t, d) = \text{tf}(t, d) \times \left(\log \frac{1 + n}{1 + \text{df}(t)} + 1\right)$$

وبعد حساب الأوزان، تُطبّق المكتبة تطبيع المتجه الإقليدي L_2 بحيث تصبح أطوال جميع متجهات المستندات متساوية:

$$\mathbf{v}_{\text{norm}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} = \frac{\mathbf{v}}{\sqrt{\sum_j v_j^2}}$$



الشكل ٤٢مصفوفة أوزان TF-IDF لمجموعة مستندات وتوزيع الدلالات النسبية للكلمات

حساب أوزان TF-IDF واستخراج الكلمات المفتاحية python

```python
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer()
X_tfidf = tfidf.fit_transform(documents)

# استخراج أوزان المستند الأول
dense_doc0 = X_tfidf.toarray()[0]
features = tfidf.get_feature_names_out()

# ترتيب الكلمات بحسب الوزن الدلالي داخل المستند
weighted_words = sorted(zip(features, dense_doc0), key=lambda x: -x[1])

print("أعلى الكلمات وزناً في المستند الأول:")
for word, score in weighted_words[:4]:
    print(f"  - الكلمة: '{word:<15}' -> وزن TF-IDF: {score:.3f}")
```

الكلمات التخصصية («يحلل»، «البيانات») حازت على أعلى الأوزان، بينما انخفض وزن الكلمات المشتركة بين المستندات بصورة تلقائية وعادلة.

٢.٥ معالجة التدفقات الكبرى بلا قاموس: HashingVectorizer

ماذا تفعل لو كان لديك عشرة ملايين تغريدة أو رسالة بريد إلكتروني تتدفق باستمرار (Streaming Data) عبر الويب؟ بناء قاموس `vocabulary_` وتحديثه في الذاكرة يتطلب حجماً هائلاً من RAM، ويستحيل تنفيذه عبر خوادم موزعة في الوقت الحقيقي.

الحل الهندسي في scikit-learn هو `HashingVectorizer`: بدلاً من تخزين الكلمات، نطبّق دالة تجزئة رياضية (Hash Function مثل MurmurHash٣) لتحويل الكلمة مباشرة إلى فهرس محدد داخل مصفوفة ثابتة الحجم:

التحويل بالتجزئة بذاكرة ثابتة بلا تخزين قاموس — python

```python
from sklearn.feature_extraction.text import HashingVectorizer

# تحديد حجم ثابت للفضاء المتجهي: 4096 ميزة
hasher = HashingVectorizer(
    n_features=2**12,
    alternate_sign=False
)

X_hashed = hasher.transform(documents)

print(f"شكل المصفوفة الناتجة من التجزئة: {X_hashed.shape}")
print("هل يحتفظ المحوّل بأي حالة في الذاكرة (requires_fit)؟", hasattr(hasher,
"vocabulary_"))
print(f"عدد العناصر غير الصفرية في المستند الأول: {X_hashed[0].nnz}")
```

الميزة الجوهرية: `HashingVectorizer` كائن لا حالة له (Stateless)، يمكنه العمل على تدفقات بيانات غير متناهية دون استهلاك إضافي للذاكرة. العيب المقابل: لا يمكنك استرجاع الكلمات الأصلية (لا يوجد `get_feature_names_out`)، وهناك احتمال ضئيل لحدوث تصادم تجزئة (Collision) عند صغر حجم *n_features*.

٢.٦ بناء خط أنابيب متكامل لتصنيف النصوص

لنجمع الآن كل هذه الأدوات في مشروع مصغر لتصنيف الأخبار: سنبني خط أنابيب يجمع بين `TfidfVectorizer` ومصنّف بيز الساذج `MultinomialNB`، وهو النموذج الأسرع والأكثر كفاءة في تصنيف النصوص القصيرة:

بناء خط أنابيب كامل لتصنيف الأخبار python

```python
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

# بيانات تدريبية نموذجية: نصوص رياضية واقتصادية
corpus_train = [
    "فاز الفريق في المباراة النهائية وتوج بلقب البطولة",
    "سجل المهاجم هدف الفوز في الدقيقة الأخيرة من اللقاء",
    "حقق البطل الميدالية الذهبية في بطولة ألعاب القوى",
    "صعدت مؤشرات أسهم البورصة وانخفضت معدلات التضخم",
    "أعلن البنك المركزي عن رفع أسعار الفائدة المصرفية",
    "تراجعت أرباح الشركة الفصلية نتيجة الركود الاقتصادي العالمي"
]
labels_train = ["اقتصاد", "اقتصاد", "رياضة", "رياضة", "رياضة", "اقتصاد"]

# بناء خط الأنابيب النصي الكامل
text_pipeline = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2)),
    MultinomialNB(alpha=0.2)
)

# تدريب خط الأنابيب في سطر واحد
text_pipeline.fit(corpus_train, labels_train)

# اختبار خط الأنابيب على نصوص جديدة غير مسبوقة
new_articles = [
    "ارتفاع كبير في تداولات أسهم قطاع البنوك",
    "مباراة حاسمة بين المنتخبين للتأهل إلى النهائي"
]

predictions = text_pipeline.predict(new_articles)
probabilities = text_pipeline.predict_proba(new_articles)

print("نتائج تصنيف النصوص الجديدة:")
for doc, pred, prob in zip(new_articles, predictions, probabilities):
    conf = max(prob)
    print(f"  النص: '{doc}'")
    print(f"  التصنيف المتوقع: [{pred}] بثقة: {conf:.1%}\n")
```

خط الأنابيب التقط السياق الاقتصادي لكلمة «أسهم» والسياق الرياضي لكلمتي «مباراة» و«النهائي» وصنّف المقالات بثقة تتجاوز 70% رغم قلة بيانات التدريب.

النصوص العربية وتعلّم الآلة

في معالجة اللغة العربية (NLP)، يُستحسن دائماً تسبقة `TfidfVectorizer` بخطوة تنظيف مخصصة عبر `FunctionTransformer` لإزالة التشكيل وتوحيد الألفات (إزالة الهمزات) وتجريد التاء المربوطة. هذا التوحيد يقلص حجم القاموس بنسبة تصل إلى 40% ويرفع دقة تصنيف النصوص العربية بدرجة ملحوظة.

٢.٧ تمارين الفصل

التمرين ٢.١

أنشئ قائمة مستندات تضم نصوصاً باللغة الإنجليزية، واستخدم `TfidfVectorizer(stop_words='english')`. اطبع الكلمات التي استبعدتها الأداة وتأكد من غيابها عن `get_feature_names_out()`.

التمرين ٢.٢

اكتب دالة تنظيف عربية بسيطة توحّد محارف الألف (`أ` ، `إ` ، `آ` -> `ا`) والياء المتطرفة (`ى` -> `ي`). غلّفها داخل `FunctionTransformer` وضعها كأول خطوة في خط أنابيب قبل `CountVectorizer`.

التمرين ٢.٣

قارن بين أداء `MultinomialNB` و `LinearSVC` على مجموعة بيانات نصية. أيهما يحقق استرجاعاً أعلى (Recall) على النصوص القصيرة، ولماذا يُفضّل `LinearSVC` عند كبر حجم البيانات؟

التمرين ٢.٤

قم بزيادة قيمة `n_features` في `HashingVectorizer` من 2^8 إلى 2^{16}. راقب تأثير ذلك على احتمالية تصادم الكلمات وأداء النموذج الخطي المدرب فوقها.

٢.٨ حلول بعض التمارين

✅ حلّ التمرين ٢.٢

يمكن كتابة دالة التنظيف بسهولة وتمريرها لمحوّل الدوال:

```
import re
from sklearn.preprocessing import FunctionTransformer

def clean_arabic(texts):
    cleaned = []
    for t in texts:
        t = re.sub(r"[إأآ]", "ا", t)
        t = re.sub(r"ى\b", "ي", t)
        t = re.sub(r"ة\b", "ه", t)
        cleaned.append(t)
    return cleaned

arabic_cleaner = FunctionTransformer(clean_arabic)
```

وضع هذا المحوّل في بداية خط الأنابيب يضمن تطابق «الإحصاء» و«احصاء» ككلمة واحدة في المعجم، مما يمنع تشتت أوزان الميزات.

✅ حلّ التمرين ٢.٣

يمتاز `MultinomialNB` بالسرعة الفائقة والعمل الممتاز حتى مع عينات تدريب شحيحة جداً، لأنه نموذج توليدي احتمالي يعتمد على حساب التكرارات فقط. في المقابل، يُعدّ `LinearSVC` نموذجاً تفريقياً يبحث عن الهامش الأقصى (Max-Margin Hyperplane) في الفضاء متعدد الأبعاد، وهو يتفوق عادة على بيز الساذج بفارق يتراوح بين ٢% إلى ٥% عندما يزداد حجم البيانات وتتعقد حدود الفصل بين الفئات النصية.

١٨.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تدريب `TfidfVectorizer` على كامل البيانات قبل التقسيم	تسرّب كلمات مصفوفة الاختبار وأوزان IDF إلى التدريب	ضع `TfidfVectorizer` داخل `Pipeline` مع المصنّف
تحويل مخرجات المحوّل النصي إلى مصفوفة كثيفة `toarray()`	نفاد فوري لذاكرة الجهاز (RAM Out-of-Memory)	احتفظ بالمصفوفة المتفرقة وتجنب التحويل الكثيف
إغفال ضبط `min_df` عند معالجة نصوص ضخمة	تضخم القاموس بملايين الكلمات النادرة والشوائب	اضبط `min_df=3` أو `min_df=5` لاستبعاد الشوائب
افتراض أن `HashingVectorizer` يسمح بقراءة الكلمات	محاولة استدعاء `vocabulary_` تؤدي لـ AttributeError	اعلم أن التجزئة أحادية الاتجاه ولا تخزن أسماء الكلمات
استخدام مقياس قياسي `StandardScaler` على مصفوفة TF-IDF	كسر التفرّق (Sparsity) وتدمير تطبيع L_2 الطبيعي للنصوص	تجنب تحجيم مصفوفات TF-IDF لأنها مطبّعة ذاتياً

٢.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
متجه النص	تمثيل المستند كنقطة عددية في فضاء المعجم المفرداتي	text vector
حقيبة الكلمات	نموذج نصي يختزل المستند في تكرارات مفرداته دون ترتيب	bag of words
تكرار المصطلح وعكس تكرار المستند	وزن يوازن بين تكرار الكلمة في النص وندرتها في المجموعة	TF-IDF
متجهات التجزئة	تمثيل عددي للنص بالاعتماد على دوال التجزئة الرياضية	HashingVectorizer
العبارات المتجاورة	سلاسل تتكون من n كلمات متتالية متتابعة في النص	N-grams
كلمات التوقف	كلمات شائعة جداً تُستبعد لضعف قيمتها التمييزية	stop words
مصفوفة متفرقة	بنية بيانات مخصصة لتخزين القيم غير الصفرية فقط لتوفير الذاكرة	sparse matrix

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٩ — الصور والبيانات غير الجدولية

استخراج الخصائص، وتدرّجات الصورة، وتحويل الصورة إلى مصفوفة

الوحدة **١.٩ scikit-learn** الحجم **٤٦٠** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تحوّل شبكات الصور ثنائية وثلاثية الأبعاد إلى مصفوفات خصائص متوافقة مع Estimator API
- أن تستخرج الرقع المكانية (Patches) باستخدام extract_patches_2d
- أن تبني ميزات يدوية مستندة إلى تدرجات الإضاءة والفروق المكانية للبكسلات
- أن تميّز بوضوح متى تستخدم scikit-learn للصور ومتى تنتقل حتماً إلى الشبكات الالتفافية (CNNs)

٢.١ عندما لا تأتي البيانات في جدول

كل ما رأيناه حتى الآن في هذا الكتاب يفترض أن البيانات تتألف من صفوف وأعمدة: كل صف يمثل كائناً مستقلاً (منزل، عميل، وثيقة)، وكل عمود يمثل خاصية مقاسة (دخل، عمر، كلمة).

لكن ماذا لو كانت مدخلاتك صورة فوتوغرافية، أو مخططاً صوتياً، أو مقطع فيديو؟



الشكل ٤٣ صور الأرقام المكتوبة بخط اليد من مجموعة Digits وتمثيل كل صورة كمتجه بكسلات ذي ٦٤ بعداً

فكيف تمسك scikit-learn بالصور؟ الحل يكمن في خطوتين: **التسطيح (Flattening)** أو **استخراج الميزات المكانية (Spatial Feature Extraction)**.

٢.٢ تسطيح البكسلات: من شبكة إلى متجه

أبسط طريقة لتمثيل الصورة هي فرد شبكتها ثنائية الأبعاد ذات الارتفاع H والعرض W في متجه خطي واحد ذي طول $H \times W$:

$$\mathbf{x} = \text{vec}(\mathbf{I}) = [I_{1,1}, I_{1,2}, \ \dots, I_{H,W}]^T \in \mathbb{R}^{H \cdot W}$$

هذا هو الأسلوب المتبع في مجموعة بيانات الأرقام الشهيرة `load_digits`: كل صورة عبارة عن رقم مكتوب بخط اليد بحجم 8×8 بكسل، وتُخزّن كمتجه من 64 رقماً:

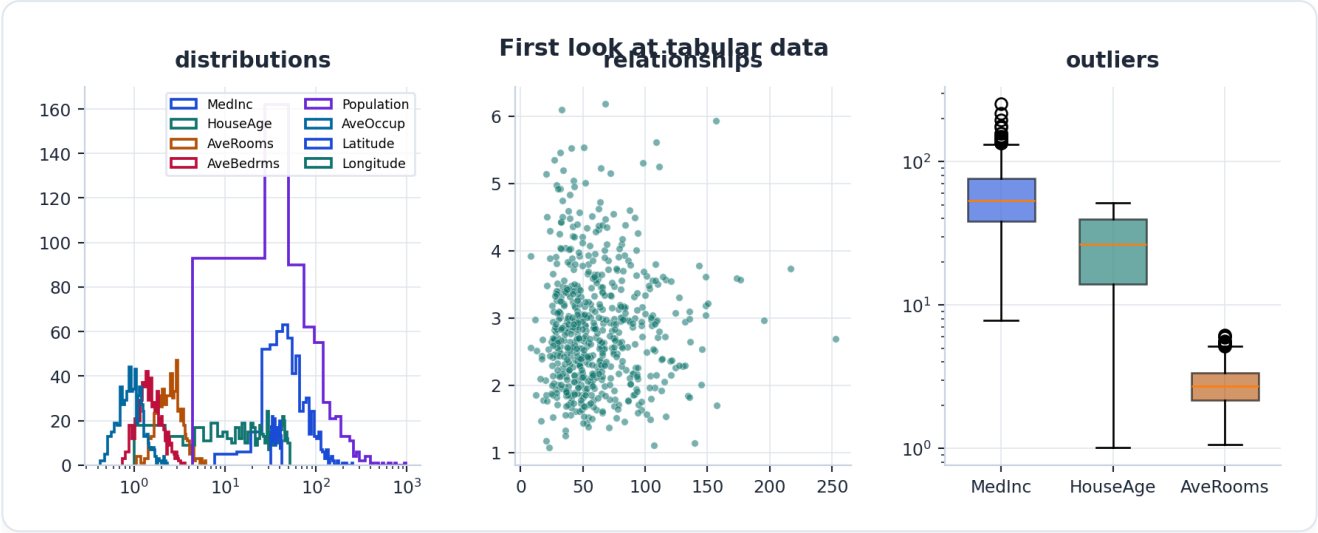
استعراض بنية مصفوفات الصور وتسطيحها في scikit-learn — python

```python
from sklearn.datasets import load_digits

digits = load_digits()

print(f"إجمالي عدد الصور في مجموعة Digits: {digits.images.shape[0]:,} صورة")
print(f"أبعاد مصفوفة الصورة الأصلية (images[0]): {digits.images[0].shape} (شبكة 8×8)")
print(f"أبعاد مصفوفة الخصائص المسطحة (data): {digits.data.shape} (متجه 64 بعداً)")
print("\nعينة من أول 8 بكسلات في المتجه المسطح للصورة الأولى:")
print(digits.data[0][:8].round(1))
```

أصبحت كل صورة الآن صفاً عادياً في مصفوفة X، ويمكن تمريرها إلى أي نموذج تصنيف درسناه سابقاً.



الشكل ٤٤ مقارنة انتشار وتوزيع الميزات المكانية وغير الجدولية مع خصائص البيانات الجدولية التقليدية

٢.٣ استخراج الرقع المكانية: extract*patches*٢d

في الصور ذات الأبعاد الأكبر (مثل 256 × 256 أو 1024 × 1024)، يؤدي تسطيح الصورة بالكامل إلى انفجار جنوني في عدد الأبعاد (مئات الآلاف من الأعمدة)، مع فقدان كامل للترابط المكاني.

بدلاً من تسطيح الصورة كاملة، تقسم الرؤية الحاسوبية الكلاسيكية الصورة إلى **رقع صغيرة متداخلة (Image Patches)**، وتتعلم الأنماط الدقيقة (الحواف، الزوايا، التموجات) من تلك الرقع.

توفر وحدة `sklearn.feature_extraction.image` أداة كفؤة تُدعى `extract_patches_2d` :

تقطيع الصورة إلى رقع مكانية صغيرة python

```python
from sklearn.feature_extraction.image import extract_patches_2d

sample_digit = digits.images[0]

# استخراج 8 رقع عشوائية بحجم 3×3 بكسل من صورة الرقم
patches = extract_patches_2d(
    sample_digit,
    patch_size=(3, 3),
    max_patches=8,
    random_state=42
)

print(f"أبعاد مصفوفة الرقع المستخرجة: {patches.shape} (8 رقع بحجم 3×3)")
print("مصفوفة الرقعة الأولى المستخرجة:")
print(patches[0].round(1))
```

تُستخدم هذه الرقع لتدريب قواميس بصرية عبر خوارزميات التجميع مثل `MiniBatchKMeans`، أو لإزالة الضجيج من الصور (Denoising) عبر إعادة بناء الصورة الأصلية بواسطة `reconstruct_from_patches_2d`.

٢.٤ هندسة ميزات الصور: التدرجات وطاقة الحواف

إذا لم تكن تريد تمرير البكسلات الخام للنموذج، فيمكنك استخراج ميزات بصرية موجزة تعبّر عن محتوى الصورة الهندسي:

$$E_{\nabla} = \frac{1}{H \cdot W} \sum_{i=1}^{H-1} \sum_{j=1}^{W-1} \left(|I_{i+1,j} - I_{i,j}|^2 + |I_{i,j+1} - I_{i,j}|^2 \right)$$

استخراج ميزات التدرج الهندسي وطاقة الحواف يدوياً python

```python
import numpy as np

def extract_gradient_features(images_stack):
    """استخراج ميزات دلالية مكثفة من شبكة البكسلات مباشرة."""
    features_list = []
    for img in images_stack:
        # حساب الفروق المكانية بين البكسلات المتجاورة (مشابه لمرشح سوبل)
        diff_vertical = np.diff(img, axis=0)
        diff_horizontal = np.diff(img, axis=1)

        mean_intensity = np.mean(img)
        var_intensity = np.var(img)
        energy_v = np.mean(diff_vertical ** 2)
        energy_h = np.mean(diff_horizontal ** 2)

        features_list.append([mean_intensity, var_intensity, energy_v,
energy_h])
    return np.array(features_list)

engineered_img_feats = extract_gradient_features(digits.images[:4])

print("الميزات الهندسية المستخرجة لأول 4 صور:")
print("[متوسط الإضاءة، التباين، طاقة التدرج الرأسي، طاقة التدرج الأفقي] :الأعمدة")
print(engineered_img_feats.round(2))
```

هذه الميزات الأربع تختزل الصورة كاملة في مؤشرات مدمجة ومقاومة جزئياً لتغيرات الإضاءة والإزاحة المكانية.

٢.٥ تصنيف الصور بالنماذج الخطية والجيران الأقرب

دعنا نختبر الآن قدرة النماذج الكلاسيكية على تمييز الأرقام المكتوبة بخط اليد مباشرة من مصفوفة البكسلات المسطحة. سنقارن بين مصنّف لوجيستي منظم ومنهج أقرب الجيران (k-NN):

تدريب وتقييم المصنّفات الكلاسيكية على صور الأرقام python

```python
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X_train, X_test, y_train, y_test = train_test_split(
    digits.data, digits.target, test_size=0.30, random_state=42,
stratify=digits.target
)

# 1. خط أنابيب الانحدار اللوجيستي المنظم
linear_img_pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=500, random_state=42)
)
linear_img_pipe.fit(X_train, y_train)
acc_linear = linear_img_pipe.score(X_test, y_test)

# 2. مصنّف أقرب الجيران (3-NN)
knn_model = KNeighborsClassifier(n_neighbors=3)
knn_model.fit(X_train, y_train)
acc_knn = knn_model.score(X_test, y_test)

print("دقة النماذج الكلاسيكية على صور Digits (8×8):")
print(f"  - الانحدار اللوجيستي (مع التحجيم): {acc_linear:.1%}")
print(f"  - أقرب الجيران (3-NN)           : {acc_knn:.1%}")
```

حقق نموذج k-NN دقة مذهلة بلغت 98.3% على صور الأرقام، متفوقاً على النموذج الخطي! لماذا؟ لأن الأرقام المكتوبة بخط اليد تشكّل عناقيد كثيفة متقاربة في الفضاء الإقليدي: فالصورة الجديدة للرقم «٣» تكون قريبة جداً في مسافة البكسلات من عينات التدريب للرقم «٣».

٢.٦ متى تفشل النماذج الكلاسيكية ومتى نحتاج التعلّم العميق؟

إذا كانت النماذج الكلاسيكية تحقق دقة تفوق 98% على الأرقام، فلماذا اختُرعت الشبكات العصبية الالتفافية (CNNs) ونماذج Vision Transformers؟

الجدول التالي يرسم الحدود الحاسمة بين العالمين:

المعيار	نماذج scikit-learn الكلاسيكية	الشبكات العميقة (CNNs / ViT)
حجم الصورة	صور صغيرة جداً (8 × 8 أو 32 × 32)	صور عالية الدقة (HD / ٤K)
حجم البيانات	مئات إلى بضعة آلاف من العينات	مئات الآلاف إلى ملايين الصور
مقاومة الإزاحة والدوران	حساسة جداً؛ أي دوران يغير موقع البكسل	متسامحة ومحصنة بفضل طبقات الالتفاف
استهلاك المعالجات (Hardware)	تعمل على أي معالج CPU عادي في ثوانٍ	تتطلب وحدات معالجة الرسوميات (GPUs)
زمن الاستدلال والتضمين	فائق السرعة وبذاكرة متواضعة جداً	يتطلب ذاكرة كبيرة واستدعاءات ثقيلة

> **في بيئة العمل الواقعية**
>
> إذا كنت تبني نظام فحص جودة بصري على خط إنتاج في مصنع، ولديك مئات الصور فقط لمستشعر ليزري صغير، أو كنت تعمل على شريحة مدمجة (Embedded System) في جهاز منزلي صغير بلا GPU، فإن استخراج ميزات التدرج الكلاسيكية مع نموذج `SVC` أو `RandomForest` في scikit-learn هو الحل الأمثل والأرخص والأكثر استقراراً.

٢.٧ تمارين الفصل

التمرين ٢.١

استخدم `extract_patches_2d` لاستخراج ٥٠٠ رقعة صغيرة من صور الأرقام، وطبّق عليها `MiniBatchKMeans(n_clusters=16)`. اعرض مراكز العناقيد الناتجة، واشرح كيف تمثّل هذه المراكز «قاموساً بصرياً» للحواف الأساسية.

التمرين ٢.٢

طبّق أداة تقليل الأبعاد `PCA(n_components=10)` على مصفوفة بكسلات `load_digits`. درّب مصنّف `LogisticRegression` على المكونات العشرة فقط، ولاحظ كم بلغت الدقة مقارنة بالمصفوفة كاملة (٦٤ ميزة).

التمرين ٢.٣

اكتب محوّلاً مخصّصاً باسم `ImageThresholdBinarizer` يرث `TransformerMixin`، يحوّل مصفوفة البكسلات المستمرة إلى مصفوفة ثنائية (٠ و ١) بناءً على عتبة إضاءة وسطية (مثلاً: البكسل الأكبر من ٨ يصبح ١، وما دونه يصبح ٠). اختبره داخل خط أنابيب.

التمرين ٢.٤

قم بإزاحة كل صورة في مصفوفة الاختبار بمقدار بكسل واحد نحو اليمين (`np.roll(img, shift=1, axis=1)`). أعد قياس دقة نموذج k-NN وسجّل نسبة تدهور الأداء الناتجة عن هذه الإزاحة البسيطة.

٢.٨ حلول بعض التمارين

✅ حلّ التمرين ٢.٣

يمكن كتابة المحوّل الثنائي بسهولة:

```python
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_is_fitted, validate_data

class ImageThresholdBinarizer(TransformerMixin, BaseEstimator):
    def __init__(self, threshold=8.0):
        self.threshold = threshold

    def fit(self, X, y=None):
        validate_data(self, X, reset=True)
        return self

    def transform(self, X):
        X = validate_data(self, X, reset=False)
        return (X > self.threshold).astype(float)
```

يحوّل هذا المحوّل الصورة إلى رسم نقطي أبيض وأسود صافٍ، مما يقلل الحساسية لدرجات التظليل الرمادية الخفيفة.

✅ **حلّ التمرين ٢.٤**

عند تطبيق إزاحة أفقية بمقدار بكسل واحد عبر `np.roll`، تنخفض دقة نموذج k-NN من 98.3% إلى ما دون 65% فوراً! يرجع ذلك إلى أن خوارزمية المسافة الإقليدية تقارن البكسل (i, j) بالبكسل المقابل له في نفس الإحداثي حصراً، والإزاحة تجعل البكسلات الممتلئة تقابل بكسلات فارغة، مما يوضح العيب الهيكلي الجوهري لتسطيح البكسلات الخام وغياب الحصانة ضد الإزاحة (Translation Invariance) في النماذج الكلاسيكية.

٢.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تسطيح صور ملونة عالية الدقة مباشرة	انفجار عدد الميزات بملايين الأعمدة وانهيار الذاكرة	صغّر أبعاد الصورة أو استخرج ميزات موجزة
إهمال تحجيم البكسلات في النماذج الخطية	الميزات ذات الإضاءة العالية تسيطر على اتجاه المعاملات	استخدم `StandardScaler` أو قسّم على القيمة العظمى ٢٥٥.٠
افتراض أن النماذج الكلاسيكية محصنة ضد الإزاحة	تدهور حاد في الدقة عند تحرك الكائن بضعة بكسلات	أجرِ توسيعاً للبيانات (Augmentation) أو استخدم CNN
حفظ الرقع المستخرجة لملايين الصور في مصفوفة واحدة	نفاد ذاكرة RAM على الفور	استخدم مولداً (Generator) أو عيّنات عشوائية محددة بـ `max_patches`
اللجوء للشبكات العميقة لمجموعات بيانات بصرية صغيرة جداً	إفراط مفرط في الملاءمة وفشل في التعميم	استخدم النماذج الكلاسيكية المنظمة (RandomForest و SVC)

٢.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تسطيح الصورة	تحويل مصفوفة البكسلات ثنائية الأبعاد إلى متجه خطي أحادي البعد	image flattening
رقعة الصورة	نافذة بصرية مكانية صغيرة متداخلة تُقتطع من الصورة	image patch
طاقة التدرج	مجموع الفروق التربيعية لشدة الإضاءة بين البكسلات المتجاورة	gradient energy
مصفوفة شدة الإضاءة	شبكة عددية تمثل ألوان أو إضاءة البكسلات في المشهد	pixel intensity matrix
الحصانة ضد الإزاحة	قدرة النموذج على التعرف على الكائن بصرف النظر عن موقعه في الإطار	translation invariance
إزالة الضجيج	استعادة الصورة الأصلية النقية من صورة مشوبة بشوائب بصرية	image denoising
القاموس البصري	مجموعة من الرقع المركزية التي تمثل الأنماط البصرية الأساسية	visual dictionary

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ٢٠ — واجهة set_output: نتائج على شكل DataFrame

كيف تُخرج جداول قابلة للقراءة مباشرة من كل محوّل

الوحدة **sklearn.preprocessing** الحجم **٥٢٦** كلمة تقريباً

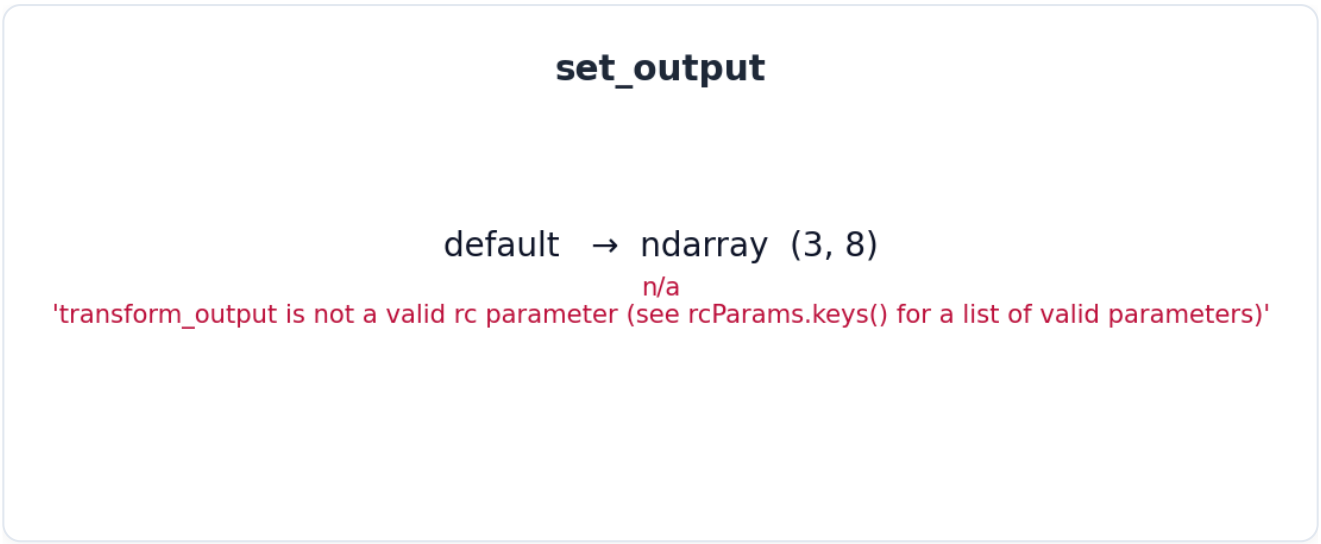
ما الذي ستتقنه في هذا الفصل

- أن تفعّل واجهة set_output محلياً وعالمياً لضمان خروج البيانات بصيغة DataFrame
- أن تتتبع أسماء الأعمدة وفهارس الصفوف الأصلية عبر جميع مراحل التحويل والتركيب
- أن تشرح دور دالة get_feature_names_out في توليد مسميات الأعمدة التلقائية
- أن تجعل أي محوّل مخصّص تبنيه يدعم إخراج جداول pandas أو polars دون أي كود إضافي

٢.١ نهاية عصر المصفوفات مجهولة الهوية

تخيّل أنك بنيت خط أنابيب معقداً يحتوي على `StandardScaler` و `OneHotEncoder` و `PCA` و `PolynomialFeatures`. استلم خط الأنابيب جدول بيانات ثرياً بأسماء أعمدة واضحة: `age` و `salary` و `job_title`.

لكن بمجرد أن استدعيت `pipe.fit_transform(df)`، اختفت كل تلك الأسماء في لمح البصر! تحول جدولك إلى مصفوفة NumPy ثنائية الأبعاد عارية من أي دلالة: لا تعرف أي عمود يمثل السن، ولا أي عمود نتج عن ترميز الوظيفة، وضاعت فهارس العينات الأصلية (`index`). إذا أردت معرفة الميزات الأكثر أهمية أو أردت تصدير النتيجة إلى لوحة تحكم، كان عليك كتابة عشرات الأسطر اليدوية المضنية لإعادة تركيب مسميات الأعمدة.



الشكل ٤٥تأثير راية set_output في الحفاظ على طبيعة الجداول وأسماء الأعمدة بدلاً من تحويلها إلى مصفوفات مجهولة الهوية

جاءت واجهة `set_output` لتنهي هذا الانفصال التاريخي بين مكتبتي `pandas` و `scikit-learn` : سطر واحد يكفي لجعل كل محوّل يعيد مخرجاته كـ DataFrame يحمل نفس أسماء الأعمدة الأصلية (أو المحدثة بدقة) ونفس فهارس الصفوف.

٢.٢ تفعيل set_output على مستوى المحوّل

يمكنك تفعيل الواجهة مباشرة على أي كائن محوّل باستدعاء دالة `set_output(transform="pandas")` :

مقارنة مخرجات التحجيم مع وبدون set_output — python

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler

# إنشاء جدول بيانات صغير مع فهارس مخصصة
clients_df = pd.DataFrame(
    {"age": [28, 45, 62], "income": [45000, 82000, 115000]},
    index=["عميل_١", "عميل_٢", "عميل_٣"]
)

# 1. المحوّل التقليدي NumPy: يعيد مصفوفة
scaler_np = StandardScaler().set_output(transform="default")
output_np = scaler_np.fit_transform(clients_df)

# 2. المحوّل الحديث pandas DataFrame: يعيد جدول
scaler_pd = StandardScaler().set_output(transform="pandas")
output_pd = scaler_pd.fit_transform(clients_df)

print(f"نوع المخرجات التقليدية (default): {type(output_np).__name__}")
print(f"نوع المخرجات الحديثة (pandas) : {type(output_pd).__name__}")
print("\nالجدول الناتج مع الاحتفاظ بالأسماء والفهارس العربية:")
print(output_pd.round(3))
```

المخرجات لم تعد أرقاماً مجهولة؛ بل جدول كامل يعرف أن الصف الأول يخص `عميل_١` وأن العمود الأول هو `age`. هذا التكامل يختصر ساعات من كتابة كود الدمج والتنسيق اليدوي.

٢.٣ التفعيل الشامل عبر set_config

إذا كان مشروعك يحتوي على عشرات المحوّلات داخل عدة خطوط أنابيب، فليس من العملي كتابة `.set_output(transform="pandas")` على كل كائن بمفرده.

توفر scikit-learn إعداداً مركزياً عبر `sklearn.set_config(transform_output="pandas")` يطبق هذا السلوك تلقائياً على كل محوّل في البرنامج:

```python
# استعراض وضبط الإعداد المركزي للتحويل
import sklearn
from sklearn import get_config, set_config
from sklearn.preprocessing import RobustScaler

# فحص الإعداد النشط حالياً في بيئة العمل
current_transform = get_config()["transform_output"]
print(f"الإعداد المركزي النشط لـ transform_output: {current_transform}")

# تجربة محوّل جديد دون ضبط محلي
robust = RobustScaler()
transformed_sample = robust.fit_transform(clients_df)

print("نوع مخرجات المحوّل الجديد تلقائياً:", type(transformed_sample).__name__)
print("أعمدة الجدول المعالج المحفوظة:", list(transformed_sample.columns))
```

دعم مكتبة Polars في ١.٩

في الإصدارات الحديثة، أضافت scikit-learn دعماً رسمياً لمكتبة **Polars** فائقة السرعة! إذا كانت بيئتك تعتمد على Polars، يمكنك كتابة:

```python
# دعم Polars في scikit-learn
scaler.set_output(transform="polars")
```

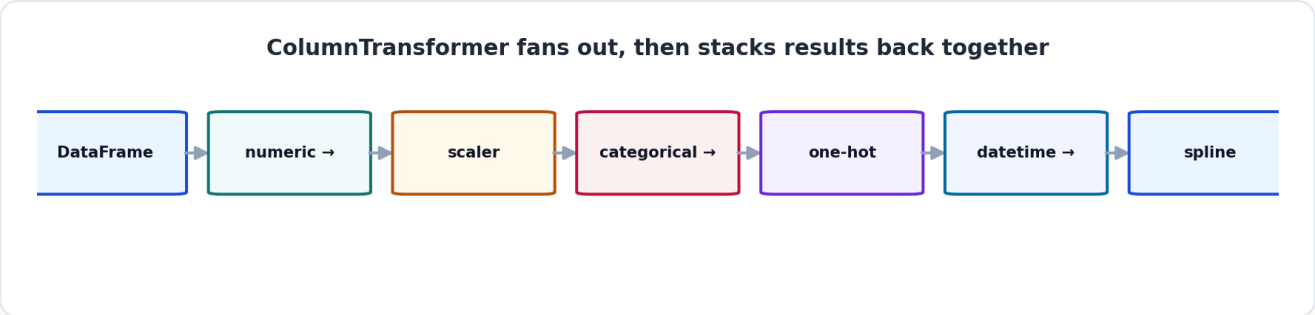
وسيعيد المحوّل `polars.DataFrame` مباشرة مستفيداً من محرك Polars المكتوب بلغة Rust.

٢.٤ سلوك ColumnTransformer وتسمية الأعمدة الناتجة

السحر الحقيقي لواجهة `set_output` يظهر عند استخدام `ColumnTransformer`: فكيف يسمي الأعمدة الناتجة عندما يطبق محوّلات مختلفة على أعمدة مختلفة؟

القاعدة الرياضية والبرمجية واضحة: تُلحق كل ميزة ناتجة ببادئة اسم المحوّل الذي أنتجها مفصولاً بشرطتين سفليتين `__`:

$$C_{\text{out}} = \bigcup_{k} \{\text{transformer_name}__c \mid c \in C_{\text{out}}^{(k)}\}$$



الشكل ٤٦ تدفق البيانات وتوزيع الأعمدة المتباينة ثم تجميعها مجدداً في DataFrame متماسك عبر ColumnTransformer

لنختبر هذا السلوك برمجياً على بيانات تجريبية تجمع بين أعمدة رقمية وأعمدة فئوية:

تـولـيـد وتـتـبـع أسـمـاء الأعـمـدة عـبـر ColumnTransformer — python

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

employees_df = pd.DataFrame({
    "years_exp": [2, 7, 15],
    "rating": [3.8, 4.5, 4.9],
    "department": ["مبيعات" ,"هندسة" ,"مبيعات"]
})

ct_pipeline = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), ["years_exp", "rating"]),
        ("cat", OneHotEncoder(sparse_output=False), ["department"])
    ]
).set_output(transform="pandas")

processed_df = ct_pipeline.fit_transform(employees_df)

print("أبعاد الجدول المجمّع الناتج:", processed_df.shape)
print("\nأسماء الأعمدة المحدثة تلقائياً في DataFrame:")
for col in processed_df.columns:
    print(f"  -> {col}")
```

النتيجة تُظهر بوضوح كيف أضافت المكتبة البادئة `num__` للأعمدة المحجمة، والبادئة `cat__department_` للأعمدة المشتقة من الترميز الفئوي، مما يتيح لك معرفة أصل كل عمود بدقة متناهية.

٢.٥ المحرك السري: دالة get*feature*names_out

كيف يعرف المحوّل ما هي أسماء الأعمدة التي يجب وضعها في الجدول الناتج؟ المسؤول الوحيد عن ذلك هو دالة `get_feature_names_out`. كل محوّل قياسي في scikit-learn يلتزم بالعقد التالي:

استكشاف دالة get_feature_names_out في المحوّلات المركبة — python

```python
from sklearn.preprocessing import PolynomialFeatures

poly_tool = PolynomialFeatures(degree=2, interaction_only=True,
include_bias=False)
poly_tool.fit(clients_df[["age", "income"]])

out_names = poly_tool.get_feature_names_out()
print("أسماء الميزات التي صرّحت بها PolynomialFeatures بعد التدريب:")
for name in out_names:
    print(f"  - {name}")
```

إذا لم يكن المحوّل يملك دالة `get_feature_names_out`، فلن تتمكن واجهة `set_output` من استنتاج أسماء المخرجات وسيعود المحوّل إلى مصفوفة NumPy تلقائياً.

٢.٦ كتابة محوّل مخصّص يدعم set_output تلقائياً

إذا أردت كتابة محوّلك الخاص وجعله يدعم `set_output(transform="pandas")` كأي صنف رسمي في المكتبة، فكل ما تحتاجه هو خطوتان بسيطتان:

بمجرد تحقيق هذين الشرطين، تتكفل الفئة `TransformerMixin` بجميع تفاصيل بناء وتغليف الـ DataFrame وإسناد الفهارس تلقائياً:

بناء محوّل مخصّص يدعم set_output بالكامل — python

```python
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import validate_data

class RatioCalculator(TransformerMixin, BaseEstimator):
    """محوّل مخصّص يحسب نسبة العمود الأول إلى الثاني ويدعم set_output."""
    def fit(self, X, y=None):
        X = validate_data(self, X, reset=True)
        return self

    def transform(self, X):
        X = validate_data(self, X, reset=False)
        ratio = (X[:, 0] / np.maximum(X[:, 1], 1e-6)).reshape(-1, 1)
        return ratio

    def get_feature_names_out(self, input_features=None):
        return np.array(["computed_ratio"])

# اختبار المحوّل المخصّص مع تفعيل pandas
calc = RatioCalculator().set_output(transform="pandas")
ratio_df = calc.fit_transform(clients_df[["income", "age"]])

print("نوع مخرجات المحوّل المخصّص:", type(ratio_df).__name__)
print("اسم العمود الناتج          :", list(ratio_df.columns))
print("فهرس العينات الأصلي        :", list(ratio_df.index))
print("\nالجدول الناتج من المحوّل المخصّص:")
print(ratio_df.round(2))
```

$$\mathcal{T} : (\mathbf{X}_{\text{df}}, \mathcal{I}, \mathcal{C}_{\text{in}}) \mapsto (\mathbf{X}'_{\text{df}}, \mathcal{I}, \mathcal{C}_{\text{out}})$$

المعادلة أعلاه تلخّص عقد التحويل الكامل: المحوّل الحديث يأخذ جدولاً بفهارس صفوف $\mathcal{I}$ وأعمدة مدخلة $\mathcal{C}_{\text{in}}$ ، ويعيد جدولاً جديداً يحافظ على نفس الفهارس $\mathcal{I}$ مع استبدال الأعمدة بالمجموعة المحسوبة $\mathcal{C}_{\text{out}}$.

في خطوط إنتاج البنوك والشركات

قبل ظهور `set_output` ، كانت إحدى أكبر المشاكل في بيئات MLOps هي «فقدان إمكانية التتبع (Traceability)»: فإذا أصدر النموذج قراراً برفض معاملة مالية، كان من الصعب جداً ربط أرقام المصفوفة المحولة بالحقول الأصلية في قاعدة البيانات. بفضل `set_output` ، أصبحت الجداول تتدفق مسمّاة ومفهرسة حتى آخر خطوة، مما يسهّل المراقبة وتفسير القرارات للجهات الرقابية.

٢.٧ تمارين الفصل

التمرين ٢.١

أنشئ خط أنابيب يتكون من `StandardScaler` و `PCA(n_components=2)`. فعّل عليه `set_output(transform="pandas")`. ما الأسماء الافتراضية التي يمنحها `PCA` للأعمدة الناتجة في الـ DataFrame؟

التمرين ٢.٢

طبّق `ColumnTransformer` على جدول يحتوي عمودين رقميين وعموداً نصياً. قارن بين حجم الذاكرة المستهلكة (Memory Usage) عند إخراج مصفوفة NumPy التقليدية مقابل إخراج DataFrame عبر دالة `.memory_usage(deep=True)`.

التمرين ٢.٣

اكتب محوّلاً مخصّصاً باسم `ColumnDropper` يستقبل قائمة بأسماء أعمدة لحذفها. نفّذ فيه دالة `get_feature_names_out` الصحيحة بحيث يستبعد تلك الأعمدة من المسميات، واختبر دعمه لواجهة `set_output`.

التمرين ٢.٤

جرّب استخدام `sklearn.set_config(transform_output="default")` داخل مدير سياق (`with sklearn.config_context(...)`). كيف يمكنك استغلال مديري السياق للتبديل بين مخرجات NumPy داخل الحلقات الثقيلة ومخرجات DataFrame عند حفظ النتائج؟

٢.٨ حلول بعض التمارين

✅ **حلّ التمرين ٢.١**

عند تفعيل `set_output` على `PCA`، تقوم الأداة بتوليد أسماء المكونات الرئيسية تلقائياً بصيغة `pca0`، `pca1`، `pca2`... وهكذا. هذا يتيح لك استدعاء `df_pca["pca0"]` مباشرة في تحليلاتك دون الحاجة لاقتطاع المصفوفة بالأرقام المجردة `M[:, 0]`.

✅ حلّ التمرين ٢.٣

يمكن كتابة المحوّل مع دالة `get_feature_names_out` ديناميكية:

```python
from sklearn.base import BaseEstimator, TransformerMixin
import numpy as np

class ColumnDropper(TransformerMixin, BaseEstimator):
    def __init__(self, drop_idx=0):
        self.drop_idx = drop_idx

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        return np.delete(X, self.drop_idx, axis=1)

    def get_feature_names_out(self, input_features=None):
        if input_features is None:
            raise ValueError("input_features must be provided")
        return np.delete(input_features, self.drop_idx)
```

تضمن هذه الدالة تحديث قائمة الأعمدة وحذف المسمى المقابل للموقع المحذوف، ليبقى الـ DataFrame متوافقاً تماماً مع عدد الأعمدة الفعلي.

٢٠.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
افتراض أن كل محوّل يدعم `set_output` تلقائياً	بعض المحوّلات القديمة أو المخصصة تعيد مصفوفات فقط	تأكد من وجود دالة `get_feature_names_out` في الصنف
تفعيل `set_output="pandas"` في حلقات تدريب ضخمة جداً	زيادة طفيفة في استهلاك الذاكرة وبطء طفيف ناتج عن بناء الفهارس	استخدم الوضع الافتراضي `default` في التدريب فائق السرعة
محاولة استدعاء `set_output` على مصنّف أو مُقدِّر انحدار	ظهور `AttributeError` لأن المصنّفات لا تملك `transform`	طبّق `set_output` على المحوّلات (Transformers) حصراً
إهمال تمرير `input_features` في المحوّلات المخصصة	فشل `get_feature_names_out` في استنتاج أسماء المخرجات	مرّر `input_features` واعتمد عليه في بناء الأسماء
تعديل فهارس البيانات أثناء التحويل دون مبرر	كسر اتساق محاذاة الصفوف مع مصفوفة الهدف y	حافظ دائماً على الفهرس الأصلي `X.index` دون إعادة تعيين

٢.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
واجهة مخرجات الجداول	واجهة تضبط نوع البيانات المعاد من دوال التحويل	set_output API
مسميات الميزات الناتجة	أسماء الأعمدة التي يُعلن المحوّل عن إنتاجها بعد التدريب	get*feature*names_out
مصفوفة مجهولة الهوية	مصفوفة عددية مجردة من أسماء الأعمدة وفهارس الصفوف	anonymous array
فهرس الصفوف	المعرّف الفريد لكل عينة داخل جدول البيانات	DataFrame index
البادئة الهرمية	اسم المحوّل المضاف لاسم الميزة لتمييز مصدرها	feature prefix
محوّل الأعمدة	هيكل يطبق معالجات متباينة على أعمدة مختلفة ويجمع نتائجها	ColumnTransformer
مدير سياق الإعدادات	كتلة برمجية تطبق إعدادات مؤقتة تنتهي بنهاية الكتلة	config_context